SMART SECTION REBUILD V5.1

Practice Solutions + Mini Projects Integrated

Ahetesham Khan | Python + Data + AI Engineer Interview Master

01 **LEARN**
Definition + easy explanation + example

02 **INTERVIEW**
Questions paired with model answers

03 **CODE**
Coding lab followed by solved practice

04 **THINK**
Logic + debugging before building

05 **BUILD**
Mini Project first, then Simple / Medium / Hard project labs

NEW SECTION ORDER

Section -> Interview Q&A -> Coding Lab -> Logic/Debug -> Solved Practice -> Mini Project -> Project Lab ->

Every mini project includes purpose, use case, architecture, files, runnable code, sample output, interview explanation and edge cases.

What changed in this rebuild

Practice answers and mini projects now live directly under the matching section.

01

SOLVED PRACTICE

Every coding lab now gets a dedicated solution spread. Question -> answer -> expected result -> why it works.

02

MINI PROJECT

Immediately after solved practice, a runnable mini project shows what the topic is actually used for.

03

PROJECT DETAIL

Purpose, "kis kaam aata hai?", architecture, files, build target, sample output and interview explanation are grouped together.

04

NUMPY / PANDAS

Beginner -> interview -> practical project flow stays attached to the relevant data sections.

05

RESUME ALIGNMENT

Chatbot, n8n Agentic AI, Power BI and analytics project stories remain tied to interview-ready explanations.

06

NAVIGATION

Final page numbers are based on the V5 PDF viewer order; section bookmarks are rebuilt.

HOW TO STUDY ONE SECTION

1. Read the concept once.
2. Speak the interview answers out loud.
3. Solve coding questions yourself before checking the solution.
4. Run the mini project, change one input, then explain one failure.

V5 PAGE MAP

Use the viewer page number. Each unit now has Solved Practice + Mini Project directly below its Logic/Debug Lab.

01 **Variables & Data Types**
L 12 | S 13 | M 14 | P 15

02 **Operators**
L 24 | S 25 | M 26 | P 27

03 **Input & Output**
L 35 | S 36 | M 37 | P 38

04 **Strings**
L 48 | S 49 | M 50 | P 51

05 **String Methods**
L 64 | S 65 | M 66 | P 67

06 **if / elif / else**
L 76 | S 77 | M 78 | P 79

07 **for & while Loops**
L 91 | S 92 | M 93 | P 94

08 **Lists**
L 102 | S 103 | M 104 | P 105

09 **List Methods**
L 115 | S 116 | M 117 | P 118

10 **Tuples**
L 126 | S 127 | M 128 | P 129

11 **Sets**
L 137 | S 138 | M 139 | P 140

12 **Dictionaries**
L 150 | S 151 | M 152 | P 153

13 **OOP (Object Oriented Programming)**
L 168 | S 169 | M 170 | P 171

14 **Functions & Reusable Python**
L 196 | S 197 | M 198 | P 199

15 **Exceptions, Files & Modules**
L 206 | S 207 | M 208 | P 209

16 **Advanced Python & Performance**
L 216 | S 217 | M 218 | P 219

24 **APIs, Flask/FastAPI & Production Python**
L 262 | S 263 | M 264 | P 265

27 **AI Engineer + Data Scientist Interview Mastery**
L 272 | S 273 | M 274 | P 275

28 **Coding Patterns, Projects & Final Practice**
L 282 | S 283 | M 284 | P 285

17 **NumPy for AI & Data Science**
L 307 | S 308 | M 309 | P 310

18 **Pandas for Data Science**
L 319 | S 320 | M 321 | P 322

19 **SQL + Databases**
L 331 | S 332 | M 333 | P 334

20 **Machine Learning + scikit-learn**
L 351 | S 352 | M 353 | P 354

21 **NLP + Transformers**
L 363 | S 364 | M 365 | P 366

22 **Generative AI, LLMs & RAG**
L 375 | S 376 | M 377 | P 378

23 **Agentic AI + LangChain + n8n**
L 387 | S 388 | M 389 | P 390

25 **Data Analytics + Power BI**
L 399 | S 400 | M 401 | P 402

26 **MLOps + Docker + Git + AWS**
L 411 | S 412 | M 413 | P 414

29 **Resume Project Interview Labs**
L 423 | S 424 | M 425 | P 426

LEGEND

L = Logic/Debug | S = Solved Practice | M = Mini Project | P = Existing Project Lab

Project Lab still contains Simple / Medium / Hard options; Mini Project is the first runnable build before it.

SECTION 01

Variables & Data Types

01

Level: Beginner | Time: ~45 min | 10 topics

Har program data ke saath kaam karta hai. Variable us data ko yaad rakhta hai, aur data type batata hai ki data **kis kism** ka hai (number, text, True/False...). Ye Python ki sabse pehli aur sabse important building block hai.

REAL-LIFE EXAMPLE SE SAMAJHO

Variable ek **label lage dabbe** jaisa hai. Dabbe ke upar naam likha hai (name), andar value rakhi hai ("Ashu"). Dabba badal sakte ho, label wahi rehta hai.

IS SECTION ME KYA SEEKHOGE

- ✓ Variable banana aur naming rules samajhna
- ✓ 8 main data types pehchanna
- ✓ type() se check karna aur type casting karna
- ✓ Dynamic typing aur mutable/immutable ka basic idea

TOPIC ROADMAP

1.1 Variable Assignment
1.2 Data Types
1.3 type()
1.4 Type Casting
1.5 Multiple Assignment
1.6 Dynamic Typing
1.7 None type
1.8 Naming Rules & Conventions ★
1.9 Unpacking with * (star) ★
1.10 Mutable vs Immutable + id() ★

★ = Advanced topic. Pehli baar padhte waqt inhe skip karke dusre round me karo.

1.1 Variable Assignment

CORE

Variable ek naam hota hai jo kisi value ko store karta hai. Python me type batana zaroori nahi, bas naam likho, = lagao aur value do.

STEP BY STEP: KAISE CHALTA HAI?

1. Python pehle **right side** ki value banata hai ("Ashu").
2. Phir **left side** ka naam us value par label ki tarah chipka deta hai.
3. print() naam se value nikal kar screen par dikhata hai.

main.py

```
name = "Ashu"
age = 22
height = 5.9
print(name, age, height)
```

▶ OUTPUT

```
Ashu 22 5.9
```

TIP

Naam hamesha **snake_case** me likho: student_name, total_marks. Naam number se start nahi ho sakta aur usme space nahi hoti.

GALTI SE BACHO

= ka matlab "barabar hai" nahi, "assign karo" hai. Barabar check karne ke liye == use hota hai.

1.2 Data Types

CORE

Python ke main built-in types: **int** (poora number), **float** (decimal), **str** (text), **bool** (True/False), aur collections: list, tuple, set, dict.

main.py

```
a = 10          # int
b = 10.5        # float
c = "hello"     # str
d = True        # bool
print(type(a), type(b), type(c), type(d))
```

▶ OUTPUT

```
<class 'int'> <class 'float'> <class 'str'> <class 'bool'>
```

YAAD RAKHO

int aur float = numbers, str = text, bool = sirf True ya False (T/F capital hona zaroori hai).

1.3 type() CORE

Kisi bhi variable ka data type check karne ke liye **type()** use hota hai. Debugging me sabse zyada kaam aata hai.

main.py

```
x = 5
print(type(x))

# Isinstance: True/False me answer
print(isinstance(x, int))
```

▶ OUTPUT

```
<class 'int'>
True
```

TIP

Agar program unexpected error de raha hai, sabse pehle **print(type(variable))** chalao. Aksar type galat hota hai.

1.4 Type Casting CORE

Ek data type ko dusre me convert karna. **int()**, **float()**, **str()**, **bool()** is kaam ke liye hain.

STEP BY STEP: KAISE CHALTA HAI?

1. "10" ek **string** hai, isliye "10" + 5 error deta.
2. int("10") usse asli number 10 bana deta hai.
3. Ab 10 + 5 = 15 ban jata hai.

main.py

```
x = "10"
y = int(x)
print(y + 5)

print(float("3.5"))
print(str(99) + " marks")
print(int(3.9))     # decimal kat jata hai, round nahi hota
```

▶ OUTPUT

```
15
3.5
99 marks
3
```

GALTI SE BACHO

int("abc") ya **int("3.5")** ValueError deta hai. Text me sirf poora number ho tabhi int() chalega.

1.5 Multiple Assignment

CORE

Ek hi line me kai variables ko values de sakte ho. Ye code chhota aur clean banata hai.

main.py

```
a, b, c = 1, 2, 3
print(a, b, c)

x = y = z = 0     # teeno ki value 0
print(x, y, z)
```

▶ OUTPUT

```
1 2 3
0 0 0
```

1.6 Dynamic Typing

CORE

Python me variable ka type **value** se decide hota hai, variable se nahi. Isliye ek hi naam ko baad me dusre type ki value bhi de sakte ho.

main.py

```
x = 5
print(type(x))
x = "hello"
print(type(x))
```

▶ OUTPUT

```
<class 'int'>
<class 'str'>
```

YAAD RAKHO

Type value ke saath jaata hai, naam ke saath nahi. Ye Python ki flexibility hai, lekin galtiyon ka reason bhi.

1.7 None type

CORE

None ka matlab "koi value nahi". Jab variable banana hai par abhi value nahi pata, tab use hota hai.

main.py

```
x = None
print(x)
print(type(x))

print(x is None)
```

▶ OUTPUT

```
None
<class 'NoneType'>
True
```

TIP

None check karne ke liye **is None** use karo, **== None** nahi. Ye Python ka standard tarika hai.

1.8 Naming Rules & Conventions ★ ADVANCED

Naam letters, digits aur underscore se bane ho sakte hain, par digit se start nahi. Python case-sensitive hai: **age** aur **Age** alag hain.

main.py

```
student_name = "Ashu"      # good
total_marks = 450          # good
_temp = 1                  # valid
MAX_SPEED = 120            # constant (convention)

# 2name = "x"    -> SyntaxError
# my name = "x" -> SyntaxError
# class = 5      -> SyntaxError (keyword)
print(student_name, MAX_SPEED)
```

▶ OUTPUT

```
Ashu 120
```

TIP

Constants ko **CAPITAL_LETTERS** me likhte hain. Python usse rokta nahi, ye sirf logon ke liye ek signal hai ki "isko badalna mat".

1.9 Unpacking with * (star) ★ ADVANCED

Kai values me se kuch alag nikalna ho aur baaki ek list me rakhna ho to * use karo.

main.py

```
first, *middle, last = [10, 20, 30, 40, 50]
print(first)
print(middle)
print(last)
```

▶ OUTPUT

```
10
[20, 30, 40]
50
```

1.10 Mutable vs Immutable + id() ★ ADVANCED

id() memory address batata hai. Immutable (int, str, tuple) badle nahi jaate, naya object banta hai. Mutable (list, dict, set) wahi object badalte hain.

main.py

```
a = "hi"
print(id(a) == id(a))

nums = [1, 2]
before = id(nums)
nums.append(3)
print(id(nums) == before)   # same object, sirf andar change

s = "hi"
s = s + "!"                 # naya string object bana
```

▶ OUTPUT

```
True
True
```

YAAD RAKHO

Immutable: int, float, str, bool, tuple | **Mutable:** list, dict, set. Ye difference aage aliasing bugs samajhne me kaam aayega.

QUICK RECAP: SECTION 01

- ✓ Variable = naam + value; = assign karta hai, == compare karta hai
- ✓ Main types: int, float, str, bool, list, tuple, set, dict, None
- ✓ type() check karta hai, int()/float()/str() convert karte hain
- ✓ Python dynamic typing use karta hai: type value ke saath badalta hai

PRACTICE TIME Khud try karo, phir Appendix C me solution dekho

EASY	**Q1.** Apna naam, age aur city 3 variables me store karo aur ek line me print karo.
MEDIUM	**Q2.** Do numbers string me hain: a = "12", b = "8". Unka sum (numeric) print karo.
HARD	**Q3.** Teesra variable use kiye bina a = 5 aur b = 9 ko swap karo.

INTERVIEW LAB / Variables & Data Types

Values ko store karna, type samajhna, conversion aur mutability ka base. • Answer formula: definition → tiny example → practical use. Word-by-word memorize mat karo; apni language me explain karo.

Q01

Variable kya hota hai?

BEST ANSWER

Variable ek naam/reference hai jo kisi value ko access karne deta hai. Example: name = "Ashu" me name variable hai aur "Ashu" value hai.

TIP: 20-30 sec me clearly bolo.

Q02

Python ke common data types kaun se hain?

BEST ANSWER

int, float, str, bool ke saath list, tuple, set aur dict common built-in types hain. NoneType bhi hota hai jab value nahi hai.

TIP: 20-30 sec me clearly bolo.

Q03

Python dynamically typed kya hota hai?

BEST ANSWER

Variable ka type pehle declare nahi karna padta. Value badalne par same name ke saath type bhi badal sakta hai.

TIP: 20-30 sec me clearly bolo.

Q04

type() aur isinstance() me fark?

BEST ANSWER

type(x) exact type batata hai. isinstance(x, T) check karta hai ki object T ya uske compatible parent type ka instance hai.

TIP: 20-30 sec me clearly bolo.

Q05

Mutable vs immutable?

BEST ANSWER

Mutable object ko same object me change kar sakte hain, jaise list/dict/set. Immutable me change karne par naya object banta hai, jaise int/str/tuple.

TIP: 20-30 sec me clearly bolo.

Q06

= aur == me fark?

BEST ANSWER

= assignment karta hai. == values compare karta hai aur True/False return karta hai.

TIP: 20-30 sec me clearly bolo.

Mini speaking drill: Har answer bina PDF dekhe 2 baar bolo. Dusri baar ek real example add karo.

CODING LAB / Variables & Data Types

Rule: Pehle problem ko 10 sec me apne words me bolo, phir code likho. Copy-paste ke bajaye khud type karo.

EASY

Name, age, city store karke f-string me profile print karo.

WRITE CODE

EASY

a = "12", b = "8" ko numeric sum me convert karke print karo.

WRITE CODE

MEDIUM

Teesre variable ke bina a = 5 aur b = 9 swap karo.

WRITE CODE

MEDIUM

first, *middle, last = [10,20,30,40,50] ka output predict karo.

WRITE CODE

HARD

Ek variable ka type print karo, phir uski value ko string me badal kar type dobara check karo.

WRITE CODE

DEBUG

int("3.5") error kyun dega? Sahi conversion ka code likho.

WRITE CODE

STARTER KIT - TRY WITHOUT LOOKING UP

```
type(x) isinstance(x, int) int(x) a,b=b,a
```

3-STEP DEBUG LOOP

- 1. Error/bug ko reproduce karo
- 2. Values/types/conditions print karke isolate karo
- 3. Ek change karo, rerun karo, phir reason explain karo

Interview coding line: "Main pehle input/edge case define karunga, phir simple correct solution likhunga, phir complexity aur edge cases check karunga."

LOGIC LAB / Variables & Data Types

Goal: code run karne se pehle output predict karo, phir dry-run karke reason bolo. Ye interview confidence aur debugging dono improve karta hai.

Predict

x = 5; x = "5"; print(type(x)) -> ?

CHECK

str

Trace

a=[1,2]; b=a; b.append(3); print(a)

CHECK

[1, 2, 3]

Think

x=None ho to x == None aur x is None me preferred check kaun sa hai?

CHECK

is None

Why

int(3.9) ka result kya hai?

CHECK

3 - truncation, round nahi

Fix

student name = "A" me problem kya hai?

CHECK

Space allowed nahi; student_name use karo

Explain

Interview me "Python me type variable ke saath hota hai" ko ek line me explain karo.

CHECK

Type value se decide hota hai, name se nahi.

5 COMMON MISTAKES TO WATCH

1. input ko number samajhna
2. = ko == samajhna
3. invalid identifier
4. mutable aliasing
5. None ko == se check karna

Mastery check: "Main is topic ko bina dekhe explain kar sakta hoon + 1 small program likh sakta hoon + 2 outputs predict kar sakta hoon."

SOLVED PRACTICE - Variables & Data Types

Every coding-lab question now has its answer, expected result and a quick check - all in one place.

EASY

Name, age, city store karke f-string me profile print karo.

```
name="Ahetesham"
age=23
city="Pune"
print(f"{name} | {age} | {city}")
```

EXPECTED RESULT

Ahetesham | 23 | Pune

EASY

a="12", b="8" ko numeric sum me convert karke print karo.

```
a="12"
b="8"
print(int(a)+int(b))
```

EXPECTED RESULT

20

MEDIUM

Teesre variable ke bina a=5 aur b=9 swap karo.

```
a,b=5,9
a,b=b,a
print(a,b)
```

EXPECTED RESULT

9 5

MEDIUM

first,*middle,last = [10,20,30,40,50] ka output predict

```
first,*middle,last=[10,20,30,40,50]
print(first,middle,last)
```

EXPECTED RESULT

10 [20, 30, 40] 50

HARD

Ek variable ka type print karo, phir value ko string me badal kar

```
x=5
print(type(x).__name__)
x=str(x)
print(type(x).__name__)
```

EXPECTED RESULT

int str

DEBUG

int("3.5") error kyun? Sahi conversion likho.

```
x="3.5"
print(float(x))
```

EXPECTED RESULT

3.5; int("3.5") ValueError deta hai.

PRACTICE HABIT

Practice rule: solve first, compare second, then change one input. Mark the question only after you can explain why the output changes.

MINI PROJECT - Profile Card CLI

Variables & Data Types ka concept ek runnable, interview-ready mini build me convert karo.

YE KYA KARTA HAI?

Name, age aur city ko clean f-string me show karta hai.

KIS KAAM AATA HAI?

Beginner projects, CLI tools, user metadata.

ARCHITECTURE

Input -> cast/validate -> variables -> formatted output

FILES / BUILD TARGET

app.py Target: 3 fields + 1 validation + 1 clean output

CORE IMPLEMENTATION - TYPE + RUN

```
name = input("Name: ").strip()
age = int(input("Age: "))
city = input("City: ").strip()
print(f"{name} | {age} | {city}")
```

SAMPLE OUTPUT `Name: Ahetesham | Age: 23 | City: Pune Printed result: Ahetesham | 23 | Pune`

INTERVIEW EXPLANATION

Project answer: Variables references hote hain; f-string readable output deta hai. Debug cue: age me invalid text aaye to ValueError; input validate ya try/except add karo. Follow-up: What would you improve first if this had 10x traffic/data?

BUILD STEPS

1. Create the file(s) shown above.
2. Run the exact sample once.
3. Change one input and observe the output.
4. Add one validation or failure test.

EDGE CASES / DEBUG

Test invalid input, boundary values and empty/missing data where relevant. Keep validation at the system boundary.

Variables & Data Types

Use this pack after the section labs: explain -> build -> debug -> defend.

SECTION ANCHOR

Definition: Values ko store karna aur unki types ko safely handle karna Python ki foundation hai.

Easy explanation: Variable = named reference; type value se decide hota hai.

Example: `name = "Ahetesham"; age = 22; print(f"{name} - {age}")`

SIMPLE

Profile Card CLI

Kya karta hai: Name, age aur city ko clean f-string me show karta hai.

Kis kaam aata hai: Beginner projects, CLI tools, user metadata.

Architecture: Input -> cast/validate -> variables -> formatted output

Files: `app.py`

Build target: 3 fields + 1 validation + 1 clean output

MEDIUM

Expense Splitter

Kya karta hai: Team expense ko numeric types me convert karke per-person amount nikalta hai.

Kis kaam aata hai: Finance utilities, billing scripts, data-entry tools.

Architecture: Input -> float/int cast -> calculation -> rounded output

Files: `splitter.py`

Build target: Handle zero people and two-decimal output

HARD

Config Type Validator

Kya karta hai: Nested config me required keys aur expected types validate karta hai.

Kis kaam aata hai: AI/API configs, environment settings, reliable application startup.

Architecture: Config -> schema rules -> validation -> fail-fast startup

Files: `config.py`

Build target: Return clear errors instead of cryptic failures

Project rule: code khud type karo, output predict karo, edge case test karo, phir 60-second explanation do.

Variables & Data Types - implementation lab

Three core implementations + project-tailored interview explanation + debugging cue.

SIMPLE | PROFILE CARD CLI

```
name = input("Name: ").strip()
age = int(input("Age: "))
city = input("City: ").strip()
print(f"{name} | {age} | {city}")
```

INTERVIEW DEFENSE

Project answer: Variables references hote hain; f-string readable output deta hai.

Debug cue: age me invalid text aaye to ValueError; input validate ya try/except add karo.

Follow-up: What would you improve first if this had 10x traffic/data?

MEDIUM | EXPENSE SPLITTER

```
total = float(input("Total: "))
people = int(input("People: "))
if people <= 0: raise ValueError("people > 0")
print(f"Per person: {total/people:.2f}")
```

INTERVIEW DEFENSE

Project answer: Type casting ke baad arithmetic safe hoti hai; division-by-zero edge case explicitly handle kiya.

Debug cue: String + number error aaye to type() se inspect karo.

Follow-up: What would you improve first if this had 10x traffic/data?

HARD | CONFIG TYPE VALIDATOR

```
cfg = {"model":"gpt", "timeout":30, "debug":False}
required = {"model":str,"timeout":int,"debug":bool}
for k,t in required.items():
    assert k in cfg and isinstance(cfg[k], t), k
```

INTERVIEW DEFENSE

Project answer: Production code me config boundary par validate karna runtime bugs ko early stop karta hai.

Debug cue: Missing key aur wrong type ko separate error messages do.

Follow-up: What would you improve first if this had 10x traffic/data?

SELF-CHECK

Can I explain the project without notes? Can I name one failure? Can I state time/space or cost trade-offs?

SECTION 02

Operators

Level: Beginner | Time: ~40 min | 10 topics

Operators values par kaam karke naya result dete hain. Har condition, har calculation aur har loop me inka use hota hai. Inko pakka kar loge to aage ka code aasaan lagega.

REAL-LIFE EXAMPLE SE SAMAJHO

Operators **calculator ke buttons** jaise hain: + - * / math karte hain, == != compare karte hain, and/or/not decisions ko jodte hain.

IS SECTION ME KYA SEEKHOGE

- ✓ Arithmetic, comparison aur logical operators
- ✓ Assignment shortcuts (+=, *=)
- ✓ in / not in aur is / is not ka fark
- ✓ Operator precedence (kaun pehle chalega)

TOPIC ROADMAP

2.1 Arithmetic Operators
2.2 Comparison Operators
2.3 Logical Operators
2.4 Assignment Operators
2.5 Membership Operators
2.6 Identity Operators
2.7 Operator Precedence ★
2.8 Chained Comparison ★
2.9 divmod(), round(), abs() ★
2.10 Short-circuit Evaluation ★

★ = Advanced topic. Pehli baar padhte waqt inhe skip karke dusre round me karo.

2.1 Arithmetic Operators

CORE

Basic math: **+ - * / % ** //**. Yaad rakho: **/** hamesha float deta hai, **//** floor division (poora hissa), **%** remainder.

main.py

```
a = 10
b = 3
print(a + b, a - b, a * b)
print(a / b)     # float
print(a % b)     # remainder
print(a ** b)    # power: 10 ka cube
print(a // b)    # quotient
```

▶ OUTPUT

```
13 7 30
3.3333333333333335
1
1000
3
```

TIP

Even/odd check: **n % 2 == 0** matlab even. Ye trick har jagah kaam aati hai.

2.2 Comparison Operators

CORE

Do values ko compare karke **True/False** deta hai: == != > < >= <=

main.py

```
a = 10
b = 20
print(a == b, a != b, a < b, a >= b)
```

▶ OUTPUT

```
False True True False
```

GALTI SE BACHO

= assignment hai aur == comparison. if x = 5: likhoge to SyntaxError aayega.

2.3 Logical Operators

CORE

Multiple conditions ko jodte hain: **and** (dono True), **or** (koi ek True), **not** (ulta).

main.py

```
a = True
b = False
print(a and b, a or b, not a)

age = 20
has_id = True
print(age >= 18 and has_id)
```

▶ OUTPUT

```
False True False
True
```

2.4 Assignment Operators

CORE

Value update karne ke shortcuts: **= += -= *= /= //= %= **=**

main.py

```
x = 5
x += 3      # x = x + 3
print(x)
x *= 2      # x = x * 2
print(x)
x -= 6
print(x)
```

▶ OUTPUT

```
8
16
10
```

2.5 Membership Operators

CORE

Check karta hai ki value kisi sequence (list, string, set...) me hai ya nahi: **in**, **not in**

main.py

```
fruits = ["apple", "banana"]
print("apple" in fruits)
print("mango" not in fruits)
print("py" in "python")
```

▶ OUTPUT

```
True
True
True
```

2.6 Identity Operators

CORE

Check karta hai ki do variables **same object** hain ya nahi: **is**, **is not**. Ye value nahi, memory identity dekhta hai.

main.py

```
a = [1, 2]
b = a
c = [1, 2]
print(a is b)     # same object
print(a is c)     # alag object
print(a == c)     # value same hai
```

▶ OUTPUT

```
True
False
True
```

YAAD RAKHO

== value compare karta hai, **is** object compare karta hai. Zyada tar jagah == hi chahiye. **is** sirf None ke liye.

2.7 Operator Precedence

★ ADVANCED

Kaun sa operator pehle chalega? Order: **()** → ****** → *** / // %** → **+ -** → comparison → **not** → **and** → **or**.

main.py

```
print(2 + 3 * 4)        # pehle * phir +
print((2 + 3) * 4)      # bracket pehle
print(2 ** 3 ** 2)      # right se left: 3**2=9, 2**9
print(not True or True) # not pehle
```

▶ OUTPUT

```
14
20
512
True
```

TIP

Shak ho to **bracket () laga do**. Code padhne me bhi aasaan hota hai.

2.8 Chained Comparison ★ ADVANCED

Python me math jaisa likh sakte ho: **1 < x < 10**. Dusri languages me ye and ke saath likhna padta hai.

main.py

```
x = 7
print(1 < x < 10)
print(0 <= x <= 5)

marks = 85
print(80 <= marks < 90)
```

▶ OUTPUT

```
True
False
True
```

2.9 divmod(), round(), abs() ★ ADVANCED

Kaam ke built-in math functions: **divmod** (quotient + remainder ek saath), **round**, **abs**.

main.py

```
print(divmod(17, 5))
print(round(3.14159, 2))
print(abs(-25))
print(-7 // 2)      # floor: negative side ki taraf jaata hai
```

▶ OUTPUT

```
(3, 2)
3.14
25
-4
```

GALTI SE BACHO

-7 // 2 ka answer -3 nahi, **-4** hai, kyunki floor division hamesha neeche ki taraf round karta hai.

2.10 Short-circuit Evaluation ★ ADVANCED

and/**or** result pata chalte hi ruk jaate hain, baaki check nahi karte. Isse errors bhi bach sakte hain.

main.py

```
x = 0
# x != 0 False hai, isliye 10 / x chalta hi nahi
print(x != 0 and 10 / x > 1)

name = ""
print(name or "Guest")     # khaali ho to default
```

▶ OUTPUT

```
False
Guest
```

QUICK RECAP: SECTION 02

- ✓ / float deta hai, // quotient, % remainder, ** power
- ✓ == value compare, is object compare
- ✓ and / or / not conditions ko jodte hain
- ✓ Shak ho to bracket lagao (precedence)

PRACTICE TIME Khud try karo, phir Appendix C me solution dekho

EASY	**Q1.** 17 ko 5 se divide karke quotient aur remainder print karo.
MEDIUM	**Q2.** n = 42 even hai ya nahi, True/False print karo.
HARD	**Q3.** age = 25 ke liye check karo ki 18 aur 60 ke beech (dono included) hai ya nahi. Chained comparison use karo.

INTERVIEW LAB / Operators

Calculation, comparison, conditions aur expression ka execution order. • Answer formula: definition → tiny example → practical use. Word-by-word memorize mat karo; apni language me explain karo.

Q01

/ aur // me fark?

BEST ANSWER

/ true division deta hai aur normally float result deta hai. // floor division quotient ka floor deta hai.

TIP: 20-30 sec me clearly bolo.

Q02

% operator kaha use hota hai?

BEST ANSWER

Remainder nikalne ke liye. Even/odd, cycle aur divisibility checks me bahut useful hai.

TIP: 20-30 sec me clearly bolo.

Q03

== aur is me fark?

BEST ANSWER

== value equality dekhta hai. is object identity dekhta hai - same object hai ya nahi.

TIP: 20-30 sec me clearly bolo.

Q04

and, or, not kya karte hain?

BEST ANSWER

and me sab conditions true chahiye, or me ek true kaafi hai, not boolean result ko invert karta hai.

TIP: 20-30 sec me clearly bolo.

Q05

Operator precedence kya hai?

BEST ANSWER

Ye decide karta hai kaun pehle evaluate hoga. Doubt ho to parentheses use karna readable aur safe hota hai.

TIP: 20-30 sec me clearly bolo.

Q06

Short-circuit evaluation kya hai?

BEST ANSWER

and/or result jaldi pata chalne par baaki expression evaluate nahi karte. Isse unnecessary work aur kuch errors avoid hote hain.

TIP: 20-30 sec me clearly bolo.

Mini speaking drill: Har answer bina PDF dekhe 2 baar bolo. Dusri baar ek real example add karo.

CODING LAB / Operators

Rule: Pehle problem ko 10 sec me apne words me bolo, phir code likho. Copy-paste ke bajaye khud type karo.

EASY

17 ko 5 se divide karke quotient aur remainder print karo.

WRITE CODE

EASY

n = 42 ka even/odd boolean check likho.

WRITE CODE

MEDIUM

age = 25 ko 18 aur 60 ke beech inclusive check karo.

WRITE CODE

MEDIUM

divmod(29, 6) use karke quotient aur remainder alag variables me lo.

WRITE CODE

HARD

x = 0 ke liye aisa condition likho jisme 10/x tabhi chale jab x zero na ho.

WRITE CODE

DEBUG

print(2 + 3 * 4) aur print((2+3)*4) ke alag outputs explain karo.

WRITE CODE

STARTER KIT - TRY WITHOUT LOOKING UP

```
// % == is and or not divmod(a,b)
```

3-STEP DEBUG LOOP

- 1. Error/bug ko reproduce karo
- 2. Values/types/conditions print karke isolate karo
- 3. Ek change karo, rerun karo, phir reason explain karo

Interview coding line: "Main pehle input/edge case define karunga, phir simple correct solution likhunga, phir complexity aur edge cases check karunga."

LOGIC LAB / Operators

Goal: code run karne se pehle output predict karo, phir dry-run karke reason bolo. Ye interview confidence aur debugging dono improve karta hai.

Predict

-7 // 2 -> ?

CHECK

-4

Trace

a=10,b=20; print(a < b and b < 30)

CHECK

True

Think

x=0; x != 0 and 10/x > 1 chalega?

CHECK

Nahi - short-circuit

Predict

2 ** 3 ** 2 -> ?

CHECK

512

Fix

if x = 5: me kya galat hai?

CHECK

Comparison ke liye ==

Explain

17 % 5 ka result 2 kyun?

CHECK

5*3=15, remainder 2

5 COMMON MISTAKES TO WATCH

1. / ko integer samajhna
2. == vs is
3. wrong precedence
4. negative floor ko truncate samajhna
5. and/or ka short-circuit ignore karna

Mastery check: "Main is topic ko bina dekhe explain kar sakta hoon + 1 small program likh sakta hoon + 2 outputs predict kar sakta hoon."

SOLVED PRACTICE - Operators

Every coding-lab question now has its answer, expected result and a quick check - all in one place.

EASY

17 ko 5 se divide karke quotient aur remainder print karo.

```
q,r=divmod(17,5)
print(q,r)
```

EXPECTED RESULT

3 2

EASY

n=42 ka even/odd boolean check likho.

```
n=42
print(n%2==0)
```

EXPECTED RESULT

True

MEDIUM

age=25 ko 18 aur 60 ke beech inclusive check karo.

```
age=25
print(18<=age<=60)
```

EXPECTED RESULT

True

MEDIUM

divmod(29,6) se quotient aur remainder variables me lo.

```
q,r=divmod(29,6)
print(q,r)
```

EXPECTED RESULT

4 5

HARD

x=0 ke liye 10/x tabhi chale jab x zero na ho.

```
x=0
print(x!=0 and 10/x>1)
```

EXPECTED RESULT

False

DEBUG

2+3*4 aur (2+3)*4 ke outputs explain karo.

```
print(2+3*4)
print((2+3)*4)
```

EXPECTED RESULT

14 20

PRACTICE HABIT

Practice rule: solve first, compare second, then change one input. Mark the question only after you can explain why the output changes.

MINI PROJECT - Bill Calculator

Operators ka concept ek runnable, interview-ready mini build me convert karo.

YE KYA KARTA HAI?

Subtotal, tax aur total amount calculate karta hai.

KIS KAAM AATA HAI?

Billing, e-commerce and small utilities.

ARCHITECTURE

Inputs -> arithmetic -> formatted receipt

FILES / BUILD TARGET

bill.py Target: Tax percentage configurable ho

CORE IMPLEMENTATION - TYPE + RUN

```
subtotal = 1200
tax = 0.18
total = subtotal * (1 + tax)
print(f"Total: {total:.2f}")
```

SAMPLE OUTPUT `Total: 1416.00 (1200 + 18% tax)`

INTERVIEW EXPLANATION

Project answer: Arithmetic operators calculation karte hain; f-string display ko readable banata hai. Debug cue: Percent ko 18 ki jagah 0.18 chahiye. Follow-up: What would you improve first if this had 10x traffic/data?

BUILD STEPS

1. Create the file(s) shown above.
2. Run the exact sample once.
3. Change one input and observe the output.
4. Add one validation or failure test.

EDGE CASES / DEBUG

Test invalid input, boundary values and empty/missing data where relevant. Keep validation at the system boundary.

Operators

Use this pack after the section labs: explain -> build -> debug -> defend.

SECTION ANCHOR

Definition: Arithmetic, comparison aur logical expressions har decision aur calculation ko power karte hain.

Easy explanation: Operators values ko transform ya compare karte hain.

Example: `score = 82; print(80 <= score < 90)`

SIMPLE

Bill Calculator

Kya karta hai: Subtotal, tax aur total amount calculate karta hai.

Kis kaam aata hai: Billing, e-commerce and small utilities.

Architecture: Inputs -> arithmetic -> formatted receipt

Files: `bill.py`

Build target: Tax percentage configurable ho

MEDIUM

Eligibility Checker

Kya karta hai: Age, ID aur score ko combine karke eligibility decide karta hai.

Kis kaam aata hai: Rule-based screening and workflow routing.

Architecture: Inputs -> comparisons -> and/or -> decision

Files: `eligibility.py`

Build target: Boundary values like 18 and 60 test karo

HARD

Rule Engine Core

Kya karta hai: Multiple business rules ko ordered predicates me evaluate karta hai.

Kis kaam aata hai: Fraud/risk routing, support triage, workflow automation.

Architecture: Event -> rule list -> first/priority match -> action

Files: `rules.py`

Build target: Rules ko data-driven banana

Project rule: code khud type karo, output predict karo, edge case test karo, phir 60-second explanation do.

Operators - implementation lab

Three core implementations + project-tailored interview explanation + debugging cue.

SIMPLE | BILL CALCULATOR

```
subtotal = 1200
tax = 0.18
total = subtotal * (1 + tax)
print(f"Total: {total:.2f}")
```

INTERVIEW DEFENSE

Project answer: Arithmetic operators calculation karte hain; f-string display ko readable banata hai.

Debug cue: Percent ko 18 ki jagah 0.18 chahiye.

Follow-up: What would you improve first if this had 10x traffic/data?

MEDIUM | ELIGIBILITY CHECKER

```
age, score = 24, 78
has_id = True
eligible = 18 <= age <= 60 and score >= 60 and has_id
print(eligible)
```

INTERVIEW DEFENSE

Project answer: Chained comparison readable hai; and tab useful hai jab all conditions required hon.

Debug cue: Operator precedence unclear ho to parentheses add karo.

Follow-up: What would you improve first if this had 10x traffic/data?

HARD | RULE ENGINE CORE

```
rules = [(lambda x: x["amount"] > 50000, "review"),
         (lambda x: x["vip"], "priority"),
         (lambda x: True, "normal")]
for ok, action in rules:
    if ok(event): break
```

INTERVIEW DEFENSE

Project answer: Operator logic ko rules me isolate karne se decisions testable aur extensible bante hain.

Debug cue: Rule order galat ho to broad rule early match kar sakta hai.

Follow-up: What would you improve first if this had 10x traffic/data?

SELF-CHECK

Can I explain the project without notes? Can I name one failure? Can I state time/space or cost trade-offs?

SECTION 03

Input & Output

Level: Beginner | Time: ~35 min | 8 topics

Ab tak values code me likhi thi. Ab user se data lena aur result sundar tarike se dikhana seekhenge. Ye har interactive program ki base hai.

REAL-LIFE EXAMPLE SE SAMAJHO

Program aur user ki **baat-cheet**: input() se program sunta hai, print() se program bolta hai.

IS SECTION ME KYA SEEKHOGE

- ✓ print() ke sep aur end
- ✓ input() hamesha string deta hai
- ✓ f-string se professional output
- ✓ Ek line me multiple inputs lena

TOPIC ROADMAP

3.1 print()
3.2 print() with sep & end
3.3 input()
3.4 Type Casting Input
3.5 Formatted Output (f-string)
3.6 f-string Formatting Tricks ★
3.7 Multiple Inputs in One Line ★
3.8 print(*list) & Unpacking ★

★ = Advanced topic. Pehli baar padhte waqt inhe skip karke dusre round me karo.

3.1 print() CORE

Screen par output dikhane ke liye. Ek saath kai values comma se de sakte ho.

main.py

```
print("Hello, Python!")
print("Age:", 22)
print()          # khaali line
print("Bye")
```

▶ OUTPUT

```
Hello, Python!
Age: 22

Bye
```

3.2 print() with sep & end

CORE

sep values ke beech ka separator set karta hai, **end** line ke last me kya aayega wo set karta hai (default \n).

main.py

```
print("A", "B", "C", sep="-", end="!\n")
print("Loading", end="...")
print("done")
```

▶ OUTPUT

```
A-B-C!
Loading...done
```

TIP

end="" se agla print usi line me aata hai. Progress bars aur patterns me kaam aata hai.

3.3 input()

CORE

User se keyboard se data lene ke liye. **Hamesha string** return karta hai, chahe user number hi type kare.

main.py

```
name = input("Enter your name: ")
print("Hello,", name)
```

▶ OUTPUT

```
Enter your name: Ashu
Hello, Ashu
```

GALTI SE BACHO

Sabse common galti: **input()** se number lekar seedha math karna. "5" + "3" = "53" ho jata hai, 8 nahi!

3.4 Type Casting Input

CORE

Number chahiye to **int()** ya **float()** se convert karo. Ek hi line me input aur conversion likh sakte ho.

STEP BY STEP: KAISE CHALTA HAI?

1. input() se "22" (string) aata hai.
2. int("22") usse number 22 banata hai.
3. age + 1 = 23 print hota hai.

main.py

```
age = int(input("Enter your age: "))
print(age + 1)
```

▶ OUTPUT

```
Enter your age: 22
23
```

3.5 Formatted Output (f-string) CORE

String ke aage **f** lagao aur variables ko **{ }** me likho. Sabse clean aur modern tarika.

main.py

```
name = "Ashu"
marks = 95
print(f"{name} scored {marks} marks")
print(f"Next year: {marks + 5}")   # expression bhi chalta hai
```

▶ OUTPUT

```
Ashu scored 95 marks
Next year: 100
```

3.6 f-string Formatting Tricks ★ ADVANCED

{value:.2f} decimal places, **{value:>8}** right align, **{value:,}** comma separator, **{value:.1%}** percentage.

main.py

```
price = 1499.5
print(f"Price: {price:.2f}")
print(f"Big: {1234567:,}")
print(f"|{42:>6}|{42:<6}|{42:^6}|")
print(f"Score: {0.856:.1%}")
```

▶ OUTPUT

```
Price: 1499.50
Big: 1,234,567
|    42|42    |  42  |
Score: 85.6%
```

3.7 Multiple Inputs in One Line ★ ADVANCED

User ek line me kai values de to **split()** se tod kar **map()** se convert karo.

main.py

```
a, b = map(int, input("Enter two numbers: ").split())
print("Sum:", a + b)
```

▶ OUTPUT

```
Enter two numbers: 4 9
Sum: 13
```

3.8 print(*list) & Unpacking ★ ADVANCED

List ke items ko bina bracket ke print karne ke liye * lagao.

main.py

```
nums = [1, 2, 3]
print(nums)
print(*nums)
print(*nums, sep=" | ")
```

▶ OUTPUT

```
[1, 2, 3]
1 2 3
1 | 2 | 3
```

QUICK RECAP: SECTION 03

- ✓ input() hamesha string deta hai, int()/float() se convert karo
- ✓ print me sep aur end control karte hain
- ✓ f"{x}" sabse best formatting hai; {x:.2f} decimals ke liye
- ✓ map(int, input().split()) multiple numbers ke liye

PRACTICE TIME Khud try karo, phir Appendix C me solution dekho

EASY	**Q1.** Naam input lo (name = "Ashu" maan lo) aur f-string se "Welcome, Ashu!" print karo.
MEDIUM	**Q2.** Do numbers a = 7, b = 3 ka average f-string se sirf 2 decimal places me print karo.
HARD	**Q3.** Ek line me 3 numbers input lo ("4 5 6") aur unka sum print karo.

INTERVIEW LAB / Input & Output

User se input lena, clean output banana aur values ko format karna. • Answer formula: definition → tiny example → practical use. Word-by-word memorize mat karo; apni language me explain karo.

Q01

input() kya return karta hai?

BEST ANSWER

input() hamesha string return karta hai, chahe user number type kare. Number chahiye to int() ya float() use karo.

TIP: 20-30 sec me clearly bolo.

Q02

sep aur end kya hain?

BEST ANSWER

sep multiple print values ke beech separator control karta hai. end line ke end ka text control karta hai; default newline hai.

TIP: 20-30 sec me clearly bolo.

Q03

f-string kyun use karte hain?

BEST ANSWER

Variables aur expressions ko readable tarike se string me embed karne ke liye. Ye modern aur concise hai.

TIP: 20-30 sec me clearly bolo.

Q04

split() + map() ka common use?

BEST ANSWER

Ek line ke space-separated inputs ko todkar har item ko same type me convert karna, jaise map(int, input().split()).

TIP: 20-30 sec me clearly bolo.

Q05

f-string me :.2f kya karega?

BEST ANSWER

Number ko 2 decimal places tak format karega.

TIP: 20-30 sec me clearly bolo.

Q06

print(*list) kya karta hai?

BEST ANSWER

List ke items ko unpack karke alag arguments ki tarah print karta hai.

TIP: 20-30 sec me clearly bolo.

Mini speaking drill: Har answer bina PDF dekhe 2 baar bolo. Dusri baar ek real example add karo.

CODING LAB / Input & Output

Rule: Pehle problem ko 10 sec me apne words me bolo, phir code likho. Copy-paste ke bajaye khud type karo.

EASY

Name input lo aur f-string se Welcome message print karo.

WRITE CODE

EASY

Do numbers input lekar sum aur average print karo.

WRITE CODE

MEDIUM

3 numbers ek line me input lo aur total print karo.

WRITE CODE

MEDIUM

price ko 2 decimals aur total ko comma format me print karo.

WRITE CODE

HARD

A, B, C ko sep="-" aur end="!" ke saath print karo; output predict karo.

WRITE CODE

DEBUG

"5" + "3" ka output 53 kyun aata hai? Numeric 8 kaise paoge?

WRITE CODE

STARTER KIT - TRY WITHOUT LOOKING UP

```
input() int(input()) f"{x}" split() map()
```

3-STEP DEBUG LOOP

- 1. Error/bug ko reproduce karo
- 2. Values/types/conditions print karke isolate karo
- 3. Ek change karo, rerun karo, phir reason explain karo

Interview coding line: "Main pehle input/edge case define karunga, phir simple correct solution likhunga, phir complexity aur edge cases check karunga."

LOGIC LAB / Input & Output

Goal: code run karne se pehle output predict karo, phir dry-run karke reason bolo. Ye interview confidence aur debugging dono improve karta hai.

Predict

a,b = map(int, "4 9".split()) -> sum?

CHECK

13

Trace

print("A","B","C",sep="-",end="! ")

CHECK

A-B-C!

Think

input se "22" aaya. age+1 se pehle kya karna hai?

CHECK

int conversion

Predict

f"{0.856:.1%}" -> ?

CHECK

85.6%

Fix

input().split() ka role kya hai?

CHECK

String ko pieces me todna

Explain

Professional output ke liye f-string ka fayda?

CHECK

Readable, concise formatting

5 COMMON MISTAKES TO WATCH

1. input ko int samajhna
2. string + number
3. format specifier miss karna
4. split se pehle conversion confusion
5. sep/end ka default bhoolna

Mastery check: "Main is topic ko bina dekhe explain kar sakta hoon + 1 small program likh sakta hoon + 2 outputs predict kar sakta hoon."

SOLVED PRACTICE - Input & Output

Every coding-lab question now has its answer, expected result and a quick check - all in one place.

EASY

Name input lo aur f-string se Welcome message print karo.

```
name=input().strip()
print(f"Welcome, {name}!")
```

EXPECTED RESULT

Welcome, Ahetesham!

EASY

Do numbers input lekar sum aur average print karo.

```
a,b=10,20
print(a+b,(a+b)/2)
```

EXPECTED RESULT

30 15.0

MEDIUM

3 numbers ek line me input lo aur total print karo.

```
a,b,c=map(int,"4 5 6".split())
print(a+b+c)
```

EXPECTED RESULT

15

MEDIUM

price ko 2 decimals aur total ko comma format me print karo.

```
price=1499.5
total=1234567
print(f"{price:.2f}")
print(f"{total:,}")
```

EXPECTED RESULT

1499.50 1,234,567

HARD

A,B,C ko sep="-" aur end="!" ke saath print karo.

```
print("A","B","C",sep="-",end="!")
```

EXPECTED RESULT

A-B-C!

DEBUG

"5"+"3" ka output 53 kyun? Numeric 8 kaise?

```
print("5"+"3")
print(int("5")+int("3"))
```

EXPECTED RESULT

53 8

PRACTICE HABIT

Practice rule: solve first, compare second, then change one input. Mark the question only after you can explain why the output changes.

MINI PROJECT - Welcome CLI

Input & Output ka concept ek runnable, interview-ready mini build me convert karo.

YE KYA KARTA HAI?

User name lekar personalized greeting deta hai.

KIS KAAM AATA HAI?

CLI tools aur beginner programs.

ARCHITECTURE

Keyboard -> input -> strip -> output

FILES / BUILD TARGET

main.py Target: Blank name ko reject karo

CORE IMPLEMENTATION - TYPE + RUN

```
name = input("Name: ").strip()
if not name:
    print("Name required")
else:
    print(f"Welcome, {name}!")
```

SAMPLE OUTPUT `Welcome, Ahetesham!`

INTERVIEW EXPLANATION

Project answer: Input boundary par strip aur empty-check simple quality improvement hai. Debug cue: input() output ko number samajhkar arithmetic mat karo. Follow-up: What would you improve first if this had 10x traffic/data?

BUILD STEPS

1. Create the file(s) shown above.
2. Run the exact sample once.
3. Change one input and observe the output.
4. Add one validation or failure test.

EDGE CASES / DEBUG

Test invalid input, boundary values and empty/missing data where relevant. Keep validation at the system boundary.

Input & Output

Use this pack after the section labs: explain -> build -> debug -> defend.

SECTION ANCHOR

Definition: Program aur user ke beech boundary: input lo, validate karo, useful output do.

Easy explanation: input() string deta hai; print()/f-string presentation control karte hain.

Example: `a, b = map(int, input().split()); print(a+b)`

SIMPLE

Welcome CLI

Kya karta hai: User name lekar personalized greeting deta hai.

Kis kaam aata hai: CLI tools aur beginner programs.

Architecture: Keyboard -> input -> strip -> output

Files: `main.py`

Build target: Blank name ko reject karo

MEDIUM

Quiz Score Reporter

Kya karta hai: Multiple answers ka score nikal kar percentage report karta hai.

Kis kaam aata hai: Training apps, assessments, small dashboards.

Architecture: Answers -> scoring loop -> percentage -> report

Files: `quiz.py`

Build target: Zero-question case handle karo

HARD

Interactive Log Reporter

Kya karta hai: Line-by-line log input ko category counts me convert karta hai.

Kis kaam aata hai: Support/monitoring scripts aur debugging utilities.

Architecture: Raw lines -> parse -> classify -> counters -> summary

Files: `report.py`

Build target: Malformed lines skip karke reason record karo

Project rule: code khud type karo, output predict karo, edge case test karo, phir 60-second explanation do.

Input & Output - implementation lab

Three core implementations + project-tailored interview explanation + debugging cue.

SIMPLE | WELCOME CLI

```
name = input("Name: ").strip()
if not name:
    print("Name required")
else:
    print(f"Welcome, {name}!")
```

INTERVIEW DEFENSE

Project answer: Input boundary par strip aur empty-check simple quality improvement hai.

Debug cue: input() output ko number samajhkar arithmetic mat karo.

Follow-up: What would you improve first if this had 10x traffic/data?

MEDIUM | QUIZ SCORE REPORTER

```
answers = [True, False, True, True]
score = sum(answers)
total = len(answers)
pct = (score/total*100) if total else 0
print(f"Score: {pct:.1f}%")
```

INTERVIEW DEFENSE

Project answer: Output formatting aur empty-input edge case dono important hain.

Debug cue: sum(bool_list) works because True/False map to 1/0.

Follow-up: What would you improve first if this had 10x traffic/data?

HARD | INTERACTIVE LOG REPORTER

```
counts = {}
for line in lines:
    parts = line.split(" ", 2)
    if len(parts) < 3: continue
    level = parts[1]
    counts[level] = counts.get(level, 0) + 1
print(counts)
```

INTERVIEW DEFENSE

Project answer: Aap raw input ko structured output me transform kar rahe ho; ye API/log pipelines ka same pattern hai.

Debug cue: split assumptions fail ho sakte hain; len(parts) guard rakho.

Follow-up: What would you improve first if this had 10x traffic/data?

SELF-CHECK

Can I explain the project without notes? Can I name one failure? Can I state time/space or cost trade-offs?

SECTION 04

Strings

Level: Beginner | Time: ~50 min | 11 topics

Text har jagah hai: naam, message, password, file path. Strings me indexing aur slicing ek baar samajh gaye to lists aur tuples me bhi wahi kaam aayega.

REAL-LIFE EXAMPLE SE SAMAJHO

String **moti ki mala** hai: har moti (character) ki ek position (index) hoti hai, 0 se shuru. Mala ka ek hissa kaat sakte ho, par ek moti badal nahi sakte.

IS SECTION ME KYA SEEKHOGE

- ✓ Strings banana (single, double, triple quotes)
- ✓ Indexing aur slicing (start:end:step)
- ✓ Concatenation, repetition, len()
- ✓ Escape characters aur immutability

TOPIC ROADMAP

4.1 Creating Strings
4.2 Indexing
4.3 Slicing
4.4 Concatenation
4.5 Repetition
4.6 len()
4.7 Escape Characters
4.8 Slicing with Step & Reverse ★
4.9 Strings are Immutable ★
4.10 Looping through a String & in ★
4.11 Raw Strings (r"") ★

★ = Advanced topic. Pehli baar padhte waqt inhe skip karke dusre round me karo.

4.1 Creating Strings

CORE

String single ' ', double " " ya triple ''' ''' quotes me likhi ja sakti hai. Triple quotes multi-line ke liye.

main.py

```
s1 = 'hello'
s2 = "world"
s3 = '''multi
line'''
print(s1, s2)
print(s3)

print("It's fine")     # andar apostrophe: double quotes use karo
```

▶ OUTPUT

```
hello world
multi
line
It's fine
```

4.2 Indexing CORE

Har character ka index hota hai, **0 se start**. Negative index end se ginta hai (-1 = last).

STEP BY STEP: KAISE CHALTA HAI?

1. P=0, y=1, t=2, h=3, o=4, n=5
2. Peeche se: n=-1, o=-2, h=-3, t=-4, y=-5, P=-6
3. text[3] = "h"

main.py

```
text = "Python"
print(text[0])
print(text[-1])
print(text[3])
```

▶ OUTPUT

```
P
n
h
```

GALTI SE BACHO

Range se bahar ka index **IndexError** deta hai: "Python"[10] error hai.

4.3 Slicing CORE

Substring nikalne ke liye **[start:end]**. Start included hota hai, **end excluded**.

main.py

```
text = "Python"
print(text[0:3])
print(text[2:])
print(text[:4])
print(text[-3:])
```

▶ OUTPUT

```
Pyt
thon
Pyth
hon
```

YAAD RAKHO

[start:end] me end wala index **shamil nahi** hota. text[0:3] = index 0, 1, 2.

4.4 Concatenation CORE

Do strings ko **+** operator se jodna.

main.py

```
first = "Hello"
second = "World"
print(first + " " + second)
```

▶ OUTPUT

```
Hello World
```

GALTI SE BACHO

String + number nahi chalta: "Age: " + 22 **TypeError** deta hai. f-string ya str(22) use karo.

4.5 Repetition CORE

String ko * se kai baar repeat karna.

main.py

```
text = "ab"
print(text * 3)
print("-" * 20)      # line banane ka mast tarika
```

▶ OUTPUT

```
ababab
--------------------
```

4.6 len() CORE

String me total characters ginta hai (space bhi count hota hai).

main.py

```
text = "Python"
print(len(text))
print(len("Hi there"))
```

▶ OUTPUT

```
6
8
```

4.7 Escape Characters — CORE

Special characters ke liye backslash: **\n** new line, **\t** tab, \\ backslash, \" quote.

main.py

```
print("Line1\nLine2")
print("Name:\tAshu")
print("Path: C:\\Users")
print("She said \"Hi\"")
```

▶ OUTPUT

```
Line1
Line2
Name:	Ashu
Path: C:\Users
She said "Hi"
```

4.8 Slicing with Step & Reverse — ★ ADVANCED

Teesra number step hai: **[start:end:step]**. Step -1 se string ulti ho jati hai.

main.py

```
text = "Python"
print(text[::2])      # har doosra character
print(text[::-1])     # reverse
print(text[1:5:2])
```

▶ OUTPUT

```
Pto
nohtyP
yh
```

TIP

text[::-1] reverse karne ka sabse famous trick hai. Palindrome check me kaam aata hai: text == text[::-1].

4.9 Strings are Immutable — ★ ADVANCED

String banne ke baad uska ek character badal nahi sakte. Naya string banana padta hai.

main.py

```
s = "hello"
try:
    s[0] = "J"
except TypeError as e:
    print("Error:", e)

s = "J" + s[1:]      # naya string
print(s)
```

▶ OUTPUT

```
Error: 'str' object does not support item assignment
Jello
```

4.10 Looping through a String & in

★ ADVANCED

String ke har character par loop chala sakte ho, aur **in** se substring check kar sakte ho.

main.py

```
for ch in "Hey":
    print(ch)

print("ell" in "hello")
print("xyz" in "hello")
```

▶ OUTPUT

```
H
e
y
True
False
```

4.11 Raw Strings (r"")

★ ADVANCED

Backslash ko normal character ki tarah rakhne ke liye string ke aage **r** lagao. File paths me kaam aata hai.

main.py

```
path = r"C:\new\test"
print(path)

normal = "C:\new"       # \n new line ban gaya
print(normal)
```

▶ OUTPUT

```
C:\new\test
C:
ew
```

QUICK RECAP: SECTION 04

- ✓ Index 0 se start, negative index end se
- ✓ [start:end:step], end excluded, [::-1] reverse
- ✓ + jodta hai, * repeat karta hai, len() length deta hai
- ✓ Strings immutable hain, badalne ke liye naya string banta hai

PRACTICE TIME Khud try karo, phir Appendix C me solution dekho

EASY	**Q1.** "Programming" ka pehla aur aakhri character print karo.
MEDIUM	**Q2.** "Python" ko reverse karke print karo.
HARD	**Q3.** Check karo ki "madam" palindrome hai ya nahi (True/False).

INTERVIEW LAB / Strings

Text ko index, slice, join aur search karke process karna. • Answer formula: definition → tiny example → practical use. Word-by-word memorize mat karo; apni language me explain karo.

Q01

String indexing kis se start hoti hai?

BEST ANSWER

Index 0 se start hota hai. Negative indexing me -1 last character hota hai.

TIP: 20-30 sec me clearly bolo.

Q02

Slicing me end included hota hai?

BEST ANSWER

Nahi. [start:end] me start included aur end excluded hota hai.

TIP: 20-30 sec me clearly bolo.

Q03

String immutable kyun yaad rakhna hai?

BEST ANSWER

Existing string ka character direct replace nahi kar sakte. Naya string banana padta hai.

TIP: 20-30 sec me clearly bolo.

Q04

[::-1] kya karta hai?

BEST ANSWER

String ko reverse karta hai because step -1 hota hai.

TIP: 20-30 sec me clearly bolo.

Q05

String me substring kaise check karte hain?

BEST ANSWER

in operator se, jaise "py" in "python".

TIP: 20-30 sec me clearly bolo.

Q06

len() kya count karta hai?

BEST ANSWER

String ke total characters, spaces samet.

TIP: 20-30 sec me clearly bolo.

Mini speaking drill: Har answer bina PDF dekhe 2 baar bolo. Dusri baar ek real example add karo.

CODING LAB / Strings

Rule: Pehle problem ko 10 sec me apne words me bolo, phir code likho. Copy-paste ke bajaye khud type karo.

EASY

"Programming" ka first aur last character print karo.

WRITE CODE

EASY

"Python" reverse print karo.

WRITE CODE

MEDIUM

"madam" palindrome hai ya nahi check karo.

WRITE CODE

MEDIUM

Kisi string me vowels count karo.

WRITE CODE

HARD

Text ko 3-3 characters ki slices me inspect karke output predict karo.

WRITE CODE

DEBUG

s="hello"; s[0]="H" error kyun? Sahi tarika likho.

WRITE CODE

STARTER KIT - TRY WITHOUT LOOKING UP

```
s[0] s[-1] s[a:b] s[::-1] len(s) in
```

3-STEP DEBUG LOOP

- 1. Error/bug ko reproduce karo
- 2. Values/types/conditions print karke isolate karo
- 3. Ek change karo, rerun karo, phir reason explain karo

Interview coding line: "Main pehle input/edge case define karunga, phir simple correct solution likhunga, phir complexity aur edge cases check karunga."

LOGIC LAB / Strings

Goal: code run karne se pehle output predict karo, phir dry-run karke reason bolo. Ye interview confidence aur debugging dono improve karta hai.

Predict

"Python"[-1] -> ?

CHECK

n

Predict

"Python"[1:4] -> ?

CHECK

yth

Trace

"Python"[::-1] -> ?

CHECK

nohtyP

Think

"Hi there" ki len()?

CHECK

8

Fix

IndexError kab aayega?

CHECK

Range se bahar index par

Explain

Slice ka end exclude kyun useful hai?

CHECK

Ranges ko predictable aur chain karna easy hota hai

5 COMMON MISTAKES TO WATCH

1. index 1 se start samajhna
2. slice end include samajhna
3. negative index bhoolna
4. immutable string modify karna
5. space ko len me ignore karna

Mastery check: "Main is topic ko bina dekhe explain kar sakta hoon + 1 small program likh sakta hoon + 2 outputs predict kar sakta hoon."

SOLVED PRACTICE - Strings

Every coding-lab question now has its answer, expected result and a quick check - all in one place.

EASY

"Programming" ka first aur last character print karo.

```
s="Programming"
print(s[0],s[-1])
```

EXPECTED RESULT

P g

EASY

"Python" reverse print karo.

```
print("Python"[::-1])
```

EXPECTED RESULT

nohtyP

MEDIUM

"madam" palindrome hai ya nahi check karo.

```
s="madam"
print(s==s[::-1])
```

EXPECTED RESULT

True

MEDIUM

Kisi string me vowels count karo.

```
s="ahetesham".lower()
print(sum(ch in "aeiou" for ch in s))
```

EXPECTED RESULT

4

HARD

Text ko 3-3 characters ki slices me inspect karo.

```
s="abcdefgh"
for i in range(0,len(s),3): print(s[i:i+3])
```

EXPECTED RESULT

abc def gh

DEBUG

s[0]="H" error kyun? Sahi tarika.

```
s="hello"
s="H"+s[1:]
print(s)
```

EXPECTED RESULT

Hello; strings immutable hain.

PRACTICE HABIT

Practice rule: solve first, compare second, then change one input. Mark the question only after you can explain why the output changes.

MINI PROJECT - Username Cleaner

Strings ka concept ek runnable, interview-ready mini build me convert karo.

YE KYA KARTA HAI?

Username ko trim aur lowercase karta hai.

KIS KAAM AATA HAI?

Login forms aur user-profile normalization.

ARCHITECTURE

Raw text -> strip -> lower -> validate

FILES / BUILD TARGET

clean.py Target: Spaces aur empty string handle karo

CORE IMPLEMENTATION - TYPE + RUN

```
raw = "  Ahetesham_Khan  "
user = raw.strip().lower()
print(user)
```

SAMPLE OUTPUT `ahetesham_khan`

INTERVIEW EXPLANATION

Project answer: Normalization consistent comparisons ke liye useful hai. Debug cue: Invisible spaces ke liye repr(text) use karo. Follow-up: What would you improve first if this had 10x traffic/data?

BUILD STEPS

1. Create the file(s) shown above.
2. Run the exact sample once.
3. Change one input and observe the output.
4. Add one validation or failure test.

EDGE CASES / DEBUG

Test invalid input, boundary values and empty/missing data where relevant. Keep validation at the system boundary.

Strings

Use this pack after the section labs: explain -> build -> debug -> defend.

SECTION ANCHOR

Definition: Text processing ka base: slicing, indexing, formatting aur search operations.

Easy explanation: String immutable hoti hai; change karne par nayi string banti hai.

Example: `text = "Python"; print(text[0], text[-1])`

SIMPLE

Username Cleaner

Kya karta hai: Username ko trim aur lowercase karta hai.

Kis kaam aata hai: Login forms aur user-profile normalization.

Architecture: Raw text -> strip -> lower -> validate

Files: `clean.py`

Build target: Spaces aur empty string handle karo

MEDIUM

Text Statistics

Kya karta hai: Words, characters aur first/last token count karta hai.

Kis kaam aata hai: Content analytics and NLP preprocessing.

Architecture: Text -> split -> aggregate -> report

Files: `stats.py`

Build target: Empty text safely handle karo

HARD

Prompt Normalizer

Kya karta hai: User prompts ko normalize karke predictable downstream input banata hai.

Kis kaam aata hai: LLM apps me input hygiene aur deduplication.

Architecture: User text -> normalize -> policy checks -> model layer

Files: `prompt_utils.py`

Build target: Collapse repeated whitespace and limit length

Project rule: code khud type karo, output predict karo, edge case test karo, phir 60-second explanation do.

Strings - implementation lab

Three core implementations + project-tailored interview explanation + debugging cue.

SIMPLE | USERNAME CLEANER

```
raw = "  Ahetesham_Khan  "
user = raw.strip().lower()
print(user)
```

INTERVIEW DEFENSE

Project answer: Normalization consistent comparisons ke liye useful hai.

Debug cue: Invisible spaces ke liye repr(text) use karo.

Follow-up: What would you improve first if this had 10x traffic/data?

MEDIUM | TEXT STATISTICS

```
text = "python makes ai practical"
words = text.split()
print(len(text), len(words), words[0] if words else None)
```

INTERVIEW DEFENSE

Project answer: String ops se lightweight preprocessing ho jata hai; heavy NLP ke liye specialized tools use karo.

Debug cue: Empty list par words[0] IndexError dega.

Follow-up: What would you improve first if this had 10x traffic/data?

HARD | PROMPT NORMALIZER

```
import re
def normalize(text, max_len=2000):
    text = re.sub(r"\s+", " ", text).strip()
    return text[:max_len]
print(normalize(" hello   ai "))
```

INTERVIEW DEFENSE

Project answer: Prompt cleaning quality and observability improve kar sakta hai, but it should not silently change user intent.

Debug cue: Over-aggressive cleaning se meaningful formatting lose ho sakti hai.

Follow-up: What would you improve first if this had 10x traffic/data?

SELF-CHECK

Can I explain the project without notes? Can I name one failure? Can I state time/space or cost trade-offs?

SECTION 05

String Methods

Level: Beginner | Time: ~60 min | 23 topics

Python me strings par kaam karne ke liye ready-made methods hote hain. Ye original string ko **badalte nahi**, hamesha naya string return karte hain, isliye result ko variable me store karna zaroori hai.

REAL-LIFE EXAMPLE SE SAMAJHO

String methods **Swiss army knife** ke tools hain. Har tool ek chhota kaam karta hai: kaatna, jodna, badalna, dhoondhna.

IS SECTION ME KYA SEEKHOGE

- ✓ Case badalna (upper, lower, title)
- ✓ Saaf karna aur todna (strip, split, join)
- ✓ Dhoondhna aur badalna (find, replace, count)
- ✓ Validation methods (isdigit, isalpha, startswith)

TOPIC ROADMAP

5.1 upper()
5.2 lower()
5.3 strip()
5.4 lstrip() / rstrip()
5.5 split()
5.6 join()
5.7 replace()
5.8 find()
5.9 index()
5.10 count()
5.11 startswith()
5.12 endswith()
5.13 format()
5.14 f-string
5.15 title()
5.16 capitalize()
5.17 swapcase()
5.18 isalpha() / isdigit() / isalnum()
5.19 zfill()
5.20 center() / ljust() / rjust()
5.21 Method Chaining ★
5.22 partition() & splitlines() ★
5.23 Mini Program: Word Counter ★

★ = Advanced topic. Pehli baar padhte waqt inhe skip karke dusre round me karo.

5.1 upper() CORE

Sare characters ko **UPPERCASE** me convert karta hai.

main.py

```
text = "hello world"
print(text.upper())
print(text)      # original nahi badla
```

▶ OUTPUT

```
HELLO WORLD
hello world
```

YAAD RAKHO

String methods original string ko change nahi karte, naya string return karte hain. **text = text.upper()** likhna padega.

5.2 lower() CORE

Sare characters ko lowercase me convert karta hai. Comparison se pehle use karo.

main.py

```
text = "HELLO WORLD"
print(text.lower())

email = "Ashu@Gmail.COM"
print(email.lower())
```

▶ OUTPUT

```
hello world
ashu@gmail.com
```

5.3 strip() CORE

Start aur end se extra spaces (ya given characters) hata deta hai. User input saaf karne ke liye must.

main.py

```
text = "  hello  "
print(text.strip())
print("xxhixx".strip("x"))
```

▶ OUTPUT

```
hello
hi
```

5.4 lstrip() / rstrip() CORE

Sirf left ya sirf right side se spaces/characters remove karta hai. repr() se quotes dikhte hain.

main.py

```
text = "  hello  "
print(repr(text.lstrip()))
print(repr(text.rstrip()))
```

▶ OUTPUT

```
'hello  '
'  hello'
```

5.5 split() CORE

String ko separator ke basis par tod kar **list** bana deta hai. Bina argument ke spaces par todta hai.

main.py

```
text = "apple,banana,mango"
print(text.split(","))
print("I love Python".split())
```

▶ OUTPUT

```
['apple', 'banana', 'mango']
['I', 'love', 'Python']
```

5.6 join() CORE

List of strings ko separator ke saath jodkar single string banata hai. **split() ka ulta.**

main.py

```
fruits = ["apple", "banana", "mango"]
print(", ".join(fruits))
print("-".join("abc"))
```

▶ OUTPUT

```
apple, banana, mango
a-b-c
```

GALTI SE BACHO

join() sirf strings jodta hai. Numbers ki list ho to pehle convert karo: **",".join(str(n) for n in nums)**.

5.7 replace() CORE

Ek part dhoondh kar dusre se badal deta hai. Default me saare matches badalta hai.

main.py

```
text = "I like cats and cats"
print(text.replace("cats", "dogs"))
print(text.replace("cats", "dogs", 1))   # sirf pehla
```

▶ OUTPUT

```
I like dogs and dogs
I like dogs and cats
```

5.8 find() CORE

Substring ka pehla index deta hai, na mile to **-1**.

main.py

```
text = "hello world"
print(text.find("world"))
print(text.find("xyz"))
```

▶ OUTPUT

```
6
-1
```

5.9 index() CORE

find() jaisa hi hai, par na milne par **ValueError** deta hai.

main.py

```
text = "hello world"
print(text.index("world"))
try:
    text.index("xyz")
except ValueError as e:
    print("Error:", e)
```

▶ OUTPUT

```
6
Error: substring not found
```

TIP

Agar "na mile to error chahiye" ho to index(), warna find() safe hai.

5.10 count() CORE

Substring kitni baar aaya hai, uski ginti deta hai.

main.py

```
text = "banana"
print(text.count("a"))
print(text.count("an"))
```

▶ OUTPUT

```
3
2
```

5.11 startswith() CORE

Check karta hai string given prefix se start hoti hai ya nahi.

main.py

```
text = "hello world"
print(text.startswith("hello"))
print("report.pdf".startswith(("data", "report")))   # tuple bhi chalta hai
```

▶ OUTPUT

```
True
True
```

5.12 endswith() CORE

Check karta hai string given suffix par end hoti hai ya nahi. File extension check ke liye best.

main.py

```
text = "hello world"
print(text.endswith("world"))
print("photo.jpg".endswith((".jpg", ".png")))
```

▶ OUTPUT

```
True
True
```

5.13 format() CORE

String ke andar **{ }** placeholders me values insert karta hai. Purana tarika, f-string usse better hai.

main.py

```
name = "Ashu"
age = 22
print("My name is {} and age is {}".format(name, age))
print("{1} is {0}".format("cool", "Python"))
```

▶ OUTPUT

```
My name is Ashu and age is 22
Python is cool
```

5.14 f-string CORE

format() ka modern, fast aur readable alternative. Naye code me yahi use karo.

main.py

```
name = "Ashu"
age = 22
print(f"My name is {name} and age is {age}")
```

▶ OUTPUT

```
My name is Ashu and age is 22
```

5.15 title() CORE

Har word ka pehla letter capital kar deta hai.

main.py

```
text = "hello world"
print(text.title())
```

▶ OUTPUT

```
Hello World
```

5.16 capitalize() CORE

Sirf string ke pehle letter ko capital karta hai, baaki lowercase.

main.py

```
text = "hello WORLD"
print(text.capitalize())
```

▶ OUTPUT

```
Hello world
```

5.17 swapcase() CORE

Upper ko lower aur lower ko upper me swap kar deta hai.

main.py

```
text = "Hello World"
print(text.swapcase())
```

▶ OUTPUT

```
hELLO wORLD
```

5.18 isalpha() / isdigit() / isalnum() CORE

Check karta hai string sirf letters / sirf digits / letters+digits hai ya nahi. Input validation me kaam aata hai.

main.py

```
print("hello".isalpha())
print("123".isdigit())
print("abc123".isalnum())
print("abc 123".isalnum())   # space hai, isliye False
```

▶ OUTPUT

```
True
True
True
False
```

TIP

Number input lene se pehle **text.isdigit()** check karo taaki int() crash na kare.

5.19 zfill() CORE

Left side me 0 se padding karke total length fix karta hai. Roll numbers, time format me kaam aata hai.

main.py

```
text = "42"
print(text.zfill(5))
print(str(7).zfill(2) + ":" + str(5).zfill(2))
```

▶ OUTPUT

```
00042
07:05
```

5.20 center() / ljust() / rjust() CORE

String ko fixed width me align karta hai. Padding character bhi de sakte ho.

main.py

```
text = "hi"
print(repr(text.center(10, "*")))
print(repr(text.ljust(10, "*")))
print(repr(text.rjust(10, "*")))
```

▶ OUTPUT

```
'****hi****'
'hi********'
'********hi'
```

5.21 Method Chaining ★ ADVANCED

Ek method ka result seedha dusre method ko de sakte ho. Left se right chalta hai.

main.py

```
raw = "   pYtHoN Is FuN   "
clean = raw.strip().lower().title()
print(clean)
print(raw.strip().replace(" ", "_").upper())
```

▶ OUTPUT

```
Python Is Fun
PYTHON_IS_FUN
```

5.22 partition() & splitlines() ★ ADVANCED

partition string ko 3 hisson me todta hai (pehle, separator, baad). **splitlines** multi-line text ko lines ki list me todta hai.

main.py

```
email = "ashu@gmail.com"
user, at, domain = email.partition("@")
print(user, domain)

text = "line1\nline2\nline3"
print(text.splitlines())
```

▶ OUTPUT

```
ashu gmail.com
['line1', 'line2', 'line3']
```

5.23 Mini Program: Word Counter ★ ADVANCED

Ab tak seekhe methods ko jodkar ek real kaam: sentence me words ginna aur sabse lamba word dhoondhna.

main.py

```
sentence = "  Python is simple and powerful  "
words = sentence.strip().split()
print("Total words:", len(words))
print("Longest:", max(words, key=len))
print("Vowels:", sum(sentence.lower().count(v) for v in "aeiou"))
```

▶ OUTPUT

```
Total words: 5
Longest: powerful
Vowels: 8
```

QUICK RECAP: SECTION 05

- ✓ Methods naya string dete hain, original nahi badalta
- ✓ strip() saaf karta hai, split() todta hai, join() jodta hai
- ✓ find() = -1 safe, index() = ValueError
- ✓ Method chaining: text.strip().lower().title()

PRACTICE TIME Khud try karo, phir Appendix C me solution dekho

EASY	**Q1.** " Hello Python " ko strip karke uppercase me print karo.
MEDIUM	**Q2.** "a-b-c-d" ko "-" par todo aur phir " + " se jodkar print karo.
HARD	**Q3.** Sentence "the quick brown fox" ke har word ka pehla letter capital karo, bina title() use kiye.

INTERVIEW LAB / String Methods

Cleaning, splitting, searching, validation aur text normalization. • Answer formula: definition → tiny example → practical use. Word-by-word memorize mat karo; apni language me explain karo.

Q01

strip() kya karta hai?

BEST ANSWER

String ke start aur end se whitespace hata deta hai. Beech ki spaces nahi hatata.

TIP: 20-30 sec me clearly bolo.

Q02

split() aur join() ka relation?

BEST ANSWER

split string ko list me todta hai; join list ke strings ko separator ke saath ek string me jodta hai.

TIP: 20-30 sec me clearly bolo.

Q03

find() aur index() me fark?

BEST ANSWER

find() not-found par -1 deta hai. index() not-found par ValueError raise karta hai.

TIP: 20-30 sec me clearly bolo.

Q04

replace() original ko modify karta hai?

BEST ANSWER

Nahi. String immutable hai, replace() naya string return karta hai.

TIP: 20-30 sec me clearly bolo.

Q05

Method chaining kya hai?

BEST ANSWER

Ek method ka result agle method ko dena, jaise text.strip().lower(). Ye compact aur readable ho sakta hai.

TIP: 20-30 sec me clearly bolo.

Q06

isdigit()/isalpha()/isalnum() ka use?

BEST ANSWER

Input validation ke liye. Ye batate hain characters digits, letters ya letters+digits hain ya nahi.

TIP: 20-30 sec me clearly bolo.

Mini speaking drill: Har answer bina PDF dekhe 2 baar bolo. Dusri baar ek real example add karo.

CODING LAB / String Methods

Rule: Pehle problem ko 10 sec me apne words me bolo, phir code likho. Copy-paste ke bajaye khud type karo.

EASY

" Ashu " ko clean karke lowercase karo.

WRITE CODE

EASY

"Python is easy" ko words ki list me todkar print karo.

WRITE CODE

MEDIUM

"banana" me "a" kitni baar hai? count() use karo.

WRITE CODE

MEDIUM

Email string ka user part aur domain partition() se nikalo.

WRITE CODE

HARD

Word counter banao: text -> strip -> lower -> split -> frequency.

WRITE CODE

DEBUG

text=text.upper() na likhne par original text kyun same rehta hai?

WRITE CODE

STARTER KIT - TRY WITHOUT LOOKING UP

```
strip() lower() split() join() replace() get/count style thinking
```

3-STEP DEBUG LOOP

- 1. Error/bug ko reproduce karo
- 2. Values/types/conditions print karke isolate karo
- 3. Ek change karo, rerun karo, phir reason explain karo

Interview coding line: "Main pehle input/edge case define karunga, phir simple correct solution likhunga, phir complexity aur edge cases check karunga."

LOGIC LAB / String Methods

Goal: code run karne se pehle output predict karo, phir dry-run karke reason bolo. Ye interview confidence aur debugging dono improve karta hai.

Predict

" Hi ".strip() -> ?

CHECK

"Hi"

Predict

"a=b".partition("=") -> ?

CHECK

("a","=","b")

Trace

"Hello".lower().replace("hello","hi") -> ?

CHECK

hi

Think

find() not-found par -1 kyun useful hai?

CHECK

Direct condition check possible

Fix

index() safely use karna ho to kya options?

CHECK

Check membership ya try/except

Explain

Text normalization ka simple pipeline?

CHECK

strip -> case normalize -> split/validate

5 COMMON MISTAKES TO WATCH

1. methods result store na karna
2. find/index confuse karna
3. split/join direction ulta
4. validation without strip
5. case normalization bhoolna

Mastery check: "Main is topic ko bina dekhe explain kar sakta hoon + 1 small program likh sakta hoon + 2 outputs predict kar sakta hoon."

SOLVED PRACTICE - String Methods

Every coding-lab question now has its answer, expected result and a quick check - all in one place.

EASY

" Ashu " ko clean karke lowercase karo.

```
print(" Ashu ".strip().lower())
```

EXPECTED RESULT

ashu

EASY

"Python is easy" ko words list me todkar print karo.

```
print("Python is easy".split())
```

EXPECTED RESULT

['Python', 'is', 'easy']

MEDIUM

"banana" me "a" kitni baar hai? count() use karo.

```
print("banana".count("a"))
```

EXPECTED RESULT

3

MEDIUM

Email se user part aur domain partition() se nikalo.

```
user,sep,domain="ahetesham@gmail.com".partition("@")
print(user,domain)
```

EXPECTED RESULT

ahetesham gmail.com

HARD

Word counter: strip -> lower -> split -> frequency.

```
from collections import Counter
text="AI ai Python"
print(Counter(text.strip().lower().split()))
```

EXPECTED RESULT

Counter({'ai': 2, 'python': 1})

DEBUG

text=text.upper() na likhne par original same kyun?

```
text="hello"
text.upper()
print(text)
text=text.upper()
print(text)
```

EXPECTED RESULT

hello HELLO

PRACTICE HABIT

Practice rule: solve first, compare second, then change one input. Mark the question only after you can explain why the output changes.

MINI PROJECT - Email Normalizer

String Methods ka concept ek runnable, interview-ready mini build me convert karo.

YE KYA KARTA HAI?

Email ko lowercase aur whitespace-clean banata hai.

KIS KAAM AATA HAI?

Forms, CSV imports, contact pipelines.

ARCHITECTURE

Raw email -> strip/lower -> basic validation

FILES / BUILD TARGET

email.py Target: Exactly one @ check

CORE IMPLEMENTATION - TYPE + RUN

```
email = " Ahetesham@GMAIL.com "
email = email.strip().lower()
valid = email.count("@") == 1
print(valid, email)
```

SAMPLE OUTPUT

```
True ahetesham@gmail.com
```

INTERVIEW EXPLANATION

Project answer: Method chaining short aur readable hoti hai; validation ko business rule ke hisaab se extend kar sakte ho. Debug cue: Count check complete email validation nahi hai. Follow-up: What would you improve first if this had 10x traffic/data?

BUILD STEPS

1. Create the file(s) shown above.
2. Run the exact sample once.
3. Change one input and observe the output.
4. Add one validation or failure test.

EDGE CASES / DEBUG

Test invalid input, boundary values and empty/missing data where relevant. Keep validation at the system boundary.

String Methods

Use this pack after the section labs: explain -> build -> debug -> defend.

SECTION ANCHOR

Definition: Reusable string methods se cleaning, matching aur parsing fast ho jata hai.

Easy explanation: Methods like strip, split, replace, startswith and join text ko transform karte hain.

Example: `email = " A@B.COM "; print(email.strip().lower())`

SIMPLE

Email Normalizer

Kya karta hai: Email ko lowercase aur whitespace-clean banata hai.

Kis kaam aata hai: Forms, CSV imports, contact pipelines.

Architecture: Raw email -> strip/lower -> basic validation

Files: `email.py`

Build target: Exactly one @ check

MEDIUM

Keyword Tagger

Kya karta hai: Text me predefined keywords ko detect karke tags banata hai.

Kis kaam aata hai: Support ticket routing aur lightweight NLP.

Architecture: Text -> lower -> keyword search -> tags

Files: `tags.py`

Build target: Case-insensitive matching

HARD

Support Message Normalizer

Kya karta hai: Support messages ko normalize karke category hints extract karta hai.

Kis kaam aata hai: AI chat support pre-processing and ticket triage.

Architecture: Message -> normalize -> split -> rules -> structured record

Files: `support.py`

Build target: Return text + category + urgency hint

Project rule: code khud type karo, output predict karo, edge case test karo, phir 60-second explanation do.

String Methods - implementation lab

Three core implementations + project-tailored interview explanation + debugging cue.

SIMPLE | EMAIL NORMALIZER

```
email = input().strip().lower()
valid = email.count("@") == 1
print(valid, email)
```

INTERVIEW DEFENSE

Project answer: Method chaining short aur readable hoti hai; validation ko business rule ke hisaab se extend kar sakte ho.

Debug cue: Count check complete email validation nahi hai.

Follow-up: What would you improve first if this had 10x traffic/data?

MEDIUM | KEYWORD TAGGER

```
text = "need python api help".lower()
keywords = {"python":"tech", "api":"backend", "help":"support
"}
tags = [tag for k,tag in keywords.items() if k in text]
print(tags)
```

INTERVIEW DEFENSE

Project answer: Simple rule-based matching explainable hota hai, but semantic variation miss kar sakta hai.

Debug cue: Substring false positives ko tokenization se reduce karo.

Follow-up: What would you improve first if this had 10x traffic/data?

HARD | SUPPORT MESSAGE NORMALIZER

```
def normalize(msg):
    s = " ".join(msg.lower().split())
    urgent = any(w in s for w in ("urgent","down","blocked"))
    return {"text":s,"urgent":urgent}
print(normalize("URGENT   API down"))
```

INTERVIEW DEFENSE

Project answer: Is layer ka output later Flask/LLM/N8N workflow me pass ho sakta hai.

Debug cue: Keep raw text if auditing/reprocessing may be required.

Follow-up: What would you improve first if this had 10x traffic/data?

SELF-CHECK

Can I explain the project without notes? Can I name one failure? Can I state time/space or cost trade-offs?

SECTION 06

if / elif / else

Level: Beginner | Time: ~45 min | 9 topics

Ab tak code upar se neeche seedha chalta tha. Conditions se program **decision** leta hai: kaun sa code chalega aur kaun sa skip hoga. Python me **indentation (4 spaces)** hi block banata hai, isliye spacing ka dhyan rakho.

REAL-LIFE EXAMPLE SE SAMAJHO

Ye **chauraha ka signal** hai: agar green hai to chalo, warna ruko. Program condition dekhkar rasta chunta hai.

IS SECTION ME KYA SEEKHOGE

- ✓ if, if-else, if-elif-else ka flow
- ✓ Nested if aur ternary operator
- ✓ Truthy/falsy values
- ✓ match-case (Python 3.10+)

TOPIC ROADMAP

6.1 if statement
6.2 if-else
6.3 if-elif-else
6.4 Nested if
6.5 Ternary Operator
6.6 Truthy & Falsy Values ★
6.7 Combining Conditions & in ★
6.8 match-case (Python 3.10+) ★
6.9 pass Statement ★

★ = Advanced topic. Pehli baar padhte waqt inhe skip karke dusre round me karo.

6.1 if statement

CORE

Condition **True** ho tabhi indented block chalta hai. Condition ke baad **:** lagana zaroori hai.

STEP BY STEP: KAISE CHALTA HAI?

1. Python condition **age >= 18** check karta hai.
2. True hai, isliye indented (4 spaces) lines chalti hain.
3. Indent se bahar wali line hamesha chalti hai.

main.py

```
age = 20
if age >= 18:
    print("You are an adult")
print("Done")      # ye hamesha chalega
```

▶ OUTPUT

```
You are an adult
Done
```

GALTI SE BACHO

Colon **:** bhoolna ya indentation galat karna sabse common error hai (SyntaxError / IndentationError).

6.2 if-else

CORE

Condition True ho to **if** block, False ho to **else** block chalta hai. Dono me se sirf ek chalta hai.

main.py

```
age = 15
if age >= 18:
    print("Adult")
else:
    print("Minor")
```

▶ OUTPUT

```
Minor
```

6.3 if-elif-else

CORE

Kai conditions ko upar se neeche check karta hai. **Pehli True condition** ka block chalta hai, baaki skip.

STEP BY STEP: KAISE CHALTA HAI?

1. marks >= 90? 75 nahi, skip.
2. marks >= 60? Haan, True! "Grade B" print.
3. Baaki elif/else check hi nahi hote.

main.py

```
marks = 75
if marks >= 90:
    print("Grade A")
elif marks >= 60:
    print("Grade B")
else:
    print("Grade C")
```

▶ OUTPUT

```
Grade B
```

TIP

Order matter karta hai. Bade se chhota ya chhote se bada, dono me se ek clear order rakho.

6.4 Nested if

CORE

Ek if ke andar dusra if. Jab pehli condition ke baad aur check karna ho.

main.py

```
num = 15
if num > 0:
    if num % 2 == 0:
        print("Positive Even")
    else:
        print("Positive Odd")
else:
    print("Zero or Negative")
```

▶ OUTPUT

```
Positive Odd
```

6.5 Ternary Operator

CORE

if-else ko ek line me likhne ka tarika: **value_if_true if condition else value_if_false**.

main.py

```
age = 20
status = "Adult" if age >= 18 else "Minor"
print(status)
```

▶ OUTPUT

```
Adult
```

6.6 Truthy & Falsy Values

★ ADVANCED

Python har value ko condition me True ya False maanta hai. **Falsy:** 0, 0.0, "", [], {}, (), set(), None, False. Baaki sab **Truthy**.

main.py

```
items = []
if items:
    print("Kuch hai")
else:
    print("List khaali hai")

name = "Ashu"
if name:
    print("Naam mila:", name)

print(bool(0), bool(""), bool("a"), bool([0]))
```

▶ OUTPUT

```
List khaali hai
Naam mila: Ashu
False False True True
```

TIP

if items: likhna **if len(items) > 0:** se zyada Pythonic hai.

6.7 Combining Conditions & in

★ ADVANCED

and / or se conditions jodo, aur **in** se ek se zyada values ek saath check karo.

main.py

```
day = "Sat"
if day == "Sat" or day == "Sun":
    print("Weekend")

if day in ("Sat", "Sun"):      # chhota tarika
    print("Weekend again")

age, has_id = 20, True
if age >= 18 and has_id:
    print("Entry allowed")
```

▶ OUTPUT

```
Weekend
Weekend again
Entry allowed
```

6.8 match-case (Python 3.10+)

★ ADVANCED

Switch jaisa feature: ek value ko kai patterns se match karta hai. _ default case hai.

main.py

```
day = 3
match day:
    case 1:
        print("Monday")
    case 2:
        print("Tuesday")
    case 3:
        print("Wednesday")
    case _:
        print("Other day")
```

▶ OUTPUT

```
Wednesday
```

6.9 pass Statement

★ ADVANCED

Kabhi block abhi khaali rakhna ho (baad me likhoge) to **pass** likhte hain, warna error aata hai.

main.py

```
x = 10
if x > 5:
    pass         # TODO: baad me likhenge
else:
    print("small")
print("No error")
```

▶ OUTPUT

```
No error
```

QUICK RECAP: SECTION 06

- ✓ Colon aur 4-space indentation zaroori hai
- ✓ elif me pehli True condition chalti hai, baaki skip
- ✓ Ternary: a if cond else b
- ✓ Khaali cheezein (0, "", [], None) falsy hoti hain

PRACTICE TIME Khud try karo, phir Appendix C me solution dekho

EASY	**Q1.** n = -4 positive, negative ya zero hai? Print karo.
MEDIUM	**Q2.** year = 2024 leap year hai ya nahi? (4 se divisible, par 100 se nahi; ya 400 se divisible)
HARD	**Q3.** marks = 82 par ternary se "Pass" (>= 40) ya "Fail" print karo, aur phir 90+ ho to "Topper" bhi print karo.

INTERVIEW LAB / if / elif / else

Program ko decision making aur multiple conditions ka flow samjhana. • Answer formula: definition → tiny example → practical use. Word-by-word memorize mat karo; apni language me explain karo.

Q01

if block ka basic rule?

BEST ANSWER

Condition True ho to indented block chalta hai. Colon aur correct indentation required hai.

TIP: 20-30 sec me clearly bolo.

Q02

elif kaise decide hota hai?

BEST ANSWER

Conditions upar se neeche check hoti hain. Pehli True branch execute hoti hai aur remaining branches skip hoti hain.

TIP: 20-30 sec me clearly bolo.

Q03

Nested if kab use karein?

BEST ANSWER

Jab pehle decision ke baad dependent second check karna ho.

TIP: 20-30 sec me clearly bolo.

Q04

Ternary operator kya hai?

BEST ANSWER

Simple if-else ko one line me likhne ka syntax: value_if_true if condition else value_if_false.

TIP: 20-30 sec me clearly bolo.

Q05

Truthy/Falsy kya hota hai?

BEST ANSWER

Condition context me values ko boolean maana jata hai. 0, "", empty collections, None aur False falsy examples hain.

TIP: 20-30 sec me clearly bolo.

Q06

match-case kya hai?

BEST ANSWER

Python 3.10+ ka pattern-matching structure jo ek value ke multiple cases handle kar sakta hai.

TIP: 20-30 sec me clearly bolo.

Mini speaking drill: Har answer bina PDF dekhe 2 baar bolo. Dusri baar ek real example add karo.

CODING LAB / if / elif / else

Rule: Pehle problem ko 10 sec me apne words me bolo, phir code likho. Copy-paste ke bajaye khud type karo.

EASY

n positive, negative ya zero print karo.

WRITE CODE

EASY

marks se A/B/C/D grade decide karo.

WRITE CODE

MEDIUM

Leap year check implement karo.

WRITE CODE

MEDIUM

age + has_id ke basis par entry allow/deny karo.

WRITE CODE

HARD

82 marks par Pass/Fail ternary aur 90+ par Topper message print karo.

WRITE CODE

DEBUG

if x > 5: line me indentation missing ho to error kyun?

WRITE CODE

STARTER KIT - TRY WITHOUT LOOKING UP

```
if / elif / else and/or ternary match/case
```

3-STEP DEBUG LOOP

- 1. Error/bug ko reproduce karo
- 2. Values/types/conditions print karke isolate karo
- 3. Ek change karo, rerun karo, phir reason explain karo

Interview coding line: "Main pehle input/edge case define karunga, phir simple correct solution likhunga, phir complexity aur edge cases check karunga."

LOGIC LAB / if / elif / else

Goal: code run karne se pehle output predict karo, phir dry-run karke reason bolo. Ye interview confidence aur debugging dono improve karta hai.

Predict

age=15; age>=18 -> ?

CHECK

False

Trace

marks=75 with >=90 then >=60

CHECK

Grade B

Think

if items: empty list par kya branch chalegi?

CHECK

else

Predict

bool([0]) -> ?

CHECK

True - list non-empty hai

Fix

elif order galat hone se kya problem?

CHECK

Broad condition early match kar sakti hai

Explain

Decision problem solve karne ka flow?

CHECK

Condition define -> branches order -> edge cases test

5 COMMON MISTAKES TO WATCH

1. colon bhoolna
2. indentation mismatch
3. elif order
4. truthy/falsy ignore karna
5. complex nested if overuse karna

Mastery check: "Main is topic ko bina dekhe explain kar sakta hoon + 1 small program likh sakta hoon + 2 outputs predict kar sakta hoon."

SOLVED PRACTICE - if / elif / else

Every coding-lab question now has its answer, expected result and a quick check - all in one place.

EASY

n positive, negative ya zero print karo.

```
n=-2
if n>0: print("positive")
elif n<0: print("negative")
else: print("zero")
```

EXPECTED RESULT

negative

EASY

marks se A/B/C/D grade decide karo.

```
m=85
if m>=80: g="A"
elif m>=60: g="B"
elif m>=40: g="C"
else: g="D"
print(g)
```

EXPECTED RESULT

A

MEDIUM

Leap year check implement karo.

```
y=2024
print(y%400==0 or (y%4==0 and y%100!=0))
```

EXPECTED RESULT

True

MEDIUM

age + has_id ke basis par entry allow/deny karo.

```
age=22; has_id=True
print("allow" if age>=18 and has_id else "deny")
```

EXPECTED RESULT

allow

HARD

82 marks par Pass/Fail ternary aur 90+ Topper message.

```
m=82
print("Pass" if m>=40 else "Fail")
if m>=90: print("Topper")
```

EXPECTED RESULT

Pass

DEBUG

if x>5: me indentation missing ho to error kyun?

```
x=7
if x>5:
    print("yes")
```

EXPECTED RESULT

Indented block required after colon.

PRACTICE HABIT

Practice rule: solve first, compare second, then change one input. Mark the question only after you can explain why the output changes.

MINI PROJECT - Grade Calculator

if / elif / else ka concept ek runnable, interview-ready mini build me convert karo.

YE KYA KARTA HAI?

Marks ko grade bands me map karta hai.

KIS KAAM AATA HAI?

Student tools, screening rules.

ARCHITECTURE

Marks -> ordered conditions -> grade

FILES / BUILD TARGET

grade.py Target: 0-100 validation

CORE IMPLEMENTATION - TYPE + RUN

```
m = 85
if not 0 <= m <= 100:
    raise ValueError("0..100")
if m >= 80:
    grade = "A"
elif m >= 60:
    grade = "B"
else:
    grade = "C"
print(grade)
```

SAMPLE OUTPUT A

INTERVIEW EXPLANATION

Project answer: Validation first; then mutually exclusive ordered rules. Debug cue: Condition ordering se 90 ko lower band me accidentally route kar sakte ho. Follow-up: What would you improve first if this had 10x traffic/data?

BUILD STEPS

1. Create the file(s) shown above.
2. Run the exact sample once.
3. Change one input and observe the output.
4. Add one validation or failure test.

EDGE CASES / DEBUG

Test invalid input, boundary values and empty/missing data where relevant. Keep validation at the system boundary.

if / elif / else

Use this pack after the section labs: explain -> build -> debug -> defend.

SECTION ANCHOR

Definition: Conditions program ke decision points define karti hain.

Easy explanation: Compare -> decide -> action; boundaries and default paths explicit rakho.

Example: `marks = 85; print("A" if marks >= 80 else "B")`

SIMPLE

Grade Calculator

Kya karta hai: Marks ko grade bands me map karta hai.

Kis kaam aata hai: Student tools, screening rules.

Architecture: Marks -> ordered conditions -> grade

Files: `grade.py`

Build target: 0-100 validation

MEDIUM

Ticket Router

Kya karta hai: Support ticket ko billing/tech/general category me route karta hai.

Kis kaam aata hai: AI chat support aur automation workflows.

Architecture: Ticket -> rules -> queue/agent assignment

Files: `router.py`

Build target: Default queue always exists

HARD

Risk Decision Service

Kya karta hai: Multiple signals se review/allow/block decision produce karta hai.

Kis kaam aata hai: Fraud-like workflows, access control, automation gates.

Architecture: Request -> validations -> risk rules -> decision -> audit log

Files: `risk.py`

Build target: Separate decision reason from final action

Project rule: code khud type karo, output predict karo, edge case test karo, phir 60-second explanation do.

if / elif / else - implementation lab

Three core implementations + project-tailored interview explanation + debugging cue.

SIMPLE | GRADE CALCULATOR

```
m = 85
if not 0 <= m <= 100: raise ValueError("0..100")
if m >= 80: grade = "A"
elif m >= 60: grade = "B"
else: grade = "C"
print(grade)
```

INTERVIEW DEFENSE

Project answer: Validation first; then mutually exclusive ordered rules.

Debug cue: Condition ordering se 90 ko lower band me accidentally route kar sakte ho.

Follow-up: What would you improve first if this had 10x traffic/data?

MEDIUM | TICKET ROUTER

```
t = "refund for order".lower()
if "refund" in t or "payment" in t: q = "billing"
elif "error" in t or "api" in t: q = "tech"
else: q = "general"
print(q)
```

INTERVIEW DEFENSE

Project answer: Rule-based routing deterministic and easy to audit hota hai.

Debug cue: Overlapping keywords ko priority rules se handle karo.

Follow-up: What would you improve first if this had 10x traffic/data?

HARD | RISK DECISION SERVICE

```
risk = 0
if amount > 50000: risk += 2
if new_device: risk += 1
if bad_ip: risk += 3
if risk >= 4: action = "review"
elif risk >= 6: action = "block"
else: action = "allow"
```

INTERVIEW DEFENSE

Project answer: Production rule engines should make thresholds and reason codes explicit.

Debug cue: Check mutually consistent ranges; here review must be evaluated after block threshold.

Follow-up: What would you improve first if this had 10x traffic/data?

SELF-CHECK

Can I explain the project without notes? Can I name one failure? Can I state time/space or cost trade-offs?

SECTION 07

for & while Loops

Level: Beginner **Time: ~75 min** **15 topics**

1000 baar print karna ho to 1000 lines nahi likhte, loop likhte hain. Loops data ko process karne ki backbone hain: lists, strings, files, sab par.

REAL-LIFE EXAMPLE SE SAMAJHO

Loop ek **chakki** hai: ek kaam baar-baar karti hai. **for** tab jab pata ho kitni baar, **while** tab jab tak condition sahi hai.

IS SECTION ME KYA SEEKHOGE

- ✓ for loop aur range()
- ✓ while loop aur infinite loop se bachna
- ✓ break, continue, loop-else
- ✓ enumerate(), zip(), list comprehension

TOPIC ROADMAP

7.1 for loop
7.2 while loop
7.3 range()
7.4 break
7.5 continue
7.6 else with loop
7.7 Nested loop
7.8 enumerate()
7.9 zip()
7.10 List Comprehension
7.11 range() with Step & Reverse ★
7.12 Accumulator Pattern ★
7.13 while True + break ★
7.14 Pattern Printing ★
7.15 Comprehension with Condition ★

★ = Advanced topic. Pehli baar padhte waqt inhe skip karke dusre round me karo.

7.1 for loop

CORE

Ek sequence (list, string, range) ke **har item** par ek-ek karke chalta hai.

STEP BY STEP: KAISE CHALTA HAI?

1. Pehli baar: fruit = "apple", body chalti hai.
2. Dusri baar: fruit = "banana".
3. Teesri baar: fruit = "mango". Items khatam, loop ruk jata hai.

main.py

```
fruits = ["apple", "banana", "mango"]
for fruit in fruits:
    print(fruit)
```

▶ OUTPUT

```
apple
banana
mango
```

7.2 while loop

CORE

Jab tak condition **True** hai, tab tak chalta rehta hai. Counter update karna mat bhoolo.

STEP BY STEP: KAISE CHALTA HAI?

1. i = 1: condition (1 <= 3) True, print 1, i = 2.
2. i = 2: True, print 2, i = 3.
3. i = 3: True, print 3, i = 4.
4. i = 4: condition False, loop khatam.

main.py

```
i = 1
while i <= 3:
    print(i)
    i += 1
```

▶ OUTPUT

```
1
2
3
```

GALTI SE BACHO

i += 1 bhool gaye to loop **kabhi nahi rukega** (infinite loop). Rokne ke liye Ctrl + C dabao.

7.3 range() CORE

Numbers ki sequence banata hai. **range(stop)**, **range(start, stop)**, **range(start, stop, step)**. Stop included nahi hota.

main.py

```
for i in range(1, 5):
    print(i)

print(list(range(5)))
print(list(range(0, 10, 3)))
```

▶ OUTPUT

```
1
2
3
4
[0, 1, 2, 3, 4]
[0, 3, 6, 9]
```

7.4 break CORE

Loop ko **turant rok** deta hai, chahe condition True ho.

main.py

```
for i in range(1, 10):
    if i == 4:
        break
    print(i)
print("Loop se bahar")
```

▶ OUTPUT

```
1
2
3
Loop se bahar
```

7.5 continue CORE

Current round **skip** karke loop ke next round par chala jata hai.

main.py

```
for i in range(1, 6):
    if i == 3:
        continue
    print(i)
```

▶ OUTPUT

```
1
2
4
5
```

YAAD RAKHO

break = poora loop khatam. **continue** = sirf is round ko skip.

7.6 else with loop

CORE

Loop bina break hue poora complete ho jaye to **else** block chalta hai. Search jaise kaam me useful.

main.py

```
for i in range(3):
    print(i)
else:
    print("Loop finished")

for n in [1, 3, 5]:
    if n % 2 == 0:
        print("Even mila")
        break
else:
    print("Koi even nahi mila")
```

▶ OUTPUT

```
0
1
2
Loop finished
Koi even nahi mila
```

7.7 Nested loop

CORE

Loop ke andar loop. Andar wala loop, bahar ke har round me poora chalta hai. Table/grid ke liye.

STEP BY STEP: KAISE CHALTA HAI?

1. Bahar i = 1: andar j = 1, 2 chalta hai.
2. Bahar i = 2: andar phir se j = 1, 2 chalta hai.
3. Total rounds = 2 x 2 = 4.

main.py

```
for i in range(1, 3):
    for j in range(1, 3):
        print(i, j)
```

▶ OUTPUT

```
1 1
1 2
2 1
2 2
```

7.8 enumerate() CORE

Loop me item ke saath uska **index** bhi deta hai. Ab i = 0; i += 1 likhne ki zarurat nahi.

main.py

```
fruits = ["apple", "banana"]
for index, fruit in enumerate(fruits):
    print(index, fruit)

for n, fruit in enumerate(fruits, start=1):
    print(n, fruit)
```

▶ OUTPUT

```
0 apple
1 banana
1 apple
2 banana
```

7.9 zip() CORE

Do ya zyada lists ko **saath-saath**, pair-pair karke loop karta hai. Chhoti list par ruk jata hai.

main.py

```
names = ["A", "B"]
ages = [20, 25]
for n, a in zip(names, ages):
    print(n, a)
```

▶ OUTPUT

```
A 20
B 25
```

7.10 List Comprehension CORE

Ek line me loop chala kar naya list banane ka short tarika: **[expression for item in iterable]**.

main.py

```
squares = [x * x for x in range(1, 5)]
print(squares)

# Loop wala lamba tarika (same result):
squares2 = []
for x in range(1, 5):
    squares2.append(x * x)
print(squares2)
```

▶ OUTPUT

```
[1, 4, 9, 16]
[1, 4, 9, 16]
```

7.11 range() with Step & Reverse ★ ADVANCED

Ulta ginne ke liye negative step: **range(10, 0, -1)**.

main.py

```
print(list(range(10, 0, -2)))
for i in range(3, 0, -1):
    print(i)
print("Liftoff!")
```

▶ OUTPUT

```
[10, 8, 6, 4, 2]
3
2
1
Liftoff!
```

7.12 Accumulator Pattern ★ ADVANCED

Loop ke bahar variable banao aur har round me update karo. Sum, count, max nikalne ka standard tarika.

main.py

```
nums = [4, 9, 2, 7]
total = 0
biggest = nums[0]
for n in nums:
    total += n
    if n > biggest:
        biggest = n
print("Sum:", total)
print("Max:", biggest)
print("Average:", total / len(nums))
```

▶ OUTPUT

```
Sum: 22
Max: 9
Average: 5.5
```

7.13 while True + break ★ ADVANCED

Jab pata nahi kitni baar chalega (jaise menu, password check), tab **while True** me break se bahar niklo.

main.py

```
attempts = 0
while True:
    attempts += 1
    if attempts == 3:
        print("3rd attempt par ruk gaye")
        break
print("Total:", attempts)
```

▶ OUTPUT

```
3rd attempt par ruk gaye
Total: 3
```

7.14 Pattern Printing ★ ADVANCED

Nested loop aur * repeat se patterns bante hain. Interviews me bahut puchhe jaate hain.

main.py

```
rows = 4
for i in range(1, rows + 1):
    print("*" * i)

for i in range(1, rows + 1):
    print(" " * (rows - i) + "*" * (2 * i - 1))
```

▶ OUTPUT

```
*
**
***
****
   *
  ***
 *****
*******
```

7.15 Comprehension with Condition ★ ADVANCED

Comprehension me **if** lagakar filter kar sakte ho: **[x for x in data if condition]**.

main.py

```
nums = range(1, 11)
evens = [n for n in nums if n % 2 == 0]
print(evens)

words = ["hi", "python", "go", "loops"]
print([w.upper() for w in words if len(w) > 2])
```

▶ OUTPUT

```
[2, 4, 6, 8, 10]
['PYTHON', 'LOOPS']
```

QUICK RECAP: SECTION 07

- ✓ for = pata hai kitni baar, while = jab tak condition True
- ✓ range(start, stop, step); stop included nahi
- ✓ break rokta hai, continue skip karta hai
- ✓ enumerate = index + item, zip = pairs, comprehension = one-line list

PRACTICE TIME Khud try karo, phir Appendix C me solution dekho

EASY	**Q1.** 1 se 5 tak ke numbers ka sum for loop se nikalo.
MEDIUM	**Q2.** FizzBuzz: 1 se 15 tak print karo. 3 ka multiple = Fizz, 5 ka = Buzz, dono ka = FizzBuzz.
HARD	**Q3.** Ek list [3, 8, 12, 5, 20] me se sirf wo numbers ek naye list me nikalo jo 5 se bade hain, comprehension use karke.

INTERVIEW LAB / for & while Loops

Repeat work, search, accumulation, nesting aur comprehensions. • Answer formula: definition → tiny example → practical use. Word-by-word memorize mat karo; apni language me explain karo.

Q01

for aur while me simple difference?

BEST ANSWER

for iterable ke items par iterate karta hai. while condition True rehne tak repeat karta hai.

TIP: 20-30 sec me clearly bolo.

Q02

range() me stop included hota hai?

BEST ANSWER

Nahi. stop excluded hota hai.

TIP: 20-30 sec me clearly bolo.

Q03

break vs continue?

BEST ANSWER

break poora loop rokta hai. continue current iteration skip karke next iteration par jata hai.

TIP: 20-30 sec me clearly bolo.

Q04

enumerate() kyun useful?

BEST ANSWER

Item ke saath index deta hai without manual counter.

TIP: 20-30 sec me clearly bolo.

Q05

zip() kya karta hai?

BEST ANSWER

Multiple iterables ko position-wise pair karta hai aur shortest iterable par rukta hai.

TIP: 20-30 sec me clearly bolo.

Q06

Accumulator pattern kya hai?

BEST ANSWER

Loop ke bahar result variable initialize karke har iteration me update karna - sum, count, max jaise problems me.

TIP: 20-30 sec me clearly bolo.

Mini speaking drill: Har answer bina PDF dekhe 2 baar bolo. Dusri baar ek real example add karo.

CODING LAB / for & while Loops

Rule: Pehle problem ko 10 sec me apne words me bolo, phir code likho. Copy-paste ke bajaye khud type karo.

EASY

1 se 10 tak numbers print karo.

WRITE CODE

EASY

1 se 50 tak even numbers print karo.

WRITE CODE

MEDIUM

1 se 15 tak FizzBuzz likho.

WRITE CODE

MEDIUM

List me target item search karo aur milte hi break karo.

WRITE CODE

HARD

2 x 3 star pattern nested loop se print karo.

WRITE CODE

DEBUG

while loop me i += 1 miss ho to kya hoga?

WRITE CODE

STARTER KIT - TRY WITHOUT LOOKING UP

```
for while range() break continue enumerate() zip()
```

3-STEP DEBUG LOOP

- 1. Error/bug ko reproduce karo
- 2. Values/types/conditions print karke isolate karo
- 3. Ek change karo, rerun karo, phir reason explain karo

Interview coding line: "Main pehle input/edge case define karunga, phir simple correct solution likhunga, phir complexity aur edge cases check karunga."

LOGIC LAB / for & while Loops

Goal: code run karne se pehle output predict karo, phir dry-run karke reason bolo. Ye interview confidence aur debugging dono improve karta hai.

Predict

list(range(5)) -> ?

CHECK

[0,1,2,3,4]

Trace

for i in range(1,4): print(i)

CHECK

1 2 3

Think

break kab use karoge?

CHECK

Jab required result mil jaye aur loop continue karna unnecessary ho

Predict

range(10,0,-2) -> ?

CHECK

10,8,6,4,2

Fix

while infinite loop diagnose ka first step?

CHECK

Condition + update variable check karo

Explain

List comprehension kab use karein?

CHECK

Short, readable transformation/filtering ke liye

5 COMMON MISTAKES TO WATCH

1. off-by-one range
2. infinite while
3. break/continue confuse karna
4. manual counters with enumerate
5. nested loops ko unreadable banana

Mastery check: "Main is topic ko bina dekhe explain kar sakta hoon + 1 small program likh sakta hoon + 2 outputs predict kar sakta hoon."

SOLVED PRACTICE - for & while Loops

Every coding-lab question now has its answer, expected result and a quick check - all in one place.

EASY

1 se 10 tak numbers print karo.

```
for i in range(1,11): print(i,end=" ")
```

EXPECTED RESULT

1 2 3 4 5 6 7 8 9 10

EASY

1 se 50 tak even numbers print karo.

```
print([i for i in range(2,51,2)])
```

EXPECTED RESULT

[2, 4, ..., 50]

MEDIUM

1 se 15 tak FizzBuzz likho.

```
for i in range(1,16):
    print("Fizz"*(i%3==0)+"Buzz"*(i%5==0) or i)
```

EXPECTED RESULT

Fizz/Buzz rules apply; 15 = FizzBuzz

MEDIUM

List me target item search karo aur milte hi break.

```
items=["a","b","c"]; target="b"
for x in items:
    if x==target: print("found"); break
```

EXPECTED RESULT

found

HARD

2 x 3 star pattern nested loop se print karo.

```
for _ in range(2): print("***")
```

EXPECTED RESULT

*** ***

DEBUG

while loop me i += 1 miss ho to kya hoga?

```
i=0
while i<3:
    print(i); i+=1
```

EXPECTED RESULT

Finite loop: 0 1 2; increment prevents infinite loop.

PRACTICE HABIT

Practice rule: solve first, compare second, then change one input. Mark the question only after you can explain why the output changes.

MINI PROJECT - Number Analyzer

for & while Loops ka concept ek runnable, interview-ready mini build me convert karo.

YE KYA KARTA HAI?

1..n me even/odd counts nikalta hai.

KIS KAAM AATA HAI?

Logic practice and basic analytics.

ARCHITECTURE

Range -> loop -> counters -> output

FILES / BUILD TARGET

count.py Target: n < 1 safe handling

CORE IMPLEMENTATION - TYPE + RUN

```
n = 10
even = odd = 0
for x in range(1, n + 1):
    even += x % 2 == 0
    odd += x % 2 == 1
print(even, odd)
```

SAMPLE OUTPUT `5 5 (five even, five odd numbers from 1..10)`

INTERVIEW EXPLANATION

Project answer: Loop state ko counters me maintain kiya; boolean arithmetic simple trick hai. Debug cue: Range endpoint inclusive nahi hota; n+1 use kiya. Follow-up: What would you improve first if this had 10x traffic/data?

BUILD STEPS

1. Create the file(s) shown above.
2. Run the exact sample once.
3. Change one input and observe the output.
4. Add one validation or failure test.

EDGE CASES / DEBUG

Test invalid input, boundary values and empty/missing data where relevant. Keep validation at the system boundary.

PROJECT LAB 07

for & while Loops

Use this pack after the section labs: explain -> build -> debug -> defend.

SECTION ANCHOR

Definition: Repeated work ko structured iteration se process karna.

Easy explanation: Loop state + termination condition + edge cases.

Example: `for x in [1,2,3]: print(x)`

SIMPLE

Number Analyzer

Kya karta hai: 1..n me even/odd counts nikalta hai.

Kis kaam aata hai: Logic practice and basic analytics.

Architecture: Range -> loop -> counters -> output

Files: `count.py`

Build target: n < 1 safe handling

MEDIUM

Batch Data Cleaner

Kya karta hai: Records ko one-by-one validate aur normalize karta hai.

Kis kaam aata hai: CSV/JSON preprocessing before analytics or ML.

Architecture: Records -> loop -> validate -> cleaned list

Files: `cleaner.py`

Build target: Invalid rows skip with reason

HARD

Log Stream Aggregator

Kya karta hai: Large log stream ko bounded memory me aggregate karta hai.

Kis kaam aata hai: Monitoring and observability pipelines.

Architecture: Stream -> loop -> parse -> counters -> periodic flush

Files: `stream.py`

Build target: No full file load into memory

Project rule: code khud type karo, output predict karo, edge case test karo, phir 60-second explanation do.

for & while Loops - implementation lab

Three core implementations + project-tailored interview explanation + debugging cue.

SIMPLE | NUMBER ANALYZER

```
n = 10
even = odd = 0
for x in range(1, n+1):
    even += x % 2 == 0
    odd += x % 2 == 1
print(even, odd)
```

INTERVIEW DEFENSE

Project answer: Loop state ko counters me maintain kiya; boolean arithmetic simple trick hai.

Debug cue: Range endpoint inclusive nahi hota; n+1 use kiya.

Follow-up: What would you improve first if this had 10x traffic/data?

MEDIUM | BATCH DATA CLEANER

```
cleaned=[]
for r in rows:
    if not r.get("email"): continue
    r["email"] = r["email"].strip().lower()
    cleaned.append(r)
```

INTERVIEW DEFENSE

Project answer: Batch cleaning is a direct bridge from Python basics to Pandas pipelines.

Debug cue: Mutating shared dicts can affect upstream data; copy when needed.

Follow-up: What would you improve first if this had 10x traffic/data?

HARD | LOG STREAM AGGREGATOR

```
counts={}
for line in file:
    level = parse_level(line)
    counts[level] = counts.get(level, 0) + 1
    if sum(counts.values()) % 10000 == 0:
        flush(counts); counts.clear()
```

INTERVIEW DEFENSE

Project answer: Streaming mindset large data ke liye memory spikes avoid karta hai.

Debug cue: Flush interval aur lost state handling test karo.

Follow-up: What would you improve first if this had 10x traffic/data?

SELF-CHECK

Can I explain the project without notes? Can I name one failure? Can I state time/space or cost trade-offs?

SECTION 08

Lists

Level: Beginner | Time: ~50 min | 9 topics

List Python ka sabse zyada use hone wala data structure hai. Ye kai values ko ek naam se store karti hai. Indexing aur slicing strings jaisi hi hai, bas yaha items **badal** bhi sakte hain (mutable).

REAL-LIFE EXAMPLE SE SAMAJHO

List ek **shopping list** jaisi hai: order me items, kabhi bhi add/remove/badal sakte ho, aur duplicates bhi allowed.

IS SECTION ME KYA SEEKHOGE

- ✓ List banana aur items access karna
- ✓ Slicing aur nested lists
- ✓ Mutability aur alias/copy ka fark
- ✓ sum(), min(), max(), len()

TOPIC ROADMAP

8.1 Creating Lists
8.2 Indexing & Negative Indexing
8.3 Slicing
8.4 Nested Lists
8.5 len()
8.6 Slicing with Step & Reverse ★
8.7 Alias vs Copy (Important!) ★
8.8 sum(), min(), max(), sorted() ★
8.9 in operator & Looping ★

★ = Advanced topic. Pehli baar padhte waqt inhe skip karke dusre round me karo.

8.1 Creating Lists

CORE

List ek **ordered, changeable** collection hai jo **[ ]** me likhi jati hai. Alag-alag types mix kar sakte ho.

main.py

```
fruits = ["apple", "banana", "mango"]
print(fruits)

mixed = [1, "two", 3.0, True]
print(mixed)
print(list("abc"))
```

▶ OUTPUT

```
['apple', 'banana', 'mango']
[1, 'two', 3.0, True]
['a', 'b', 'c']
```

8.2 Indexing & Negative Indexing CORE

Items index se access hote hain. Negative index end se ginta hai. Item badal bhi sakte ho.

main.py

```
fruits = ["apple", "banana", "mango"]
print(fruits[0])
print(fruits[-1])

fruits[1] = "kiwi"      # item badla
print(fruits)
```

▶ OUTPUT

```
apple
mango
['apple', 'kiwi', 'mango']
```

8.3 Slicing CORE

List ka ek hissa nikalne ke liye **[start:end]**. Result hamesha nayi list hota hai.

main.py

```
nums = [10, 20, 30, 40, 50]
print(nums[1:4])
print(nums[:2])
print(nums[-2:])
```

▶ OUTPUT

```
[20, 30, 40]
[10, 20]
[40, 50]
```

8.4 Nested Lists CORE

List ke andar list rakh sakte ho (2D structure, jaise table ya grid).

STEP BY STEP: KAISE CHALTA HAI?

1. matrix[1] = doosri row = [3, 4]
2. [3, 4][0] = 3
3. Isliye matrix[1][0] = 3 (pehle row, phir column).

main.py

```
matrix = [[1, 2], [3, 4]]
print(matrix[1][0])

for row in matrix:
    print(row)
```

▶ OUTPUT

```
3
[1, 2]
[3, 4]
```

8.5 len() CORE

List me kitne items hain, uski ginti deta hai.

main.py

```
fruits = ["apple", "banana", "mango"]
print(len(fruits))
print(len([]))
```

▶ OUTPUT

```
3
0
```

8.6 Slicing with Step & Reverse ★ ADVANCED

Strings ki tarah lists me bhi **[::step]** aur **[::-1]** chalta hai.

main.py

```
nums = [1, 2, 3, 4, 5, 6]
print(nums[::2])
print(nums[::-1])
```

▶ OUTPUT

```
[1, 3, 5]
[6, 5, 4, 3, 2, 1]
```

8.7 Alias vs Copy (Important!) ★ ADVANCED

b = a naya list nahi banata, ek hi list ke do naam ban jate hain. Ek me change karoge to dusre me bhi dikhega.

main.py

```
a = [1, 2, 3]
b = a           # alias (same list)
c = a.copy()    # asli copy
b.append(99)
print(a)        # a bhi badal gaya!
print(c)        # c safe hai
```

▶ OUTPUT

```
[1, 2, 3, 99]
[1, 2, 3]
```

GALTI SE BACHO

Ye interview aur real projects dono ka famous bug hai. Independent copy chahiye to hamesha **.copy()** ya **list(a)** use karo.

8.8 sum(), min(), max(), sorted()

★ ADVANCED

Numbers ki list par kaam ke built-in functions.

main.py

```
nums = [4, 9, 2, 7]
print(sum(nums))
print(min(nums), max(nums))
print(sorted(nums))
print(nums)         # original safe
```

▶ OUTPUT

```
22
2 9
[2, 4, 7, 9]
[4, 9, 2, 7]
```

8.9 in operator & Looping

★ ADVANCED

Item list me hai ya nahi check karna, aur list par loop chalana.

main.py

```
fruits = ["apple", "banana"]
print("apple" in fruits)

for f in fruits:
    print(f.upper())
```

▶ OUTPUT

```
True
APPLE
BANANA
```

QUICK RECAP: SECTION 08

- ✓ List = ordered + changeable + duplicates allowed
- ✓ Index 0 se, slicing me end excluded, [::-1] reverse
- ✓ b = a alias banata hai, .copy() asli copy
- ✓ sum, min, max, len, sorted built-in hain

PRACTICE TIME Khud try karo, phir Appendix C me solution dekho

EASY	**Q1.** fruits = ["apple", "banana", "mango"] ka pehla aur aakhri item print karo.
MEDIUM	**Q2.** Ek list [5, 3, 8, 1] ka sabse bada, sabse chhota aur average print karo.
HARD	**Q3.** matrix = [[1, 2, 3], [4, 5, 6]] ke saare elements ka sum nested loop se nikalo.

INTERVIEW LAB / Lists

Ordered mutable data, indexing, nested structures aur safe copying. • Answer formula: definition → tiny example → practical use. Word-by-word memorize mat karo; apni language me explain karo.

Q01

List ki main properties?

BEST ANSWER

Ordered, mutable collection. Duplicates allowed hote hain aur mixed data types rakh sakti hai.

TIP: 20-30 sec me clearly bolo.

Q02

List indexing strings jaisi hai?

BEST ANSWER

Haan. Index 0 se start aur negative indexing end se. Slicing bhi same pattern follow karti hai.

TIP: 20-30 sec me clearly bolo.

Q03

Nested list kya hai?

BEST ANSWER

List ke andar list. 2D table/matrix jaisa structure represent kar sakti hai.

TIP: 20-30 sec me clearly bolo.

Q04

b = a aur b = a.copy() me fark?

BEST ANSWER

b=a same object ka alias hai. copy() nayi shallow list banata hai.

TIP: 20-30 sec me clearly bolo.

Q05

sum/min/max/sorted ka role?

BEST ANSWER

Numeric list par aggregate aur sorted result nikalne ke built-ins hain. sorted original ko change nahi karta.

TIP: 20-30 sec me clearly bolo.

Q06

List mutable hone ka practical impact?

BEST ANSWER

Ek reference se change karoge to same list ke doosre aliases par bhi effect aa sakta hai.

TIP: 20-30 sec me clearly bolo.

Mini speaking drill: Har answer bina PDF dekhe 2 baar bolo. Dusri baar ek real example add karo.

CODING LAB / Lists

Rule: Pehle problem ko 10 sec me apne words me bolo, phir code likho. Copy-paste ke bajaye khud type karo.

EASY

Fruits list ka first aur last item print karo.

WRITE CODE

EASY

[5,3,8,1] ka min, max aur average nikalo.

WRITE CODE

MEDIUM

Nested matrix ke saare elements ka sum nested loop se nikalo.

WRITE CODE

MEDIUM

List reverse karo aur original ka effect inspect karo.

WRITE CODE

HARD

Alias aur copy dono bana kar ek item append karo; dono outputs compare karo.

WRITE CODE

DEBUG

b=a ke baad b.append(99) se a kyun change hoti hai?

WRITE CODE

STARTER KIT - TRY WITHOUT LOOKING UP

```
list slicing nested index copy() sum() min() max()
```

3-STEP DEBUG LOOP

- 1. Error/bug ko reproduce karo
- 2. Values/types/conditions print karke isolate karo
- 3. Ek change karo, rerun karo, phir reason explain karo

Interview coding line: "Main pehle input/edge case define karunga, phir simple correct solution likhunga, phir complexity aur edge cases check karunga."

LOGIC LAB / Lists

Goal: code run karne se pehle output predict karo, phir dry-run karke reason bolo. Ye interview confidence aur debugging dono improve karta hai.

Predict

[10,20,30,40,50][1:4] -> ?

CHECK

[20,30,40]

Trace

matrix=[[1,2],[3,4]]; matrix[1][0]

CHECK

3

Think

copy() shallow kyun kaha jata hai?

CHECK

Nested objects same references ho sakte hain

Predict

[1,2,3][::-1] -> ?

CHECK

[3,2,1]

Fix

Average me empty list ho to kya edge case?

CHECK

Division by zero; validate non-empty

Explain

List ko choose kab karoge?

CHECK

Jab ordered, changeable collection chahiye

5 COMMON MISTAKES TO WATCH

1. alias vs copy
2. index out of range
3. empty list average
4. nested index confusion
5. sorted ko in-place samajhna

Mastery check: "Main is topic ko bina dekhe explain kar sakta hoon + 1 small program likh sakta hoon + 2 outputs predict kar sakta hoon."

SOLVED PRACTICE - Lists

Every coding-lab question now has its answer, expected result and a quick check - all in one place.

EASY

Fruits list ka first aur last item print karo.

```
fruits=["apple","banana","mango"]
print(fruits[0],fruits[-1])
```

EXPECTED RESULT

apple mango

EASY

[5,3,8,1] ka min,max,average.

```
x=[5,3,8,1]
print(min(x),max(x),sum(x)/len(x))
```

EXPECTED RESULT

1 8 4.25

MEDIUM

Nested matrix ke saare elements ka sum nested loop se.

```
m=[[1,2],[3,4]]
t=0
for row in m:
    for x in row: t+=x
print(t)
```

EXPECTED RESULT

10

MEDIUM

List reverse karo aur original effect inspect karo.

```
a=[1,2,3]
b=a[::-1]
print(a,b)
```

EXPECTED RESULT

[1, 2, 3] [3, 2, 1]

HARD

Alias aur copy bana kar append karo; outputs compare.

```
import copy
a=[1,2]; b=a; c=a.copy()
b.append(3)
print(a,c)
```

EXPECTED RESULT

a changes; c stays independent.

DEBUG

b=a ke baad b.append(99) se a kyun change?

```
a=[1,2]; b=a
b.append(99)
print(a)
```

EXPECTED RESULT

b aur a same list object ko refer karte hain.

PRACTICE HABIT

Practice rule: solve first, compare second, then change one input. Mark the question only after you can explain why the output changes.

MINI PROJECT - Shopping Cart

Lists ka concept ek runnable, interview-ready mini build me convert karo.

YE KYA KARTA HAI?

Items add karke total quantity calculate karta hai.

KIS KAAM AATA HAI?

Basic e-commerce workflows.

ARCHITECTURE

Input -> list -> aggregate -> output

FILES / BUILD TARGET

cart.py Target: Duplicate items allowed

CORE IMPLEMENTATION - TYPE + RUN

```
cart = ["book", "pen", "book"]
print(len(cart), cart.count("book"))
```

SAMPLE OUTPUT `3 2 (3 cart items, 2 books)`

INTERVIEW EXPLANATION

Project answer: List duplicates preserve karti hai aur order useful hai. Debug cue: Missing index par IndexError; membership check use karo. Follow-up: What would you improve first if this had 10x traffic/data?

BUILD STEPS

1. Create the file(s) shown above.
2. Run the exact sample once.
3. Change one input and observe the output.
4. Add one validation or failure test.

EDGE CASES / DEBUG

Test invalid input, boundary values and empty/missing data where relevant. Keep validation at the system boundary.

Lists

Use this pack after the section labs: explain -> build -> debug -> defend.

SECTION ANCHOR

Definition: Ordered mutable collections; batches, queues aur feature rows me common.

Easy explanation: List = ordered sequence; index/slice se data access.

Example: `nums = [1,2,3]; print(nums[0], nums[-1])`

SIMPLE

Shopping Cart

Kya karta hai: Items add karke total quantity calculate karta hai.

Kis kaam aata hai: Basic e-commerce workflows.

Architecture: Input -> list -> aggregate -> output

Files: `cart.py`

Build target: Duplicate items allowed

MEDIUM

Batch Builder

Kya karta hai: Inputs ko fixed-size batches me split karta hai.

Kis kaam aata hai: ML inference, embeddings and API request batching.

Architecture: Records -> list -> slicing -> batch loop

Files: `batch.py`

Build target: Last partial batch preserve karo

HARD

Dataset Batch Builder

Kya karta hai: Feature rows ko training/inference batches me package karta hai.

Kis kaam aata hai: AI pipelines and offline inference.

Architecture: Raw rows -> validation -> list of batches -> consumer

Files: `dataset.py`

Build target: Empty dataset and uneven final batch support

Project rule: code khud type karo, output predict karo, edge case test karo, phir 60-second explanation do.

Lists - implementation lab

Three core implementations + project-tailored interview explanation + debugging cue.

SIMPLE | SHOPPING CART

```
cart = ["book","pen","book"]
print(len(cart), cart.count("book"))
```

INTERVIEW DEFENSE

Project answer: List duplicates preserve karti hai aur order useful hai.

Debug cue: Missing index par IndexError; membership check use karo.

Follow-up: What would you improve first if this had 10x traffic/data?

MEDIUM | BATCH BUILDER

```
def batches(items, size):
    return [items[i:i+size] for i in range(0,len(items),size)
]
print(batches(list(range(7)), 3))
```

INTERVIEW DEFENSE

Project answer: Batching throughput improve kar sakti hai while controlling request size.

Debug cue: size <= 0 should raise an error.

Follow-up: What would you improve first if this had 10x traffic/data?

HARD | DATASET BATCH BUILDER

```
def make_batches(rows, bs=32):
    if bs <= 0: raise ValueError
    for i in range(0, len(rows), bs):
        yield rows[i:i+bs]
for batch in make_batches(rows): process(batch)
```

INTERVIEW DEFENSE

Project answer: Generator version memory-efficient hai compared with returning all batches.

Debug cue: Verify consumer handles empty list and final short batch.

Follow-up: What would you improve first if this had 10x traffic/data?

SELF-CHECK

Can I explain the project without notes? Can I name one failure? Can I state time/space or cost trade-offs?

SECTION 09

List Methods

Level: Intermediate | Time: ~60 min | 16 topics

Lists ko rozana badalna padta hai: naye items jodna, hatana, sort karna. Ye methods **list ko andar se hi badal dete hain** (in-place), aur zyada tar **None** return karte hain.

REAL-LIFE EXAMPLE SE SAMAJHO

List methods ek **remote control** ke buttons hain: add, remove, sort, reverse, sab ek click me.

IS SECTION ME KYA SEEKHOGE

- ✓ Add: append, extend, insert
- ✓ Remove: remove, pop, clear
- ✓ Search: index, count
- ✓ Order: sort, reverse, sorted, key=

TOPIC ROADMAP

9.1 append()
9.2 extend()
9.3 insert()
9.4 remove()
9.5 pop()
9.6 clear()
9.7 index()
9.8 count()
9.9 sort()
9.10 sort(reverse=True)
9.11 reverse()
9.12 copy()
9.13 sorted() vs sort() ★
9.14 sort with key= (custom sorting) ★
9.15 Stack (LIFO) using List ★
9.16 Remove Duplicates, Order Safe ★

★ = Advanced topic. Pehli baar padhte waqt inhe skip karke dusre round me karo.

9.1 append() CORE

List ke **end** me ek single item add karta hai.

main.py

```
fruits = ["apple", "banana"]
fruits.append("mango")
print(fruits)

fruits.append(["x", "y"])     # poori list ek item ban gayi
print(fruits)
```

▶ OUTPUT

```
['apple', 'banana', 'mango']
['apple', 'banana', 'mango', ['x', 'y']]
```

GALTI SE BACHO

nums = nums.append(5) mat likho! append() None return karta hai, list gayab ho jayegi. Bas **nums.append(5)** likho.

9.2 extend() CORE

Dusri list ke **saare items** end me jodta hai (append ki tarah ek list nahi).

main.py

```
fruits = ["apple", "banana"]
fruits.extend(["mango", "orange"])
print(fruits)
```

▶ OUTPUT

```
['apple', 'banana', 'mango', 'orange']
```

9.3 insert() CORE

Specific index par item add karta hai: **insert(index, item)**. Baaki items aage khisak jate hain.

main.py

```
fruits = ["apple", "banana", "orange"]
fruits.insert(2, "mango")
print(fruits)
fruits.insert(0, "kiwi")      # sabse aage
print(fruits)
```

▶ OUTPUT

```
['apple', 'banana', 'mango', 'orange']
['kiwi', 'apple', 'banana', 'mango', 'orange']
```

9.4 remove() CORE

Pehla **matching value** remove karta hai.

main.py

```
fruits = ["apple", "banana", "mango", "banana"]
fruits.remove("banana")
print(fruits)
```

▶ OUTPUT

```
['apple', 'mango', 'banana']
```

GALTI SE BACHO

Jo item list me nahi hai use remove karoge to **ValueError** aayega. Pehle **if item in list** check karo.

9.5 pop() CORE

Given **index** (default last) ka item remove karke **return** bhi karta hai.

main.py

```
fruits = ["apple", "banana", "mango"]
item = fruits.pop()
print(item)
print(fruits)
print(fruits.pop(0))     # pehla item
```

▶ OUTPUT

```
mango
['apple', 'banana']
apple
```

YAAD RAKHO

remove(value) value se hatata hai. **pop(index)** position se hatata hai aur item wapas deta hai.

9.6 clear() CORE

List ke saare items ek saath hata deta hai. List khaali reh jati hai.

main.py

```
fruits = ["apple", "banana"]
fruits.clear()
print(fruits)
```

▶ OUTPUT

```
[]
```

9.7 index() CORE

Item ka **pehla index** return karta hai. Na mile to ValueError.

main.py

```
fruits = ["apple", "banana", "mango"]
print(fruits.index("mango"))
```

▶ OUTPUT

```
2
```

9.8 count() CORE

Item list me kitni baar aaya hai, uski ginti.

main.py

```
nums = [1, 2, 2, 3, 2]
print(nums.count(2))
```

▶ OUTPUT

```
3
```

9.9 sort()

CORE

List ko **ascending** order me sort karta hai. Original list badal jati hai (in-place).

main.py

```
nums = [3, 1, 4, 1, 5]
nums.sort()
print(nums)

words = ["banana", "apple", "cherry"]
words.sort()
print(words)
```

▶ OUTPUT

```
[1, 1, 3, 4, 5]
['apple', 'banana', 'cherry']
```

9.10 sort(reverse=True)

CORE

List ko **descending** order me sort karta hai.

main.py

```
nums = [3, 1, 4, 1, 5]
nums.sort(reverse=True)
print(nums)
```

▶ OUTPUT

```
[5, 4, 3, 1, 1]
```

9.11 reverse()

CORE

Items ka order ulta kar deta hai (sort nahi karta, sirf ulta).

main.py

```
nums = [1, 2, 3]
nums.reverse()
print(nums)
```

▶ OUTPUT

```
[3, 2, 1]
```

9.12 copy()

CORE

List ki **shallow copy** banata hai taki original safe rahe.

main.py

```
fruits = ["apple", "banana"]
new_list = fruits.copy()
new_list.append("mango")
print(fruits)
print(new_list)
```

▶ OUTPUT

```
['apple', 'banana']
['apple', 'banana', 'mango']
```

9.13 sorted() vs sort()

★ ADVANCED

sort() list ko badal deta hai aur None deta hai. **sorted()** original ko chhode bina nayi sorted list deta hai (kisi bhi iterable par).

main.py

```
nums = [3, 1, 2]
new = sorted(nums)
print(new)
print(nums)              # original wahi
print(sorted("python"))   # string par bhi chalta hai
```

▶ OUTPUT

```
[1, 2, 3]
[3, 1, 2]
['h', 'n', 'o', 'p', 't', 'y']
```

9.14 sort with key= (custom sorting)

★ ADVANCED

key batata hai ki sorting kis basis par ho. **lambda** ek chhota anonymous function hai.

main.py

```
words = ["banana", "kiwi", "apple", "fig"]
words.sort(key=len)
print(words)

students = [("Ravi", 82), ("Sita", 95), ("Amit", 70)]
students.sort(key=lambda s: s[1], reverse=True)
print(students)
```

▶ OUTPUT

```
['fig', 'kiwi', 'apple', 'banana']
[('Sita', 95), ('Ravi', 82), ('Amit', 70)]
```

TIP

lambda s: s[1] ka matlab: "har item s ko lo aur uska index 1 wala part sorting ke liye use karo".

9.15 Stack (LIFO) using List

★ ADVANCED

Stack = last in, first out. Plates ki pile jaisa. **append()** se push, **pop()** se pop.

main.py

```
stack = []
stack.append("a")
stack.append("b")
stack.append("c")
print(stack.pop())    # c pehle nikla
print(stack.pop())
print(stack)
```

▶ OUTPUT

```
c
b
['a']
```

9.16 Remove Duplicates, Order Safe ★ ADVANCED

set() duplicate hata deta hai par order bigad jata hai. **dict.fromkeys()** order bacha ke duplicates hata deta hai.

main.py

```
nums = [3, 1, 3, 2, 1]
print(list(set(nums)))              # order guarantee nahi
print(list(dict.fromkeys(nums)))    # order safe
```

▶ OUTPUT

```
[1, 2, 3]
[3, 1, 2]
```

QUICK RECAP: SECTION 09

- ✓ append (1 item), extend (many), insert (index par)
- ✓ remove (value se), pop (index se + return), clear (sab)
- ✓ sort() in-place, sorted() nayi list
- ✓ Methods jaise append/sort None return karte hain, unhe variable me assign mat karo

PRACTICE TIME Khud try karo, phir Appendix C me solution dekho

EASY	**Q1.** Ek khaali list banao, usme 10, 20, 30 append karo aur print karo.
MEDIUM	**Q2.** ["pear", "fig", "banana", "kiwi"] ko length ke hisab se descending sort karo.
HARD	**Q3.** [1, 2, 2, 3, 1, 4] se duplicates hatao aur order same rakho.

INTERVIEW LAB / List Methods

List ko add/remove/search/sort karke practical data processing karna. • Answer formula: definition → tiny example → practical use. Word-by-word memorize mat karo; apni language me explain karo.

Q01

append vs extend?

BEST ANSWER

append ek single item add karta hai; extend iterable ke items ko alag-alag add karta hai.

TIP: 20-30 sec me clearly bolo.

Q02

remove vs pop?

BEST ANSWER

remove value ke first matching item ko hataata hai. pop index se item hataata hai aur removed value return karta hai.

TIP: 20-30 sec me clearly bolo.

Q03

sort vs sorted?

BEST ANSWER

sort list ko in-place modify karta hai aur None return karta hai. sorted nayi sorted list deta hai aur any iterable par chal sakta hai.

TIP: 20-30 sec me clearly bolo.

Q04

sort(key=...) ka use?

BEST ANSWER

Custom sorting criterion define karne ke liye, jaise strings ko length ya tuples ko score ke basis par.

TIP: 20-30 sec me clearly bolo.

Q05

List ko stack ke roop me kaise use karte hain?

BEST ANSWER

append() push aur pop() pop. Isse LIFO behaviour milta hai.

TIP: 20-30 sec me clearly bolo.

Q06

Order-safe duplicate removal?

BEST ANSWER

dict.fromkeys(items) se insertion order preserve karte hue duplicates remove kar sakte hain.

TIP: 20-30 sec me clearly bolo.

Mini speaking drill: Har answer bina PDF dekhe 2 baar bolo. Dusri baar ek real example add karo.

CODING LAB / List Methods

Rule: Pehle problem ko 10 sec me apne words me bolo, phir code likho. Copy-paste ke bajaye khud type karo.

EASY

Empty list me 10,20,30 append karo.

WRITE CODE

EASY

List se last item pop karke print karo.

WRITE CODE

MEDIUM

Words ko length ke descending order me sort karo.

WRITE CODE

MEDIUM

Students tuples ko marks ke descending order me sort karo.

WRITE CODE

HARD

[1,2,2,3,1,4] se duplicates hatao, order same rakho.

WRITE CODE

DEBUG

nums = nums.append(5) galat kyun?

WRITE CODE

STARTER KIT - TRY WITHOUT LOOKING UP

```
append() extend() pop() remove() sort() sorted() key=
```

3-STEP DEBUG LOOP

- 1. Error/bug ko reproduce karo
- 2. Values/types/conditions print karke isolate karo
- 3. Ek change karo, rerun karo, phir reason explain karo

Interview coding line: "Main pehle input/edge case define karunga, phir simple correct solution likhunga, phir complexity aur edge cases check karunga."

LOGIC LAB / List Methods

Goal: code run karne se pehle output predict karo, phir dry-run karke reason bolo. Ye interview confidence aur debugging dono improve karta hai.

Predict

["a","b"].append(["c","d"]) -> ?

CHECK

["a","b",["c","d"]]

Trace

nums=[3,1,2]; x=nums.sort(); print(x)

CHECK

None

Think

sorted("python") ka output kis type ka hai?

CHECK

List of characters

Predict

["pear","fig"].sort(key=len) -> ?

CHECK

["fig","pear"]

Fix

remove("x") on missing item?

CHECK

ValueError; check membership

Explain

Stack ka core rule?

CHECK

LIFO: last in, first out

5 COMMON MISTAKES TO WATCH

1. append vs extend
2. method return None
3. remove missing item
4. sort mutates list
5. stack top = last item

Mastery check: "Main is topic ko bina dekhe explain kar sakta hoon + 1 small program likh sakta hoon + 2 outputs predict kar sakta hoon."

SOLVED PRACTICE - List Methods

Every coding-lab question now has its answer, expected result and a quick check - all in one place.

EASY

Empty list me 10,20,30 append karo.

```
x=[]
x.append(10); x.append(20); x.append(30)
print(x)
```

EXPECTED RESULT

[10, 20, 30]

EASY

List se last item pop karke print karo.

```
x=[10,20,30]
print(x.pop())
```

EXPECTED RESULT

30

MEDIUM

Words ko length descending order me sort karo.

```
w=["ai","python","api"]
print(sorted(w,key=len,reverse=True))
```

EXPECTED RESULT

['python', 'api', 'ai']

MEDIUM

Student tuples ko marks descending sort karo.

```
s=[("A",70),("B",95),("C",85)]
print(sorted(s,key=lambda x:x[1],reverse=True))
```

EXPECTED RESULT

[('B', 95), ('C', 85), ('A', 70)]

HARD

[1,2,2,3,1,4] duplicates hatao, order same rakho.

```
x=[1,2,2,3,1,4]; out=[]
for v in x:
    if v not in out: out.append(v)
print(out)
```

EXPECTED RESULT

[1, 2, 3, 4]

DEBUG

nums=nums.append(5) galat kyun?

```
nums=[1,2]
nums.append(5)
print(nums)
```

EXPECTED RESULT

append() list ko mutate karta hai aur None return karta hai.

PRACTICE HABIT

Practice rule: solve first, compare second, then change one input. Mark the question only after you can explain why the output changes.

MINI PROJECT - Task Queue

List Methods ka concept ek runnable, interview-ready mini build me convert karo.

YE KYA KARTA HAI?

Pending tasks ko append aur pop ke through manage karta hai.

KIS KAAM AATA HAI?

CLI automation and small queues.

ARCHITECTURE

Task source -> list -> append/pop -> worker

FILES / BUILD TARGET

queue.py Target: Empty queue handling

CORE IMPLEMENTATION - TYPE + RUN

```
queue = []
queue.append("email")
queue.append("report")
task = queue.pop(0)
print(task, queue)
```

SAMPLE OUTPUT `email ['report']`

INTERVIEW EXPLANATION

Project answer: List queue is fine for small workloads; real FIFO at scale usually needs deque. Debug cue: pop(0) is O(n), so repeated front pops can become slow. Follow-up: What would you improve first if this had 10x traffic/data?

BUILD STEPS

1. Create the file(s) shown above.
2. Run the exact sample once.
3. Change one input and observe the output.
4. Add one validation or failure test.

EDGE CASES / DEBUG

Test invalid input, boundary values and empty/missing data where relevant. Keep validation at the system boundary.

List Methods

Use this pack after the section labs: explain -> build -> debug -> defend.

SECTION ANCHOR

Definition: Mutating and querying lists safely: append, extend, insert, remove, pop, sort, reverse.

Easy explanation: Method choice depends on mutation, ordering and return value.

Example: `items = [3,1]; items.sort(); print(items)`

SIMPLE

Task Queue

Kya karta hai: Pending tasks ko append aur pop ke through manage karta hai.

Kis kaam aata hai: CLI automation and small queues.

Architecture: Task source -> list -> append/pop -> worker

Files: `queue.py`

Build target: Empty queue handling

MEDIUM

Top Scores

Kya karta hai: Scores collect, sort aur top-k return karta hai.

Kis kaam aata hai: Leaderboards and ranking utilities.

Architecture: Scores -> append -> sort/reverse -> slice

Files: `scores.py`

Build target: k > len(scores) safe

HARD

Transformation Pipeline

Kya karta hai: List operations ko composable cleaning stages me chain karta hai.

Kis kaam aata hai: Preprocessing and ETL prototypes.

Architecture: Raw list -> map/filter style stages -> final list

Files: `pipeline.py`

Build target: Keep each stage testable

Project rule: code khud type karo, output predict karo, edge case test karo, phir 60-second explanation do.

List Methods - implementation lab

Three core implementations + project-tailored interview explanation + debugging cue.

SIMPLE | TASK QUEUE

```
queue=[]
queue.append("email")
queue.append("report")
task = queue.pop(0)
print(task, queue)
```

INTERVIEW DEFENSE

Project answer: List queue is fine for small workloads; real FIFO at scale usually needs deque.

Debug cue: pop(0) is O(n), so repeated front pops can become slow.

Follow-up: What would you improve first if this had 10x traffic/data?

MEDIUM | TOP SCORES

```
scores=[72,91,88,95]
scores.sort(reverse=True)
print(scores[:3])
```

INTERVIEW DEFENSE

Project answer: Sort gives O(n log n); for huge streams top-k heap can be better.

Debug cue: sort() returns None; it mutates in place.

Follow-up: What would you improve first if this had 10x traffic/data?

HARD | TRANSFORMATION PIPELINE

```
rows=[" A ","","," B "]
rows=[r.strip() for r in rows]
rows=[r for r in rows if r]
rows.sort()
print(rows)
```

INTERVIEW DEFENSE

Project answer: Readable stage separation makes debugging easier than one giant expression.

Debug cue: Track length/content after every stage when rows unexpectedly disappear.

Follow-up: What would you improve first if this had 10x traffic/data?

SELF-CHECK

Can I explain the project without notes? Can I name one failure? Can I state time/space or cost trade-offs?

SECTION 10

Tuples

Level: Intermediate | Time: ~35 min | 9 topics

Tuple list jaisa hi hai, bas **immutable**. Jab data fixed rehna chahiye (coordinates, RGB color, DB row) tab tuple use karte hain. Ye list se thoda fast bhi hota hai.

REAL-LIFE EXAMPLE SE SAMAJHO

Tuple ek **seal-band packet** hai: ek baar pack ho gaya to andar ka saman badal nahi sakte.

IS SECTION ME KYA SEEKHOGE

- ✓ Tuple banana, indexing, slicing
- ✓ Immutability aur unpacking
- ✓ Single-item tuple ki trick
- ✓ namedtuple aur function se multiple values return karna

TOPIC ROADMAP

10.1 Creating Tuples
10.2 Indexing & Slicing
10.3 Immutability
10.4 Tuple Unpacking
10.5 count() & index()
10.6 Why use Tuples?
10.7 Single-item Tuple (comma!) ★
10.8 Return Multiple Values ★
10.9 namedtuple ★

★ = Advanced topic. Pehli baar padhte waqt inhe skip karke dusre round me karo.

10.1 Creating Tuples

CORE

Tuple list jaisa hai par **immutable** hota hai, **( )** me likha jata hai.

main.py

```
point = (10, 20)
print(point)
print(type(point))

rgb = 255, 128, 0         # bracket ke bina bhi tuple
print(rgb)
```

▶ OUTPUT

```
(10, 20)
<class 'tuple'>
(255, 128, 0)
```

10.2 Indexing & Slicing CORE

Tuple me bhi list jaisi hi indexing aur slicing chalti hai.

main.py

```
point = (10, 20, 30)
print(point[0])
print(point[1:])
print(point[-1])
```

▶ OUTPUT

```
10
(20, 30)
30
```

10.3 Immutability CORE

Tuple banne ke baad uske items **badle nahi** ja sakte.

main.py

```
point = (10, 20)
try:
    point[0] = 99
except TypeError as e:
    print("Error:", e)
```

▶ OUTPUT

```
Error: 'tuple' object does not support item assignment
```

10.4 Tuple Unpacking CORE

Tuple ki values ko seedhe alag-alag variables me nikal sakte ho. Variables ki ginti items ke barabar honi chahiye.

main.py

```
point = (10, 20)
x, y = point
print(x, y)

name, age, city = ("Ashu", 22, "Delhi")
print(name, city)
```

▶ OUTPUT

```
10 20
Ashu Delhi
```

10.5 count() & index() CORE

Tuple me sirf ye 2 methods hote hain (kyunki badal nahi sakta).

main.py

```
nums = (1, 2, 2, 3)
print(nums.count(2))
print(nums.index(3))
```

▶ OUTPUT

```
2
3
```

10.6 Why use Tuples? CORE

Jab data ko accidentally change hone se bachana ho. Tuples dictionary keys bhi ban sakte hain, list nahi.

main.py

```
coordinates = (28.6, 77.2)     # lat, long: badalna nahi chahiye
print(coordinates)

locations = {(28.6, 77.2): "Delhi"}
print(locations[(28.6, 77.2)])
```

▶ OUTPUT

```
(28.6, 77.2)
Delhi
```

10.7 Single-item Tuple (comma!) ★ ADVANCED

Ek item ka tuple banane ke liye **comma zaroori** hai. Bina comma ke wo sirf bracket wala number/string hai.

main.py

```
a = (5)
b = (5,)
print(type(a))
print(type(b))
```

▶ OUTPUT

```
<class 'int'>
<class 'tuple'>
```

GALTI SE BACHO

(5) tuple nahi hai, sirf int hai. **(5,)** tuple hai.

10.8 Return Multiple Values ★ ADVANCED

Function ek se zyada values tuple ke roop me return kar sakta hai. Unpacking se alag alag mil jati hain.

main.py

```
def min_max(nums):
    return min(nums), max(nums)

low, high = min_max([4, 9, 2, 7])
print(low, high)
print(min_max([4, 9, 2, 7]))
```

▶ OUTPUT

```
2 9
(2, 9)
```

10.9 namedtuple ★ ADVANCED

Tuple ke items ko **naam** de deta hai taki t[0] ki jagah t.x likh sako. Code padhne me aasaan.

main.py

```
from collections import namedtuple

Point = namedtuple("Point", ["x", "y"])
p = Point(10, 20)
print(p.x, p.y)
print(p)
```

▶ OUTPUT

```
10 20
Point(x=10, y=20)
```

QUICK RECAP: SECTION 10

- ✓ Tuple = immutable list, ( ) me likhte hain
- ✓ Unpacking: x, y = point
- ✓ Single item ke liye (5,) comma chahiye
- ✓ Fixed data, dict keys aur multiple return values ke liye best

PRACTICE TIME Khud try karo, phir Appendix C me solution dekho

EASY	**Q1.** t = (5, 10, 15) ka pehla aur aakhri item print karo.
MEDIUM	**Q2.** Tuple ("Ashu", 22, "Delhi") ko 3 variables me unpack karke f-string se print karo.
HARD	**Q3.** Ek function likho jo list ka sum aur average tuple me return kare.

INTERVIEW LAB / Tuples

Fixed data, unpacking aur multiple-value returns. • Answer formula: definition → tiny example → practical use. Word-by-word memorize mat karo; apni language me explain karo.

Q01

Tuple aur list me main difference?

BEST ANSWER

Tuple immutable hoti hai, list mutable. Fixed data ke liye tuple useful hai.

TIP: 20-30 sec me clearly bolo.

Q02

Tuple indexing/slicing chalti hai?

BEST ANSWER

Haan, list jaisi indexing aur slicing hoti hai.

TIP: 20-30 sec me clearly bolo.

Q03

Single-item tuple ka syntax?

BEST ANSWER

(5,) - comma required hai. Sirf (5) integer expression hai.

TIP: 20-30 sec me clearly bolo.

Q04

Tuple unpacking kya hai?

BEST ANSWER

Tuple/iterable ke items ko multiple variables me assign karna, jaise name, age = data.

TIP: 20-30 sec me clearly bolo.

Q05

Function multiple values kaise return kar sakta hai?

BEST ANSWER

return a, b likhne par Python tuple return karta hai, jise unpack kiya ja sakta hai.

TIP: 20-30 sec me clearly bolo.

Q06

namedtuple kab useful?

BEST ANSWER

Fixed record ko named fields ke saath readable banana ho aur full class ki zarurat na ho.

TIP: 20-30 sec me clearly bolo.

Mini speaking drill: Har answer bina PDF dekhe 2 baar bolo. Dusri baar ek real example add karo.

CODING LAB / Tuples

Rule: Pehle problem ko 10 sec me apne words me bolo, phir code likho. Copy-paste ke bajaye khud type karo.

EASY

(5,10,15) ka first aur last item print karo.

WRITE CODE

EASY

("Ashu",22,"Pune") ko 3 variables me unpack karo.

WRITE CODE

MEDIUM

Tuple of numbers ka count aur index methods use karo.

WRITE CODE

MEDIUM

Function likho jo min aur max tuple me return kare.

WRITE CODE

HARD

Single-item tuple aur normal parenthesized integer ka type compare karo.

WRITE CODE

DEBUG

t=(1,2,3); t[0]=9 error kyun?

WRITE CODE

STARTER KIT - TRY WITHOUT LOOKING UP

```
tuple unpacking (x,) count() index() return a,b
```

3-STEP DEBUG LOOP

- 1. Error/bug ko reproduce karo
- 2. Values/types/conditions print karke isolate karo
- 3. Ek change karo, rerun karo, phir reason explain karo

Interview coding line: "Main pehle input/edge case define karunga, phir simple correct solution likhunga, phir complexity aur edge cases check karunga."

LOGIC LAB / Tuples

Goal: code run karne se pehle output predict karo, phir dry-run karke reason bolo. Ye interview confidence aur debugging dono improve karta hai.

Predict

type((5)) -> ?

CHECK

int

Predict

type((5,)) -> ?

CHECK

tuple

Trace

a,b=(10,20); print(a,b)

CHECK

10 20

Think

Tuple ka immutability practical kab help karta hai?

CHECK

Fixed record ko accidental change se protect karne me

Fix

x,y,z = (1,2) problem?

CHECK

Number of variables/items mismatch

Explain

Tuple ko DB row/coordinates jaisi cheezon ke liye kyun use kar sakte hain?

CHECK

Fixed-position data represent karta hai

5 COMMON MISTAKES TO WATCH

1. single-item comma bhoolna
2. unpacking count mismatch
3. immutability ignore karna
4. tuple methods overexpect karna
5. list aur tuple use-case confuse karna

Mastery check: "Main is topic ko bina dekhe explain kar sakta hoon + 1 small program likh sakta hoon + 2 outputs predict kar sakta hoon."

SOLVED PRACTICE - Tuples

Every coding-lab question now has its answer, expected result and a quick check - all in one place.

EASY

(5,10,15) ka first aur last item print karo.

```
t=(5,10,15)
print(t[0],t[-1])
```

EXPECTED RESULT

5 15

EASY

("Ashu",22,"Pune") ko 3 variables me unpack.

```
name,age,city=("Ashu",22,"Pune")
print(name,age,city)
```

EXPECTED RESULT

Ashu 22 Pune

MEDIUM

Tuple of numbers ka count aur index methods use karo.

```
t=(1,2,2,3)
print(t.count(2),t.index(3))
```

EXPECTED RESULT

2 3

MEDIUM

Function jo min aur max tuple me return kare.

```
def minmax(t): return min(t),max(t)
print(minmax((4,9,2)))
```

EXPECTED RESULT

(2, 9)

HARD

Single-item tuple vs normal parenthesized integer type.

```
a=(5,)
b=(5)
print(type(a).__name__,type(b).__name__)
```

EXPECTED RESULT

tuple int

DEBUG

t=(1,2,3); t[0]=9 error kyun?

```
t=(1,2,3)
# t[0]=9  -> TypeError
print(t)
```

EXPECTED RESULT

Tuple immutable hota hai; reassignment allowed nahi.

PRACTICE HABIT

Practice rule: solve first, compare second, then change one input. Mark the question only after you can explain why the output changes.

MINI PROJECT - Coordinate Record

Tuples ka concept ek runnable, interview-ready mini build me convert karo.

YE KYA KARTA HAI?

Latitude/longitude ko immutable pair me store karta hai.

KIS KAAM AATA HAI?

Geo data and API responses.

ARCHITECTURE

Input -> tuple -> unpack -> consumer

FILES / BUILD TARGET

geo.py Target: Unpack safely

CORE IMPLEMENTATION - TYPE + RUN

```
point = (18.52, 73.85)
lat, lon = point
print(lat, lon)
```

SAMPLE OUTPUT `18.52 73.85`

INTERVIEW EXPLANATION

Project answer: Tuple suitable when the record should not be mutated accidentally. Debug cue: Unpacking count must match tuple length. Follow-up: What would you improve first if this had 10x traffic/data?

BUILD STEPS

1. Create the file(s) shown above.
2. Run the exact sample once.
3. Change one input and observe the output.
4. Add one validation or failure test.

EDGE CASES / DEBUG

Test invalid input, boundary values and empty/missing data where relevant. Keep validation at the system boundary.

Tuples

Use this pack after the section labs: explain -> build -> debug -> defend.

SECTION ANCHOR

Definition: Fixed, ordered records ke liye useful; unpacking aur immutability common interview topics hain.

Easy explanation: Tuple = ordered immutable sequence.

Example: `point = (18.5, 73.8); lat, lon = point`

SIMPLE

Coordinate Record

Kya karta hai: Latitude/longitude ko immutable pair me store karta hai.

Kis kaam aata hai: Geo data and API responses.

Architecture: Input -> tuple -> unpack -> consumer

Files: `geo.py`

Build target: Unpack safely

MEDIUM

API Status Record

Kya karta hai: Status code, latency aur success ko compact tuple me return karta hai.

Kis kaam aata hai: Internal helper functions and metrics.

Architecture: API call -> tuple result -> caller unpacks

Files: `client.py`

Build target: Document tuple order

HARD

Immutable Config Snapshot

Kya karta hai: Critical runtime settings ka fixed snapshot banata hai.

Kis kaam aata hai: Logging, reproducibility and config auditing.

Architecture: Validated config -> tuple snapshot -> audit/storage

Files: `config.py`

Build target: Stable order and explicit names

Project rule: code khud type karo, output predict karo, edge case test karo, phir 60-second explanation do.

Tuples - implementation lab

Three core implementations + project-tailored interview explanation + debugging cue.

SIMPLE | COORDINATE RECORD

```
point = (18.52, 73.85)
lat, lon = point
print(lat, lon)
```

INTERVIEW DEFENSE

Project answer: Tuple suitable when the record should not be mutated accidentally.

Debug cue: Unpacking count must match tuple length.

Follow-up: What would you improve first if this had 10x traffic/data?

MEDIUM | API STATUS RECORD

```
def result():
    return 200, 0.31, True
status, latency, ok = result()
print(status, latency, ok)
```

INTERVIEW DEFENSE

Project answer: Tuples are lightweight but less self-documenting than dataclasses for complex records.

Debug cue: Wrong positional unpacking is an easy source of bugs.

Follow-up: What would you improve first if this had 10x traffic/data?

HARD | IMMUTABLE CONFIG SNAPSHOT

```
config = ("prod", 30, True)
ENV, TIMEOUT, DEBUG = config
print(ENV, TIMEOUT, DEBUG)
```

INTERVIEW DEFENSE

Project answer: For larger configs, frozen dataclass is often clearer; tuple is best for tiny stable records.

Debug cue: Prefer named structures when positional meaning becomes unclear.

Follow-up: What would you improve first if this had 10x traffic/data?

SELF-CHECK

Can I explain the project without notes? Can I name one failure? Can I state time/space or cost trade-offs?

SECTION 11

Sets

Level: Intermediate | Time: ~40 min | 10 topics

Set **unique** items ka collection hai. Duplicates hatane aur do groups ko compare karne (common, alag, sab milake) me ye bahut kaam aata hai. Membership check (in) list se kaafi tez hota hai.

REAL-LIFE EXAMPLE SE SAMAJHO

Set ek **guest list** jaisi hai: har naam sirf ek baar, order ki koi guarantee nahi. Bas ye pata hota hai ki koi list me hai ya nahi.

IS SECTION ME KYA SEEKHOGE

- ✓ Set banana aur duplicates hatana
- ✓ add, remove, discard
- ✓ union, intersection, difference
- ✓ Set operators | & - ^

TOPIC ROADMAP

11.1 Creating Sets
11.2 add() & remove()
11.3 Removing Duplicates
11.4 union()
11.5 intersection()
11.6 difference()
11.7 Set Operators | & - ^ ★
11.8 discard() vs remove() ★
11.9 issubset(), issuperset(), isdisjoint() ★
11.10 Set Comprehension & Practical Use ★

★ = Advanced topic. Pehli baar padhte waqt inhe skip karke dusre round me karo.

11.1 Creating Sets

CORE

Set **unordered** collection hai jisme **duplicates allowed nahi**. **{ }** me likha jata hai.

main.py

```
fruits = {"apple", "banana", "mango", "apple"}
print(len(fruits))     # apple ek hi baar
print("apple" in fruits)

empty = set()          # khaali set: {} nahi, set() likho
print(type(empty))
```

▶ OUTPUT

```
3
True
<class 'set'>
```

GALTI SE BACHO

{} khaali dictionary banata hai, set nahi. Khaali set ke liye **set()** likho.

YAAD RAKHO

Set ka order fixed nahi hota, isliye print me items kisi bhi order me aa sakte hain. Index se access nahi hota.

11.2 add() & remove() CORE

Set me item **add** karo ya **remove** karo.

main.py

```
fruits = {"apple", "banana"}
fruits.add("mango")
fruits.remove("apple")
print(sorted(fruits))
```

▶ OUTPUT

```
['banana', 'mango']
```

11.3 Removing Duplicates CORE

List ko set me convert karne se duplicates **automatically** hat jate hain.

main.py

```
nums = [1, 2, 2, 3, 3, 3]
unique = set(nums)
print(unique)
print(len(unique))
```

▶ OUTPUT

```
{1, 2, 3}
3
```

11.4 union() CORE

Do sets ke **sare unique items** milakar naya set banata hai.

main.py

```
a = {1, 2, 3}
b = {3, 4, 5}
print(a.union(b))
```

▶ OUTPUT

```
{1, 2, 3, 4, 5}
```

11.5 intersection() CORE

Sirf wo items jo **dono sets me common** hain.

main.py

```
a = {1, 2, 3}
b = {2, 3, 4}
print(a.intersection(b))
```

▶ OUTPUT

```
{2, 3}
```

11.6 difference() CORE

Pehle set ke wo items jo **dusre me nahi** hain.

main.py

```
a = {1, 2, 3}
b = {2, 3, 4}
print(a.difference(b))
print(b.difference(a))     # order matter karta hai
```

▶ OUTPUT

```
{1}
{4}
```

11.7 Set Operators | & - ^ ★ ADVANCED

Methods ke shortcut operators: **|** union, **&** intersection, **-** difference, **^** symmetric difference (jo sirf ek me hai).

main.py

```
a = {1, 2, 3}
b = {2, 3, 4}
print(a | b)
print(a & b)
print(a - b)
print(a ^ b)
```

▶ OUTPUT

```
{1, 2, 3, 4}
{2, 3}
{1}
{1, 4}
```

11.8 discard() vs remove() ★ ADVANCED

remove() na mile to KeyError deta hai. **discard()** chup-chap ignore kar deta hai.

main.py

```
s = {1, 2, 3}
s.discard(99)        # koi error nahi
print(s)
try:
    s.remove(99)
except KeyError:
    print("KeyError: 99 nahi mila")
```

▶ OUTPUT

```
{1, 2, 3}
KeyError: 99 nahi mila
```

11.9 issubset(), issuperset(), isdisjoint()

★ ADVANCED

Do sets ka relation check karte hain.

main.py

```
a = {1, 2}
b = {1, 2, 3, 4}
print(a.issubset(b))
print(b.issuperset(a))
print(a.isdisjoint({8, 9}))    # koi common nahi
```

▶ OUTPUT

```
True
True
True
```

11.10 Set Comprehension & Practical Use

★ ADVANCED

Comprehension set me bhi chalta hai. Real use: do lists me common items dhoondhna.

main.py

```
print({x % 3 for x in range(10)})

class_a = ["Ravi", "Sita", "Amit"]
class_b = ["Sita", "Neha", "Amit"]
print(set(class_a) & set(class_b))
```

▶ OUTPUT

```
{0, 1, 2}
{'Amit', 'Sita'}
```

QUICK RECAP: SECTION 11

- ✓ Set = unique items, unordered, index nahi
- ✓ Khaali set: set(), khaali dict: {}
- ✓ | union, & common, - difference, ^ sirf ek me
- ✓ Duplicates hatane aur fast in-check ke liye best

PRACTICE TIME Khud try karo, phir Appendix C me solution dekho

EASY	**Q1.** [5, 1, 5, 2, 1] me kitne unique numbers hain? Print karo.
MEDIUM	**Q2.** Do lists me common items nikalo: [1, 2, 3, 4] aur [3, 4, 5, 6].
HARD	**Q3.** Ek string "programming" me kaun kaun se unique letters hain, sorted order me print karo.

INTERVIEW LAB / Sets

Unique values, membership aur set relationships. • Answer formula: definition → tiny example → practical use. Word-by-word memorize mat karo; apni language me explain karo.

Q01

Set ki main property?

BEST ANSWER

Set unique items ka collection hai. Duplicates automatically collapse hote hain.

TIP: 20-30 sec me clearly bolo.

Q02

Set ka common use?

BEST ANSWER

Duplicates remove karna aur membership checks/union/intersection jaise operations.

TIP: 20-30 sec me clearly bolo.

Q03

union/intersection/difference?

BEST ANSWER

Union sab items, intersection common items, difference first set ke unique items deta hai.

TIP: 20-30 sec me clearly bolo.

Q04

discard vs remove?

BEST ANSWER

remove missing item par KeyError deta hai; discard missing item par error nahi deta.

TIP: 20-30 sec me clearly bolo.

Q05

issubset aur issuperset?

BEST ANSWER

A subset B ka matlab A ke saare elements B me hain. Superset iska reverse relation check karta hai.

TIP: 20-30 sec me clearly bolo.

Q06

Set comprehension kya hai?

BEST ANSWER

Loop se unique values ka set concise syntax me banana, jaise {x*x for x in nums}.

TIP: 20-30 sec me clearly bolo.

Mini speaking drill: Har answer bina PDF dekhe 2 baar bolo. Dusri baar ek real example add karo.

CODING LAB / Sets

Rule: Pehle problem ko 10 sec me apne words me bolo, phir code likho. Copy-paste ke bajaye khud type karo.

EASY

[5,1,5,2,1] me unique count print karo.

WRITE CODE

EASY

Do lists ke common items nikalo.

WRITE CODE

MEDIUM

Do sets ka union, intersection aur difference nikalo.

WRITE CODE

MEDIUM

String "programming" ke unique letters sorted order me print karo.

WRITE CODE

HARD

Subset/superset check ka mini permission example banao.

WRITE CODE

DEBUG

Set ko index se access karne ki koshish kyun nahi kar sakte?

WRITE CODE

STARTER KIT - TRY WITHOUT LOOKING UP

```
set() add() union() intersection() difference() subset
```

3-STEP DEBUG LOOP

- 1. Error/bug ko reproduce karo
- 2. Values/types/conditions print karke isolate karo
- 3. Ek change karo, rerun karo, phir reason explain karo

Interview coding line: "Main pehle input/edge case define karunga, phir simple correct solution likhunga, phir complexity aur edge cases check karunga."

LOGIC LAB / Sets

Goal: code run karne se pehle output predict karo, phir dry-run karke reason bolo. Ye interview confidence aur debugging dono improve karta hai.

Predict

set([1,1,2,3]) -> ?

CHECK

{1,2,3}

Trace

{1,2,3} & {2,3,4} -> ?

CHECK

{2,3}

Think

Set membership list se useful kab?

CHECK

Jab uniqueness + membership important ho

Predict

{1,2}.issubset({1,2,3}) -> ?

CHECK

True

Fix

a.remove(99) missing item par?

CHECK

KeyError

Explain

Set me order-based logic kyun avoid karna chahiye?

CHECK

Set is primarily for membership/relationship, not positional access

5 COMMON MISTAKES TO WATCH

1. set indexing
2. remove vs discard
3. difference direction
4. assuming order
5. duplicates count karna

Mastery check: "Main is topic ko bina dekhe explain kar sakta hoon + 1 small program likh sakta hoon + 2 outputs predict kar sakta hoon."

SOLVED PRACTICE - Sets

Every coding-lab question now has its answer, expected result and a quick check - all in one place.

EASY

[5,1,5,2,1] me unique count print karo.

```
x=[5,1,5,2,1]
print(len(set(x)))
```

EXPECTED RESULT

3

EASY

Do lists ke common items nikalo.

```
a=[1,2,3]; b=[2,3,4]
print(sorted(set(a)&set(b)))
```

EXPECTED RESULT

[2, 3]

MEDIUM

Union, intersection, difference nikalo.

```
a={1,2,3}; b={2,3,4}
print(a|b,a&b,a-b)
```

EXPECTED RESULT

{1,2,3,4} {2,3} {1}

MEDIUM

"programming" ke unique letters sorted order me.

```
print(sorted(set("programming")))
```

EXPECTED RESULT

['a','g','i','m','n','o','p','r']

HARD

Subset/superset permission example.

```
user={"read","write"}; required={"read"}
print(required.issubset(user))
```

EXPECTED RESULT

True

DEBUG

Set ko index se access kyun nahi?

```
s={10,20,30}
# s[0] -> TypeError
print(len(s))
```

EXPECTED RESULT

Set ordered indexing contract nahi deta.

PRACTICE HABIT

Practice rule: solve first, compare second, then change one input. Mark the question only after you can explain why the output changes.

MINI PROJECT - Unique Tags

Sets ka concept ek runnable, interview-ready mini build me convert karo.

YE KYA KARTA HAI?

Duplicate tags ko remove karta hai.

KIS KAAM AATA HAI?

Metadata cleanup and categorization.

ARCHITECTURE

Raw list -> set -> normalized tags

FILES / BUILD TARGET

tags.py Target: Preserve deterministic output by sorting

CORE IMPLEMENTATION - TYPE + RUN

```
tags = ["ai", "python", "ai"]
unique = sorted(set(tags))
print(unique)
```

SAMPLE OUTPUT `['ai', 'python']`

INTERVIEW EXPLANATION

Project answer: Set is for uniqueness/membership; sorting restores predictable presentation order. Debug cue: Set itself is unordered; do not rely on display order. Follow-up: What would you improve first if this had 10x traffic/data?

BUILD STEPS

1. Create the file(s) shown above.
2. Run the exact sample once.
3. Change one input and observe the output.
4. Add one validation or failure test.

EDGE CASES / DEBUG

Test invalid input, boundary values and empty/missing data where relevant. Keep validation at the system boundary.

Sets

Use this pack after the section labs: explain -> build -> debug -> defend.

SECTION ANCHOR

Definition: Unique membership and set operations ke liye fast, clean abstraction.

Easy explanation: Set duplicates remove karta hai; union/intersection/difference powerful hain.

Example: `tags = {"ai","python","ai"}; print(tags)`

SIMPLE

Unique Tags

Kya karta hai: Duplicate tags ko remove karta hai.

Kis kaam aata hai: Metadata cleanup and categorization.

Architecture: Raw list -> set -> normalized tags

Files: `tags.py`

Build target: Preserve deterministic output by sorting

MEDIUM

Skill Gap Finder

Kya karta hai: Required aur existing skills ka difference nikalta hai.

Kis kaam aata hai: Learning roadmaps and resume gap analysis.

Architecture: Required set + current set -> difference -> report

Files: `gap.py`

Build target: Case normalization

HARD

Record Dedup Pipeline

Kya karta hai: IDs ke basis par duplicates remove karke clean batch banata hai.

Kis kaam aata hai: ETL ingestion and webhook duplicate handling.

Architecture: Records -> seen IDs set -> clean output -> audit counts

Files: `dedupe.py`

Build target: Keep first valid record

Project rule: code khud type karo, output predict karo, edge case test karo, phir 60-second explanation do.

Sets - implementation lab

Three core implementations + project-tailored interview explanation + debugging cue.

SIMPLE | UNIQUE TAGS

```
tags = ["ai","python","ai"]
unique = sorted(set(tags))
print(unique)
```

INTERVIEW DEFENSE

Project answer: Set is for uniqueness/membership; sorting restores predictable presentation order.

Debug cue: Set itself is unordered; do not rely on display order.

Follow-up: What would you improve first if this had 10x traffic/data?

MEDIUM | SKILL GAP FINDER

```
required={"python","sql","numpy","pandas"}
current={"python","sql"}
print(sorted(required-current))
```

INTERVIEW DEFENSE

Project answer: Set difference directly models the question “what is missing?”.

Debug cue: Normalize case before comparison.

Follow-up: What would you improve first if this had 10x traffic/data?

HARD | RECORD DEDUP PIPELINE

```
seen=set(); clean=[]
for row in rows:
    rid=row["id"]
    if rid in seen: continue
    seen.add(rid); clean.append(row)
print(len(clean))
```

INTERVIEW DEFENSE

Project answer: Seen-set gives average O(1) membership and prevents duplicate processing.

Debug cue: Decide whether you want first-write-wins or latest-write-wins.

Follow-up: What would you improve first if this had 10x traffic/data?

SELF-CHECK

Can I explain the project without notes? Can I name one failure? Can I state time/space or cost trade-offs?

SECTION 12

Dictionaries

Level: Intermediate | Time: ~60 min | 13 topics

Dictionary data ko **key: value** pairs me store karti hai. Real world ke records (student, product, user profile) ko represent karne ka best tarika. Key se lookup bahut fast hota hai.

REAL-LIFE EXAMPLE SE SAMAJHO

Dictionary ek **phone contact book** hai: naam (key) se number (value) turant mil jata hai, poori list scan nahi karni padti.

IS SECTION ME KYA SEEKHOGE

- ✓ Dictionary banana, access, add, update, delete
- ✓ keys(), values(), items() aur looping
- ✓ get(), setdefault(), update()
- ✓ Counting aur nested dictionaries

TOPIC ROADMAP

12.1 Creating Dictionaries
12.2 Accessing Values
12.3 Adding / Updating
12.4 del / pop()
12.5 keys(), values(), items()
12.6 Looping through Dictionary
12.7 Dictionary Comprehension
12.8 Checking Keys with in ★
12.9 update() & Merging ★
12.10 setdefault() ★
12.11 Counting Words (Frequency) ★
12.12 Nested Dictionaries ★
12.13 Sort Dictionary by Value ★

★ = Advanced topic. Pehli baar padhte waqt inhe skip karke dusre round me karo.

12.1 Creating Dictionaries

CORE

Dictionary **key-value pairs** store karti hai, **{ }** me. Keys unique hoti hain aur immutable honi chahiye (str, int, tuple).

main.py

```
student = {"name": "Ashu", "age": 22}
print(student)
print(type(student))
```

▶ OUTPUT

```
{'name': 'Ashu', 'age': 22}
<class 'dict'>
```

12.2 Accessing Values

CORE

Key se value nikalo: **d[key]** ya **d.get(key)**. Key na ho to [ ] error deta hai, get() None deta hai.

main.py

```
student = {"name": "Ashu", "age": 22}
print(student["name"])
print(student.get("age"))
print(student.get("city"))            # None
print(student.get("city", "Unknown"))  # default value
```

▶ OUTPUT

```
Ashu
22
None
Unknown
```

GALTI SE BACHO

student["city"] jaisa missing key **KeyError** deta hai. Safe access ke liye **.get()** use karo.

12.3 Adding / Updating

CORE

Nayi key add karni ho ya purani ki value badalni ho, dono ka syntax same hai: **d[key] = value**.

main.py

```
student = {"name": "Ashu"}
student["age"] = 22            # nayi key
student["name"] = "Ahetesham"  # purani key update
print(student)
```

▶ OUTPUT

```
{'name': 'Ahetesham', 'age': 22}
```

12.4 del / pop()

CORE

Ek key-value pair hatane ke liye. **pop()** value wapas bhi deta hai.

main.py

```
student = {"name": "Ashu", "age": 22, "city": "Delhi"}
removed = student.pop("age")
print(removed)
del student["city"]
print(student)
```

▶ OUTPUT

```
22
{'name': 'Ashu'}
```

12.5 keys(), values(), items() CORE

Sari keys, sari values, ya (key, value) pairs nikalne ke liye.

main.py

```
student = {"name": "Ashu", "age": 22}
print(student.keys())
print(student.values())
print(student.items())
```

▶ OUTPUT

```
dict_keys(['name', 'age'])
dict_values(['Ashu', 22])
dict_items([('name', 'Ashu'), ('age', 22)])
```

12.6 Looping through Dictionary CORE

items() se key aur value dono ek saath milte hain.

main.py

```
student = {"name": "Ashu", "age": 22}
for key, value in student.items():
    print(key, ":", value)

for key in student:          # sirf keys
    print(key)
```

▶ OUTPUT

```
name : Ashu
age : 22
name
age
```

12.7 Dictionary Comprehension CORE

Ek line me loop chala kar naya dictionary banana: **{k: v for ...}**.

main.py

```
squares = {x: x * x for x in range(1, 4)}
print(squares)

words = ["hi", "python"]
print({w: len(w) for w in words})
```

▶ OUTPUT

```
{1: 1, 2: 4, 3: 9}
{'hi': 2, 'python': 6}
```

12.8 Checking Keys with in ★ ADVANCED

in dictionary me sirf **keys** check karta hai, values nahi.

main.py

```
d = {"a": 1, "b": 2}
print("a" in d)
print(1 in d)            # 1 value hai, key nahi
print(1 in d.values())
```

▶ OUTPUT

```
True
False
True
```

12.9 update() & Merging ★ ADVANCED

Ek dictionary me dusri ke items jodna. Python 3.9+ me | operator bhi hai.

main.py

```
a = {"x": 1, "y": 2}
b = {"y": 20, "z": 30}
a.update(b)            # y overwrite ho gaya
print(a)

merged = {"p": 1} | {"q": 2}
print(merged)
```

▶ OUTPUT

```
{'x': 1, 'y': 20, 'z': 30}
{'p': 1, 'q': 2}
```

12.10 setdefault() ★ ADVANCED

Key ho to uski value deta hai, na ho to default value ke saath key bana deta hai. Grouping me useful.

main.py

```
groups = {}
for word in ["apple", "avocado", "banana", "blueberry", "cherry"]:
    groups.setdefault(word[0], []).append(word)
print(groups)
```

▶ OUTPUT

```
{'a': ['apple', 'avocado'], 'b': ['banana', 'blueberry'], 'c': ['cherry']}
```

12.1 1 Counting Words (Frequency)

★ ADVANCED

Dictionary ka sabse popular use: kaun sa item kitni baar aaya.

STEP BY STEP: KAISE CHALTA HAI?

1. Har word ke liye current count nikalo: get(word, 0).
2. Usme +1 karo aur wapas store karo.
3. Pehli baar aane par get() default 0 deta hai, isliye KeyError nahi aata.

main.py

```
text = "to be or not to be"
count = {}
for word in text.split():
    count[word] = count.get(word, 0) + 1
print(count)
```

▶ OUTPUT

```
{'to': 2, 'be': 2, 'or': 1, 'not': 1}
```

12.1 2 Nested Dictionaries

★ ADVANCED

Dictionary ke andar dictionary. JSON data aur real records aise hi hote hain.

main.py

```
users = {
    "ashu": {"age": 22, "skills": ["Python", "HTML"]},
    "riya": {"age": 24, "skills": ["SQL"]},
}
print(users["ashu"]["skills"][0])
for name, info in users.items():
    print(name, info["age"])
```

▶ OUTPUT

```
Python
ashu 22
riya 24
```

12.1 3 Sort Dictionary by Value

★ ADVANCED

sorted() ke saath key aur items() milake dictionary ko value ke basis par sort karte hain.

main.py

```
marks = {"Ravi": 82, "Sita": 95, "Amit": 70}
ranked = sorted(marks.items(), key=lambda kv: kv[1], reverse=True)
print(ranked)
print(dict(ranked))
```

▶ OUTPUT

```
[('Sita', 95), ('Ravi', 82), ('Amit', 70)]
{'Sita': 95, 'Ravi': 82, 'Amit': 70}
```

QUICK RECAP: SECTION 12

- ✓ Dictionary = key: value, key unique aur immutable
- ✓ d[key] error de sakta hai, d.get(key, default) safe hai
- ✓ items() se key aur value dono ke saath loop
- ✓ get(), setdefault(), update() kaam ke helpers hain

PRACTICE TIME Khud try karo, phir Appendix C me solution dekho

EASY	**Q1.** Ek dictionary banao jisme tumhara name, age, city ho, aur city print karo.
MEDIUM	**Q2.** String "banana" me har letter ki frequency dictionary me nikalo.
HARD	**Q3.** marks = {"A": 55, "B": 90, "C": 72} me se sabse zyada marks wale student ka naam nikalo.

INTERVIEW LAB / Dictionaries

Key-value records, safe lookup, counting aur nested data. • Answer formula: definition → tiny example → practical use. Word-by-word memorize mat karo; apni language me explain karo.

Q01

Dictionary kya hoti hai?

BEST ANSWER

Key-value pairs ka mapping. Real-world records jaise user, product, student data represent karne me useful.

TIP: 20-30 sec me clearly bolo.

Q02

d[key] vs d.get(key)?

BEST ANSWER

d[key] missing key par KeyError de sakta hai. get() missing key par default/None de sakta hai.

TIP: 20-30 sec me clearly bolo.

Q03

items() ka use?

BEST ANSWER

Key aur value dono ke saath loop karne ke liye, jaise for k,v in d.items().

TIP: 20-30 sec me clearly bolo.

Q04

in dictionary me kya check karta hai?

BEST ANSWER

By default keys check karta hai, values nahi.

TIP: 20-30 sec me clearly bolo.

Q05

Frequency counting ka pattern?

BEST ANSWER

freq[x] = freq.get(x, 0) + 1 - pehle current count lo, phir 1 add karo.

TIP: 20-30 sec me clearly bolo.

Q06

Dictionary comprehension kya hai?

BEST ANSWER

Expression + loop se new dict banana, jaise {x:x*x for x in nums}.

TIP: 20-30 sec me clearly bolo.

Mini speaking drill: Har answer bina PDF dekhe 2 baar bolo. Dusri baar ek real example add karo.

CODING LAB / Dictionaries

Rule: Pehle problem ko 10 sec me apne words me bolo, phir code likho. Copy-paste ke bajaye khud type karo.

EASY

Name, age, city dict banao aur city print karo.

WRITE CODE

EASY

marks dict se highest scorer ka naam nikalo.

WRITE CODE

MEDIUM

"banana" me letter frequency dictionary banao.

WRITE CODE

MEDIUM

Student records ki nested dictionary se ek value access karo.

WRITE CODE

HARD

Scores ko value ke descending order me sorted result me print karo.

WRITE CODE

DEBUG

Missing key ko safe access karne ka best basic pattern likho.

WRITE CODE

STARTER KIT - TRY WITHOUT LOOKING UP

```
d[key] get() items() in update() comprehension frequency
```

3-STEP DEBUG LOOP

- 1. Error/bug ko reproduce karo
- 2. Values/types/conditions print karke isolate karo
- 3. Ek change karo, rerun karo, phir reason explain karo

Interview coding line: "Main pehle input/edge case define karunga, phir simple correct solution likhunga, phir complexity aur edge cases check karunga."

LOGIC LAB / Dictionaries

Goal: code run karne se pehle output predict karo, phir dry-run karke reason bolo. Ye interview confidence aur debugging dono improve karta hai.

Predict

{"a":1,"b":2}["a"] -> ?

CHECK

1

Trace

d={}; d["x"]=d.get("x",0)+1 -> ?

CHECK

{"x":1}

Think

"name" in {"name":"A"} True ya False?

CHECK

True - key check

Predict

max({"A":55,"B":90}, key={"A":55,"B":90}.get) -> ?

CHECK

B

Fix

d["missing"] error se bachne ka simple way?

CHECK

d.get("missing", default)

Explain

Dictionary ko list ke instead kab choose karoge?

CHECK

Jab key-based lookup/records chahiye

5 COMMON MISTAKES TO WATCH

1. get vs []
2. in checks keys
3. frequency initialization
4. nested keys confusion
5. sorting key direction

Mastery check: "Main is topic ko bina dekhe explain kar sakta hoon + 1 small program likh sakta hoon + 2 outputs predict kar sakta hoon."

SOLVED PRACTICE - Dictionaries

Every coding-lab question now has its answer, expected result and a quick check - all in one place.

EASY

Name, age, city dict banao aur city print karo.

```
d={"name":"Ahetesham","age":23,"city":"Pune"}
print(d["city"])
```

EXPECTED RESULT

Pune

EASY

Marks dict se highest scorer ka naam.

```
m={"A":82,"B":95,"C":88}
print(max(m,key=m.get))
```

EXPECTED RESULT

B

MEDIUM

"banana" ki letter frequency dictionary.

```
d={}
for ch in "banana": d[ch]=d.get(ch,0)+1
print(d)
```

EXPECTED RESULT

{'b':1,'a':3,'n':2}

MEDIUM

Nested student dictionary se value access.

```
s={"A":{"marks":{"math":90}}}
print(s["A"]["marks"]["math"])
```

EXPECTED RESULT

90

HARD

Scores ko value descending sorted result me print.

```
scores={"A":82,"B":95,"C":88}
print(sorted(scores.items(),key=lambda x:x[1],reverse=True))
```

EXPECTED RESULT

[('B',95),('C',88),('A',82)]

DEBUG

Missing key safe access pattern.

```
d={"name":"A"}
print(d.get("city","unknown"))
```

EXPECTED RESULT

unknown

PRACTICE HABIT

Practice rule: solve first, compare second, then change one input. Mark the question only after you can explain why the output changes.

MINI PROJECT - Contact Book

Dictionaries ka concept ek runnable, interview-ready mini build me convert karo.

YE KYA KARTA HAI?

Name ko phone/email record se map karta hai.

KIS KAAM AATA HAI?

CLI contact managers and lookup utilities.

ARCHITECTURE

Key -> record -> lookup/update

FILES / BUILD TARGET

contacts.py Target: Safe get() lookup

CORE IMPLEMENTATION - TYPE + RUN

```
contacts = {"ali": {"email": "a@x.com"}}
print(contacts.get("ali", {}).get("email"))
```

SAMPLE OUTPUT `a@x.com`

INTERVIEW EXPLANATION

Project answer: Nested dicts model JSON-like records but typed models are safer for large systems. Debug cue: d[key] can raise KeyError; get() is safer for optional fields. Follow-up: What would you improve first if this had 10x traffic/data?

BUILD STEPS

1. Create the file(s) shown above.
2. Run the exact sample once.
3. Change one input and observe the output.
4. Add one validation or failure test.

EDGE CASES / DEBUG

Test invalid input, boundary values and empty/missing data where relevant. Keep validation at the system boundary.

Dictionaries

Use this pack after the section labels: explain -> build -> debug -> defend.

SECTION ANCHOR

Definition: Key-value structure: JSON payloads, configs, counters and records ka core tool.

Easy explanation: Dict gives named access; keys should be stable and hashable.

Example: `user = {"name":"Ahetesham","role":"AI Engineer"}; print(user["role"])`

SIMPLE

Contact Book

Kya karta hai: Name ko phone/email record se map karta hai.

Kis kaam aata hai: CLI contact managers and lookup utilities.

Architecture: Key -> record -> lookup/update

Files: `contacts.py`

Build target: Safe get() lookup

MEDIUM

Word Frequency

Kya karta hai: Text ke words ka frequency counter banata hai.

Kis kaam aata hai: NLP preprocessing and analytics.

Architecture: Text -> tokens -> dict counter -> top words

Files: `freq.py`

Build target: Case/ punctuation normalization

HARD

Webhook Payload Validator

Kya karta hai: Nested API/webhook payload me required fields and types validate karta hai.

Kis kaam aata hai: n8n/webhook/API integration boundaries.

Architecture: JSON -> schema checks -> normalized dict -> workflow

Files: `validate.py`

Build target: Return structured validation errors

Project rule: code khud type karo, output predict karo, edge case test karo, phir 60-second explanation do.

Dictionaries - implementation lab

Three core implementations + project-tailored interview explanation + debugging cue.

SIMPLE | CONTACT BOOK

```
contacts={"ali":{"email":"a@x.com"}}
print(contacts.get("ali",{}).get("email"))
```

INTERVIEW DEFENSE

Project answer: Nested dicts model JSON-like records but typed models are safer for large systems.

Debug cue: d[key] can raise KeyError; get() is safer for optional fields.

Follow-up: What would you improve first if this had 10x traffic/data?

MEDIUM | WORD FREQUENCY

```
counts={}
for w in "ai is ai practical".split():
    counts[w]=counts.get(w,0)+1
print(counts)
```

INTERVIEW DEFENSE

Project answer: get(key,0) is a common frequency-map pattern.

Debug cue: Normalize punctuation before counting if required.

Follow-up: What would you improve first if this had 10x traffic/data?

HARD | WEBHOOK PAYLOAD VALIDATOR

```
def validate(p):
    errs=[]
    if not isinstance(p.get("event"), str): errs.append("even
t")
    if not isinstance(p.get("data"), dict): errs.append("data
")
    return errs
print(validate(payload))
```

INTERVIEW DEFENSE

Project answer: Validate once at system boundaries so downstream steps can assume a contract.

Debug cue: Log field names, not secrets or raw credentials.

Follow-up: What would you improve first if this had 10x traffic/data?

SELF-CHECK

Can I explain the project without notes? Can I name one failure? Can I state time/space or cost trade-offs?

SECTION 13

OOP
(Object Oriented Programming)

Level: Advanced | Time: ~120 min | 19 topics

OOP me data (attributes) aur kaam (methods) ko ek saath ek **class** me bandh dete hain. Bade projects ko organized, reusable aur maintain karne layak banane ka ye best tarika hai. Python ke saare built-in types (str, list, dict) bhi classes hi hain.

REAL-LIFE EXAMPLE SE SAMAJHO

Class ek **ghar ka naksha (blueprint)** hai, object uska **asli ghar**. Ek naksha se kai ghar ban sakte hain, har ghar ka apna rang, apna saman.

IS SECTION ME KYA SEEKHOGE

- ✓ Class, object, __init__, self
- ✓ Instance/class/static methods
- ✓ Inheritance, overriding, polymorphism, super()
- ✓ Encapsulation, property, dunder methods, dataclass

TOPIC ROADMAP

13.1 class & object
13.2 __init__()
13.3 instance method
13.4 class variable
13.5 class method
13.6 static method
13.7 inheritance
13.8 method overriding
13.9 polymorphism
13.10 encapsulation
13.11 __str__()
13.12 property decorator
13.13 super() ★
13.14 __repr__ vs __str__ ★
13.15 isinstance() & issubclass() ★
13.16 property setter (Validation) ★
13.17 Magic (Dunder) Methods ★
13.18 Composition (has-a) ★
13.19 @dataclass ★

★ = Advanced topic. Pehli baar padhte waqt inhe skip karke dusre round me karo.

13.1 class & object

CORE

Class ek blueprint hai. **Object** uska real instance hai. Naam ka pehla letter capital rakhte hain (CamelCase).

main.py

```
class Car:
    pass

my_car = Car()
other = Car()
print(type(my_car))
print(my_car is other)    # alag objects
```

▶ OUTPUT

```
<class '__main__.Car'>
False
```

13.2 __init__()

CORE

Constructor method. Object banate hi apne aap call hota hai. **self** us object ko refer karta hai jo abhi ban raha hai.

STEP BY STEP: KAISE CHALTA HAI?

1. Car("Tesla") likhte hi Python ek khaali object banata hai.
2. __init__ automatically call hota hai, self = naya object, brand = "Tesla".
3. self.brand = brand se value object ke andar save hoti hai.

main.py

```
class Car:
    def __init__(self, brand):
        self.brand = brand

c = Car("Tesla")
print(c.brand)
```

▶ OUTPUT

```
Tesla
```

GALTI SE BACHO

Method ke pehle parameter me **self** likhna mat bhoolo, warna TypeError aayega.

13.3 instance method

CORE

Class ke andar normal function jo **object ke apne data** par kaam karta hai. Pehla parameter hamesha self.

main.py

```
class Car:
    def __init__(self, brand):
        self.brand = brand

    def show(self):
        print(f"Brand: {self.brand}")

c = Car("Tesla")
c.show()
Car("BMW").show()
```

▶ OUTPUT

```
Brand: Tesla
Brand: BMW
```

13.4 class variable

CORE

Ye variable **sabhi objects ke beech shared** hota hai. Instance variable (self.x) har object ka apna hota hai.

main.py

```
class Car:
    wheels = 4              # class variable

    def __init__(self, brand):
        self.brand = brand  # instance variable

c1 = Car("Tesla")
c2 = Car("BMW")
print(c1.wheels, c2.wheels)
Car.wheels = 6              # sabke liye badla
print(c1.wheels, c2.wheels)
```

▶ OUTPUT

```
4 4
6 6
```

13.5 class method

CORE

@classmethod se decorate hota hai. Pehla parameter **cls** (class), object nahi. Alternative constructors me kaam aata hai.

main.py

```
class Car:
    wheels = 4

    @classmethod
    def info(cls):
        print(f"Wheels: {cls.wheels}")

Car.info()
```

▶ OUTPUT

```
Wheels: 4
```

13.6 static method

CORE

@staticmethod ko na self chahiye na cls. Ye class ke andar rakha ek normal utility function hai.

main.py

```
class MathUtil:
    @staticmethod
    def add(a, b):
        return a + b

print(MathUtil.add(3, 4))
```

▶ OUTPUT

```
7
```

YAAD RAKHO

Instance method = object data (self). Class method = class data (cls). Static method = kisi data ki zarurat nahi.

13.7 inheritance

CORE

Child class parent ki properties aur methods **reuse** kar leti hai. Syntax: **class Child(Parent)**.

main.py

```
class Animal:
    def speak(self):
        print("Animal speaks")

class Dog(Animal):
    pass

d = Dog()
d.speak()
```

▶ OUTPUT

```
Animal speaks
```

13.8 method overriding CORE

Child class parent ke method ko **apne tarike se dobara likh** deti hai.

main.py

```
class Animal:
    def speak(self):
        print("Animal speaks")

class Dog(Animal):
    def speak(self):
        print("Dog barks")

Dog().speak()
Animal().speak()
```

▶ OUTPUT

```
Dog barks
Animal speaks
```

13.9 polymorphism CORE

Same method name, alag-alag classes me alag behaviour. Caller ko type ki fikar nahi karni padti.

main.py

```
class Cat:
    def sound(self):
        print("Meow")

class Dog:
    def sound(self):
        print("Woof")

for animal in [Cat(), Dog()]:
    animal.sound()
```

▶ OUTPUT

```
Meow
Woof
```

13.10 encapsulation

CORE

Data ko **private** (**__var**) rakh kar directly access hone se bachana. Sirf methods ke through access do.

main.py

```
class Account:
    def __init__(self, balance):
        self.__balance = balance

    def get_balance(self):
        return self.__balance

a = Account(1000)
print(a.get_balance())
try:
    print(a.__balance)
except AttributeError as e:
    print("Error:", e)
```

▶ OUTPUT

```
1000
Error: 'Account' object has no attribute '__balance'
```

TIP

Python me true private nahi hota, __ naam badal deta hai (name mangling). Ye ek warning hai: "bahar se mat chhuo".

13.11 __str__()

CORE

print(object) karne par kya text dikhega, wo define karta hai. Nahi likha to ajeeb address dikhta hai.

main.py

```
class Car:
    def __init__(self, brand):
        self.brand = brand

    def __str__(self):
        return f"Car: {self.brand}"

print(Car("Tesla"))
```

▶ OUTPUT

```
Car: Tesla
```

13.12 property decorator

CORE

@property se method ko variable jaisa access karne dete hain (bina bracket ke).

main.py

```
class Circle:
    def __init__(self, radius):
        self.radius = radius

    @property
    def area(self):
        return 3.14 * self.radius ** 2

c = Circle(2)
print(c.area)
c.radius = 3
print(c.area)      # apne aap update
```

▶ OUTPUT

```
12.56
28.26
```

13.13 super()

★ ADVANCED

Child class me parent ka method/constructor call karne ke liye. Parent ka code dobara likhna nahi padta.

main.py

```
class Animal:
    def __init__(self, name):
        self.name = name

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)     # parent ka __init__
        self.breed = breed

d = Dog("Bruno", "Labrador")
print(d.name, d.breed)
```

▶ OUTPUT

```
Bruno Labrador
```

13.14 __repr__ vs __str__

★ ADVANCED

__str__ users ke liye readable text hai. **__repr__** developers ke liye exact representation (debugging, lists me dikhta hai).

main.py

```
class Car:
    def __init__(self, brand):
        self.brand = brand
    def __str__(self):
        return f"Car: {self.brand}"
    def __repr__(self):
        return f"Car(brand={self.brand!r})"

c = Car("Tesla")
print(c)
print([c])       # list me repr dikhta hai
```

▶ OUTPUT

```
Car: Tesla
[Car(brand='Tesla')]
```

13.15 isinstance() & issubclass()

★ ADVANCED

Object kis class ka hai (ya uske parent ka) check karne ke liye.

main.py

```
class Animal: pass
class Dog(Animal): pass

d = Dog()
print(isinstance(d, Dog))
print(isinstance(d, Animal))     # parent ka bhi True
print(issubclass(Dog, Animal))
```

▶ OUTPUT

```
True
True
True
```

13.16 property setter (Validation) ★ ADVANCED

@x.setter se value set karte waqt check laga sakte ho. Galat data andar ghusne se rukta hai.

main.py

```
class Person:
    def __init__(self, age):
        self.age = age            # setter call hoga

    @property
    def age(self):
        return self._age

    @age.setter
    def age(self, value):
        if value < 0:
            raise ValueError("Age negative nahi ho sakti")
        self._age = value

p = Person(20)
try:
    p.age = -5
except ValueError as e:
    print("Error:", e)
print(p.age)
```

▶ OUTPUT

```
Error: Age negative nahi ho sakti
20
```

13.17 Magic (Dunder) Methods ★ ADVANCED

Double-underscore methods Python ke built-in behaviour ko customize karte hain: **__len__** (len()), **__eq__** (==), **__add__** (+).

main.py

```
class Cart:
    def __init__(self, items):
        self.items = items
    def __len__(self):
        return len(self.items)
    def __add__(self, other):
        return Cart(self.items + other.items)

a = Cart(["pen", "book"])
b = Cart(["bag"])
print(len(a))
print(len(a + b))
```

▶ OUTPUT

```
2
3
```

13.18 Composition (has-a)

★ ADVANCED

Har jagah inheritance nahi. Jab "is-a" nahi balki "has-a" ho (Car **has an** Engine), tab ek object ko dusre ke andar rakho.

main.py

```
class Engine:
    def start(self):
        return "Engine started"

class Car:
    def __init__(self):
        self.engine = Engine()      # has-a
    def start(self):
        return self.engine.start()

print(Car().start())
```

▶ OUTPUT

```
Engine started
```

13.19 @dataclass

★ ADVANCED

Sirf data rakhne wali classes ke liye shortcut. __init__, __repr__, __eq__ apne aap ban jate hain.

main.py

```
from dataclasses import dataclass

@dataclass
class Student:
    name: str
    marks: int = 0

s = Student("Ashu", 95)
print(s)
print(s == Student("Ashu", 95))
```

▶ OUTPUT

```
Student(name='Ashu', marks=95)
True
```

QUICK RECAP: SECTION 13

- ✓ Class = blueprint, object = instance, self = current object
- ✓ __init__ constructor hai; instance/class/static methods ka fark
- ✓ Inheritance + super() reuse ke liye, polymorphism flexible code ke liye
- ✓ Encapsulation (__var, @property), __str__/__repr__ aur @dataclass professional code ke liye

PRACTICE TIME Khud try karo, phir Appendix C me solution dekho

EASY	**Q1.** Ek Person class banao jisme name aur age ho, aur ek method introduce() ho jo "Hi, I am Ashu" print kare.
MEDIUM	**Q2.** BankAccount class banao: deposit(), withdraw() aur balance. Balance se zyada withdraw hone par "Insufficient funds" print ho.
HARD	**Q3.** Shape parent class (area() method) banao. Rectangle aur Circle child classes me area() override karo aur ek loop me dono ka area print karo.

INTERVIEW LAB / OOP (Object Oriented Programming)

Classes, objects, reuse, behavior, validation aur professional object design. • Answer formula: definition → tiny example → practical use. Word-by-word memorize mat karo; apni language me explain karo.

Q01

Class aur object kya hain?

BEST ANSWER

Class blueprint hai; object us blueprint ka real instance. Ek class se multiple objects ban sakte hain.

TIP: 20-30 sec me clearly bolo.

Q02

self kya hota hai?

BEST ANSWER

Instance method me current object ka reference. self.brand jaise attributes us object ke data ko access karte hain.

TIP: 20-30 sec me clearly bolo.

Q03

Instance/class/static method difference?

BEST ANSWER

Instance method self use karta hai, classmethod cls se class data handle karta hai, staticmethod ko self/cls ki zarurat nahi.

TIP: 20-30 sec me clearly bolo.

Q04

Inheritance aur overriding?

BEST ANSWER

Inheritance me child parent ka code reuse karta hai. Overriding me child same method ko apne behavior se redefine karta hai.

TIP: 20-30 sec me clearly bolo.

Q05

Polymorphism kya hai?

BEST ANSWER

Same interface/method call ko different classes apne behavior ke saath implement karti hain.

TIP: 20-30 sec me clearly bolo.

Q06

Encapsulation/property ka use?

BEST ANSWER

Data ko controlled access dena. @property getter ko attribute-style access deta hai; setter validation add kar sakta hai.

TIP: 20-30 sec me clearly bolo.

Mini speaking drill: Har answer bina PDF dekhe 2 baar bolo. Dusri baar ek real example add karo.

CODING LAB / OOP (Object Oriented Programming)

Rule: Pehle problem ko 10 sec me apne words me bolo, phir code likho. Copy-paste ke bajaye khud type karo.

EASY

Person class with name, age aur introduce() banao.

WRITE CODE

EASY

BankAccount me deposit/withdraw methods banao.

WRITE CODE

MEDIUM

Animal parent aur Dog/ Cat child classes se method overriding karo.

WRITE CODE

MEDIUM

Shape base class, Rectangle/Circle area() implement karo.

WRITE CODE

HARD

Private balance + @property + setter validation wala Account banao.

WRITE CODE

DEBUG

super().__init__() kab use karoge?

WRITE CODE

STARTER KIT - TRY WITHOUT LOOKING UP

```
class __init__ self super() @property inheritance override
```

3-STEP DEBUG LOOP

- 1. Error/bug ko reproduce karo
- 2. Values/types/conditions print karke isolate karo
- 3. Ek change karo, rerun karo, phir reason explain karo

Interview coding line: "Main pehle input/edge case define karunga, phir simple correct solution likhunga, phir complexity aur edge cases check karunga."

LOGIC LAB / OOP (Object Oriented Programming)

Goal: code run karne se pehle output predict karo, phir dry-run karke reason bolo. Ye interview confidence aur debugging dono improve karta hai.

Predict

Dog(Animal) aur Dog inherits Animal -> isinstance(dog, Animal)?

CHECK

True

Trace

Car().start() where Car has Engine -> kya concept?

CHECK

Composition / has-a

Think

__str__ vs __repr__?

CHECK

str user-readable; repr debug/developer representation

Predict

len(custom_object) tabhi chalega jab?

CHECK

__len__ defined ho

Fix

Setter me negative age allow kar di to bug kya?

CHECK

Invalid state object me aa sakta hai

Explain

OOP ka benefit simple words me?

CHECK

Data + related behavior ko organized reusable unit me rakhna

5 COMMON MISTAKES TO WATCH

1. self bhoolna
2. class vs instance data
3. inheritance overuse
4. private ko truly inaccessible samajhna
5. __str__/__repr__ confuse karna

Mastery check: "Main is topic ko bina dekhe explain kar sakta hoon + 1 small program likh sakta hoon + 2 outputs predict kar sakta hoon."

SOLVED PRACTICE - OOP (Object Oriented Programm

Every coding-lab question now has its answer, expected result and a quick check - all in one place.

EASY

Person class with name, age aur introduce().

```
class Person:
    def __init__(self,name,age): self.name,self.age=name,age
    def introduce(self): return f"I am {self.name}"
p=Person("Ahetesham",23); print(p.introduce())
```

EXPECTED RESULT

I am Ahetesham

EASY

BankAccount deposit/withdraw methods.

```
class BankAccount:
    def __init__(self,b=0): self.balance=b
    def deposit(self,x): self.balance+=x
a=BankAccount(100); a.deposit(50); print(a.balance)
```

EXPECTED RESULT

150

MEDIUM

Animal parent + Dog/Cat overriding.

```
class Animal:
    def sound(self): return "..."
class Dog(Animal):
    def sound(self): return "woof"
print(Dog().sound())
```

EXPECTED RESULT

woof

MEDIUM

Shape base class + Rectangle/Circle area().

```
class Shape: pass
class Rectangle(Shape):
    def __init__(self,w,h): self.w,self.h=w,h
    def area(self): return self.w*self.h
print(Rectangle(4,5).area())
```

EXPECTED RESULT

20

HARD

Private balance + property setter validation.

```
class Account:
    def __init__(self,b): self._balance=b
    @property
    def balance(self): return self._balance
    @balance.setter
    def balance(self,v):
        if v<0: raise ValueError("negative")
        self._balance=v
print(Account(100).balance)
```

EXPECTED RESULT

100

DEBUG

super().__init__() kab use karoge?

```
class Child(Parent):
    def __init__(self,x,y):
        super().__init__(x)
        self.y=y
```

EXPECTED RESULT

Parent initializer ko child se reuse karna ho.

PRACTICE HABIT

Practice rule: solve first, compare second, then change one input. Mark the question only after you can explain why the output changes.

MINI PROJECT - Bank Account

OOP (Object Oriented Programming) ka concept ek runnable, interview-ready mini build me convert karo.

YE KYA KARTA HAI?

Balance, deposit aur withdraw ko class me package karta hai.

KIS KAAM AATA HAI?

OOP practice and transactional logic.

ARCHITECTURE

Account object -> methods -> state updates

FILES / BUILD TARGET

bank.py Target: Reject invalid withdrawals

CORE IMPLEMENTATION - TYPE + RUN

```
class BankAccount:
    def __init__(self, balance=0):
        self.balance = balance
    def deposit(self, amount):
        self.balance += amount
    def withdraw(self, amount):
        if amount > self.balance:
            raise ValueError("insufficient")
        self.balance -= amount

a = BankAccount(100)
a.deposit(50)
a.withdraw(20)
print(a.balance)
```

SAMPLE OUTPUT `BankAccount balance changes safely through methods.`

INTERVIEW EXPLANATION

Project answer: Encapsulation state changes ko methods ke through control karta hai. Debug cue: Keep state transitions explicit and test both valid/invalid paths. Follow-up: What would you improve first if this had 10x traffic/data?

BUILD STEPS

1. Create the file(s) shown above.
2. Run the exact sample once.
3. Change one input and observe the output.
4. Add one validation or failure test.

EDGE CASES / DEBUG

Test invalid input, boundary values and empty/missing data where relevant. Keep validation at the system boundary.

OOP (Object Oriented Programming)

Use this pack after the section labs: explain -> build -> debug -> defend.

SECTION ANCHOR

Definition: Data + behavior ko classes me organize karke larger codebases ko modular banata hai.

Easy explanation: Class blueprint hai; object instance; encapsulation and composition improve boundaries.

Example: `class User: def __init__(self, name): self.name = name`

SIMPLE

Bank Account

Kya karta hai: Balance, deposit aur withdraw ko class me package karta hai.

Kis kaam aata hai: OOP practice and transactional logic.

Architecture: Account object -> methods -> state updates

Files: `bank.py`

Build target: Reject invalid withdrawals

MEDIUM

Chat Session

Kya karta hai: Conversation history, user ID aur session operations ko class me manage karta hai.

Kis kaam aata hai: Chatbot backends and session-aware applications.

Architecture: HTTP request -> ChatSession -> history -> persistence

Files: `session.py`

Build target: Separate in-memory state from DB adapter

HARD

Model Service Layer

Kya karta hai: Model provider calls ko interface ke peeche isolate karta hai.

Kis kaam aata hai: LLM/ML provider swapping, testing and production APIs.

Architecture: Flask/FastAPI -> service class -> provider adapter -> model

Files: `service.py | provider.py`

Build target: Provider dependency replaceable ho

Project rule: code khud type karo, output predict karo, edge case test karo, phir 60-second explanation do.

OOP (Object Oriented Programming) - implementatio

Three core implementations + project-tailored interview explanation + debugging cue.

SIMPLE | BANK ACCOUNT

```
class BankAccount:
    def __init__(self,balance=0): self.balance=balance
    def deposit(self,x): self.balance += x
    def withdraw(self,x):
        if x > self.balance: raise ValueError("insufficient")
        self.balance -= x
```

INTERVIEW DEFENSE

Project answer: Encapsulation state changes ko methods ke through control karta hai.

Debug cue: Keep state transitions explicit and test both valid/invalid paths.

Follow-up: What would you improve first if this had 10x traffic/data?

MEDIUM | CHAT SESSION

```
class ChatSession:
    def __init__(self, user_id):
        self.user_id=user_id; self.history=[]
    def add(self, role, text):
        self.history.append({"role":role,"text":text})
    def last(self): return self.history[-1] if self.history e
lse None
```

INTERVIEW DEFENSE

Project answer: This mirrors a real chatbot service boundary; persistence can be injected later.

Debug cue: Do not let session objects own database connections unnecessarily.

Follow-up: What would you improve first if this had 10x traffic/data?

HARD | MODEL SERVICE LAYER

```
class ModelService:
    def __init__(self, provider): self.provider=provider
    def answer(self, prompt):
        return self.provider.generate(prompt)
class FakeProvider:
    def generate(self,prompt): return "test"
```

INTERVIEW DEFENSE

Project answer: Dependency injection makes model code testable and provider-agnostic.

Debug cue: Mock provider se failures reproduce karo without calling real API.

Follow-up: What would you improve first if this had 10x traffic/data?

SELF-CHECK

Can I explain the project without notes? Can I name one failure? Can I state time/space or cost trade-offs?

APPENDIX A

Cheat Sheets

Exam se pehle ya code likhte waqt, yahi pages kholo

1. List vs Tuple vs Set vs Dictionary

	List	Tuple	Set	Dictionary
Syntax	[1, 2, 3]	(1, 2, 3)	{1, 2, 3}	{'a': 1}
Ordered?	Haan	Haan	Nahi	Haan (3.7+)
Changeable?	Haan	Nahi	Haan	Haan
Duplicates?	Allowed	Allowed	Nahi	Keys unique
Index access	l[0]	t[0]	Nahi	d['key']
Khaali banao	[] ya list()	() ya tuple()	set() (sirf!)	{} ya dict()
Kab use karein	Items ki changing list	Fixed data, dict key	Unique items, fast in-check	Key se lookup, records

2. String Methods Quick Table

Method	Kaam	Example	Result
upper()	Sab UPPERCASE	`"hello".upper()`	`'HELLO'`
lower()	Sab lowercase	`"HeLLo".lower()`	`'hello'`
strip()	Dono taraf ke spaces hatao	`" hi ".strip()`	`'hi'`
split()	String ko list me todo	`"a,b,c".split(",")`	`['a', 'b', 'c']`
join()	List ko string me jodo	`"-".join(["a", "b"])`	`'a-b'`
replace()	Part badlo	`"cats".replace("cat", "dog")`	`'dogs'`
find()	Index dhoondho (na mile: -1)	`"hello".find("l")`	2
count()	Kitni baar aaya	`"banana".count("a")`	3
startswith()	Prefix check	`"hello".startswith("he")`	True
endswith()	Suffix check	`"a.jpg".endswith(".jpg")`	True
title()	Har word capital	`"hello world".title()`	`'Hello World'`
capitalize()	Pehla letter capital	`"hello world".capitalize()`	`'Hello world'`
swapcase()	Case ulta	`"Hello".swapcase()`	`'hELLO'`
isdigit()	Sirf digits?	`"123".isdigit()`	True
isalpha()	Sirf letters?	`"abc1".isalpha()`	False
zfill()	Left me 0 lagao	`"42".zfill(5)`	`'00042'`

Method	Kaam	Example	Result
center()	Beech me align	`"hi".center(6, "*")`	`'**hi**'`
partition()	3 hisso me todo	`"a=b".partition("=")`	`('a', '=', 'b')`

3. List Methods Quick Table

Method	Kaam	Code (nums = [3, 1, 2])	Result
append(x)	End me 1 item	`nums.append(9)`	`nums = [3, 1, 2, 9]`
extend(list)	End me kai items	`nums.extend([7, 8])`	`nums = [3, 1, 2, 7, 8]`
insert(i, x)	Index par add	`nums.insert(1, 99)`	`nums = [3, 99, 1, 2]`
remove(x)	Value hatao	`nums.remove(1)`	`nums = [3, 2]`
pop(i)	Index se hatao + return	`nums.pop(0)`	`nums = [1, 2] (return: 3)`
index(x)	Pehla index	`nums.index(2)`	`nums = [3, 1, 2] (return: 2)`
count(x)	Kitni baar	`nums.count(2)`	`nums = [1, 2, 2] (return: 2)`
sort()	Ascending sort (in-place)	`nums.sort()`	`nums = [1, 2, 3]`
reverse()	Order ulta	`nums.reverse()`	`nums = [2, 1, 3]`
copy()	Asli copy	`nums.copy()`	`nums = [3, 1, 2] (return: [3, 1, 2])`
clear()	Sab hatao	`nums.clear()`	`nums = []`

4. Dictionary Methods Quick Table

Method	Kaam	Code (d = {"a": 1, "b": 2})	Result
d[key]	Value (missing = error)	`d["a"]`	`1`
get(key, default)	Safe value	`d.get("z", 0)`	`0`
keys()	Sari keys	`list(d.keys())`	`['a', 'b']`
values()	Sari values	`list(d.values())`	`[1, 2]`
items()	(key, value) pairs	`list(d.items())`	`[('a', 1), ('b', 2)]`
pop(key)	Hatao + value return	`dict(d).pop("a")`	`1`
update(d2)	Merge karo	`dict(d, c=3)`	`{'a': 1, 'b': 2, 'c': 3}`
in	Key hai kya?	`"a" in d`	`True`

APPENDIX B

Common Errors & Fixes

Error se darna nahi. Aakhri line padho, wahi asli baat batati hai

Python error deta hai to wo bata deta hai **kaunsi line** me aur **kya** galat hua. Neeche wale 10 errors beginners ko sabse zyada milte hain.

Error	Kab aata hai	Example	Fix
SyntaxError	Colon, bracket ya quote bhoolna	`if x > 5` `print(x)`	if x > 5: likho (colon lagao)
IndentationError	Spaces galat ya mixed	`if True:` `print("hi")`	Block ke andar 4 spaces do
NameError	Variable define nahi kiya / spelling galat	`print(nmae)`	Naam ki spelling check karo, pehle define karo
TypeError	Galat types ka operation	`"Age: " + 22`	f-string ya str(22) use karo
ValueError	Sahi type, galat value	`int("abc")`	Pehle isdigit() se check karo
IndexError	List/string me range se bahar index	`a = [1, 2]` `a[5]`	len(a) se check karo, last index = len - 1
KeyError	Dictionary me key nahi hai	`d = {"a": 1}` `d["b"]`	d.get("b") ya "b" in d
AttributeError	Object me wo method/attribute nahi	`"hi".append("x")`	Sahi type ka method use karo (dir(obj) dekho)
ZeroDivisionError	0 se divide kiya	`10 / 0`	Divide se pehle if b != 0 check karo
ModuleNotFoundError	Library install nahi hai	`import requests`	pip install requests

TIP

Debugging ka golden rule: 1) Error ki aakhri line padho. 2) Line number par jao. 3) **print()** se variable ki value aur **type()** check karo. 90% bugs yahi se mil jate hain.

APPENDIX C

Practice Solutions

Pehle khud try karo. Sirf atak jao tab hi dekho

01 Variables & Data Types

Q1 (Easy) Apna naam, age aur city 3 variables me store karo aur ek line me print karo.

solution_1_1.py

```
name = "Ashu"
age = 22
city = "Delhi"
print(name, age, city)
```

▶ OUTPUT

```
Ashu 22 Delhi
```

Q2 (Medium) Do numbers string me hain: a = "12", b = "8". Unka sum (numeric) print karo.

solution_1_2.py

```
a = "12"
b = "8"
print(int(a) + int(b))
```

▶ OUTPUT

```
20
```

Q3 (Hard) Teesra variable use kiye bina a = 5 aur b = 9 ko swap karo.

solution_1_3.py

```
a, b = 5, 9
a, b = b, a
print(a, b)
```

▶ OUTPUT

```
9 5
```

02 Operators

Q1 (Easy) 17 ko 5 se divide karke quotient aur remainder print karo.

solution_2_1.py

```
print(17 // 5, 17 % 5)
```

▶ OUTPUT

```
3 2
```

Q2 (Medium) n = 42 even hai ya nahi, True/False print karo.

solution_2_2.py

```
n = 42
print(n % 2 == 0)
```

▶ OUTPUT

```
True
```

Q3 (Hard) age = 25 ke liye check karo ki 18 aur 60 ke beech (dono included) hai ya nahi. Chained comparison use karo.

solution_2_3.py

```
age = 25
print(18 <= age <= 60)
```

▶ OUTPUT

```
True
```

03 Input & Output

Q1 (Easy) Naam input lo (name = "Ashu" maan lo) aur f-string se "Welcome, Ashu!" print karo.

solution_3_1.py

```
name = "Ashu"
print(f"Welcome, {name}!")
```

▶ OUTPUT

```
Welcome, Ashu!
```

Q2 (Medium) Do numbers a = 7, b = 3 ka average f-string se sirf 2 decimal places me print karo.

solution_3_2.py

```
a, b = 7, 3
print(f"Average: {(a + b) / 2:.2f}")
```

▶ OUTPUT

```
Average: 5.00
```

Q3 (Hard) Ek line me 3 numbers input lo ("4 5 6") aur unka sum print karo.

solution_3_3.py

```
nums = list(map(int, input().split()))
print(sum(nums))
```

▶ OUTPUT

```
15
```

04 Strings

Q1 (Easy) "Programming" ka pehla aur aakhri character print karo.

solution_4_1.py

```
word = "Programming"
print(word[0], word[-1])
```

▶ OUTPUT

```
P g
```

Q2 (Medium) "Python" ko reverse karke print karo.

solution_4_2.py

```
print("Python"[::-1])
```

▶ OUTPUT

```
nohtyP
```

Q3 (Hard) Check karo ki "madam" palindrome hai ya nahi (True/False).

solution_4_3.py

```
w = "madam"
print(w == w[::-1])
```

▶ OUTPUT

```
True
```

05 String Methods

Q1 (Easy) " Hello Python " ko strip karke uppercase me print karo.

solution_5_1.py

```
print("  Hello Python  ".strip().upper())
```

▶ OUTPUT

```
HELLO PYTHON
```

Q2 (Medium) "a-b-c-d" ko "-" par todo aur phir " + " se jodkar print karo.

solution_5_2.py

```
parts = "a-b-c-d".split("-")
print(" + ".join(parts))
```

▶ OUTPUT

```
a + b + c + d
```

Q3 (Hard) Sentence "the quick brown fox" ke har word ka pehla letter capital karo, bina title() use kiye.

solution_5_3.py

```
words = "the quick brown fox".split()
print(" ".join(w.capitalize() for w in words))
```

▶ OUTPUT

```
The Quick Brown Fox
```

06 if / elif / else

Q1 (Easy) n = -4 positive, negative ya zero hai? Print karo.

solution_6_1.py

```
n = -4
if n > 0:
    print("Positive")
elif n < 0:
    print("Negative")
else:
    print("Zero")
```

▶ OUTPUT

```
Negative
```

Q2 (Medium) year = 2024 leap year hai ya nahi? (4 se divisible, par 100 se nahi; ya 400 se divisible)

solution_6_2.py

```
year = 2024
if (year % 4 == 0 and year % 100 != 0) or year % 400 == 0:
    print("Leap year")
else:
    print("Not a leap year")
```

▶ OUTPUT

```
Leap year
```

Q3 (Hard) marks = 82 par ternary se "Pass" (>= 40) ya "Fail" print karo, aur phir 90+ ho to "Topper" bhi print karo.

solution_6_3.py

```
marks = 82
print("Pass" if marks >= 40 else "Fail")
if marks >= 90:
    print("Topper")
```

▶ OUTPUT

```
Pass
```

07 for & while Loops

Q1 (Easy) 1 se 5 tak ke numbers ka sum for loop se nikalo.

solution_7_1.py

```
total = 0
for i in range(1, 6):
    total += i
print(total)
```

▶ OUTPUT

```
15
```

Q2 (Medium) FizzBuzz: 1 se 15 tak print karo. 3 ka multiple = Fizz, 5 ka = Buzz, dono ka = FizzBuzz.

solution_7_2.py

```
for i in range(1, 16):
    if i % 15 == 0:
        print("FizzBuzz")
    elif i % 3 == 0:
        print("Fizz")
    elif i % 5 == 0:
        print("Buzz")
    else:
        print(i)
```

▶ OUTPUT

```
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
```

Q3 (Hard) Ek list [3, 8, 12, 5, 20] me se sirf wo numbers ek naye list me nikalo jo 5 se bade hain, comprehension use karke.

solution_7_3.py

```
nums = [3, 8, 12, 5, 20]
print([n for n in nums if n > 5])
```

▶ OUTPUT

```
[8, 12, 20]
```

08 Lists

Q1 (Easy) fruits = ["apple", "banana", "mango"] ka pehla aur aakhri item print karo.

solution_8_1.py

```
fruits = ["apple", "banana", "mango"]
print(fruits[0], fruits[-1])
```

▶ OUTPUT

```
apple mango
```

Q2 (Medium) Ek list [5, 3, 8, 1] ka sabse bada, sabse chhota aur average print karo.

solution_8_2.py

```
nums = [5, 3, 8, 1]
print(max(nums), min(nums), sum(nums) / len(nums))
```

▶ OUTPUT

```
8 1 4.25
```

Q3 (Hard) matrix = [[1, 2, 3], [4, 5, 6]] ke saare elements ka sum nested loop se nikalo.

solution_8_3.py

```
matrix = [[1, 2, 3], [4, 5, 6]]
total = 0
for row in matrix:
    for x in row:
        total += x
print(total)
```

▶ OUTPUT

```
21
```

09 List Methods

Q1 (Easy) Ek khaali list banao, usme 10, 20, 30 append karo aur print karo.

solution_9_1.py

```
nums = []
nums.append(10)
nums.append(20)
nums.append(30)
print(nums)
```

▶ OUTPUT

```
[10, 20, 30]
```

Q2 (Medium) ["pear", "fig", "banana", "kiwi"] ko length ke hisab se descending sort karo.

solution_9_2.py

```
words = ["pear", "fig", "banana", "kiwi"]
words.sort(key=len, reverse=True)
print(words)
```

▶ OUTPUT

```
['banana', 'pear', 'kiwi', 'fig']
```

Q3 (Hard) [1, 2, 2, 3, 1, 4] se duplicates hatao aur order same rakho.

solution_9_3.py

```
nums = [1, 2, 2, 3, 1, 4]
print(list(dict.fromkeys(nums)))
```

▶ OUTPUT

```
[1, 2, 3, 4]
```

10 Tuples

Q1 (Easy) t = (5, 10, 15) ka pehla aur aakhri item print karo.

solution_10_1.py

```
t = (5, 10, 15)
print(t[0], t[-1])
```

▶ OUTPUT

```
5 15
```

Q2 (Medium) Tuple ("Ashu", 22, "Delhi") ko 3 variables me unpack karke f-string se print karo.

solution_10_2.py

```
name, age, city = ("Ashu", 22, "Delhi")
print(f"{name} is {age} from {city}")
```

▶ OUTPUT

```
Ashu is 22 from Delhi
```

Q3 (Hard) Ek function likho jo list ka sum aur average tuple me return kare.

solution_10_3.py

```
def stats(nums):
    return sum(nums), sum(nums) / len(nums)

total, avg = stats([2, 4, 6])
print(total, avg)
```

▶ OUTPUT

```
12 4.0
```

11 Sets

Q1 (Easy) [5, 1, 5, 2, 1] me kitne unique numbers hain? Print karo.

solution_11_1.py

```
print(len(set([5, 1, 5, 2, 1])))
```

▶ OUTPUT

```
3
```

Q2 (Medium) Do lists me common items nikalo: [1, 2, 3, 4] aur [3, 4, 5, 6].

solution_11_2.py

```
a, b = [1, 2, 3, 4], [3, 4, 5, 6]
print(sorted(set(a) & set(b)))
```

▶ OUTPUT

```
[3, 4]
```

Q3 (Hard) Ek string "programming" me kaun kaun se unique letters hain, sorted order me print karo.

solution_11_3.py

```
print(sorted(set("programming")))
```

▶ OUTPUT

```
['a', 'g', 'i', 'm', 'n', 'o', 'p', 'r']
```

12 Dictionaries

Q1 (Easy) Ek dictionary banao jisme tumhara name, age, city ho, aur city print karo.

solution_12_1.py

```
me = {"name": "Ashu", "age": 22, "city": "Delhi"}
print(me["city"])
```

▶ OUTPUT

```
Delhi
```

Q2 (Medium) String "banana" me har letter ki frequency dictionary me nikalo.

solution_12_2.py

```
freq = {}
for ch in "banana":
    freq[ch] = freq.get(ch, 0) + 1
print(freq)
```

▶ OUTPUT

```
{'b': 1, 'a': 3, 'n': 2}
```

Q3 (Hard) marks = {"A": 55, "B": 90, "C": 72} me se sabse zyada marks wale student ka naam nikalo.

solution_12_3.py

```
marks = {"A": 55, "B": 90, "C": 72}
print(max(marks, key=marks.get))
```

▶ OUTPUT

```
B
```

13 OOP (OOP)

Q1 (Easy) Ek Person class banao jisme name aur age ho, aur ek method introduce() ho jo "Hi, I am Ashu" print kare.

solution_13_1.py

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def introduce(self):
        print(f"Hi, I am {self.name}")

Person("Ashu", 22).introduce()
```

▶ OUTPUT

```
Hi, I am Ashu
```

Q2 (Medium) BankAccount class banao: deposit(), withdraw() aur balance. Balance se zyada withdraw hone par "Insufficient funds" print ho.

solution_13_2.py

```
class BankAccount:
    def __init__(self, balance=0):
        self.balance = balance
    def deposit(self, amt):
        self.balance += amt
    def withdraw(self, amt):
        if amt > self.balance:
            print("Insufficient funds")
        else:
            self.balance -= amt

acc = BankAccount(500)
acc.deposit(200)
acc.withdraw(1000)
acc.withdraw(300)
print(acc.balance)
```

▶ OUTPUT

```
Insufficient funds
400
```

Q3 (Hard) Shape parent class (area() method) banao. Rectangle aur Circle child classes me area() override karo aur ek loop me dono ka area print karo.

solution_13_3.py

```
class Shape:
    def area(self):
        return 0

class Rectangle(Shape):
    def __init__(self, w, h):
        self.w, self.h = w, h
    def area(self):
        return self.w * self.h

class Circle(Shape):
    def __init__(self, r):
        self.r = r
    def area(self):
        return 3.14 * self.r ** 2

for s in [Rectangle(3, 4), Circle(2)]:
    print(s.area())
```

▶ OUTPUT

```
12
12.56
```

APPENDIX D

Mini Projects

Seekhi hui cheezein jodkar 3 real programs banao

Project code ko pehle **khud type karo**, run karo, phir usme apne changes karo (naye students, naye features). Yahi asli practice hai.

Project 1: Student Grade Calculator

Concepts: Variables, loops, if/elif, dictionaries, f-strings

Ek dictionary me students ke marks hain. Program har student ka grade nikalta hai aur class ka average dikhata hai.

project_1.py

```
marks = {"Ravi": 82, "Sita": 95, "Amit": 58, "Neha": 71}

def grade(m):
    if m >= 90:
        return "A"
    elif m >= 75:
        return "B"
    elif m >= 60:
        return "C"
    return "D"

for name, m in marks.items():
    print(f"{name:<6} {m:>3}  Grade {grade(m)}")

avg = sum(marks.values()) / len(marks)
print(f"\nClass average: {avg:.1f}")
topper = max(marks, key=marks.get)
print("Topper:", topper)
```

▶ OUTPUT

```
Ravi    82  Grade B
Sita    95  Grade A
Amit    58  Grade D
Neha    71  Grade C

Class average: 76.5
Topper: Sita
```

TIP

Challenge: Naya student add karo, aur pass/fail count bhi print karo.

Project 2: Contact Book

Concepts: Dictionaries, functions, loops, string methods

Contacts ko dictionary me store karke add, search aur list ka kaam karta hai.

project_2.py

```
contacts = {}

def add(name, phone):
    contacts[name.title()] = phone

def search(name):
    return contacts.get(name.title(), "Not found")

add("ravi", "98111 22222")
add("SITA", "98222 33333")

print(search("ravi"))
print(search("amit"))
for name, phone in sorted(contacts.items()):
    print(f"{name}: {phone}")
```

▶ OUTPUT

```
98111 22222
Not found
Ravi: 98111 22222
Sita: 98222 33333
```

TIP

Challenge: Delete function banao, aur naam se contact update karne ka option jodo.

Project 3: Bank Account (OOP)

Concepts: Classes, encapsulation, methods, error handling

Private balance, deposit/withdraw methods aur transaction history wala mini banking system.

project_3.py

```python
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.__balance = balance
        self.history = []

    def deposit(self, amt):
        self.__balance += amt
        self.history.append(f"+{amt}")

    def withdraw(self, amt):
        if amt > self.__balance:
            raise ValueError("Insufficient funds")
        self.__balance -= amt
        self.history.append(f"-{amt}")

    @property
    def balance(self):
        return self.__balance

    def __str__(self):
        return f"{self.owner}: Rs.{self.__balance}"

acc = BankAccount("Ashu", 1000)
acc.deposit(500)
acc.withdraw(300)
try:
    acc.withdraw(5000)
except ValueError as e:
    print("Error:", e)
print(acc)
print(acc.history)
```

▶ OUTPUT

```
Error: Insufficient funds
Ashu: Rs.1200
['+500', '-300']
```

TIP

Challenge: Interest add karne wala method banao, aur history me date bhi store karo.

APPENDIX E

Interview Quick-Fire

Fresher interviews me sabse zyada puchhe jaane wale sawal

1. **List aur Tuple me kya fark hai?**
List mutable hai (badal sakte hain), tuple immutable hai. Tuple thoda fast hota hai aur dict key ban sakta hai.

2. **== aur is me fark?**
== value compare karta hai, is check karta hai ki dono **same object** hain ya nahi.

3. **Mutable aur immutable types kaun se hain?**
Mutable: list, dict, set. Immutable: int, float, str, bool, tuple.

4. **b = a aur b = a.copy() me kya fark hai?**
b = a sirf doosra naam banata hai (dono same list). copy() nayi independent list banata hai.

5. **append() aur extend() me fark?**
append() ek item (list bhi ek hi item ki tarah) jodta hai. extend() dusre list ke saare items alag alag jodta hai.

6. **sort() aur sorted() me fark?**
sort() list ko in-place badalta hai aur None deta hai. sorted() nayi sorted list deta hai, original safe.

7. **Dictionary me missing key ko safely kaise access karein?**
d.get(key, default) use karo, ya pehle key in d check karo.

8. **Set kab use karte hain?**
Duplicates hatane, fast membership check, aur union/intersection jaise operations ke liye.

9. **self kya hota hai?**
Class ke method me current object ka reference. Ye batata hai ki method kis object par kaam kar raha hai.

10. **Inheritance, polymorphism aur encapsulation kya hain?**
Inheritance = parent se reuse. Polymorphism = same method name, alag behaviour. Encapsulation = data ko class ke andar chhupana aur methods se access dena.

11. **__str__ aur __repr__ me fark?**
__str__ users ke liye readable text, __repr__ developers ke liye exact debug representation.

12. **Python me list comprehension kyun use karte hain?**
Chhota, readable aur aksar tez tarika hai ek loop se naya list banane ka.

APPENDIX F

Aage Kya? Next Steps

Ye guide complete karne ke baad ka rasta

Badhai ho! Basics se OOP tak ka safar tumne pura kar liya. Ab neeche wale topics ek ek karke seekho, yahi Python developer ka asli roadmap hai.

Topic	Kya seekhna hai	Kyun zaroori
1. Functions	def, parameters, return, *args, **kwargs, lambda	Code ko reusable banata hai. Sabse pehle ye.
2. Exception Handling	try / except / finally, raise	Program ko crash hone se bachata hai.
3. File Handling	open(), with, read/write, CSV, JSON	Data save aur load karna.
4. Modules & pip	import, random, datetime, math, pip install	Doosron ka likha code use karna.
5. Libraries	requests, pandas, matplotlib, Flask / Django	Web, data aur automation ke liye.
6. Git & GitHub	commit, push, README	Projects ko portfolio me dikhane ke liye.

Project Ideas (Portfolio ke liye)

BEGINNER	Number guessing game, To-do list (CLI), Calculator, Password generator
INTERMEDIATE	Expense tracker (file me save), Quiz app, Weather app (API), Contact book with JSON
ADVANCED	Flask blog, Web scraper + CSV export, Student management (OOP + DB), Telegram bot

Keep coding, Ashu!

Roz thoda code likho, roz thoda seekho. Ek mahine baad khud ko dekhoge to yakeen nahi hoga.

APPENDIX G

AI Engineer + Data Science Master Track

Ye appended track original 137 physical pages ko touch kiye bina unke neeche add hota hai. Same Learn -> Speak -> Code -> Think -> Debug philosophy ab advanced Python, data, ML, GenAI, Agentic AI, APIs aur interview mastery tak extend hoti hai.

01	Functions & Python Core	Pages 100-105
02	Exceptions, Files & Modules	Pages 106-111
03	Advanced Python & Performance	Pages 112-117
04	NumPy for AI & Data Science	Pages 118-123
05	Pandas & Exploratory Data Analysis	Pages 124-129
06	SQL + Data Analytics with Python	Pages 130-135
07	Machine Learning with Python	Pages 136-141
08	Generative AI, LLMs & RAG	Pages 142-147
09	Agentic AI & Automation	Pages 148-153
10	APIs, Flask/FastAPI & Production Python	Pages 154-159
11	AI Engineer + Data Scientist Interview Mastery	Pages 160-165
12	Coding Patterns, Projects & Final Practice	Pages 166-171

Study Rule: Har section ke 6 pages complete karo. Coding answers ko dekho, phir same problem ko blank file me bina dekhe rewrite karo.

SECTION 01

Functions & Python Core

Level: Intermediate Time: ~90 min 7 focus topics

Reusable logic, clean design aur interview coding ka foundation.

IS TRACK ME KYA SEEKHOGE

✓ Functions
✓ parameters
✓ scope
✓ lambda
✓ recursion
✓ comprehensions
✓ *args/**kwargs

TOPIC ROADMAP

1.1 Functions
1.2 parameters
1.3 scope
1.4 lambda
1.5 recursion
1.6 comprehensions
1.7 *args/**kwargs

INTERVIEW MINDSET

Definition -> tiny example -> practical use -> trade-off/edge case. Har answer ko apni language me bolo.

01.1 Core Concepts

Functions code ko reusable banate hain. AI/data projects me aap data cleaning, API calls, scoring aur prompt-building ko functions me todte ho. Interview me function explain karte waqt inputs, output, side effects aur complexity mention karo.

PARAMETERS

Positional, keyword, default aur keyword-only parameters ko clearly distinguish karo. Default mutable values se bachna important hai.

RETURN VS PRINT

print screen par dikhata hai; return caller ko value deta hai. Reusable code me return usually better hota hai.

SCOPE / LEGB

Local -> Enclosing -> Global -> Built-in. Global state ko limit karo; dependency injection aur explicit parameters use karo.

ARGS / KWARGS

Variable number of positional/keyword arguments ke liye. API wrappers aur configurable utilities me useful.

example.py

```
from typing import Iterable

def average(values: Iterable[float]) -> float:
    values = list(values)
    if not values:
        raise ValueError("values cannot be empty")
    return sum(values) / len(values)

print(average([10, 20, 30]))
```

OUTPUT

```
20.0
```

YAAD RAKHO
Clean functions small, testable aur predictable honi chahiye. AI code me prompt building, parsing, validation aur tool execution ko separate functions rakho.

01.2 Interview Depth

Is page ka goal hai concept ko implementation decision tak le jana. Interviewer ko syntax se zyada reasoning, edge cases aur trade-offs dikhte hain.

WHAT

Kya hai aur kab use karoge?

WHY

Kaunsi problem solve hoti hai?

RISK

Common bug / failure mode kya hai?

TRADE-OFF

Simple option aur scalable option me kya difference hai?

example.py

```
from typing import Iterable

def average(values: Iterable[float]) -> float:
    values = list(values)
    if not values:
        raise ValueError("values cannot be empty")
    return sum(values) / len(values)

print(average([10, 20, 30]))
```

OUTPUT

```
20.0
```

YAAD RAKHO
One-line answer practice: definition + example + use + limitation.

INTERVIEW LAB / Functions & Python Core

Definition -> tiny example -> practical use -> trade-off. Word-by-word memorize mat karo.

Q01

Mutable default argument se issue kya ho sakta hai?

SMART ANSWER

Default object function definition time par banta hai. list jaise mutable default reuse ho sakte hain; safe pattern None default + create inside function hai.

TIP 20-30 sec: small example batao.

Q02

return aur yield me difference?

SMART ANSWER

return function ko end karke ek result deta hai. yield generator produce karta hai aur function state preserve karke next value par resume ho sakta hai.

TIP Large stream example do.

Q03

LEGB kya hai?

SMART ANSWER

Python name lookup Local, Enclosing, Global, Built-in scopes me karta hai. Shadowing se bachne ke liye clear names aur explicit dependencies rakho.

TIP A simple nested example explain karo.

Q04

lambda kab use karoge?

SMART ANSWER

Short one-expression function ke liye, especially key=, map/filter-style small transformations me. Complex logic ke liye def better readability deta hai.

TIP Overuse se readability mat girao.

Q05

Decorator real project me kaha?

SMART ANSWER

Timing, logging, authorization, retries ya caching jaisa cross-cutting behavior wrap karne ke liye.

TIP Ek @timed idea explain karo.

Q06

*args aur **kwargs?

SMART ANSWER

*args extra positional args tuple me collect karta hai; **kwargs named args dict me collect karta hai. Wrapper functions me common hai.

TIP Function signature ka example bolo.

Speaking drill: Har answer 2 baar bolo. Second round me ek project example add karo.

CODING LAB / Functions & Python Core

Pehle problem ko 10 sec me explain karo -> then code -> edge case -> complexity.

EASY

Write a function square_sum(nums) that returns sum of squares.

ANSWER

```
def square_sum(nums):
return sum(x*x for x in nums)
```

Reason: Generator expression avoids building a second list. O(n) time, O(1) extra space (excluding result object).

EASY

Write normalize_name(name) that strips spaces and title-cases it.

ANSWER

```
def normalize_name(name):
return " ".join(name.split()).title()
```

Reason: split() handles repeated whitespace robustly.
Complexity: O(n)

MEDIUM

Return first unique character or None.

ANSWER

```
def first_unique(s):
c={}
for ch in s: c[ch]=c.get(ch,0)+1
return next((ch for ch in s if c[ch]==1),
None)
```

Reason: Two passes keep logic simple and linear.
Complexity: O(n) / O(k)

MEDIUM

Write a decorator that counts calls.

ANSWER

```
def count_calls(fn):
calls=0
def wrap(*a,**kw):
nonlocal calls; calls+=1
return fn(*a,**kw)
return wrap
```

Reason: Closure stores state in the enclosing scope.
Complexity: O(1) extra per call)

HARD

Flatten nested one-level list safely.

ANSWER

```
def flatten(rows):
return [x for row in rows for x in row]
```

Reason: Nested comprehension mirrors two loops.
Complexity: O(n)

DEBUG

Fix mutable default argument in add_item.

ANSWER

```
def add_item(x, items=None):
items=[] if items is None else items
items.append(x); return items
```

Reason: Create a fresh list for each call.
Complexity: O(1) extra plus list

LOGIC LAB / Functions & Python Core

Code run karne se pehle output predict karo. Phir dry-run karke reason explain karo.

PREDICT

def f(x=1, box=[]): box.append(x); return box
print(f(1)); print(f(2))

CHECK
[1] then [1, 2]

WHY
Mutable default persists between calls.

TRACE

x=[1,2]; y=x; y.append(3); print(x)

CHECK
[1, 2, 3]

WHY
Both names reference the same list.

THINK

def outer(): x=10; def inner(): return x; return inner()

CHECK
10

WHY
Enclosing scope is visible to inner function.

PREDICT

print(sum(i*i for i in range(4)))

CHECK
14

WHY
Squares are 0+1+4+9.

FIX

def add(a,b): print(a+b) value=add(2,3)

CHECK
value is None

WHY
print has no explicit return.

EXPLAIN

What is the cleanest way to add validation without duplicating logic?

CHECK
Small reusable validator function or clear guard clauses.

WHY
Separate validation from core computation where practical.

Mastery: Explain without notes + solve one fresh problem + predict two outputs + tell one edge case.

SOLVED PRACTICE - Functions & Reusable Python

Every coding-lab question now has its answer, expected result and a quick check - all in one place.

EASY

Write a function square_sum(nums) that returns sum of squares.

```
def square_sum(nums):
return sum(x*x for x in nums)
Reason: Generator expression avoids building a
second list. O(n) time, O(1) extra space (excluding
result object).
Complexity: O(n) / O(1)
```

EXPECTED RESULT

Returns the calculated numeric result.

EASY

Write normalize_name(name) that strips spaces and title-cases it.

```
def normalize_name(name):
return " ".join(name.split()).title()
Reason: split() handles repeated whitespace robustly.
Complexity: O(n)
```

EXPECTED RESULT

Compare with the code and verify one edge case.

MEDIUM

Return first unique character or None.

```
def first_unique(s):
c={}
for ch in s: c[ch]=c.get(ch,0)+1
return next((ch for ch in s if c[ch]==1),
None)
Reason: Two passes keep logic simple and linear.
Complexity: O(n) / O(k)
```

EXPECTED RESULT

Compare with the code and verify one edge case.

MEDIUM

Write a decorator that counts calls.

```
def count_calls(fn):
calls=0
def wrap(*a,**kw):
nonlocal calls; calls+=1
return fn(*a,**kw)
return wrap
Reason: Closure stores state in the enclosing scope.
Complexity: O(1) extra per call)
```

EXPECTED RESULT

Wrapper runs and preserves the original call result.

HARD

Flatten nested one-level list safely.

```
def flatten(rows):
return [x for row in rows for x in row]
Reason: Nested comprehension mirrors two loops.
Complexity: O(n)
```

EXPECTED RESULT

Compare with the code and verify one edge case.

PRACTICE HABIT

Practice rule: solve first, compare second, then change one input. Mark the question only after you can explain why the output changes.

MINI PROJECT - Calculator Functions

Functions & Reusable Python ka concept ek runnable, interview-ready mini build me convert karo.

YE KYA KARTA HAI?

Small pure functions se arithmetic reuse karta hai.

KIS KAAM AATA HAI?

Any Python project foundation.

ARCHITECTURE

Caller -> function -> return value

FILES / BUILD TARGET

utils.py Target: No global state

CORE IMPLEMENTATION - TYPE + RUN

```
def add(a, b):
    return a + b

def safe_div(a, b):
    return None if b == 0 else a / b

print(add(2, 3), safe_div(5, 0))
```

SAMPLE OUTPUT `5 None (add(2,3), safe_div(5,0))`

INTERVIEW EXPLANATION

Project answer: return testability aur composition ke liye print se better hai. Debug cue: Unit-test zero and boundary cases. Follow-up: What would you improve first if this had 10x traffic/data?

BUILD STEPS

1. Create the file(s) shown above.
2. Run the exact sample once.
3. Change one input and observe the output.
4. Add one validation or failure test.

EDGE CASES / DEBUG

Test invalid input, boundary values and empty/missing data where relevant. Keep validation at the system boundary.

Functions & Reusable Python

Use this pack after the section labs: explain -> build -> debug -> defend.

SECTION ANCHOR

Definition: Reusable logic ko parameters, return values, decorators aur generators ke saath isolate karo.

Easy explanation: Good function = clear inputs, one responsibility, predictable output.

Example: `def add(a,b): return a+b`

SIMPLE

Calculator Functions

Kya karta hai: Small pure functions se arithmetic reuse karta hai.

Kis kaam aata hai: Any Python project foundation.

Architecture: Caller -> function -> return value

Files: `utils.py`

Build target: No global state

MEDIUM

Reusable ETL Pipeline

Kya karta hai: Load, clean aur transform steps ko functions me separate karta hai.

Kis kaam aata hai: Analytics, ML preprocessing and data pipelines.

Architecture: load() -> clean() -> transform() -> save()

Files: `pipeline.py`

Build target: Each stage independently testable

HARD

Timed API Function + Generator

Kya karta hai: Decorator timing aur generator streaming ko combine karta hai.

Kis kaam aata hai: Observability and large-result processing.

Architecture: Caller -> timed function -> generator -> consumer

Files: `perf.py`

Build target: Measure without changing business output

Project rule: code khud type karo, output predict karo, edge case test karo, phir 60-second explanation do.

Functions & Reusable Python - implementation lab

Three core implementations + project-tailored interview explanation + debugging cue.

SIMPLE | CALCULATOR FUNCTIONS

```
def add(a,b): return a+b
def safe_div(a,b):
    return None if b == 0 else a/b
print(add(2,3), safe_div(5,0))
```

INTERVIEW DEFENSE

Project answer: return testability aur composition ke liye print se better hai.

Debug cue: Unit-test zero and boundary cases.

Follow-up: What would you improve first if this had 10x traffic/data?

MEDIUM | REUSABLE ETL PIPELINE

```
def clean(rows):
    return [r for r in rows if r.get("id")]
def transform(rows):
    return [{**r,"name":r["name"].strip()} for r in rows]
rows = transform(clean(load()))
```

INTERVIEW DEFENSE

Project answer: Pipeline functions make dependencies and data flow explicit.

Debug cue: Print lengths/schema after every stage.

Follow-up: What would you improve first if this had 10x traffic/data?

HARD | TIMED API FUNCTION + GENERATOR

```
import time
def timed(fn):
  def wrap(*a,**k):
    t=time.perf_counter(); out=fn(*a,**k); print(time.perf_co
unter()-t); return out
  return wrap
@timed
def squares(n): return (i*i for i in range(n))
```

INTERVIEW DEFENSE

Project answer: Decorator cross-cutting concerns add karta hai; generator lazy output deta hai.

Debug cue: Preserve metadata with functools.wraps in production.

Follow-up: What would you improve first if this had 10x traffic/data?

SELF-CHECK

Can I explain the project without notes? Can I name one failure? Can I state time/space or cost trade-offs?

SECTION 02

Exceptions, Files & Modules

Level: Intermediate Time: ~85 min 7 focus topics

Reliable Python apps ke liye errors, files aur reusable modules ko professional tarike se handle kar

IS TRACK ME KYA SEEKHOGE

✓ try/except/else/finally
✓ custom exceptions
✓ pathlib
✓ JSON/CSV
✓ logging
✓ modules
✓ virtual environments

TOPIC ROADMAP

2.1 try/except/else/finally
2.2 custom exceptions
2.3 pathlib
2.4 JSON/CSV
2.5 logging
2.6 modules
2.7 virtual environments

INTERVIEW MINDSET

Definition -> tiny example -> practical use -> trade-off/edge case. Har answer ko apni language me bolo.

02.1 Core Concepts

Production-style Python sirf happy path nahi dekhta. Input invalid ho sakta hai, file missing ho sakti hai, network timeout aa sakta hai. Exception handling ka goal error ko chupana nahi, balki controlled behavior aur useful diagnostics dena hai.

EXCEPTION FLOW

try me risky code, except me expected recovery, else success path, finally cleanup ke liye.

FILES

pathlib + with open() use karo. Context manager resource ko close kar deta hai.

JSON / CSV

AI workflows me JSON payloads aur analytics me CSV/JSON export common hai. Schema validate karna useful hai.

LOGGING

print debugging ke bajay structured log messages use karo. Secret/API key kabhi log mat karo.

example.py

```
import json
from pathlib import Path

p = Path("profile.json")
data = {"name": "Ashu", "role": "AI Engineer"}
p.write_text(json.dumps(data, indent=2), encoding="utf-8")
loaded = json.loads(p.read_text(encoding="utf-8"))
print(loaded["role"])
```

OUTPUT

```
AI Engineer
```

YAAD RAKHO
Error handling should be specific, readable aur observable. Except Exception blindly mat use karo.

02.2 Interview Depth

Is page ka goal hai concept ko implementation decision tak le jana. Interviewer ko syntax se zyada reasoning, edge cases aur trade-offs dikhte hain.

WHAT

Kya hai aur kab use karoge?

WHY

Kaunsi problem solve hoti hai?

RISK

Common bug / failure mode kya hai?

TRADE-OFF

Simple option aur scalable option me kya difference hai?

YAAD RAKHO
One-line answer practice: definition + example + use + limitation.

INTERVIEW LAB / Exceptions, Files & Modules

Definition -> tiny example -> practical use -> trade-off. Word-by-word memorize mat karo.

Q01

Bare except avoid kyun?

SMART ANSWER

Ye har exception ko catch karta hai, including unexpected cases, aur debugging ko hide kar sakta hai. Expected exceptions specifically catch karo.

TIP ValueError vs FileNotFoundError example do.

Q02

finally kab run hota hai?

SMART ANSWER

Normal success ya exception dono paths par cleanup ke liye use hota hai, jaise file/resource handling.

TIP with statement se compare karo.

Q03

JSON aur Python dict same hain?

SMART ANSWER

JSON text serialization format hai; Python dict in-memory object hai. json.dumps/loads conversion karte hain.

TIP API payload example do.

Q04

pathlib kyun?

SMART ANSWER

Filesystem paths ko object-oriented, readable aur cross-platform style me handle karta hai.

TIP Path / operator ka example bolo.

Q05

logging vs print?

SMART ANSWER

Logging me levels, timestamps, handlers aur persistent configuration hoti hai. Production debugging ke liye more controllable hai.

TIP Secret values log na karne ka point add karo.

Q06

Custom exception kyun?

SMART ANSWER

Domain-specific failure ko clear type dene ke liye, jisse caller specific recovery strategy bana sakta hai.

TIP class InvalidRecordError(Exception): pass

Speaking drill: Har answer 2 baar bolo. Second round me ek project example add karo.

CODING LAB / Exceptions, Files & Modules

Pehle problem ko 10 sec me explain karo -> then code -> edge case -> complexity.

EASY

Read JSON text safely and return a key with default.

ANSWER

```
import json
def get_role(text):
data=json.loads(text)
return data.get("role","unknown")
```

Reason: JSON parse failure should be handled at a caller boundary when needed.
Complexity: O(n)

EASY

Write a helper that loads text file with UTF-8.

ANSWER

```
from pathlib import Path
def load_text(path):
return
Path(path).read_text(encoding="utf-8")
```

Reason: Path keeps filesystem code clear.
Complexity: O(n)

MEDIUM

Catch missing file separately from invalid JSON.

ANSWER

```
try:
text=Path("x.json").read_text(encoding="u
tf-8")
data=json.loads(text)
except FileNotFoundError:
data={}
except json.JSONDecodeError:
data={}
```

Reason: Different errors deserve different recovery paths.
Complexity: O(n)

MEDIUM

Create a custom exception for invalid score.

ANSWER

```
class InvalidScoreError(ValueError):
pass

def validate(score):
if not 0<=score<=100:
raise InvalidScoreError(score)
```

Reason: Domain-specific errors improve caller clarity.
Complexity: O(1)

HARD

Build a module-friendly config loader pattern.

ANSWER

```
def load_config(getenv):
return {"host": getenv("HOST",
"localhost"),
"debug": getenv("DEBUG", "0")=="1"}
```

Reason: Keep config access separate from business logic.
Complexity: O(1)

DEBUG

Why is `except Exception: pass` dangerous? Give a safer pattern.

ANSWER

```
try:
run_job()
except TimeoutError as exc:
logger.warning("timeout: %s", exc)
raise
```

Reason: Do not silently swallow unknown failures.
Complexity: O(1) excluding work

LOGIC LAB / Exceptions, Files & Modules

Code run karne se pehle output predict karo. Phir dry-run karke reason explain karo.

PREDICT

try: 1/0 except ZeroDivisionError: print("handled")

CHECK
handled

WHY
Specific exception is caught.

TRACE

data=json.loads("{bad}")

CHECK
JSONDecodeError

WHY
Malformed JSON text cannot parse.

THINK

with open("x.txt") as f: text=f.read()

CHECK
File is automatically closed after block

WHY
Context manager handles cleanup.

FIX

Path("a") / "b"

CHECK
a/b path object

WHY
Path composition is platform-aware.

EXPLAIN

Why not hide all errors with except Exception?

CHECK
It can swallow bugs and make root cause invisible.

WHY
Catch expected exceptions, log useful context, re-raise when needed.

PREDICT

d={"a":1}; print(json.dumps(d))

CHECK
{"a": 1}

WHY
Python dict serialized to JSON text.

Mastery: Explain without notes + solve one fresh problem + predict two outputs + tell one edge case.

SOLVED PRACTICE - Exceptions, Files & Modules

Every coding-lab question now has its answer, expected result and a quick check - all in one place.

EASY

Read JSON text safely and return a key with default.

```
import json
def get_role(text):
data=json.loads(text)
return data.get("role","unknown")
Reason: JSON parse failure should be handled at a
caller boundary when needed.
Complexity: O(n)
```

EXPECTED RESULT

Parsed dict or safe recovery path.

EASY

Write a helper that loads text file with UTF-8.

```
from pathlib import Path
def load_text(path):
return
Path(path).read_text(encoding="utf-8")
Reason: Path keeps filesystem code clear.
Complexity: O(n)
```

EXPECTED RESULT

Text is loaded with explicit UTF-8 encoding.

MEDIUM

Catch missing file separately from invalid JSON.

```
try:
text=Path("x.json").read_text(encoding="u
tf-8")
data=json.loads(text)
except FileNotFoundError:
data={}
except json.JSONDecodeError:
data={}
Reason: Different errors deserve different recovery
```

EXPECTED RESULT

Parsed dict or safe recovery path.

MEDIUM

Create a custom exception for invalid score.

```
class InvalidScoreError(ValueError):
pass
def validate(score):
if not 0<=score<=100:
raise InvalidScoreError(score)
Reason: Domain-specific errors improve caller clarity.
Complexity: O(1)
```

EXPECTED RESULT

Specific exception is handled without hiding unrelated failures.

HARD

Build a module-friendly config loader pattern.

```
def load_config(getenv):
return {"host": getenv("HOST",
"localhost"),
"debug": getenv("DEBUG", "0")=="1"}
Reason: Keep config access separate from business
logic.
Complexity: O(1)
```

EXPECTED RESULT

Compare with the code and verify one edge case.

PRACTICE HABIT

Practice rule: solve first, compare second, then change one input. Mark the question only after you can explain why the output changes.

MINI PROJECT - Safe File Reader

Exceptions, Files & Modules ka concept ek runnable, interview-ready mini build me convert karo.

YE KYA KARTA HAI?

Text file ko safely read karta hai aur missing file ko clear message deta hai.

KIS KAAM AATA HAI?

Config/log/text ingestion.

ARCHITECTURE

Path -> open -> read -> handled error

FILES / BUILD TARGET

reader.py Target: UTF-8 and missing-file handling

CORE IMPLEMENTATION - TYPE + RUN

```
from pathlib import Path
path = Path("notes.txt")
try:
    text = path.read_text(encoding="utf-8")
except FileNotFoundError:
    text = ""
print(text)
```

SAMPLE OUTPUT `Missing notes.txt -> empty string instead of crash.`

INTERVIEW EXPLANATION

Project answer: Pathlib readable hai and specific exception handling user-friendly failures deta hai. Debug cue: Catch the exception you can actually handle; do not use bare except. Follow-up: What would you improve first if this had 10x traffic/data?

BUILD STEPS

1. Create the file(s) shown above.
2. Run the exact sample once.
3. Change one input and observe the output.
4. Add one validation or failure test.

EDGE CASES / DEBUG

Test invalid input, boundary values and empty/missing data where relevant. Keep validation at the system boundary.

Exceptions, Files & Modules

Use this pack after the section labs: explain -> build -> debug -> defend.

SECTION ANCHOR

Definition: Reliable Python happy path ke saath failures, file I/O aur modular imports ko handle karta hai.

Easy explanation: Catch specific errors, validate boundaries, and keep modules focused.

Example: `try: value = int("10") except ValueError: value = 0`

SIMPLE

Safe File Reader

Kya karta hai: Text file ko safely read karta hai aur missing file ko clear message deta hai.

Kis kaam aata hai: Config/log/text ingestion.

Architecture: Path -> open -> read -> handled error

Files: `reader.py`

Build target: UTF-8 and missing-file handling

MEDIUM

JSON Config Manager

Kya karta hai: JSON config load, parse aur validation ek module me rakhta hai.

Kis kaam aata hai: AI/API applications with structured settings.

Architecture: JSON file -> parse -> validate -> config object

Files: `config_io.py`

Build target: Malformed JSON + missing keys

HARD

Resilient Log Ingestor

Kya karta hai: Large log files ko line-wise parse karke malformed lines ko quarantine karta hai.

Kis kaam aata hai: Data ingestion and observability pipelines.

Architecture: File stream -> parse -> valid store / error store

Files: `ingest.py | errors.log`

Build target: Continue processing after bad rows

Project rule: code khud type karo, output predict karo, edge case test karo, phir 60-second explanation do.

Exceptions, Files & Modules - implementation lab

Three core implementations + project-tailored interview explanation + debugging cue.

SIMPLE | SAFE FILE READER

```
from pathlib import Path
p = Path("notes.txt")
try:
    text = p.read_text(encoding="utf-8")
except FileNotFoundError:
    text = ""
print(text)
```

INTERVIEW DEFENSE

Project answer: Pathlib readable hai and specific exception handling user-friendly failures deta hai.

Debug cue: Catch the exception you can actually handle; do not use bare except.

Follow-up: What would you improve first if this had 10x traffic/data?

MEDIUM | JSON CONFIG MANAGER

```
import json
from pathlib import Path
def load_config(path):
    data=json.loads(Path(path).read_text())
    if "timeout" not in data: raise ValueError("timeout")
    return data
```

INTERVIEW DEFENSE

Project answer: Separate I/O, parsing and validation so tests can target each layer.

Debug cue: JSONDecodeError is different from FileNotFoundError and KeyError/validation errors.

Follow-up: What would you improve first if this had 10x traffic/data?

HARD | RESILIENT LOG INGESTOR

```
for line in file:
    try:
        rec = parse(line)
        save(rec)
    except (ValueError, KeyError) as e:
        bad.write(f"{line.rstrip()} | {e}\n")
```

INTERVIEW DEFENSE

Project answer: Exception handling should preserve progress while making failures observable.

Debug cue: Never silently swallow parsing errors.

Follow-up: What would you improve first if this had 10x traffic/data?

SELF-CHECK

Can I explain the project without notes? Can I name one failure? Can I state time/space or cost trade-offs?

SECTION 03

Advanced Python & Performance

Level: Advanced Time: ~110 min 8 focus topics

Interview me Python internals aur efficient coding samajhne ke liye.

IS TRACK ME KYA SEEKHOGE

✓ iterators
✓ generators
✓ decorators
✓ closures
✓ dataclasses
✓ context managers
✓ shallow/deep copy
✓ time complexity

TOPIC ROADMAP

3.1 iterators
3.2 generators
3.3 decorators
3.4 closures
3.5 dataclasses
3.6 context managers
3.7 shallow/deep copy
3.8 time complexity

INTERVIEW MINDSET

Definition -> tiny example -> practical use -> trade-off/edge case. Har answer ko apni language me bolo.

03.1 Core Concepts

Advanced Python ka focus magic ya tricks nahi, balki behavior samajhna hai. Generator memory bachata hai, decorator behavior wrap karta hai, dataclass data-centric classes ko concise banata hai.

GENERATOR

yield ek item at a time deta hai; large files/streams ke liye memory efficient.

DECORATOR

Function ko wrap karke timing, logging, auth ya retry jaisa cross-cutting behavior add kar sakte ho.

COPY SEMANTICS

Assignment reference bind karta hai. copy() shallow hai; deepcopy() nested mutable structures tak clone karta hai.

COMPLEXITY

O(1), O(n), O(log n), O(n log n), O(n^2) identify karna coding interviews ka core skill hai.

example.py

```
def count_calls(fn):
    calls = 0
    def wrapper(*args, **kwargs):
        nonlocal calls
        calls += 1
        result = fn(*args, **kwargs)
        return result, calls
    return wrapper

@count_calls
def add(a, b):
    return a + b
```

OUTPUT

```
(5, 1)
```

YAAD RAKHO
Optimization rule: pehle correctness, phir profiling, phir targeted optimization. Premature optimization avoid karo.

03.2 Interview Depth

Is page ka goal hai concept ko implementation decision tak le jana. Interviewer ko syntax se zyada reasoning, edge cases aur trade-offs dikhte hain.

WHAT

Kya hai aur kab use karoge?

WHY

Kaunsi problem solve hoti hai?

RISK

Common bug / failure mode kya hai?

TRADE-OFF

Simple option aur scalable option me kya difference hai?

example.py

```
def count_calls(fn):
    calls = 0
    def wrapper(*args, **kwargs):
        nonlocal calls
        calls += 1
        result = fn(*args, **kwargs)
        return result, calls
    return wrapper

@count_calls
def add(a, b):
    return a + b
```

OUTPUT

```
(5, 1)
```

YAAD RAKHO
One-line answer practice: definition + example + use + limitation.

INTERVIEW LAB / Advanced Python & Performance

Definition -> tiny example -> practical use -> trade-off. Word-by-word memorize mat karo.

Q01

Generator memory kaise bachata hai?

SMART ANSWER

Saara result list me materialize karne ke bajay yield se ek item at a time produce karta hai.

TIP Large file line-by-line example do.

Q02

Decorator ka basic model?

SMART ANSWER

Outer function inner wrapper return karta hai; wrapper original function call se pehle/baad behavior add karta hai.

TIP Timing/logging example do.

Q03

Shallow vs deep copy?

SMART ANSWER

Shallow copy outer container copy karta hai but nested references share ho sakte hain. Deep copy recursively nested objects clone karta hai.

TIP Nested list example explain karo.

Q04

Dataclass ka benefit?

SMART ANSWER

Data-holding classes ke boilerplate __init__, repr, comparison etc. reduce karta hai.

TIP @dataclass example do.

Q05

O(n) se O(log n) better kab?

SMART ANSWER

Large input me growth slower hoti hai, but operation aur data structure fit hona zaroori hai. Binary search classic example hai.

TIP Sorted array constraint mention karo.

Q06

Profiling kyun?

SMART ANSWER

Performance guess se nahi, measurement se optimize karo. Hot path identify karke targeted change karo.

TIP Time complexity + profiling dono mention karo.

Speaking drill: Har answer 2 baar bolo. Second round me ek project example add karo.

CODING LAB / Advanced Python & Performance

Pehle problem ko 10 sec me explain karo -> then code -> edge case -> complexity.

EASY

Create generator for squares 0..n-1.

ANSWER

```
def squares(n):
for i in range(n):
yield i*i
```

Reason: yield is lazy.
Complexity: O(n) time / O(1) extra

EASY

Write a timing decorator skeleton.

ANSWER

```
def timed(fn):
def wrap(*a,**kw):
import time
t=time.perf_counter(); r=fn(*a,**kw)
return r, time.perf_counter()-t
return wrap
```

Reason: Measure actual runtime.
Complexity: O(1) extra

MEDIUM

Demonstrate shallow copy aliasing.

ANSWER

```
import copy
a=[[1],[2]]
b=copy.copy(a)
b[0].append(9)
print(a)
```

Reason: Nested list remains shared.
Complexity: O(n) copy outer

MEDIUM

Use dataclass for a record.

ANSWER

```
from dataclasses import dataclass
@dataclass
class User:
name:str
active:bool=True
```

Reason: Removes repetitive constructor/repr boilerplate.
Complexity: O(1)

HARD

Binary search index of target in sorted list.

ANSWER

```
def binary_search(a,t):
lo,hi=0,len(a)-1
while lo<=hi:
m=(lo+hi)//2
if a[m]==t:return m
if a[m]<t:lo=m+1
else:hi=m-1
return -1
```

Reason: Each step halves the search interval.
Complexity: O(log n)

DEBUG

What is the complexity of two nested loops over n?

ANSWER

```
for i in range(n):
for j in range(n):
work(i,j)
```

Reason: If both loops run n times independently, it is O(n^2).
Complexity: O(n^2)

LOGIC LAB / Advanced Python & Performance

Code run karne se pehle output predict karo. Phir dry-run karke reason explain karo.

PREDICT

g=(i*i for i in range(3)); print(next(g), next(g))

CHECK
0 1

WHY
Generator yields lazily.

TRACE

a=[[1],[2]]; b=a.copy(); b[0].append(9); print(a)

CHECK
[[1, 9], [2]]

WHY
Nested list reference is shared.

THINK

O(n)+O(n)

CHECK
O(n)

WHY
Sequential linear passes add, they do not multiply.

PREDICT

binary search on 1,000,000 items

CHECK
About 20 comparisons (log2 1e6)

WHY
Each step halves search interval.

FIX

for i in range(n): for j in range(n): pass

CHECK
O(n^2)

WHY
Independent nested loops multiply.

EXPLAIN

Why profile before optimizing?

CHECK
To target actual hotspots instead of guessing.

WHY
Measurement guides effort.

Mastery: Explain without notes + solve one fresh problem + predict two outputs + tell one edge case.

SOLVED PRACTICE - Advanced Python & Performance

Every coding-lab question now has its answer, expected result and a quick check - all in one place.

EASY

Create generator for squares 0..n-1.

```
def squares(n):
for i in range(n):
yield i*i
Reason: yield is lazy.
Complexity: O(n) time / O(1) extra
```

EXPECTED RESULT

Values arrive lazily, one item at a time.

EASY

Write a timing decorator skeleton.

```
def timed(fn):
def wrap(*a,**kw):
import time
t=time.perf_counter(); r=fn(*a,**kw)
return r, time.perf_counter()-t
return wrap
Reason: Measure actual runtime.
Complexity: O(1) extra
```

EXPECTED RESULT

Wrapper runs and preserves the original call result.

MEDIUM

Demonstrate shallow copy aliasing.

```
import copy
a=[[1],[2]]
b=copy.copy(a)
b[0].append(9)
print(a)
Reason: Nested list remains shared.
Complexity: O(n) copy outer
```

EXPECTED RESULT

Compare with the code and verify one edge case.

MEDIUM

Use dataclass for a record.

```
from dataclasses import dataclass
@dataclass
class User:
name:str
active:bool=True
Reason: Removes repetitive constructor/repr
boilerplate.
Complexity: O(1)
```

EXPECTED RESULT

Compare with the code and verify one edge case.

HARD

Binary search index of target in sorted list.

```
def binary_search(a,t):
lo,hi=0,len(a)-1
while lo<=hi:
m=(lo+hi)//2
if a[m]==t:return m
if a[m]<t:lo=m+1
else:hi=m-1
return -1
Reason: Each step halves the search interval.
```

EXPECTED RESULT

Index of target or -1; O(log n).

PRACTICE HABIT

Practice rule: solve first, compare second, then change one input. Mark the question only after you can explain why the output changes.

MINI PROJECT - Generator Stream

Advanced Python & Performance ka concept ek runnable, interview-ready mini build me convert karo.

YE KYA KARTA HAI?

Large sequence ko lazy way me consume karta hai.

KIS KAAM AATA HAI?

Large files, API pagination, data streams.

ARCHITECTURE

Source -> generator -> consumer

FILES / BUILD TARGET

stream.py Target: Avoid building huge lists

CORE IMPLEMENTATION - TYPE + RUN

```
def rows(n):
    for i in range(n):
        yield {"id": i}

for row in rows(3):
    print(row)
```

SAMPLE OUTPUT `{'id': 0} {'id': 1} {'id': 2}`

INTERVIEW EXPLANATION

Project answer: Generator state preserve karta hai and values one-by-one deta hai. Debug cue: Once exhausted, generator must be recreated. Follow-up: What would you improve first if this had 10x traffic/data?

BUILD STEPS

1. Create the file(s) shown above.
2. Run the exact sample once.
3. Change one input and observe the output.
4. Add one validation or failure test.

EDGE CASES / DEBUG

Test invalid input, boundary values and empty/missing data where relevant. Keep validation at the system boundary.

Advanced Python & Performance

Use this pack after the section labs: explain -> build -> debug -> defend.

SECTION ANCHOR

Definition: Advanced features ko performance, memory aur maintainability ke context me use karo.

Easy explanation: Choose the simplest tool that meets the constraint: generator, caching, concurrency.

Example: `(x for x in range(10_000_000))`

SIMPLE

Generator Stream

Kya karta hai: Large sequence ko lazy way me consume karta hai.

Kis kaam aata hai: Large files, API pagination, data streams.

Architecture: Source -> generator -> consumer

Files: `stream.py`

Build target: Avoid building huge lists

MEDIUM

Caching Decorator

Kya karta hai: Repeated pure function calls ko memoize karta hai.

Kis kaam aata hai: Expensive feature calculations and repeated lookups.

Architecture: Function -> cache key -> hit/miss -> result

Files: `cache.py`

Build target: Only cache safe deterministic calls

HARD

Concurrent Fetcher

Kya karta hai: Independent I/O tasks ko concurrent workers se process karta hai.

Kis kaam aata hai: API enrichment and batch ingestion.

Architecture: Task list -> worker pool -> results -> error collection

Files: `fetch.py`

Build target: Bound concurrency and timeout handling

Project rule: code khud type karo, output predict karo, edge case test karo, phir 60-second explanation do.

Advanced Python & Performance - implementation l

Three core implementations + project-tailored interview explanation + debugging cue.

SIMPLE | GENERATOR STREAM

```
def rows(n):
    for i in range(n):
        yield {"id":i}
for row in rows(3): print(row)
```

INTERVIEW DEFENSE

Project answer: Generator state preserve karta hai and values one-by-one deta hai.

Debug cue: Once exhausted, generator must be recreated.

Follow-up: What would you improve first if this had 10x traffic/data?

MEDIUM | CACHING DECORATOR

```
from functools import lru_cache
@lru_cache(maxsize=128)
def fib(n):
    if n<2: return n
    return fib(n-1)+fib(n-2)
```

INTERVIEW DEFENSE

Project answer: Caching trades memory for latency. Key design and invalidation matter.

Debug cue: Do not cache functions with hidden mutable dependencies.

Follow-up: What would you improve first if this had 10x traffic/data?

HARD | CONCURRENT FETCHER

```
from concurrent.futures import ThreadPoolExecutor
with ThreadPoolExecutor(max_workers=8) as ex:
    futures = [ex.submit(fetch, u) for u in urls]
    results = [f.result(timeout=5) for f in futures]
```

INTERVIEW DEFENSE

Project answer: Threads help I/O-bound work; measure before parallelizing.

Debug cue: Handle per-task timeout/errors instead of failing the entire batch blindly.

Follow-up: What would you improve first if this had 10x traffic/data?

SELF-CHECK

Can I explain the project without notes? Can I name one failure? Can I state time/space or cost trade-offs?

SECTION 04

NumPy for AI & Data Science

Level: Intermediate Time: ~100 min 9 focus topics

Numeric arrays, vectorization aur ML-ready computation ka Python base.

IS TRACK ME KYA SEEKHOGE

✓ arrays
✓ shape
✓ dtype
✓ indexing
✓ broadcasting
✓ vectorization
✓ aggregation
✓ random
✓ linear algebra basics

TOPIC ROADMAP

4.1 arrays
4.2 shape
4.3 dtype
4.4 indexing
4.5 broadcasting
4.6 vectorization
4.7 aggregation
4.8 random
4.9 linear algebra basics

INTERVIEW MINDSET

Definition -> tiny example -> practical use -> trade-off/edge case. Har answer ko apni language me bolo.

04.1 Core Concepts

NumPy ka core idea Python loops ko dense numeric arrays par optimized operations se replace karna hai. AI/data tasks me feature arrays, image tensors aur embeddings ke liye ye fundamental hai.

SHAPE / DTYPE

shape dimensions batata hai; dtype memory aur numeric behavior influence karta hai.

VECTORIZATION

Element-wise operations aur ufuncs Python loops ki need reduce karte hain.

BROADCASTING

Compatible dimensions par automatic expansion allow karta hai; shape mismatch debug karna zaroori hai.

EMBEDDINGS

Embedding vectors par dot product/cosine similarity jaisi operations NumPy se samajho.

example.py

```
import numpy as np

x = np.array([[1, 2], [3, 4]])
weights = np.array([0.1, 0.2])
scores = x @ weights
print(scores)
print(x.shape, x.dtype)
```

OUTPUT

```
[0.5 1.1]
(2, 2) int64
```

YAAD RAKHO
Interview tip: shape mismatch ko mentally calculate karna aur vectorized alternative batana strong answer hai.

04.2 Interview Depth

Is page ka goal hai concept ko implementation decision tak le jana. Interviewer ko syntax se zyada reasoning, edge cases aur trade-offs dikhte hain.

WHAT

Kya hai aur kab use karoge?

WHY

Kaunsi problem solve hoti hai?

RISK

Common bug / failure mode kya hai?

TRADE-OFF

Simple option aur scalable option me kya difference hai?

YAAD RAKHO
One-line answer practice: definition + example + use + limitation.

INTERVIEW LAB / NumPy for AI & Data Science

Definition -> tiny example -> practical use -> trade-off. Word-by-word memorize mat karo.

Q01

NumPy list se different kyun?

SMART ANSWER

NumPy arrays homogeneous numeric storage aur vectorized operations provide karte hain, jo dense numeric work ke liye efficient hota hai.

TIP dtype aur shape mention karo.

Q02

Broadcasting kya hai?

SMART ANSWER

Compatible shapes ke arrays ko element-wise operations me automatically align karna.

TIP 2x2 + vector example explain karo.

Q03

shape mismatch debug kaise?

SMART ANSWER

Har operand ka shape print/check karo, intended matrix multiplication dimensions compare karo.

TIP X.shape and W.shape bolna.

Q04

Vectorization kyun?

SMART ANSWER

Python-level loops reduce karke array operations use karte hain; code concise aur numeric workloads efficient hote hain.

TIP Loop vs x+1 example do.

Q05

dtype important kyun?

SMART ANSWER

Numeric precision, memory aur allowed operations affect karta hai.

TIP float32 vs float64 conceptually explain karo.

Q06

Embedding similarity ka basic formula?

SMART ANSWER

Cosine similarity = dot(a,b) / (||a|| ||b||). Zero norm ko handle karna chahiye.

TIP Our code example reference karo.

Speaking drill: Har answer 2 baar bolo. Second round me ek project example add karo.

CODING LAB / NumPy for AI & Data Science

Pehle problem ko 10 sec me explain karo -> then code -> edge case -> complexity.

EASY

Create array and multiply every value by 2.

ANSWER

```
import numpy as np
a=np.array([1,2,3])
print(a*2)
```

Reason: Vectorized element-wise operation.
Complexity: O(n)

EASY

Column mean of 2D array.

ANSWER

```
import numpy as np
x=np.array([[1,2],[3,4]])
print(x.mean(axis=0))
```

Reason: axis=0 reduces rows, keeping columns.
Complexity: O(n)

MEDIUM

Cosine similarity with NumPy.

ANSWER

```
dot=a@b
score=dot/(np.linalg.norm(a)*np.linalg.norm(b))
```

Reason: Guard zero norms.
Complexity: O(d)

MEDIUM

Normalize rows to unit length.

ANSWER

```
norms=np.linalg.norm(x,axis=1,keepdims=True)
y=x/np.where(norms==0,1,norms)
```

Reason: keepdims preserves broadcastable shape.
Complexity: O(n*d)

HARD

Explain shape of A @ B. If A is (m,k), B must be (k,n), result is (m,n).

ANSWER

```
A=np.zeros((2,3))
B=np.zeros((3,4))
C=A@B
```

Reason: Matrix multiplication dimension rule.
Complexity: O(m*k*n)

DEBUG

Why can broadcasting fail for (3,4) and (3,)?

ANSWER

```
x.shape=(3,4), v.shape=(3,)
```

Reason: Last dimensions 4 and 3 do not align. Reshape v to (3,1) for row-wise addition if intended.
Complexity: Depends

LOGIC LAB / NumPy for AI & Data Science

Code run karne se pehle output predict karo. Phir dry-run karke reason explain karo.

PREDICT

x=np.array([1,2,3]); print(x*2)

CHECK
[2 4 6]

WHY
Vectorized element-wise multiply.

TRACE

a=np.zeros((2,3)); b=np.ones((3,)); a+b shape?

CHECK
(2,3)

WHY
Vector broadcasts across rows.

THINK

A(4,2) @ B(2,5) -> shape?

CHECK
(4,5)

WHY
Inner dimensions match 2.

FIX

A(3,4)+v(3,)

CHECK
Broadcast mismatch on last axis

WHY
Use v[:,None] for column-wise addition.

PREDICT

np.array([1,2,3]).mean()

CHECK
2.0

WHY
Arithmetic mean.

EXPLAIN

Why keep track of dtype?

CHECK
Precision and memory affect model/data computations.

WHY
Wrong dtype can create overflow or unwanted casts in real workflows.

Mastery: Explain without notes + solve one fresh problem + predict two outputs + tell one edge case.

SECTION 05

Pandas & Exploratory Data Analysis

Level: Intermediate Time: ~110 min 9 focus topics

Data cleaning, transformation, aggregation aur EDA ka interview-ready workflow.

IS TRACK ME KYA SEEKHOGE

✓ Series/DataFrame
✓ loc/iloc
✓ missing values
✓ groupby
✓ merge/join
✓ sort
✓ pivot
✓ datetime
✓ EDA

TOPIC ROADMAP

5.1 Series/DataFrame
5.2 loc/iloc
5.3 missing values
5.4 groupby
5.5 merge/join
5.6 sort
5.7 pivot
5.8 datetime
5.9 EDA

INTERVIEW MINDSET

Definition -> tiny example -> practical use -> trade-off/edge case. Har answer ko apni language me bolo.

05.1 Core Concepts

Pandas ko sirf syntax ki list mat samjho. Real analysis ka flow hota hai: inspect -> clean -> transform -> aggregate -> validate -> explain. Har step ka business purpose samjho.

INSPECT

head(), info(), describe(), isna().sum() se data ka shape aur quality dekho.

TRANSFORM

assign, rename, astype, fillna, drop_duplicates aur vectorized string/date operations use karo.

GROUPBY

Groupby + agg se KPI tables banti hain. Multiple aggregations ko named outputs ke saath readable rakho.

MERGE

Keys validate karo. One-to-one vs one-to-many relationship samajhkar accidental row explosion avoid karo.

example.py

```
import pandas as pd

df = pd.DataFrame({"team":["A","A","B"], "score":[10,20,15]})
summary = (df.groupby("team", as_index=False)
             .agg(avg_score=("score", "mean"),
                  total=("score", "sum")))
print(summary.to_string(index=False))
```

OUTPUT

```
team  avg_score  total
A         15.0     30
B         15.0     15
```

YAAD RAKHO
EDA answer formula: observation -> evidence -> likely reason -> action.

05.2 Interview Depth

Is page ka goal hai concept ko implementation decision tak le jana. Interviewer ko syntax se zyada reasoning, edge cases aur trade-offs dikhte hain.

WHAT

Kya hai aur kab use karoge?

WHY

Kaunsi problem solve hoti hai?

RISK

Common bug / failure mode kya hai?

TRADE-OFF

Simple option aur scalable option me kya difference hai?

YAAD RAKHO
One-line answer practice: definition + example + use + limitation.

INTERVIEW LAB / Pandas & Exploratory Data Analysis

Definition -> tiny example -> practical use -> trade-off. Word-by-word memorize mat karo.

Q01

loc vs iloc?

SMART ANSWER

loc label-based selection hai; iloc integer-position based selection.

TIP One row example bolo.

Q02

groupby ka mental model?

SMART ANSWER

Split data into groups -> apply aggregation/transformation -> combine result.

TIP Revenue by customer example do.

Q03

merge me row explosion kya?

SMART ANSWER

One-to-many keys duplicate left rows according to matches; counts/revenue inflate ho sakte hain.

TIP Key cardinality validate karo.

Q04

Missing values strategy?

SMART ANSWER

Drop, fill, impute ya model-specific treatment context par depend karta hai. Bas blindly fillna(0) mat karo.

TIP Meaning of missing samjho.

Q05

EDA ka output kya?

SMART ANSWER

Data quality findings, distributions, relationships, anomalies aur next analysis/model decisions.

TIP Only charts nahi.

Q06

Datetime handling?

SMART ANSWER

Parse to datetime dtype, then extract useful features or resample with clear timezone semantics where needed.

TIP to_datetime mention karo.

Speaking drill: Har answer 2 baar bolo. Second round me ek project example add karo.

CODING LAB / Pandas & Exploratory Data Analysis

Pehle problem ko 10 sec me explain karo -> then code -> edge case -> complexity.

EASY

Filter rows where score >= 50.

ANSWER

```
df[df["score"]>=50]
```

Reason: Boolean mask selects rows.
Complexity: O(n)

EASY

Group average by team.

ANSWER

```
df.groupby("team",as_index=False)["score"
].mean()
```

Reason: Group then aggregate.
Complexity: O(n)

MEDIUM

Fill a numeric missing value with median.

ANSWER

```
median=df["age"].median()
df["age"]=df["age"].fillna(median)
```

Reason: Median is robust to outliers compared with mean.
Complexity: O(n)

MEDIUM

Merge customers with orders.

ANSWER

```
out=customers.merge(orders,on="customer_i
d",how="left")
```

Reason: Left join preserves customers.
Complexity: O(n) average, data-dependent

HARD

Find top customer by total revenue.

ANSWER

```
s=(orders.groupby("customer_id")["amount"
].sum()
.sort_values(ascending=False))
print(s.index[0])
```

Reason: Aggregate first, then sort.
Complexity: O(n log n) due to sort

DEBUG

Why can groupby sum be wrong after a many-to-many merge?

ANSWER

```
Inspect key uniqueness before merge.
```

Reason: Duplicate matches can multiply rows and revenue.
Complexity: Data-dependent

LOGIC LAB / Pandas & Exploratory Data Analysis

Code run karne se pehle output predict karo. Phir dry-run karke reason explain karo.

PREDICT

df["score"].isna().sum()

CHECK
Number of missing score values

WHY
isna produces a boolean mask then sum counts True.

TRACE

groupby("team")["score"].mean()

CHECK
One average per team

WHY
Aggregation changes grain to team.

THINK

merge 1 customer with 5 orders

CHECK
Customer row can appear 5 times

WHY
One-to-many relationship expands rows.

FIX

Fill all missing categories with 0?

CHECK
Usually unsafe without meaning check

WHY
0 may mean a real category/value rather than unknown.

PREDICT

sort_values("revenue", ascending=False).iloc[0]

CHECK
Top revenue row

WHY
Sort descending and take first.

EXPLAIN

What makes EDA useful?

CHECK
It converts raw data into validated patterns and questions.

WHY
EDA should influence next decisions.

Mastery: Explain without notes + solve one fresh problem + predict two outputs + tell one edge case.

SECTION 06

SQL + Data Analytics with Python

Level: Intermediate Time: ~100 min 9 focus topics

SQL thinking ko Python analytics ke saath connect karo.

IS TRACK ME KYA SEEKHOGE

✓ SELECT/WHERE
✓ GROUP BY
✓ JOIN
✓ CASE
✓ subqueries/CTE
✓ window functions
✓ NULL
✓ indexing basics
✓ KPI logic

TOPIC ROADMAP

6.1 SELECT/WHERE
6.2 GROUP BY
6.3 JOIN
6.4 CASE
6.5 subqueries/CTE
6.6 window functions
6.7 NULL
6.8 indexing basics
6.9 KPI logic

INTERVIEW MINDSET

Definition -> tiny example -> practical use -> trade-off/edge case. Har answer ko apni language me bolo.

06.1 Core Concepts

AI Engineer aur Data Scientist ko SQL se data nikalna aana chahiye. Interview me query syntax se zyada important hota hai data grain, joins aur aggregation ko sahi samajhna.

GRAIN

Har row represent kya karti hai? Order, customer, event ya day? Grain clear na ho to KPI galat ho sakta hai.

JOINS

INNER rows match only; LEFT keeps all left rows. One-to-many join duplicate counts create kar sakta hai.

WINDOW FUNCTIONS

ROW_NUMBER, RANK, SUM OVER jaise functions group ko collapse kiye bina analytics dete hain.

NULL

NULL = unknown. = NULL nahi; IS NULL / IS NOT NULL use karo. Aggregates me NULL behavior samjho.

example.py

```
SELECT customer_id,
       COUNT(*) AS orders,
       SUM(amount) AS revenue
FROM orders
WHERE status = 'paid'
GROUP BY customer_id
HAVING SUM(amount) > 10000
ORDER BY revenue DESC;
```

OUTPUT

```
Top customers with paid revenue > 10000
```

YAAD RAKHO
SQL answer me query ke saath explain karo: data grain, filter stage, aggregation stage, result interpretation.

06.2 Interview Depth

Is page ka goal hai concept ko implementation decision tak le jana. Interviewer ko syntax se zyada reasoning, edge cases aur trade-offs dikhte hain.

WHAT

Kya hai aur kab use karoge?

WHY

Kaunsi problem solve hoti hai?

RISK

Common bug / failure mode kya hai?

TRADE-OFF

Simple option aur scalable option me kya difference hai?

YAAD RAKHO
One-line answer practice: definition + example + use + limitation.

INTERVIEW LAB / SQL + Data Analytics with Python

Definition -> tiny example -> practical use -> trade-off. Word-by-word memorize mat karo.

Q01

WHERE aur HAVING?

SMART ANSWER

WHERE rows ko grouping se pehle filter karta hai; HAVING grouped aggregate results ko filter karta hai.

TIP COUNT/SUM example do.

Q02

INNER vs LEFT JOIN?

SMART ANSWER

INNER only matching rows; LEFT keeps all left rows and unmatched right values become NULL.

TIP Use case explain karo.

Q03

Window function kyun?

SMART ANSWER

Group collapse kiye bina per-row analytics, ranking ya running total provide karti hai.

TIP ROW_NUMBER example do.

Q04

NULL ko = NULL kyun nahi?

SMART ANSWER

NULL unknown state hai; equality comparison NULL ke saath expected boolean nahi deta. IS NULL use karo.

TIP Small query bolo.

Q05

Index ka trade-off?

SMART ANSWER

Reads/filtering speed up ho sakti hai, but storage aur write/update cost add hoti hai.

TIP Selective columns concept.

Q06

SQL result validate kaise?

SMART ANSWER

Row counts, duplicates, nulls, join cardinality aur sample records check karo; KPI totals independently verify karo.

TIP Validation mindset.

Speaking drill: Har answer 2 baar bolo. Second round me ek project example add karo.

CODING LAB / SQL + Data Analytics with Python

Pehle problem ko 10 sec me explain karo -> then code -> edge case -> complexity.

EASY

Count paid orders.

ANSWER

```
SELECT COUNT(*) FROM orders WHERE
status='paid';
```

Reason: Filter then count.
Complexity: O(n) scan if no index

EASY

Revenue by customer.

ANSWER

```
SELECT customer_id, SUM(amount) AS
revenue
FROM orders GROUP BY customer_id;
```

Reason: Grouping defines KPI grain.
Complexity: O(n)

MEDIUM

Customers with revenue > 10000.

ANSWER

```
SELECT customer_id, SUM(amount) revenue
FROM orders GROUP BY customer_id
HAVING SUM(amount)>10000;
```

Reason: HAVING filters aggregate groups.
Complexity: O(n) + grouping

MEDIUM

Latest order per customer concept.

ANSWER

```
ROW_NUMBER() OVER (PARTITION BY
customer_id ORDER BY created_at DESC)
```

Reason: Window ranks rows within each customer.
Complexity: Depends

HARD

Running revenue.

ANSWER

```
SUM(amount) OVER (ORDER BY created_at
ROWS UNBOUNDED PRECEDING)
```

Reason: Window aggregate keeps row-level detail.
Complexity: Depends

DEBUG

Why does LEFT JOIN + WHERE right_table.col = ... act like inner join?

ANSWER

```
The WHERE removes NULL-extended unmatched
rows.
```

Reason: Put the right-side filter in ON when preserving unmatched left rows is required.
Complexity: Conceptual

LOGIC LAB / SQL + Data Analytics with Python

Code run karne se pehle output predict karo. Phir dry-run karke reason explain karo.

PREDICT

COUNT(*) after WHERE status=paid

CHECK
Count of filtered paid rows

WHY
WHERE runs before aggregation conceptually.

TRACE

HAVING SUM(amount)>1000

CHECK
Groups above threshold

WHY
HAVING filters aggregate results.

THINK

LEFT JOIN + WHERE right.id IS NOT NULL

CHECK
Effectively inner for that condition

WHY
WHERE removes unmatched NULLs.

FIX

COUNT(column_with_nulls)

CHECK
Counts non-NULL values

WHY
COUNT(*) and COUNT(col) differ.

PREDICT

ROW_NUMBER() OVER(PARTITION BY customer ORDER BY date DESC)

CHECK
1 marks latest row per customer

WHY
Ranking is within each partition.

EXPLAIN

Why validate SQL?

CHECK
Queries can be syntactically correct but semantically wrong.

WHY
Check grain, counts and totals.

Mastery: Explain without notes + solve one fresh problem + predict two outputs + tell one edge case.

SECTION 07

Machine Learning with Python

Level: Intermediate Time: ~120 min 8 focus topics

Data Scientist / AI Engineer interviews ke ML fundamentals aur implementation thinking.

IS TRACK ME KYA SEEKHOGE

✓ problem framing
✓ train/validation/test
✓ leakage
✓ preprocessing
✓ metrics
✓ baseline
✓ feature engineering
✓ pipelines

TOPIC ROADMAP

7.1 problem framing
7.2 train/validation/test
7.3 leakage
7.4 preprocessing
7.5 metrics
7.6 baseline
7.7 feature engineering
7.8 pipelines

INTERVIEW MINDSET

Definition -> tiny example -> practical use -> trade-off/edge case. Har answer ko apni language me bolo.

07.1 Core Concepts

ML interview ka best pattern hai: problem define -> target define -> split -> baseline -> preprocessing -> train -> validate -> metric -> error analysis -> iterate. Model name baad me aata hai.

LEAKAGE

Future/target-derived information training me aa gayi to validation unrealistically strong ho sakti hai.

METRICS

Classification me precision/recall/F1/ROC-AUC context par depend karte hain. Regression me MAE/MSE/RMSE common hain.

BASELINE

Simple baseline se compare karo. Baseline ke bina improvement ka meaning weak hota hai.

PIPELINE

Preprocessing + model ko single reproducible pipeline me rakhna train/test contamination reduce karta hai.

example.py

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ("scale", StandardScaler()),
    ("model", LogisticRegression(max_iter=1000)),
])
# X, y ko pehle train/test me split karke fit karo
# pipe.fit(X_train, y_train)
# pred = pipe.predict(X_test)
```

OUTPUT

```
Pipeline preprocessing ko model ke saath bind karta hai
```

YAAD RAKHO
Interview me metric choice ko business cost se connect karo; sirf formula mat bolo.

07.2 Interview Depth

Is page ka goal hai concept ko implementation decision tak le jana. Interviewer ko syntax se zyada reasoning, edge cases aur trade-offs dikhte hain.

WHAT

Kya hai aur kab use karoge?

WHY

Kaunsi problem solve hoti hai?

RISK

Common bug / failure mode kya hai?

TRADE-OFF

Simple option aur scalable option me kya difference hai?

YAAD RAKHO
One-line answer practice: definition + example + use + limitation.

INTERVIEW LAB / Machine Learning with Python

Definition -> tiny example -> practical use -> trade-off. Word-by-word memorize mat karo.

Q01

Data leakage kya?

SMART ANSWER

Training process me information ka use jo real prediction time par available nahi hogi. Validation result misleadingly strong ho sakta hai.

TIP Target leakage example do.

Q02

Train/validation/test?

SMART ANSWER

Train fit karta hai, validation model/threshold selection me help karta hai, test final unbiased estimate ke liye preserve karte hain.

TIP Do not tune on test.

Q03

Precision vs recall?

SMART ANSWER

Precision predicted positives me correctness, recall actual positives me coverage. Use-case ke false-positive/false-negative cost se choose karo.

TIP Fraud/spam examples.

Q04

Baseline kyun?

SMART ANSWER

Simple benchmark ke bina model improvement ka context weak hota hai.

TIP Mean predictor or simple classifier.

Q05

Pipeline kyun?

SMART ANSWER

Preprocessing and model ek reproducible object me bind hote hain; leakage risk kam hota hai.

TIP Scaler before model.

Q06

Error analysis kya?

SMART ANSWER

Wrong predictions ko categories me todkar pattern identify karna and next feature/model/data action decide karna.

TIP Confusion matrix buckets.

Speaking drill: Har answer 2 baar bolo. Second round me ek project example add karo.

CODING LAB / Machine Learning with Python

Pehle problem ko 10 sec me explain karo -> then code -> edge case -> complexity.

EASY

Split numeric data into train/test.

ANSWER

```
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)
```

Reason: Keeps evaluation data separate.
Complexity: O(n)

EASY

Standardize then fit model using Pipeline.

ANSWER

```
Pipeline([("scale",StandardScaler()),("model",LogisticRegression())])
```

Reason: Preprocessing stays inside fit/predict flow.
Complexity: Model-dependent

MEDIUM

Compute precision manually from TP,FP.

ANSWER

```
precision=TP/(TP+FP) if TP+FP else 0
```

Reason: Guard zero denominator.
Complexity: O(1)

MEDIUM

Choose MAE vs RMSE.

ANSWER

```
MAE=sum(abs(e))/n
RMSE=(sum(e*e)/n)**0.5
```

Reason: RMSE penalizes larger errors more strongly.
Complexity: O(n)

HARD

Design leakage-safe feature workflow.

ANSWER

```
Split first -> fit transforms on train only -> transform validation/test -> fit model.
```

Reason: Never learn preprocessing parameters from test.
Complexity: O(n)

DEBUG

Test accuracy is 99%. What do you inspect?

ANSWER

```
Class balance, leakage, split strategy, duplicates, baseline, confusion matrix, data shift.
```

Reason: A single metric can hide data or evaluation problems.
Complexity: Reasoning

LOGIC LAB / Machine Learning with Python

Code run karne se pehle output predict karo. Phir dry-run karke reason explain karo.

PREDICT

Train on leaked future feature

CHECK
Unrealistically strong validation possible

WHY
Leakage violates real-world availability assumptions.

TRACE

Accuracy 99% on 99:1 imbalance

CHECK
Could be poor positive-class recall

WHY
Inspect confusion matrix and class-aware metrics.

THINK

Fit scaler on full dataset before split

CHECK
Leakage risk

WHY
Validation/test statistics leak into training transform.

FIX

No baseline, model 90%

CHECK
Cannot tell if 90% is useful

WHY
Compare a simple baseline.

PREDICT

MAE=0.5 vs RMSE=1.5

CHECK
Large errors are penalized more by RMSE

WHY
Squaring increases influence of outliers.

EXPLAIN

Best next step after weak model?

CHECK
Error analysis and data/feature review

WHY
Not always a bigger model.

Mastery: Explain without notes + solve one fresh problem + predict two outputs + tell one edge case.

SECTION 08

Generative AI, LLMs & RAG

Level: Advanced Time: ~125 min 11 focus topics

LLM applications ko engineering perspective se samjho: prompts, context, retrieval, tools aur evalua

IS TRACK ME KYA SEEKHOGE

✓ tokens
✓ context
✓ prompting
✓ structured output
✓ embeddings
✓ chunking
✓ retrieval
✓ reranking concept
✓ RAG
✓ hallucination
✓ evaluation

TOPIC ROADMAP

8.1 tokens
8.2 context
8.3 prompting
8.4 structured output
8.5 embeddings
8.6 chunking
8.7 retrieval
8.8 reranking concept
8.9 RAG
8.10 hallucination
8.11 evaluation

INTERVIEW MINDSET

Definition -> tiny example -> practical use -> trade-off/edge case. Har answer ko apni language me bolo.

08.1 Core Concepts

LLM app me model se sirf prompt bhejna enough nahi. Quality ka system input validation, prompt design, relevant context retrieval, output validation, evaluation aur observability par depend karta hai.

RAG

External knowledge ko retrieve karke prompt/context me add karte ho, taaki model ko relevant evidence mil sake.

CHUNKING

Too-small chunks context lose kar sakte hain; too-large chunks noise badha sakte hain. Domain-based chunking socho.

EMBEDDINGS

Text ko vector space me map karte hain. Similar meanings often nearby vectors produce karte hain.

EVALUATION

Groundedness, retrieval relevance, answer correctness, latency aur cost ko separately measure karo.

example.py

```
from math import sqrt

def cosine(a, b):
    dot = sum(x*y for x, y in zip(a, b))
    na = sqrt(sum(x*x for x in a))
    nb = sqrt(sum(y*y for y in b))
    if na == 0 or nb == 0:
        return 0.0
    return dot / (na * nb)

print(round(cosine([1,0],[0.9,0.1]), 3))
```

OUTPUT

```
0.994
```

YAAD RAKHO
RAG is not a guarantee of truth. Retrieval quality, source quality and answer validation all matter.

08.2 Interview Depth

Is page ka goal hai concept ko implementation decision tak le jana. Interviewer ko syntax se zyada reasoning, edge cases aur trade-offs dikhte hain.

WHAT

Kya hai aur kab use karoge?

WHY

Kaunsi problem solve hoti hai?

RISK

Common bug / failure mode kya hai?

TRADE-OFF

Simple option aur scalable option me kya difference hai?

example.py

```
from math import sqrt

def cosine(a, b):
    dot = sum(x*y for x, y in zip(a, b))
    na = sqrt(sum(x*x for x in a))
    nb = sqrt(sum(y*y for y in b))
    if na == 0 or nb == 0:
        return 0.0
    return dot / (na * nb)

print(round(cosine([1,0],[0.9,0.1]), 3))
```

OUTPUT

```
0.994
```

YAAD RAKHO
One-line answer practice: definition + example + use + limitation.

INTERVIEW LAB / Generative AI, LLMs & RAG

Definition -> tiny example -> practical use -> trade-off. Word-by-word memorize mat karo.

Q01

Prompt engineering vs RAG?

SMART ANSWER

Prompt engineering instruction/context style improve karta hai. RAG external relevant information retrieve karta hai. Dono complementary hain.

TIP Need-based choice explain karo.

Q02

Embedding kya?

SMART ANSWER

Information ko numeric vector representation me map karna, jisse similarity search jaise operations possible hote hain.

TIP Cosine similarity reference.

Q03

Hallucination kaise reduce?

SMART ANSWER

Grounded context, clear instructions, tool/source constraints, output validation and evaluation use karo.

TIP No absolute guarantee.

Q04

Chunking trade-off?

SMART ANSWER

Small chunks precise but fragmented; large chunks have more context but can add noise.

TIP Overlap concept mention karo.

Q05

RAG pipeline steps?

SMART ANSWER

Ingest -> clean -> chunk -> embed -> index -> retrieve -> optionally rerank -> prompt/context -> generate -> evaluate.

TIP Full pipeline 10 sec me bolo.

Q06

LLM app evaluation?

SMART ANSWER

Task correctness, groundedness, retrieval quality, latency, cost and safety should be measured separately.

TIP Offline eval + sampled review.

Speaking drill: Har answer 2 baar bolo. Second round me ek project example add karo.

CODING LAB / Generative AI, LLMs & RAG

Pehle problem ko 10 sec me explain karo -> then code -> edge case -> complexity.

EASY

Compute cosine similarity.

ANSWER

```
def cosine(a,b):
dot=sum(x*y for x,y in zip(a,b))
return dot/(norm(a)*norm(b))
```

Reason: Add zero-norm guard.
Complexity: O(d)

EASY

Normalize a query before vector search conceptually.

ANSWER

```
q = q / (np.linalg.norm(q) or 1)
```

Reason: Unit vector simplifies cosine ranking if vectors are normalized.
Complexity: O(d)

MEDIUM

RAG retrieval contract: return top-k docs with score and id.

ANSWER

```
[(doc_id, score, text) for
doc_id,score,text in ranked[:k]]
```

Reason: Keep source metadata for traceability.
Complexity: O(n log n) sorting; can be optimized

MEDIUM

Validate structured output.

ANSWER

```
required={"answer","sources"}
if not required <= data.keys():
raise ValueError("schema error")
```

Reason: Never assume model output matches schema.
Complexity: O(k)

HARD

RAG pipeline pseudo-flow.

ANSWER

```
docs=ingest()
chunks=chunk(docs)
index=embed_and_index(chunks)
ctx=retrieve(query,index)
answer=generate(query,ctx)
```

Reason: Separate retrieval quality from generation quality.
Complexity: Pipeline-dependent

DEBUG

Why does adding more context sometimes reduce answer quality?

ANSWER

```
More context can add distractors,
conflicts, or exceed useful attention
budget.
```

Reason: Retrieve relevant evidence, not everything.
Complexity: Conceptual

LOGIC LAB / Generative AI, LLMs & RAG

Code run karne se pehle output predict karo. Phir dry-run karke reason explain karo.

PREDICT

cosine([1,0],[0,1])

CHECK
0

WHY
Orthogonal vectors have zero dot product.

TRACE

RAG retrieves irrelevant docs

CHECK
Answer can still be fluent but wrong

WHY
Generation quality cannot fix bad evidence.

THINK

More retrieved docs always better?

CHECK
No

WHY
Noise and conflicts can hurt quality.

FIX

Model returns invalid JSON

CHECK
Validate and retry/repair within bounded policy

WHY
Treat model output as untrusted input.

PREDICT

normalized vectors with highest dot product

CHECK
Highest cosine similarity

WHY
Normalization makes dot product equivalent to cosine.

EXPLAIN

Why keep source IDs?

CHECK
Traceability, citations and debugging

WHY
You need to inspect where an answer came from.

Mastery: Explain without notes + solve one fresh problem + predict two outputs + tell one edge case.

SECTION 09

Agentic AI & Automation

Level: Advanced Time: ~125 min 10 focus topics

Tools, state, planning aur workflows ke saath reliable AI agents design karo.

IS TRACK ME KYA SEEKHOGE

✓ agent loop
✓ tool calling
✓ state
✓ memory
✓ routing
✓ retries
✓ guardrails
✓ human-in-loop
✓ n8n/webhooks
✓ observability

TOPIC ROADMAP

9.1 agent loop
9.2 tool calling
9.3 state
9.4 memory
9.5 routing
9.6 retries
9.7 guardrails
9.8 human-in-loop
9.9 n8n/webhooks
9.10 observability

INTERVIEW MINDSET
Definition -> tiny example -> practical use -> trade-off/edge case. Har answer ko apni language me bolo.

09.1 Core Concepts

Agent ko chatbot samajhne ki galti mat karo. Agentic system environment observe karta hai, decide karta hai, tool invoke karta hai, result inspect karta hai aur next step choose karta hai - within defined boundaries.

TOOL CALLING

Tool ka name, input schema aur output contract clear rakho. Invalid arguments ko reject karo.

STATE

Conversation, task status, retrieved docs aur tool results ka explicit state model useful hota hai.

RETRIES

Transient failure retry ho sakta hai; permanent validation error repeat nahi karna chahiye.

HUMAN-IN-LOOP

High-risk actions, irreversible updates ya ambiguous cases me approval gate design karo.

example.py

```
from dataclasses import dataclass, field

@dataclass
class AgentState:
    goal: str
    steps: list[str] = field(default_factory=list)
    done: bool = False

def add_step(state: AgentState, step: str) -> AgentState:
    state.steps.append(step)
    return state

s = AgentState("Create report")
```

OUTPUT

```
['fetch_data', 'summarize']
```

YAAD RAKHO
Reliable agent = clear tools + bounded state + validation + observability + recovery strategy.

09.2 Interview Depth

Is page ka goal hai concept ko implementation decision tak le jana. Interviewer ko syntax se zyada reasoning, edge cases aur trade-offs dikhte hain.

WHAT

Kya hai aur kab use karoge?

WHY

Kaunsi problem solve hoti hai?

RISK

Common bug / failure mode kya hai?

TRADE-OFF

Simple option aur scalable option me kya difference hai?

example.py

```
from dataclasses import dataclass, field

@dataclass
class AgentState:
    goal: str
    steps: list[str] = field(default_factory=list)
    done: bool = False

def add_step(state: AgentState, step: str) -> AgentState:
    state.steps.append(step)
    return state

s = AgentState("Create report")
```

OUTPUT

```
['fetch_data', 'summarize']
```

YAAD RAKHO
One-line answer practice: definition + example + use + limitation.

INTERVIEW LAB / Agentic AI & Automation

Definition -> tiny example -> practical use -> trade-off. Word-by-word memorize mat karo.

Q01

Agent aur chatbot me difference?

SMART ANSWER

Chatbot often response-focused hota hai; agentic system goal ke liye state, tools aur actions use karke multi-step work execute kar sakta hai.

TIP Boundaries mention karo.

Q02

Tool schema kyun?

SMART ANSWER

Agent ko valid arguments aur expected result shape clear milta hai; validation easier hoti hai.

TIP JSON schema idea.

Q03

Agent loop kya?

SMART ANSWER

Observe state -> decide next action -> call tool -> inspect result -> update state -> stop or continue.

TIP Add max steps.

Q04

Retries kab?

SMART ANSWER

Transient timeouts/5xx jaisi recoverable failures. Invalid inputs ko retry karna usually pointless hai.

TIP Backoff concept.

Q05

Human-in-loop kab?

SMART ANSWER

High-impact, irreversible, sensitive ya ambiguous actions se pehle approval gate.

TIP Send message/delete record example.

Q06

Agent observability me kya?

SMART ANSWER

Inputs, tool calls, latency, errors, state transitions, token/cost metadata and final outcome - without logging secrets.

TIP Traceability matters.

Speaking drill: Har answer 2 baar bolo. Second round me ek project example add karo.

CODING LAB / Agentic AI & Automation

Pehle problem ko 10 sec me explain karo -> then code -> edge case -> complexity.

EASY

Define bounded agent state.

ANSWER

```
from dataclasses import dataclass
@dataclass
class State:
goal:str
steps:list
done:bool=False
```

Reason: Explicit state is easier to debug.
Complexity: O(1) object creation

EASY

Tool call should validate inputs.

ANSWER

```
if not isinstance(amount,(int,float)) or
amount<0:
raise ValueError("invalid amount")
```

Reason: Validate before external side effects.
Complexity: O(1)

MEDIUM

Implement max-step guard.

ANSWER

```
for step in range(8):
action=decide(state)
if action=="done": break
state=execute(action,state)
```

Reason: Bound loops to avoid runaway agents.
Complexity: O(steps)

MEDIUM

Retry only transient failure.

ANSWER

```
for attempt in range(3):
try: return call()
except TimeoutError: continue
raise RuntimeError("failed")
```

Reason: Do not retry validation errors.
Complexity: O(retries)

HARD

Human approval gate for irreversible action.

ANSWER

```
if action.is_irreversible:
require_approval(action)
execute(action)
```

Reason: Approval boundary belongs before side effect.
Complexity: O(1) gate

DEBUG

Agent repeats same failing tool call. What do you add?

ANSWER

```
Capture error state, classify
retryability, increment retry count,
alter plan or stop.
```

Reason: Feedback without state update creates loops.
Complexity: Conceptual

LOGIC LAB / Agentic AI & Automation

Code run karne se pehle output predict karo. Phir dry-run karke reason explain karo.

PREDICT

Agent tool limit = 5; action never done

CHECK
Stop safely at limit

WHY
Bound loops.

TRACE

Tool returns validation error twice

CHECK
Do not blindly retry

WHY
Change input/plan or stop.

THINK

Irreversible action without approval

CHECK
Risky architecture

WHY
Add human gate or policy check.

FIX

Agent repeats same tool call

CHECK
Persist error/retry count in state

WHY
The system needs feedback memory.

PREDICT

Transient timeout

CHECK
Retry with bounded attempts/backoff

WHY
Timeout may be recoverable.

EXPLAIN

Best agent architecture?

CHECK
Clear tools + bounded state + validation + observability

WHY
Avoid unconstrained autonomy.

Mastery: Explain without notes + solve one fresh problem + predict two outputs + tell one edge case.

SECTION 10

APIs, Flask/FastAPI & Production Python

Level: Advanced Time: ~105 min 9 focus topics

AI services ko real applications me expose, validate aur monitor karna.

IS TRACK ME KYA SEEKHOGE

✓ HTTP/REST
✓ JSON
✓ auth basics
✓ webhooks
✓ Flask/FastAPI concepts
✓ validation
✓ async I/O
✓ env vars
✓ testing

TOPIC ROADMAP

10.1 HTTP/REST
10.2 JSON
10.3 auth basics
10.4 webhooks
10.5 Flask/FastAPI concepts
10.6 validation
10.7 async I/O
10.8 env vars
10.9 testing

INTERVIEW MINDSET

Definition -> tiny example -> practical use -> trade-off/edge case. Har answer ko apni language me bolo.

10.1 Core Concepts

AI Engineer interview me ek common discussion hoti hai: model ko production app me kaise expose karoge? Answer should cover API contract, validation, authentication, timeouts, error codes, logging, rate limits aur tests.

REST CONTRACT

Method + route + request schema + response schema + status codes clear rakho.

VALIDATION

User input ko trust mat karo. Type, required fields, ranges aur allowed values validate karo.

TIMEOUTS

External model/API calls unlimited wait nahi karne chahiye. Timeouts aur retries define karo.

SECRETS

API keys code me hardcode mat karo. Environment/config secret store use karo; logs me mask karo.

example.py

```
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.post("/score")
def score():
    data = request.get_json(silent=True) or {}
    value = data.get("value")
    if not isinstance(value, (int, float)):
        return jsonify({"error":"value must be numeric"}), 400
    return jsonify({"score": value * 2})
```

OUTPUT

```
POST /score -> JSON response
```

YAAD RAKHO
Production answer me API correctness ke saath security, timeouts, validation, tests aur observability mention karo.

10.2 Interview Depth

Is page ka goal hai concept ko implementation decision tak le jana. Interviewer ko syntax se zyada reasoning, edge cases aur trade-offs dikhte hain.

WHAT

Kya hai aur kab use karoge?

WHY

Kaunsi problem solve hoti hai?

RISK

Common bug / failure mode kya hai?

TRADE-OFF

Simple option aur scalable option me kya difference hai?

example.py

```
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.post("/score")
def score():
    data = request.get_json(silent=True) or {}
    value = data.get("value")
    if not isinstance(value, (int, float)):
        return jsonify({"error":"value must be numeric"}), 400
    return jsonify({"score": value * 2})
```

OUTPUT

```
POST /score -> JSON response
```

YAAD RAKHO
One-line answer practice: definition + example + use + limitation.

INTERVIEW LAB / APIs, Flask/FastAPI & Production Python

Definition -> tiny example -> practical use -> trade-off. Word-by-word memorize mat karo.

Q01

REST API contract me kya?

SMART ANSWER

Route, method, auth, request schema, response schema and status codes.

TIP POST /score example.

Q02

Webhook kya?

SMART ANSWER

External event hone par aapke endpoint ko HTTP request bhejna.

TIP Signature validation mention karo.

Q03

Timeout kyun?

SMART ANSWER

External dependency slow ho to request indefinitely hang nahi honi chahiye.

TIP Set bounded timeout.

Q04

Async kab?

SMART ANSWER

I/O-heavy concurrency me useful, especially many network calls. CPU-bound work ko automatically faster nahi banata.

TIP Important distinction.

Q05

Environment variables kyun?

SMART ANSWER

Config/secrets ko code se separate rakhte hain. Production secret manager often stronger option hota hai.

TIP Do not commit .env.

Q06

API test kaise?

SMART ANSWER

Happy path + invalid body + auth failure + timeout/error + boundary values + schema checks.

TIP Automated tests mention.

Speaking drill: Har answer 2 baar bolo. Second round me ek project example add karo.

CODING LAB / APIs, Flask/FastAPI & Production Python

Pehle problem ko 10 sec me explain karo -> then code -> edge case -> complexity.

EASY

Validate numeric JSON body in Flask.

ANSWER

```
data=request.get_json(silent=True) or {}
if not isinstance(data.get("value"),(int,
float)):
return {"error":"bad value"},400
```

Reason: Reject malformed input early.
Complexity: O(1)

EASY

Load config from environment.

ANSWER

```
import os
API_URL=os.getenv("API_URL","http://local
host")
```

Reason: Keep defaults safe and non-secret.
Complexity: O(1)

MEDIUM

Make an HTTP request with timeout.

ANSWER

```
import requests
r=requests.get(url,timeout=10)
r.raise_for_status()
```

Reason: Bound waiting time and surface HTTP errors.
Complexity: Network-dependent

MEDIUM

Webhook signature concept.

ANSWER

```
expected=sign(secret, raw_body)
if not secure_compare(expected, header):
reject()
```

Reason: Authenticate event source before processing.
Complexity: O(n)

HARD

API error mapping.

ANSWER

```
try:
result=service(data)
except TimeoutError:
return {"error":"upstream timeout"},504
```

Reason: Client receives meaningful status codes.
Complexity: Service-dependent

DEBUG

Why should an API never return secret exceptions verbatim?

ANSWER

```
Internal messages may reveal credentials,
paths, stack traces or implementation
details.
```

Reason: Log internally; return safe client message.
Complexity: Security reasoning

LOGIC LAB / APIs, Flask/FastAPI & Production Python

Code run karne se pehle output predict karo. Phir dry-run karke reason explain karo.

PREDICT

Malformed JSON request

CHECK
400-style client error

WHY
Client input invalid.

TRACE

Upstream timeout

CHECK
Gateway/service may return 504-style response

WHY
Server could not complete due to dependency timeout.

THINK

Secret in source code

CHECK
Credential exposure risk

WHY
Use environment/secret management.

FIX

Webhook without signature validation

CHECK
Anyone may forge events

WHY
Authenticate webhook source.

PREDICT

Request waits forever

CHECK
Resource exhaustion risk

WHY
Set timeout.

EXPLAIN

Why status codes matter?

CHECK
Clients use them to interpret success/failure/retry behavior.

WHY
Stable API contract improves integration.

Mastery: Explain without notes + solve one fresh problem + predict two outputs + tell one edge case.

SOLVED PRACTICE - APIs, Flask/FastAPI & Productio

Every coding-lab question now has its answer, expected result and a quick check - all in one place.

EASY

Validate numeric JSON body in Flask.

```
data=request.get_json(silent=True) or {}
if not isinstance(data.get("value"),(int,
float)):
return {"error":"bad value"},400
Reason: Reject malformed input early.
Complexity: O(1)
```

EXPECTED RESULT

Parsed dict or safe recovery path.

EASY

Load config from environment.

```
import os
API_URL=os.getenv("API_URL","http://local
host")
Reason: Keep defaults safe and non-secret.
Complexity: O(1)
```

EXPECTED RESULT

Compare with the code and verify one edge case.

MEDIUM

Make an HTTP request with timeout.

```
import requests
r=requests.get(url,timeout=10)
r.raise_for_status()
Reason: Bound waiting time and surface HTTP errors.
Complexity: Network-dependent
```

EXPECTED RESULT

Compare with the code and verify one edge case.

MEDIUM

Webhook signature concept.

```
expected=sign(secret, raw_body)
if not secure_compare(expected, header):
reject()
Reason: Authenticate event source before processing.
Complexity: O(n)
```

EXPECTED RESULT

Validated request/response behavior with a bounded failure path.

HARD

API error mapping.

```
try:
result=service(data)
except TimeoutError:
return {"error":"upstream timeout"},504
Reason: Client receives meaningful status codes.
Complexity: Service-dependent
```

EXPECTED RESULT

Specific exception is handled without hiding unrelated failures.

PRACTICE HABIT

Practice rule: solve first, compare second, then change one input. Mark the question only after you can explain why the output changes.

MINI PROJECT - Flask Health API

APIs, Flask/FastAPI & Production Python ka concept ek runnable, interview-ready mini build me convert karo.

YE KYA KARTA HAI?

Minimal health endpoint expose karta hai.

KIS KAAM AATA HAI?

Any web/backend project starting point.

ARCHITECTURE

HTTP -> Flask route -> JSON response

FILES / BUILD TARGET

app.py Target: Simple /health endpoint

CORE IMPLEMENTATION - TYPE + RUN

```
from flask import Flask, jsonify
app = Flask(__name__)

@app.get("/health")
def health():
    return jsonify(status="ok")
```

SAMPLE OUTPUT `GET /health -> {"status":"ok"}`

INTERVIEW EXPLANATION

Project answer: Flask route maps HTTP path to Python function; JSON is a clean API response shape. Debug cue: Check route path, method and server bind/port. Follow-up: What would you improve first if this had 10x traffic/data?

BUILD STEPS

1. Create the file(s) shown above.
2. Run the exact sample once.
3. Change one input and observe the output.
4. Add one validation or failure test.

EDGE CASES / DEBUG

Test invalid input, boundary values and empty/missing data where relevant. Keep validation at the system boundary.

APIs, Flask/FastAPI & Production Python

Use this pack after the section labs: explain -> build -> debug -> defend.

SECTION ANCHOR

Definition: AI/data logic ko REST APIs ke through expose karna: validation, status codes, timeouts, logging aur health checks.

Easy explanation: API boundary = contract. Validate early and return predictable errors.

Example: `GET /health -> {"status":"ok"}`

SIMPLE

Flask Health API

Kya karta hai: Minimal health endpoint expose karta hai.

Kis kaam aata hai: Any web/backend project starting point.

Architecture: HTTP -> Flask route -> JSON response

Files: `app.py`

Build target: Simple /health endpoint

MEDIUM

Validated Prediction Endpoint

Kya karta hai: POST payload validate karke model/service call karta hai.

Kis kaam aata hai: ML/AI inference APIs.

Architecture: Client -> JSON validation -> service -> JSON result

Files: `app.py | service.py`

Build target: 400 for bad input, 200 for valid

HARD

Production AI API Skeleton

Kya karta hai: Health, structured errors, request IDs and latency logging add karta hai.

Kis kaam aata hai: Real AI services and interviews on production readiness.

Architecture: Client -> middleware -> validation -> service -> provider -> logs

Files: `app.py | logging.py | service.py`

Build target: Timeouts, safe errors, observable latency

Project rule: code khud type karo, output predict karo, edge case test karo, phir 60-second explanation do.

APIs, Flask/FastAPI & Production Python - implemen

Three core implementations + project-tailored interview explanation + debugging cue.

SIMPLE | FLASK HEALTH API

```
from flask import Flask, jsonify
app=Flask(__name__)
@app.get("/health")
def health():
    return jsonify(status="ok")
```

INTERVIEW DEFENSE

Project answer: Flask route maps HTTP path to Python function; JSON is a clean API response shape.

Debug cue: Check route path, method and server bind/port.

Follow-up: What would you improve first if this had 10x traffic/data?

MEDIUM | VALIDATED PREDICTION ENDPOINT

```
@app.post("/predict")
def predict():
    data=request.get_json(silent=True) or {}
    if "features" not in data: return {"error":"features"},40
0
    return {"score":service.predict(data["features"])}
```

INTERVIEW DEFENSE

Project answer: Separate request parsing, validation and model call for testability.

Debug cue: Do not assume JSON exists; validate types and shape too.

Follow-up: What would you improve first if this had 10x traffic/data?

HARD | PRODUCTION AI API SKELETON

```
@app.post("/chat")
def chat():
    start=time.perf_counter()
    try:
        payload=validate(request.get_json())
        out=service.answer(payload)
        return {"answer":out,"request_id":rid()}
    except TimeoutError:
        return {"error":"provider_timeout"},504
    finally:
        log_latency(time.perf_counter()-start)
```

INTERVIEW DEFENSE

Project answer: Production API answer should cover contract, provider failures, latency, secrets and observability.

Debug cue: Never expose API keys or raw provider stack traces to clients.

Follow-up: What would you improve first if this had 10x traffic/data?

SELF-CHECK

Can I explain the project without notes? Can I name one failure? Can I state time/space or cost trade-offs?

SECTION 11

AI Engineer + Data Scientist Interview Mast

Level: Advanced Time: ~120 min 8 focus topics

Technical answers, project explanation, system design aur behavioral interview ko ek framework me la

IS TRACK ME KYA SEEKHOGE

✓ self-introduction
✓ project story
✓ debugging
✓ trade-offs
✓ system design
✓ metrics
✓ behavioral STAR
✓ follow-ups

TOPIC ROADMAP

11.1 self-introduction
11.2 project story
11.3 debugging
11.4 trade-offs
11.5 system design
11.6 metrics
11.7 behavioral STAR
11.8 follow-ups

INTERVIEW MINDSET

Definition -> tiny example -> practical use -> trade-off/edge case. Har answer ko apni language me bolo.

11.1 Core Concepts

Interview me knowledge + communication dono measure hote hain. Strong answer concise hota hai, example deta hai, trade-off batata hai aur interviewer ke follow-up ke liye clear hooks deta hai.

PROJECT STORY

Problem -> approach -> architecture -> your contribution -> challenge -> result -> learning.

DEBUGGING STORY

Symptom reproduce -> isolate layer -> inspect types/inputs/logs -> fix -> test regression.

TRADE-OFFS

Accuracy vs latency, cost vs quality, batch vs real-time, simple vs scalable. Context ke saath answer.

BEHAVIORAL

STAR: Situation -> Task -> Action -> Result. Result measurable ho to better.

example.py

```
def project_answer(problem, approach, result):
    return (f"Problem: {problem}. "
            f"Approach: {approach}. "
            f"Result: {result}.")

print(project_answer(
    "manual support workflow",
    "API + webhook + validation + LLM step",
    "less manual repetition and better traceability"
))
```

OUTPUT

```
Problem -> Approach -> Result
```

YAAD RAKHO
Smart answer means truthful, specific aur understandable. Overclaim mat karo.

11.2 Interview Depth

Is page ka goal hai concept ko implementation decision tak le jana. Interviewer ko syntax se zyada reasoning, edge cases aur trade-offs dikhte hain.

WHAT

Kya hai aur kab use karoge?

WHY

Kaunsi problem solve hoti hai?

RISK

Common bug / failure mode kya hai?

TRADE-OFF

Simple option aur scalable option me kya difference hai?

YAAD RAKHO
One-line answer practice: definition + example + use + limitation.

INTERVIEW LAB / AI Engineer + Data Scientist Interview Mastery

Definition -> tiny example -> practical use -> trade-off. Word-by-word memorize mat karo.

Q01

Project interviewer ko kaise explain?

SMART ANSWER

Problem -> architecture -> your contribution -> one challenge -> result -> learning.

TIP Keep it 60-90 sec.

Q02

Why this technology?

SMART ANSWER

Requirement se link karo. Example: Flask for simple service, FastAPI for typed modern API style, etc. Context matters.

TIP Never say because it is popular only.

Q03

A failure you fixed?

SMART ANSWER

Symptom, root cause, fix, regression test aur prevention batao.

TIP Do not blame others.

Q04

System design answer ka order?

SMART ANSWER

Requirements -> APIs/data -> architecture -> failure/scale -> security -> observability -> trade-offs.

TIP Start simple.

Q05

Data science case study?

SMART ANSWER

Business question -> metric -> data -> cleaning -> analysis -> model/baseline -> evaluation -> deployment/monitoring if needed.

TIP Explain assumptions.

Q06

Unknown question ka answer?

SMART ANSWER

Clarify what is being asked, state what you know, reason from first principles, and be explicit about uncertainty.

TIP Do not bluff.

Speaking drill: Har answer 2 baar bolo. Second round me ek project example add karo.

CODING LAB / AI Engineer + Data Scientist Interview Mastery

Pehle problem ko 10 sec me explain karo -> then code -> edge case -> complexity.

EASY

Write 30-second project answer template.

ANSWER

```
def answer(p,a,r): return f"Problem: {p}. Approach: {a}. Result: {r}."
```

Reason: Concise structure is easy to follow.
Complexity: O(1)

EASY

Give one measurable project metric.

ANSWER

```
metric={"manual_time_reduced":0.6}
```

Reason: Metrics should be truthful and clearly defined.
Complexity: O(1)

MEDIUM

Debugging checklist function.

ANSWER

```
checks=["reproduce","inspect inputs","inspect types","isolate","fix","regression test"]
```

Reason: Process beats random edits.
Complexity: O(1)

MEDIUM

System design starting points.

ANSWER

```
requirements -> APIs -> data -> core flow -> failure -> scale -> security -> observability
```

Reason: Ordering avoids premature scaling.
Complexity: O(1)

HARD

Behavioral STAR skeleton.

ANSWER

```
S=situation; T=task; A=action; R=result
```

Reason: Keep Action focused on your contribution and Result evidence-based.
Complexity: O(1)

DEBUG

Interviewer challenges your architecture.

ANSWER

```
Clarify constraint, accept feedback, compare alternatives, explain trade-off.
```

Reason: Interview is discussion, not a script.
Complexity: Reasoning

LOGIC LAB / AI Engineer + Data Scientist Interview Mastery

Code run karne se pehle output predict karo. Phir dry-run karke reason explain karo.

PREDICT

Answer only technology names in project interview

CHECK
Weak project story

WHY
Interviewer needs problem, choices and results.

TRACE

Bug fixed but no regression test

CHECK
Bug may return

WHY
Add test for the failure condition.

THINK

System design starts with database

CHECK
May optimize wrong thing

WHY
Start with requirements and data/traffic assumptions.

FIX

Interviewer challenges your choice

CHECK
Compare alternatives and trade-offs

WHY
Discussion is more valuable than defensiveness.

PREDICT

Unknown question

CHECK
Clarify + reason + state uncertainty

WHY
Bluffing damages trust.

EXPLAIN

What makes a smart answer?

CHECK
Specific, truthful, structured and easy to follow.

WHY
Tie claims to your real work.

Mastery: Explain without notes + solve one fresh problem + predict two outputs + tell one edge case.

SOLVED PRACTICE - AI Engineer + Data Scientist Inter

Every coding-lab question now has its answer, expected result and a quick check - all in one place.

EASY

Write 30-second project answer template.

```
def answer(p,a,r): return f"Problem: {p}.
Approach: {a}. Result: {r}."
Reason: Concise structure is easy to follow.
Complexity: O(1)
```

EXPECTED RESULT

Compare with the code and verify one edge case.

EASY

Give one measurable project metric.

```
metric={"manual_time_reduced":0.6}
Reason: Metrics should be truthful and clearly defined.
Complexity: O(1)
```

EXPECTED RESULT

Compare with the code and verify one edge case.

MEDIUM

Debugging checklist function.

```
checks=["reproduce","inspect
inputs","inspect
types","isolate","fix","regression test"]
Reason: Process beats random edits.
Complexity: O(1)
```

EXPECTED RESULT

Compare with the code and verify one edge case.

MEDIUM

System design starting points.

```
requirements -> APIs -> data -> core flow
-> failure -> scale -> security ->
observability
Reason: Ordering avoids premature scaling.
Complexity: O(1)
```

EXPECTED RESULT

Compare with the code and verify one edge case.

HARD

Behavioral STAR skeleton.

```
S=situation; T=task; A=action; R=result
Reason: Keep Action focused on your contribution and
Result evidence-based.
Complexity: O(1)
```

EXPECTED RESULT

Compare with the code and verify one edge case.

PRACTICE HABIT

Practice rule: solve first, compare second, then change one input. Mark the question only after you can explain why the output changes.

MINI PROJECT - 30-Second Answer Builder

AI Engineer + Data Scientist Interview Mastery ka concept ek runnable, interview-ready mini build me convert karo.

YE KYA KARTA HAI?

Technical question ko concise structured response me convert karta hai.

KIS KAAM AATA HAI?

Interview speaking practice.

ARCHITECTURE

Question -> definition -> example -> use -> edge case

FILES / BUILD TARGET

answers.py Target: No memorized paragraphs

CORE IMPLEMENTATION - TYPE + RUN

```
def answer(defn, example, use, edge):
    return (f"{defn}. Example: {example}. "
            f"Use: {use}. Edge: {edge}.")

print(answer("A variable stores a value", "x=1", "state", "wrong
type"))
```

SAMPLE OUTPUT `Problem: ... Approach: ... Result: ...`

INTERVIEW EXPLANATION

Project answer: Use your own language and one real example rather than reciting a textbook. Debug cue: Too much detail before the direct answer loses clarity. Follow-up: What would you improve first if this had 10x traffic/data?

BUILD STEPS

1. Create the file(s) shown above.
2. Run the exact sample once.
3. Change one input and observe the output.
4. Add one validation or failure test.

EDGE CASES / DEBUG

Test invalid input, boundary values and empty/missing data where relevant. Keep validation at the system boundary.

AI Engineer + Data Scientist Interview Mastery

Use this pack after the section labs: explain -> build -> debug -> defend.

SECTION ANCHOR

Definition: Technical knowledge ko structured answers, project stories and trade-offs me convert karo.

Easy explanation: Best interview answer = definition -> example -> use -> limitation.

Example: `Problem -> Approach -> Architecture -> Result -> Limitation -> Next`

SIMPLE

30-Second Answer Builder

Kya karta hai: Technical question ko concise structured response me convert karta hai.

Kis kaam aata hai: Interview speaking practice.

Architecture: Question -> definition -> example -> use -> edge case

Files: `answers.py`

Build target: No memorized paragraphs

MEDIUM

Project Story Generator

Kya karta hai: Resume bullet ko interview story me convert karta hai.

Kis kaam aata hai: Project rounds and recruiter screens.

Architecture: Resume bullet -> problem -> architecture -> contribution -> result

Files: `story.py`

Build target: Truthful measurable claims only

HARD

System Design Drill Generator

Kya karta hai: Interview system-design prompt ko assumptions, bottlenecks and trade-offs me break karta hai.

Kis kaam aata hai: AI Engineer/Backend/DS system discussions.

Architecture: Requirements -> data flow -> components -> failure modes -> scaling

Files: `design_notes.md`

Build target: State assumptions before choosing tech

Project rule: code khud type karo, output predict karo, edge case test karo, phir 60-second explanation do.

AI Engineer + Data Scientist Interview Mastery - imp

Three core implementations + project-tailored interview explanation + debugging cue.

SIMPLE | 30-SECOND ANSWER BUILDER

```
def answer(defn, example, use, edge):
    return f"{defn}. Example: {example}. Use: {use}. Edge: {e
dge}."
```

INTERVIEW DEFENSE

Project answer: Use your own language and one real example rather than reciting a textbook.

Debug cue: Too much detail before the direct answer loses clarity.

Follow-up: What would you improve first if this had 10x traffic/data?

MEDIUM | PROJECT STORY GENERATOR

```
def project_story(problem, stack, contribution, result):
    return {"problem":problem,"stack":stack,
            "contribution":contribution,"result":result}
```

INTERVIEW DEFENSE

Project answer: Mention what you personally built, not only technologies used.

Debug cue: If a metric cannot be defended, rephrase it as an observed/qualitative result.

Follow-up: What would you improve first if this had 10x traffic/data?

HARD | SYSTEM DESIGN DRILL GENERATOR

```
assumptions={"traffic":"low-medium","latency":"interactive","
data":"SQL"}
components=["API","service","DB","queue","observability"]
print(assumptions, components)
```

INTERVIEW DEFENSE

Project answer: Do not jump to tools; start from requirements, data, failure modes and traffic.

Debug cue: Challenge your own architecture with retries, duplicates, timeouts and cost.

Follow-up: What would you improve first if this had 10x traffic/data?

SELF-CHECK

Can I explain the project without notes? Can I name one failure? Can I state time/space or cost trade-offs?

SECTION 12

Coding Patterns, Projects & Final Practice

Level: Advanced Time: ~140 min 9 focus topics

Interview coding ko patterns se approach karna aur AI/data portfolio projects ko explain karna.

IS TRACK ME KYA SEEKHOGE

✓ hash maps
✓ two pointers
✓ sliding window
✓ stack/queue
✓ binary search
✓ recursion
✓ BFS/DFS concept
✓ testing
✓ mini projects

TOPIC ROADMAP

12.1 hash maps
12.2 two pointers
12.3 sliding window
12.4 stack/queue
12.5 binary search
12.6 recursion
12.7 BFS/DFS concept
12.8 testing
12.9 mini projects

INTERVIEW MINDSET

Definition -> tiny example -> practical use -> trade-off/edge case. Har answer ko apni language me bolo.

12.1 Core Concepts

Coding problem ko sirf memorize nahi karna. Constraints padho, brute force socho, bottleneck identify karo, pattern map karo, complexity state karo aur edge cases test karo.

HASH MAP

Frequency/count/lookup questions me dictionary often O(1) average lookup provide karta hai.

TWO POINTERS

Sorted arrays aur opposite/same-direction scans ke liye useful.

SLIDING WINDOW

Contiguous subarray/substring problems me window maintain karo; state update efficient rakho.

TESTING

Normal + empty + single item + duplicates + extreme values + invalid input test set rakho.

example.py

```
def first_unique(text):
    counts = {}
    for ch in text:
        counts[ch] = counts.get(ch, 0) + 1
    for ch in text:
        if counts[ch] == 1:
            return ch
    return None

print(first_unique("swiss"))
```

OUTPUT

```
w
```

YAAD RAKHO
Pattern: count first, scan second. O(n) time, O(k) space.

12.2 Interview Depth

Is page ka goal hai concept ko implementation decision tak le jana. Interviewer ko syntax se zyada reasoning, edge cases aur trade-offs dikhte hain.

WHAT

Kya hai aur kab use karoge?

WHY

Kaunsi problem solve hoti hai?

RISK

Common bug / failure mode kya hai?

TRADE-OFF

Simple option aur scalable option me kya difference hai?

YAAD RAKHO
One-line answer practice: definition + example + use + limitation.

INTERVIEW LAB / Coding Patterns, Projects & Final Practice

Definition -> tiny example -> practical use -> trade-off. Word-by-word memorize mat karo.

Q01

Hash map pattern kab?

SMART ANSWER

Fast lookup or frequency counting when keys can be hashed.

TIP Two-sum / frequency example.

Q02

Sliding window kab?

SMART ANSWER

Contiguous substring/subarray with maintainable state.

TIP Longest unique substring concept.

Q03

Two pointers kab?

SMART ANSWER

Sorted sequence or left/right scan where pointer movement can eliminate work.

TIP Pair sum example.

Q04

Testing edge cases?

SMART ANSWER

Empty, one item, duplicates, negative/zero, max size, invalid types/inputs.

TIP State expected behavior.

Q05

Brute force first?

SMART ANSWER

Constraint size padho; simple brute force helps validate logic, then optimize if needed.

TIP Explain trade-off.

Q06

Coding answer structure?

SMART ANSWER

Clarify input/output -> examples -> approach -> code -> complexity -> tests.

TIP Speak while coding lightly.

Speaking drill: Har answer 2 baar bolo. Second round me ek project example add karo.

CODING LAB / Coding Patterns, Projects & Final Practice

Pehle problem ko 10 sec me explain karo -> then code -> edge case -> complexity.

EASY

Two-sum using hash map.

ANSWER

```
seen={}
for i,x in enumerate(nums):
if target-x in seen: return
[seen[target-x],i]
seen[x]=i
```

Reason: One pass average O(n).
Complexity: O(n) / O(n)

EASY

Reverse string.

ANSWER

```
s[::-1]
```

Reason: Built-in slicing is simple.
Complexity: O(n)

MEDIUM

Sliding window maximum sum of k.

ANSWER

```
window=sum(a[:k]); best=window
for i in range(k,len(a)):
window+=a[i]-a[i-k];
best=max(best,window)
```

Reason: Maintain current window incrementally.
Complexity: O(n)

MEDIUM

Unique characters in substring concept.

ANSWER

```
window=set(); left=0
# move right, shrink while duplicate
```

Reason: Window state gives linear scan rather than nested restart.
Complexity: O(n)

HARD

Binary search.

ANSWER

```
lo,hi=0,len(a)-1
while lo<=hi:
mid=(lo+hi)//2
# compare and shrink half
```

Reason: Requires sorted data and monotonic decision.
Complexity: O(log n)

DEBUG

What should you say before coding?

ANSWER

```
Clarify constraints, input/output,
examples, edge cases, expected
complexity.
```

Reason: Shows structured thinking.
Complexity: Interview process

LOGIC LAB / Coding Patterns, Projects & Final Practice

Code run karne se pehle output predict karo. Phir dry-run karke reason explain karo.

PREDICT

seen={}; nums=[2,7,11,15]; target=9

CHECK
Return indices [0,1] in two-sum

WHY
7 complements 2.

TRACE

Sliding window adds new right, removes old left

CHECK
Window sum stays current

WHY
Incremental update avoids recomputing entire window.

THINK

Sorted array + two pointers

CHECK
Can reduce pair search from O(n^2) to O(n)

WHY
Pointers eliminate impossible regions.

FIX

Empty input

CHECK
Return defined neutral/None/error per contract

WHY
Edge behavior must be intentional.

PREDICT

Binary search on unsorted data

CHECK
Not valid in general

WHY
Sorted/monotonic condition is required.

EXPLAIN

Coding interview first sentence?

CHECK
Clarify inputs, outputs and constraints.

WHY
Then choose pattern and complexity.

Mastery: Explain without notes + solve one fresh problem + predict two outputs + tell one edge case.

SOLVED PRACTICE - Coding Patterns, Projects & Fina

Every coding-lab question now has its answer, expected result and a quick check - all in one place.

EASY

Two-sum using hash map.

```
seen={}
for i,x in enumerate(nums):
if target-x in seen: return
[seen[target-x],i]
seen[x]=i
Reason: One pass average O(n).
Complexity: O(n) / O(n)
```

EXPECTED RESULT

Compare with the code and verify one edge case.

EASY

Reverse string.

```
s[::-1]
Reason: Built-in slicing is simple.
Complexity: O(n)
```

EXPECTED RESULT

Compare with the code and verify one edge case.

MEDIUM

Sliding window maximum sum of k.

```
window=sum(a[:k]); best=window
for i in range(k,len(a)):
window+=a[i]-a[i-k];
best=max(best,window)
Reason: Maintain current window incrementally.
Complexity: O(n)
```

EXPECTED RESULT

Largest value; empty input handled as None when specified.

MEDIUM

Unique characters in substring concept.

```
window=set(); left=0
# move right, shrink while duplicate
Reason: Window state gives linear scan rather than
nested restart.
Complexity: O(n)
```

EXPECTED RESULT

Compare with the code and verify one edge case.

HARD

Binary search.

```
lo,hi=0,len(a)-1
while lo<=hi:
mid=(lo+hi)//2
# compare and shrink half
Reason: Requires sorted data and monotonic decision.
Complexity: O(log n)
```

EXPECTED RESULT

Index of target or -1; O(log n).

PRACTICE HABIT

Practice rule: solve first, compare second, then change one input. Mark the question only after you can explain why the output changes.

MINI PROJECT - Two-Sum

Coding Patterns, Projects & Final Practice ka concept ek runnable, interview-ready mini build me convert karo.

YE KYA KARTA HAI?

Target sum ka index pair hash map se find karta hai.

KIS KAAM AATA HAI?

Classic interview pattern and lookup problems.

ARCHITECTURE

Array -> seen map -> complement lookup

FILES / BUILD TARGET

two_sum.py Target: One pass O(n)

CORE IMPLEMENTATION - TYPE + RUN

```
def two_sum(nums, target):
    seen = {}
    for i, x in enumerate(nums):
        if target - x in seen:
            return seen[target - x], i
        seen[x] = i
    return None

print(two_sum([2, 7, 11, 15], 9))
```

SAMPLE OUTPUT `two_sum([2,7,11,15],9) -> (0,1)`

INTERVIEW EXPLANATION

Project answer: Store seen values so the complement lookup is average O(1). Debug cue: Handle no-solution and duplicate-value cases. Follow-up: What would you improve first if this had 10x traffic/data?

BUILD STEPS

1. Create the file(s) shown above.
2. Run the exact sample once.
3. Change one input and observe the output.
4. Add one validation or failure test.

EDGE CASES / DEBUG

Test invalid input, boundary values and empty/missing data where relevant. Keep validation at the system boundary.

Coding Patterns, Projects & Final Practice

Use this pack after the section labs: explain -> build -> debug -> defend.

SECTION ANCHOR

Definition: Interview coding me patterns constraints aur complexity ke basis par choose karo.

Easy explanation: Clarify -> example -> pattern -> code -> complexity -> tests.

Example: `Two-sum: hash map; sliding window: contiguous range; binary search: sorted/monotonic`

SIMPLE

Two-Sum

Kya karta hai: Target sum ka index pair hash map se find karta hai.

Kis kaam aata hai: Classic interview pattern and lookup problems.

Architecture: Array -> seen map -> complement lookup

Files: `two_sum.py`

Build target: One pass O(n)

MEDIUM

Sliding Window Analyzer

Kya karta hai: Fixed-size or variable-size window ka aggregate efficiently update karta hai.

Kis kaam aata hai: Logs, strings, time-window analytics.

Architecture: Sequence -> window state -> incremental update -> best

Files: `window.py`

Build target: O(n) for fixed-size sum

HARD

Binary Search Service

Kya karta hai: Sorted data me O(log n) lookup perform karta hai.

Kis kaam aata hai: Search-heavy interview tasks and threshold problems.

Architecture: Sorted input -> mid -> shrink half -> result

Files: `search.py`

Build target: Correct boundaries and termination

Project rule: code khud type karo, output predict karo, edge case test karo, phir 60-second explanation do.

Coding Patterns, Projects & Final Practice - impleme

Three core implementations + project-tailored interview explanation + debugging cue.

SIMPLE | TWO-SUM

```
def two_sum(nums,target):
    seen={}
    for i,x in enumerate(nums):
        if target-x in seen: return seen[target-x], i
        seen[x]=i
    return None
```

INTERVIEW DEFENSE

Project answer: Store seen values so the complement lookup is average O(1).

Debug cue: Handle no-solution and duplicate-value cases.

Follow-up: What would you improve first if this had 10x traffic/data?

MEDIUM | SLIDING WINDOW ANALYZER

```
def max_sum(a,k):
    w=sum(a[:k]); best=w
    for i in range(k,len(a)):
        w += a[i]-a[i-k]
        best=max(best,w)
    return best
```

INTERVIEW DEFENSE

Project answer: Maintain state instead of recomputing each window.

Debug cue: Validate k in [1, len(a)] before slicing.

Follow-up: What would you improve first if this had 10x traffic/data?

HARD | BINARY SEARCH SERVICE

```
def bs(a,x):
    lo,hi=0,len(a)-1
    while lo<=hi:
        m=(lo+hi)//2
        if a[m]==x:return m
        if a[m]<x:lo=m+1
        else:hi=m-1
    return -1
```

INTERVIEW DEFENSE

Project answer: Binary search requires a sorted or monotonic search space.

Debug cue: Most bugs are boundary/loop-condition mistakes.

Follow-up: What would you improve first if this had 10x traffic/data?

SELF-CHECK

Can I explain the project without notes? Can I name one failure? Can I state time/space or cost trade-offs?

FINAL CHEAT SHEET / Python Interview Core

FUNCTIONS	def f(x=..., *, mode=...):	Keep interfaces explicit; return values; avoid global state.
EXCEPTIONS	try -> except specific -> else -> finally	Catch what you can handle; log context; do not hide bugs.
COLLECTIONS	list / tuple / set / dict	Choose by access, ordering, mutability and uniqueness needs.
ITERATION	for / while / enumerate / zip	Prefer readable iteration; generator for streams.
COMPLEXITY	O(1), O(log n), O(n), O(n log n), O(n^2)	State time + space for interview code.
DEBUGGING	reproduce -> isolate -> inspect -> fix -> regression test	Types, inputs, conditions and boundaries first.

FINAL CHEAT SHEET / AI + Data

NUMPY	shape, dtype, axis, broadcasting	Think dimensions before writing the operation.
PANDAS	inspect -> clean -> transform -> aggregate -> validate	Always know the row grain.
SQL	WHERE -> GROUP BY -> HAVING -> ORDER BY	JOIN cardinality can change counts.
ML	split -> baseline -> preprocess -> train -> evaluate	Avoid leakage; choose metric by error cost.
RAG	ingest -> chunk -> embed -> retrieve -> generate -> evaluate	Quality depends on retrieval + generation.
AGENTS	state -> decide -> tool -> observe -> stop	Bound steps, validate tools, add approval for high-impact actions.

FINAL CHEAT SHEET / Interview Answer Templates

WHY PYTHON?	Readable, productive, rich ecosystem, strong AI/data tooling.	Add one real project example.
PROJECT	Problem -> Approach -> Contribution -> Challenge -> Result -> Learning	Keep it 60-90 seconds.
DEBUGGING	Symptom -> Reproduce -> Isolate -> Root cause -> Fix -> Test	Describe what you actually did.
TRADE-OFF	Accuracy vs latency; cost vs quality; simple vs scalable	Give context; avoid absolute answers.
UNKNOWN	Clarify -> reason from principles -> state assumption	Never invent facts.
CODING	Clarify -> examples -> approach -> code -> complexity -> tests	Speak your reasoning without over-talking.

30-DAY INTERVIEW + CODING PLAN

DAYS 1-7

Python core + functions + exceptions

2 concept pages/day + 5 coding problems/day + 10 speaking answers.

DAYS 8-14

Advanced Python + NumPy + Pandas

1 mini data task/day + 5 output predictions + 10 interview answers.

DAYS 15-21

SQL + ML + GenAI/RAG

1 SQL set/day + 1 EDA task + 1 GenAI architecture explanation.

DAYS 22-26

Agents + APIs + automation

Build one bounded agent flow + one API endpoint + test failure paths.

DAYS 27-30

Mock interviews + revision

3 timed coding sessions + 3 project explanations + 2 full mock interviews.

Daily minimum: 45 min learning + 45 min coding + 20 min speaking/logic + 10 min revision.

FINAL SELF-TEST / 50-QUESTION CHECKPOINT

1. Explain mutable vs immutable with one bug example.
2. Why is input() string by default?
3. Difference between == and is?
4. What does LEGB mean?
5. When would you use a generator?
6. Why avoid mutable default arguments?
7. How do you handle a JSONDecodeError?
8. Why use pathlib?
9. What is shallow copy aliasing?
10. What is O(n log n)?
11. What is NumPy broadcasting?
12. axis=0 vs axis=1?
13. Why does shape matter in matrix multiplication?
14. What is vectorization?
15. What is a DataFrame?
16. How do you inspect missing values in Pandas?
17. Explain groupby.
18. What is merge cardinality?
19. WHERE vs HAVING?
20. INNER vs LEFT JOIN?
21. What is a window function?
22. What is data leakage?
23. Why keep a baseline?
24. Precision vs recall?
25. Why use pipelines?
26. What is an embedding?
27. What is RAG?
28. Chunking trade-off?
29. How do you evaluate an LLM app?
30. What is hallucination?
31. Agent vs chatbot?
32. What is tool calling?
33. Why bound agent steps?
34. When to use human-in-loop?
35. What should be in agent state?
36. What is a webhook?
37. Why use request timeouts?
38. How do you protect webhook endpoints?
39. Why keep secrets out of source?
40. What is API validation?
41. Explain one project in 60 seconds.
42. Describe one bug you fixed.
43. Explain a trade-off you made.
44. How do you answer an unknown question?
45. What is your coding interview process?
46. Two-sum pattern?
47. Sliding window pattern?
48. Two pointers?
49. Binary search requirement?
50. How do you test edge cases?

Pass rule: 45/50 answers spoken clearly + 10 coding tasks solved without notes + 5 fresh debugging problems.

MASTER UPGRADE • RESUME-ALIGNED

Your Interview Skill Map

Built directly around the skills and projects listed in your resume.

CORE PYTHON

Python → functions → OOP → exceptions → files → modules → iterators → de
typing → testing → clean code
Goal: syntax se aage jaakar production-quality reasoning, debugging aur ir
explanation.

DATA + ANALYTICS

NumPy → Pandas → EDA → statistics → SQL → Excel → Power BI/DAX → busin
Goal: messy data ko inspect, clean, transform, validate aur explain karna.

AI / ML

Scikit-learn → ML workflow → evaluation → TensorFlow/PyTorch concepts →
spaCy → Transformers
Goal: model choice, preprocessing, leakage, metrics aur deployment reaso

GENAI + AGENTS

OpenAI/Gemini APIs → LLMs → embeddings → RAG → LangChain → Agentic A
Zapier → webhooks → REST APIs
Goal: AI system ko architecture, tools, state, errors aur safety ke saath expl
karna.

ENGINEERING

Flask → REST → MySQL/SQLite → Git/GitHub → Docker → AWS basics → MLO
logging/testing
Goal: “AI model banaya” se “AI application ship ki” tak.

RESUME PROJECT LAB

AI chatbot • Agentic n8n automation • Power BI dashboard • 30+ analytics
challenges
Every project gets: 60-sec pitch → architecture → code reasoning → failure c
interviewer follow-ups.

FUNCTIONS & REUSABLE PYTHON • Learn 1

Easy Hinglish • practical mental model • interview depth

Functions ka mental model

Function input leta hai, work karta hai aur optional output return karta hai. AI projects me reusable preprocessing, validation, API calls aur business logic ko functions me isolate karna maintainability improve karta hai.
Syntax: def name(parameters): ... return value.
Important: return aur print same nahi hain — return value caller ko deta hai.

Parameters deeply

Positional, keyword, default, *args, **kwargs aur keyword-only parameters ko samjho. Production APIs me explicit keyword-only arguments readability aur m prevention ke liye useful ho sakte hain.

Scope + LEGB

Python name lookup: Local → Enclosing → Global → Built-in. Global state ko unnecessarily use mat karo. Function ko dependencies parameters ke throug testing ko easy banata hai.

Lambda, map, filter, sorted

Lambda small expression ke liye; map/filter lazy iterators return karte hain. Readability first: complex lambda ko normal def me convert karo.

Recursion + base case

Recursive function ko base case aur progress chahiye. Interview me stack depth aur iterative alternative discuss karo.

SMART STUDY CHECK

Bina code dekhe: 1) concept 20 sec me explain karo 2) tiny example likho 3) one failure case bolo 4) one real AI/data use-case batao.

FUNCTIONS & REUSABLE PYTHON • Learn 2

Easy Hinglish • practical mental model • interview depth

Decorators

Decorator function ko wrap karke behavior add karta hai: logging, timing, authorization, retry jaise cross-cutting concerns. @syntax sirf readable wrapp syntax hai.

Generators

yield se lazy values produce hoti hain. Large files, streaming data aur pipeline me memory efficient. list() se generator ko materialize karne par memory ber khatam ho sakta hai.

Typing + dataclass

Type hints intent communicate karte hain; runtime validation automatically r hoti. dataclass data-centric classes ko concise banata hai.

Testing functions

Pure functions ko unit-test karna easy hota hai. Inputs: normal, empty, invalid boundary. AI/data code me deterministic preprocessing functions alag test ka

Interview-ready function design

Answer pattern: responsibility → inputs → output → edge cases → complexity → test. “Function ka naam kya karega?” se start nahi, “problem ko kaise isolate karoge?” se start karo.

SMART STUDY CHECK

Bina code dekhe: 1) concept 20 sec me explain karo 2) tiny example likho 3) one failure case bolo 4) one real AI/data use-case batao.

INTERVIEW LAB • FUNCTIONS & REUSABLE PYTHON

Smart answer = definition + example + practical use + edge case + follow-up

Q1 • return aur print me difference?

BEST ANSWER
print screen par output dikhata hai; return caller ko value deta hai, jise variable me store ya further process kiya ja sakta hai.
FOLLOW-UP / TIP
Example: result = add(2,3).
SPEAK: 30-45 sec, then give one project example.

Q2 • *args kya hai?

BEST ANSWER
Variable number of positional arguments ko tuple ke form me receive karta hai.
FOLLOW-UP / TIP
Use when API ko flexible positional inputs chahiye.
SPEAK: 30-45 sec, then give one project example.

Q3 • **kwargs?

BEST ANSWER
Variable number of keyword arguments ko dictionary ke form me receive karta
FOLLOW-UP / TIP
Configuration-style functions me useful.
SPEAK: 30-45 sec, then give one project example.

Q4 • mutable default argument se problem?

BEST ANSWER
Default object function definition par ek baar create hota hai, isliye calls ke beech state leak ho sakta hai.
FOLLOW-UP / TIP
Use None default, then create inside.
SPEAK: 30-45 sec, then give one project example.

INTERVIEW UPGRADE

If interviewer asks “why?”, add a trade-off. If asks “how?”, give implementation steps. If asks “what if it fails?”, explain the failure path.

INTERVIEW LAB • FUNCTIONS & REUSABLE PYTHON

Smart answer = definition + example + practical use + edge case + follow-up

Q5 • decorator ka practical use?

BEST ANSWER
Existing function ko modify kiye bina logging, timing, authorization ya retry behavior wrap karna.
FOLLOW-UP / TIP
AI API latency logging example.
SPEAK: 30-45 sec, then give one project example.

Q6 • generator kab?

BEST ANSWER
Jab data ko lazily stream karna ho aur poora dataset memory me nahi rakhna.
FOLLOW-UP / TIP
Large file/API stream.
SPEAK: 30-45 sec, then give one project example.

Q7 • type hints runtime validation hain?

BEST ANSWER
Nahi. Type hints documentation/tooling help dete hain; runtime validation sepa logic/library se karni hoti hai.
FOLLOW-UP / TIP
Interview trap.
SPEAK: 30-45 sec, then give one project example.

Q8 • good function ka rule?

BEST ANSWER
One clear responsibility, explicit inputs/outputs, predictable side effects, edge-case handling aur tests.
FOLLOW-UP / TIP
Service functions ko small rakho.
SPEAK: 30-45 sec, then give one project example.

INTERVIEW UPGRADE

If interviewer asks “why?”, add a trade-off. If asks “how?”, give implementation steps. If asks “what if it fails?”, explain the failure path.

CODING LAB • FUNCTIONS & REUSABLE PYTHO

Pehle solve karo → then answer compare karo → complexity + edge case explain karo.

EASY • PRACTICE

```
Write add(a,b) that returns the sum.
ANSWER
def add(a, b):
    return a + b
CHECK
Input edge cases + expected output + complexity.
```

EASY • PRACTICE

```
Return even numbers from a list using a function.
ANSWER
def evens(nums):
    return [x for x in nums if x % 2 == 0]
CHECK
Input edge cases + expected output + complexity.
```

MEDIUM • PRACTICE

```
Count word frequency.
ANSWER
def word_count(text):
    out = {}
    for w in text.lower().split():
        out[w] = out.get(w, 0) + 1
    return out
CHECK
Input edge cases + expected output + complexity.
```

MEDIUM • PRACTICE

```
Use *args to return the maximum.
ANSWER
def max_value(*args):
    if not args: return None
    return max(args)
CHECK
Input edge cases + expected output + complexity.
```

CODING INTERVIEW LINE

“Main pehle constraints aur edge cases clarify karunga, phir simplest correct solution, phir complexity aur test cases explain karunga.”

CODING LAB • FUNCTIONS & REUSABLE PYTHO

Pehle solve karo → then answer compare karo → complexity + edge case explain karo.

HARD • PRACTICE

```
Write a decorator that prints function name.
ANSWER
def log_call(fn):
    def wrapper(*args, **kwargs):
        print("CALL:", fn.__name__)
        return fn(*args, **kwargs)
    return wrapper
CHECK
Input edge cases + expected output + complexity.
```

HARD • PRACTICE

```
Generator for 1..n.
ANSWER
def numbers(n):
    for i in range(1, n + 1):
        yield i
CHECK
Input edge cases + expected output + complexity.
```

DEBUG • PRACTICE

```
Fix mutable default list.
ANSWER
def add_item(item, items=None):
    if items is None:
        items = []
    items.append(item)
    return items
CHECK
Input edge cases + expected output + complexity.
```

INTERVIEW CODE • PRACTICE

```
Validate age input without crashing.
ANSWER
def parse_age(value):
    try:
        age = int(value)
    except (TypeError, ValueError):
        return None
    return age if 0 <= age <= 120 else None
CHECK
Input edge cases + expected output + complexity.
```

CODING INTERVIEW LINE

“Main pehle constraints aur edge cases clarify karunga, phir simplest correct solution, phir complexity aur test cases explain karunga.”

LOGIC + DEBUG LAB • FUNCTIONS & REUSABLE PYTHON

Run karne se pehle predict → dry-run → reason → fix → retest.

PREDICT

What changes first: input, transformed data, or output?
CHECK / ANSWER
Trace data flow from boundary → validation → transformation → result.

TRACE

Pick one code example from today and write variable state after every step.
CHECK / ANSWER
Interviewers value reasoning, not only final output.

WHY

What is the most likely bug if the result is unexpectedly empty?
CHECK / ANSWER
Check input shape/type, filtering condition, join keys or model response.

FIX

What edge case breaks the simple solution?
CHECK / ANSWER
Empty input, NULL/None, duplicate IDs, invalid JSON, timeout or shape mismatc

DESIGN

Where should validation happen?
CHECK / ANSWER
At system boundaries before expensive or irreversible operations.

TRADE-OFF

What would you optimize first?
CHECK / ANSWER
Measure first; then target the actual bottleneck in latency, memory, cost or correctness.

MASTER CHECK

A topic is interview-ready when you can explain it simply, code it without copying, debug one failure, state a trade-off and connect it to your resume

FUNCTIONS & REUSABLE PYTHON • 20-Question Checkpo

Answer first. Then use the compact answer key.

01 return aur print me difference?

KEY: print screen par output dikhata hai; return

02 *args kya hai?

KEY: Variable number of positional arguments ko tuple

03 **kwargs?

KEY: Variable number of keyword arguments ko

04 mutable default argument se problem?

KEY: Default object function definition par ek baar

05 decorator ka practical use?

KEY: Existing function ko modify kiye bina logging,

06 generator kab?

KEY: Jab data ko lazily stream karna ho aur poora

07 type hints runtime validation hain?

KEY: Nahi. Type hints documentation/tooling help dete

08 good function ka rule?

KEY: One clear responsibility, explicit

09 Write add(a,b) that returns the sum.

KEY: Coding answer: def add(a, b):

10 Return even numbers from a list using a function.

KEY: Coding answer: def evens(nums):

11 Count word frequency.

KEY: Coding answer: def word_count(text):

12 Use *args to return the maximum.

KEY: Coding answer: def max_value(*args):

SCORE RULE

0-8: revise basics • 9-15: practice code • 16-20: explain aloud + build mini task.
Repeat until you can defend the reasoning.

NUMPY FOR AI + DATA • Learn 1

Easy Hinglish • practical mental model • interview depth

Array vs Python list

NumPy ndarray homogeneous numerical data ke liye optimized array abstrac
shape, ndim, dtype aur size ko pehle inspect karna habit banao.

Indexing + slicing

arr[row, col], slicing aur boolean masks. AI preprocessing me mask-based sele
loops se clearer aur often faster hoti hai.

Axis mental model

axis=0 generally rows ke across operation produce karta hai (columns-wise
aggregation), axis=1 columns ke across (row-wise). Shape ko draw karke rea
karo.

Vectorization

Python loop ko array operations se replace karna. Broadcasting compatible sh
ko align karta hai; incompatible shapes error dete hain.

Reshape + transpose

reshape data layout ko change karta hai without changing element count; transpose
axes swap karta hai. ML tensors me shape errors common interview topic hain.

SMART STUDY CHECK

Bina code dekhe: 1) concept 20 sec me explain karo 2) tiny example likho 3) one
failure case bolo 4) one real AI/data use-case batao.

NUMPY FOR AI + DATA • Learn 2

Easy Hinglish • practical mental model • interview depth

Statistics + missing values

mean, median, std, min/max, nan-aware functions. Missing-value strategy do
model requirement par depend karti hai.

Linear algebra basics

dot/matmul, norms, matrix shape. Neural networks me matrix multiplication c
operation hai.

Random + reproducibility

Random generators aur seeds experiments ko reproducible banane me help
Seed alone complete reproducibility guarantee nahi karta across every envir

Performance thinking

Vectorization, contiguous memory, dtype selection aur avoiding unnecessary
“Fast” bolne ke bajay benchmark/complexity explain karo.

NumPy interview mastery

shape + axis + broadcasting + vectorization + copies/views + dtype + matrix multiplication — ye seven concepts practical code ke saath rehearse karo.

SMART STUDY CHECK

Bina code dekhe: 1) concept 20 sec me explain karo 2) tiny example likho 3) one failure case bolo 4) one real AI/data use-case batao.

INTERVIEW LAB • NUMPY FOR AI + DATA

Smart answer = definition + example + practical use + edge case + follow-up

Q1 • NumPy list se alag kyun?

BEST ANSWER
Numerical arrays ke liye optimized operations, vectorization aur compact dtype provide karta hai.
FOLLOW-UP / TIP
Matrix/data preprocessing.
SPEAK: 30-45 sec, then give one project example.

Q2 • shape kya batata hai?

BEST ANSWER
Har axis ki length. Shape mismatch matrix operations aur ML pipelines me comm bug hai.
FOLLOW-UP / TIP
(3,4) = 3 rows, 4 columns.
SPEAK: 30-45 sec, then give one project example.

Q3 • axis=0 vs axis=1?

BEST ANSWER
2D array me axis=0 par rows aggregate hote hain, result columns-wise hota ha axis=1 par columns aggregate, result row-wise.
FOLLOW-UP / TIP
sum example.
SPEAK: 30-45 sec, then give one project example.

Q4 • broadcasting?

BEST ANSWER
Compatible shapes ko arithmetic ke liye align karna without explicit copying across dimensions.
FOLLOW-UP / TIP
Feature-wise normalization.
SPEAK: 30-45 sec, then give one project example.

INTERVIEW UPGRADE

If interviewer asks “why?”, add a trade-off. If asks “how?”, give implementation steps. If asks “what if it fails?”, explain the failure path.

INTERVIEW LAB • NUMPY FOR AI + DATA

Smart answer = definition + example + practical use + edge case + follow-up

Q5 • vectorization?

BEST ANSWER
Python-level loops ko array operations se replace karna.
FOLLOW-UP / TIP
Faster/clear numerical code.
SPEAK: 30-45 sec, then give one project example.

Q6 • reshape constraint?

BEST ANSWER
Total element count same rehna chahiye.
FOLLOW-UP / TIP
12 elements can become (3,4), not (5,3).
SPEAK: 30-45 sec, then give one project example.

Q7 • view vs copy?

BEST ANSWER
View underlying data share kar sakta hai; copy independent data banata hai.
FOLLOW-UP / TIP
Unexpected mutation debugging.
SPEAK: 30-45 sec, then give one project example.

Q8 • random seed kyun?

BEST ANSWER
Experiments ko repeatable banane me help karta hai.
FOLLOW-UP / TIP
Model/data split or synthetic data.
SPEAK: 30-45 sec, then give one project example.

INTERVIEW UPGRADE

If interviewer asks “why?”, add a trade-off. If asks “how?”, give implementation steps. If asks “what if it fails?”, explain the failure path.

CODING LAB • NUMPY FOR AI + DATA

Pehle solve karo → then answer compare karo → complexity + edge case explain karo.

EASY • PRACTICE

```
Create 2x3 array and print shape.
ANSWER
import numpy as np
a = np.array([[1,2,3],[4,5,6]])
print(a.shape)
CHECK
Input edge cases + expected output + complexity.
```

EASY • PRACTICE

```
Select values > 5.
ANSWER
mask = a > 5
print(a[mask])
CHECK
Input edge cases + expected output + complexity.
```

MEDIUM • PRACTICE

```
Column-wise mean.
ANSWER
print(a.mean(axis=0))
CHECK
Input edge cases + expected output + complexity.
```

MEDIUM • PRACTICE

```
Normalize each column.
ANSWER
mean = a.mean(axis=0)
std = a.std(axis=0)
scaled = (a - mean) / std
CHECK
Input edge cases + expected output + complexity.
```

CODING INTERVIEW LINE

“Main pehle constraints aur edge cases clarify karunga, phir simplest correct solution, phir complexity aur test cases explain karunga.”

CODING LAB • NUMPY FOR AI + DATA

Pehle solve karo → then answer compare karo → complexity + edge case explain karo.

HARD • PRACTICE

```
Broadcast a feature bias.
ANSWER
bias = np.array([1,2,3])
print(a + bias)
CHECK
Input edge cases + expected output + complexity.
```

HARD • PRACTICE

```
Matrix multiplication.
ANSWER
x = np.array([[1,2],[3,4]])
w = np.array([[2],[1]])
print(x @ w)
CHECK
Input edge cases + expected output + complexity.
```

DEBUG • PRACTICE

```
Why does shapes (3,2) and (3,1) fail for @?
ANSWER
Matrix multiplication needs inner dimensions equal: (3,2) @ (3,1) fai
!= 3.
CHECK
Input edge cases + expected output + complexity.
```

INTERVIEW CODE • PRACTICE

```
Compute row sums and column sums.
ANSWER
row_sum = a.sum(axis=1)
col_sum = a.sum(axis=0)
CHECK
Input edge cases + expected output + complexity.
```

CODING INTERVIEW LINE

“Main pehle constraints aur edge cases clarify karunga, phir simplest correct solution, phir complexity aur test cases explain karunga.”

LOGIC + DEBUG LAB • NUMPY FOR AI + DATA

Run karne se pehle predict → dry-run → reason → fix → retest.

PREDICT

What changes first: input, transformed data, or output?
CHECK / ANSWER
Trace data flow from boundary → validation → transformation → result.

TRACE

Pick one code example from today and write variable state after every step.
CHECK / ANSWER
Interviewers value reasoning, not only final output.

WHY

What is the most likely bug if the result is unexpectedly empty?
CHECK / ANSWER
Check input shape/type, filtering condition, join keys or model response.

FIX

What edge case breaks the simple solution?
CHECK / ANSWER
Empty input, NULL/None, duplicate IDs, invalid JSON, timeout or shape mismatc

DESIGN

Where should validation happen?
CHECK / ANSWER
At system boundaries before expensive or irreversible operations.

TRADE-OFF

What would you optimize first?
CHECK / ANSWER
Measure first; then target the actual bottleneck in latency, memory, cost or correctness.

MASTER CHECK

A topic is interview-ready when you can explain it simply, code it without copying, debug one failure, state a trade-off and connect it to your resume

SOLVED PRACTICE - NumPy for AI & Data Science

Every coding-lab question now has its answer, expected result and a quick check - all in one place.

EASY

Create 2x3 array and print shape.

```
import numpy as np
a = np.array([[1,2,3],[4,5,6]])
print(a.shape)
```

EXPECTED RESULT

Array result with explicit shape/dtype reasoning.

EASY

Select values > 5.

```
mask = a > 5
print(a[mask])
```

EXPECTED RESULT

Array result with explicit shape/dtype reasoning.

MEDIUM

Column-wise mean.

```
print(a.mean(axis=0))
```

EXPECTED RESULT

Array result with explicit shape/dtype reasoning.

MEDIUM

Normalize each column.

```
mean = a.mean(axis=0)
std = a.std(axis=0)
scaled = (a - mean) / std
```

EXPECTED RESULT

Array result with explicit shape/dtype reasoning.

HARD

Broadcast a feature bias.

```
bias = np.array([1,2,3])
print(a + bias)
```

EXPECTED RESULT

Array result with explicit shape/dtype reasoning.

HARD

Matrix multiplication.

```
x = np.array([[1,2],[3,4]])
w = np.array([[2],[1]])
print(x @ w)
```

EXPECTED RESULT

Array result with explicit shape/dtype reasoning.

DEBUG

Why does shapes (3,2) and (3,1) fail for @?

```
Matrix multiplication needs inner dimensions equal: (3,2) @ (3,1)
fails because 2
!= 3.
```

EXPECTED RESULT

Array result with explicit shape/dtype reasoning.

INTERVIEW CODE

Compute row sums and column sums.

```
row_sum = a.sum(axis=1)
col_sum = a.sum(axis=0)
```

EXPECTED RESULT

Array result with explicit shape/dtype reasoning.

PRACTICE HABIT

Practice rule: solve first, compare second, then change one input. Mark the question only after you can explain why the output changes.

MINI PROJECT - Vector Calculator

NumPy for AI & Data Science ka concept ek runnable, interview-ready mini build me convert karo.

YE KYA KARTA HAI?

Feature vector ka scale and summary nikalta hai.

KIS KAAM AATA HAI?

ML preprocessing and numeric utilities.

ARCHITECTURE

List -> ndarray -> vectorized ops -> stats

FILES / BUILD TARGET

vector.py Target: Avoid Python loops for element-wise work

CORE IMPLEMENTATION - TYPE + RUN

```
import numpy as np
x = np.array([1., 2., 3.])
print(x * 2)
print(x.mean())
```

SAMPLE OUTPUT `[2. 4. 6.] mean -> 2.0`

INTERVIEW EXPLANATION

Project answer: Vectorization keeps the operation in NumPy and makes shape/dtype explicit. Debug cue: Check x.shape and x.dtype first. Follow-up: What would you improve first if this had 10x traffic/data?

BUILD STEPS

1. Create the file(s) shown above.
2. Run the exact sample once.
3. Change one input and observe the output.
4. Add one validation or failure test.

EDGE CASES / DEBUG

Test invalid input, boundary values and empty/missing data where relevant. Keep validation at the system boundary.

NumPy for AI & Data Science

Use this pack after the section labs: explain -> build -> debug -> defend.

SECTION ANCHOR

Definition: Numeric arrays, vectorization, shape, broadcasting and linear algebra ML/AI ka computation base hai.

Easy explanation: Think shape-first and vectorize dense numeric operations.

Example: `x = np.array([[1,2],[3,4]]); print(x.shape)`

SIMPLE

Vector Calculator

Kya karta hai: Feature vector ka scale and summary nikalta hai.

Kis kaam aata hai: ML preprocessing and numeric utilities.

Architecture: List -> ndarray -> vectorized ops -> stats

Files: `vector.py`

Build target: Avoid Python loops for element-wise work

MEDIUM

Feature Normalizer

Kya karta hai: 2D feature matrix ko row-wise normalize karta hai.

Kis kaam aata hai: Embeddings and ML feature preparation.

Architecture: X -> norm -> guarded divide -> normalized X

Files: `normalize.py`

Build target: Zero vector safe

HARD

Embedding Similarity Search

Kya karta hai: Query embedding ko document vectors ke against rank karta hai.

Kis kaam aata hai: RAG retrieval and semantic search.

Architecture: Query -> embeddings -> cosine scores -> top-k docs

Files: `search.py`

Build target: Zero norms and top-k edge cases

Project rule: code khud type karo, output predict karo, edge case test karo, phir 60-second explanation do.

NumPy for AI & Data Science - implementation lab

Three core implementations + project-tailored interview explanation + debugging cue.

SIMPLE | VECTOR CALCULATOR

```
import numpy as np
x=np.array([1.,2.,3.])
print(x*2)
print(x.mean())
```

INTERVIEW DEFENSE

Project answer: Vectorization keeps the operation in NumPy and makes shape/dtype explicit.

Debug cue: Check x.shape and x.dtype first.

Follow-up: What would you improve first if this had 10x traffic/data?

MEDIUM | FEATURE NORMALIZER

```
import numpy as np
X=np.array([[3.,4.],[0.,0.]])
n=np.linalg.norm(X,axis=1,keepdims=True)
Y=X/np.where(n==0,1,n)
print(Y)
```

INTERVIEW DEFENSE

Project answer: keepdims preserves broadcastable shape; zero norm is an edge case.

Debug cue: Shape mismatch in broadcasting means inspect trailing dimensions.

Follow-up: What would you improve first if this had 10x traffic/data?

HARD | EMBEDDING SIMILARITY SEARCH

```
import numpy as np
def cosine(a,b):
    den=np.linalg.norm(a)*np.linalg.norm(b)
    return 0.0 if den==0 else float(a@b/den)
scores=[cosine(q,d) for d in docs]
print(np.argsort(scores)[::-1][:5])
```

INTERVIEW DEFENSE

Project answer: This is the numeric core behind vector retrieval; real systems add vector indexes and metadata filters.

Debug cue: Inspect embedding dimension consistency before scoring.

Follow-up: What would you improve first if this had 10x traffic/data?

SELF-CHECK

Can I explain the project without notes? Can I name one failure? Can I state time/space or cost trade-offs?

NUMPY FOR AI + DATA • 20-Question Checkpoint

Answer first. Then use the compact answer key.

01 NumPy list se alag kyun?

KEY: Numerical arrays ke liye optimized operations,

02 shape kya batata hai?

KEY: Har axis ki length. Shape mismatch matrix

03 axis=0 vs axis=1?

KEY: 2D array me axis=0 par rows aggregate hote hain,

04 broadcasting?

KEY: Compatible shapes ko arithmetic ke liye align

05 vectorization?

KEY: Python-level loops ko array operations se

06 reshape constraint?

KEY: Total element count same rehna chahiye.

07 view vs copy?

KEY: View underlying data share kar sakta hai; copy

08 random seed kyun?

KEY: Experiments ko repeatable banane me help karta

09 Create 2x3 array and print shape.

KEY: Coding answer: import numpy as np

10 Select values > 5.

KEY: Coding answer: mask = a > 5

11 Column-wise mean.

KEY: Coding answer: print(a.mean(axis=0))

12 Normalize each column.

KEY: Coding answer: mean = a.mean(axis=0)

SCORE RULE

0-8: revise basics • 9-15: practice code • 16-20: explain aloud + build mini task.
Repeat until you can defend the reasoning.

PANDAS FOR DATA SCIENCE • Learn 1

Easy Hinglish • practical mental model • interview depth

Series + DataFrame

Series one-dimensional labeled data; DataFrame tabular labeled data. DataF
grain/row meaning samajhna sabse important.

Inspect before transform

head, tail, shape, columns, dtypes, info, describe, isna. Blind cleaning mat kar
— pehle data profile banao.

Selection

loc label-based; iloc position-based. Boolean filtering ko explicit parentheses
saath likho.

Missing values

isna, fillna, dropna. Mean/median/mode fill blindly mat use karo; missingness
business meaning check karo.

Duplicates + types

duplicated/drop_duplicates, to_numeric, to_datetime, astype. Date parsing failures
ko errors/coercion strategy ke saath handle karo.

SMART STUDY CHECK

Bina code dekhe: 1) concept 20 sec me explain karo 2) tiny example likho 3) one
failure case bolo 4) one real AI/data use-case batao.

PANDAS FOR DATA SCIENCE • Learn 2

Easy Hinglish • practical mental model • interview depth

GroupBy

Split → apply → combine. Aggregation ke baad output grain clearly explain ka

Merge/join

One-to-one, one-to-many, many-to-many cardinality. Duplicate keys silently ro
count explode kar sakti hain — validation check karo.

Pivot + reshape

pivot_table, melt, concat. Dashboard-ready data me wide vs long format ka t
off samjho.

Apply vs vectorization

apply flexible hai but often slower than native vectorized operations. Interview
me first ask: can this be expressed with built-in operations?

EDA workflow

Question → profile → clean → distributions → relationships → anomalies → insights
→ validation. Plot banana goal nahi; decision-support insight goal hai.

SMART STUDY CHECK

Bina code dekhe: 1) concept 20 sec me explain karo 2) tiny example likho 3) one
failure case bolo 4) one real AI/data use-case batao.

INTERVIEW LAB • PANDAS FOR DATA SCIENCE

Smart answer = definition + example + practical use + edge case + follow-up

Q1 • loc vs iloc?

BEST ANSWER
loc labels ke basis par; iloc integer positions ke basis par.
FOLLOW-UP / TIP
df.loc[df["age"]>20].
SPEAK: 30-45 sec, then give one project example.

Q2 • groupby kya karta hai?

BEST ANSWER
Data ko groups me split karke aggregation/transformation combine karta hai.
FOLLOW-UP / TIP
Revenue by region.
SPEAK: 30-45 sec, then give one project example.

Q3 • merge ka biggest risk?

BEST ANSWER
Wrong join keys ya many-to-many relation row count ko unexpectedly multiply sakta hai.
FOLLOW-UP / TIP
Validate cardinality.
SPEAK: 30-45 sec, then give one project example.

Q4 • missing values ka best method?

BEST ANSWER
Context-dependent: drop, impute, flag, or leave missing. Data meaning aur model/business requirement dekho.
FOLLOW-UP / TIP
Never blindly fill all with zero.
SPEAK: 30-45 sec, then give one project example.

INTERVIEW UPGRADE

If interviewer asks “why?”, add a trade-off. If asks “how?”, give implementation steps. If asks “what if it fails?”, explain the failure path.

INTERVIEW LAB • PANDAS FOR DATA SCIENCE

Smart answer = definition + example + practical use + edge case + follow-up

Q5 • apply slow kyun ho sakta hai?

BEST ANSWER
Python function per row/element call hota hai; native vectorized operations ofte
more efficient.
FOLLOW-UP / TIP
Use built-ins first.
SPEAK: 30-45 sec, then give one project example.

Q6 • EDA ka purpose?

BEST ANSWER
Data quality, distributions, relationships, anomalies aur decision-relevant patterns samajhna.
FOLLOW-UP / TIP
Not just making charts.
SPEAK: 30-45 sec, then give one project example.

Q7 • datetime ka common issue?

BEST ANSWER
Mixed formats/timezones or invalid strings. Parse explicitly and validate failures.
FOLLOW-UP / TIP
pd.to_datetime.
SPEAK: 30-45 sec, then give one project example.

Q8 • row grain kya hai?

BEST ANSWER
Har row exactly kis entity/event ko represent karti hai.
FOLLOW-UP / TIP
Join and KPI correctness ke liye critical.
SPEAK: 30-45 sec, then give one project example.

INTERVIEW UPGRADE

If interviewer asks “why?”, add a trade-off. If asks “how?”, give implementation steps. If asks “what if it fails?”, explain the failure path.

CODING LAB • PANDAS FOR DATA SCIENCE

Pehle solve karo → then answer compare karo → complexity + edge case explain karo.

EASY • PRACTICE

```
Create DataFrame and inspect.
ANSWER
import pandas as pd
df = pd.DataFrame({"name":["A","B"],"score":[80,90]})
print(df.info())
CHECK
Input edge cases + expected output + complexity.
```

EASY • PRACTICE

```
Filter score >= 85.
ANSWER
print(df.loc[df["score"] >= 85])
CHECK
Input edge cases + expected output + complexity.
```

MEDIUM • PRACTICE

```
Group revenue by region.
ANSWER
df.groupby("region", as_index=False)["revenue"].sum()
CHECK
Input edge cases + expected output + complexity.
```

MEDIUM • PRACTICE

```
Fill numeric missing values with median.
ANSWER
df["age"] = df["age"].fillna(df["age"].median())
CHECK
Input edge cases + expected output + complexity.
```

CODING INTERVIEW LINE

“Main pehle constraints aur edge cases clarify karunga, phir simplest correct solution, phir complexity aur test cases explain karunga.”

CODING LAB • PANDAS FOR DATA SCIENCE

Pehle solve karo → then answer compare karo → complexity + edge case explain karo.

HARD • PRACTICE

```
Top customer by total revenue.
ANSWER
totals = df.groupby("customer")["revenue"].sum()
print(totals.idxmax(), totals.max())
CHECK
Input edge cases + expected output + complexity.
```

HARD • PRACTICE

```
Merge customers and orders safely.
ANSWER
out = customers.merge(orders, on="customer_id", how="left",
validate="one_to_many")
CHECK
Input edge cases + expected output + complexity.
```

DEBUG • PRACTICE

```
Rows doubled after merge — what do you check?
ANSWER
Check key uniqueness, join cardinality, duplicate IDs, expected grain
row counts.
CHECK
Input edge cases + expected output + complexity.
```

INTERVIEW CODE • PRACTICE

```
Monthly revenue.
ANSWER
df["date"] = pd.to_datetime(df["date"])
monthly = df.groupby(df["date"].dt.to_period("M"))["revenue"].sum()
CHECK
Input edge cases + expected output + complexity.
```

CODING INTERVIEW LINE

“Main pehle constraints aur edge cases clarify karunga, phir simplest correct solution, phir complexity aur test cases explain karunga.”

LOGIC + DEBUG LAB • PANDAS FOR DATA SCIENCE

Run karne se pehle predict → dry-run → reason → fix → retest.

PREDICT

What changes first: input, transformed data, or output?
CHECK / ANSWER
Trace data flow from boundary → validation → transformation → result.

TRACE

Pick one code example from today and write variable state after every step.
CHECK / ANSWER
Interviewers value reasoning, not only final output.

WHY

What is the most likely bug if the result is unexpectedly empty?
CHECK / ANSWER
Check input shape/type, filtering condition, join keys or model response.

FIX

What edge case breaks the simple solution?
CHECK / ANSWER
Empty input, NULL/None, duplicate IDs, invalid JSON, timeout or shape mismatc

DESIGN

Where should validation happen?
CHECK / ANSWER
At system boundaries before expensive or irreversible operations.

TRADE-OFF

What would you optimize first?
CHECK / ANSWER
Measure first; then target the actual bottleneck in latency, memory, cost or correctness.

MASTER CHECK

A topic is interview-ready when you can explain it simply, code it without copying, debug one failure, state a trade-off and connect it to your resume

SOLVED PRACTICE - Pandas for Data Science

Every coding-lab question now has its answer, expected result and a quick check - all in one place.

EASY

Create DataFrame and inspect.

```
import pandas as pd
df = pd.DataFrame({"name":["A","B"],"score":[80,90]})
print(df.info())
```

EXPECTED RESULT

DataFrame transformation/summary is produced.

EASY

Filter score >= 85.

```
print(df.loc[df["score"] >= 85])
```

EXPECTED RESULT

DataFrame transformation/summary is produced.

MEDIUM

Group revenue by region.

```
df.groupby("region", as_index=False)["revenue"].sum()
```

EXPECTED RESULT

[2, 4, 6] style filtered values.

MEDIUM

Fill numeric missing values with median.

```
df["age"] = df["age"].fillna(df["age"].median())
```

EXPECTED RESULT

DataFrame transformation/summary is produced.

HARD

Top customer by total revenue.

```
totals = df.groupby("customer")["revenue"].sum()
print(totals.idxmax(), totals.max())
```

EXPECTED RESULT

[2, 4, 6] style filtered values.

HARD

Merge customers and orders safely.

```
out = customers.merge(orders, on="customer_id", how="left",
validate="one_to_many")
```

EXPECTED RESULT

DataFrame transformation/summary is produced.

DEBUG

Rows doubled after merge - what do you check?

```
Check key uniqueness, join cardinality, duplicate IDs, expected
grain and pre/post
row counts.
```

EXPECTED RESULT

DataFrame transformation/summary is produced.

INTERVIEW CODE

Monthly revenue.

```
df["date"] = pd.to_datetime(df["date"])
monthly = df.groupby(df["date"].dt.to_period("M"))["revenue"].sum()
```

EXPECTED RESULT

[2, 4, 6] style filtered values.

PRACTICE HABIT

Practice rule: solve first, compare second, then change one input. Mark the question only after you can explain why the output changes.

MINI PROJECT - CSV Profiler

Pandas for Data Science ka concept ek runnable, interview-ready mini build me convert karo.

YE KYA KARTA HAI?

CSV ko inspect karke shape, dtypes aur missing counts report karta hai.

KIS KAAM AATA HAI?

EDA ka first pass.

ARCHITECTURE

CSV -> DataFrame -> inspect -> report

FILES / BUILD TARGET

profile.py Target: Quick dataset health summary

CORE IMPLEMENTATION - TYPE + RUN

```
import pandas as pd
df = pd.read_csv("sales.csv")
print(df.shape)
print(df.dtypes)
print(df.isna().sum())
```

SAMPLE OUTPUT `shape + dtypes + missing-value counts printed.`

INTERVIEW EXPLANATION

Project answer: Inspect before transform; otherwise cleaning decisions can hide data problems. Debug cue: Unexpected dtypes often come from dirty source values. Follow-up: What would you improve first if this had 10x traffic/data?

BUILD STEPS

1. Create the file(s) shown above.
2. Run the exact sample once.
3. Change one input and observe the output.
4. Add one validation or failure test.

EDGE CASES / DEBUG

Test invalid input, boundary values and empty/missing data where relevant. Keep validation at the system boundary.

Pandas for Data Science

Use this pack after the section labs: explain -> build -> debug -> defend.

SECTION ANCHOR

Definition: Tabular data ko inspect, clean, transform, aggregate and validate karna Data Science ka core workflow hai.

Easy explanation: Always know the row grain before merge/groupby.

Example: `df.head(); df.info(); df.isna().sum()`

SIMPLE

CSV Profiler

Kya karta hai: CSV ko inspect karke shape, dtypes aur missing counts report karta hai.

Kis kaam aata hai: EDA ka first pass.

Architecture: CSV -> DataFrame -> inspect -> report

Files: `profile.py`

Build target: Quick dataset health summary

MEDIUM

Sales EDA

Kya karta hai: Region-wise revenue and average order value nikalta hai.

Kis kaam aata hai: Business analytics and dashboard preparation.

Architecture: CSV -> clean -> groupby -> KPI table

Files: `eda.py`

Build target: Consistent numeric types

HARD

Safe Customer-Order Merge

Kya karta hai: Customer and order tables merge karke duplicate-row risk detect karta hai.

Kis kaam aata hai: Analytics pipelines and feature datasets.

Architecture: Customers + Orders -> cardinality validation -> aggregate -> output

Files: `merge.py`

Build target: Detect unexpected many-to-many joins

Project rule: code khud type karo, output predict karo, edge case test karo, phir 60-second explanation do.

Pandas for Data Science - implementation lab

Three core implementations + project-tailored interview explanation + debugging cue.

SIMPLE | CSV PROFILER

```
import pandas as pd
df=pd.read_csv("sales.csv")
print(df.shape)
print(df.dtypes)
print(df.isna().sum())
```

INTERVIEW DEFENSE

Project answer: Inspect before transform; otherwise cleaning decisions can hide data problems.

Debug cue: Unexpected dtypes often come from dirty source values.

Follow-up: What would you improve first if this had 10x traffic/data?

MEDIUM | SALES EDA

```
df["revenue"]=pd.to_numeric(df["revenue"],errors="coerce")
summary=(df.groupby("region",as_index=False)["revenue"]
        .agg(total="sum", avg="mean"))
print(summary)
```

INTERVIEW DEFENSE

Project answer: Groupby split-apply-combine hai; output grain clearly state karna chahiye.

Debug cue: Check whether each row is an order, line item, or customer before aggregating.

Follow-up: What would you improve first if this had 10x traffic/data?

HARD | SAFE CUSTOMER-ORDER MERGE

```
merged=orders.merge(customers,on="customer_id",how="left",val
idate="many_to_one")
assert merged["customer_id"].notna().all()
print(merged.shape)
```

INTERVIEW DEFENSE

Project answer: merge cardinality is a major interview and real-world bug source.

Debug cue: If rows multiply, inspect duplicate keys before blaming Pandas.

Follow-up: What would you improve first if this had 10x traffic/data?

SELF-CHECK

Can I explain the project without notes? Can I name one failure? Can I state time/space or cost trade-offs?

PANDAS FOR DATA SCIENCE • 20-Question Checkpoint

Answer first. Then use the compact answer key.

01 loc vs iloc?

KEY: loc labels ke basis par; iloc integer positions

02 groupby kya karta hai?

KEY: Data ko groups me split karke

03 merge ka biggest risk?

KEY: Wrong join keys ya many-to-many relation row

04 missing values ka best method?

KEY: Context-dependent: drop, impute, flag, or leave

05 apply slow kyun ho sakta hai?

KEY: Python function per row/element call hota hai;

06 EDA ka purpose?

KEY: Data quality, distributions, relationships,

07 datetime ka common issue?

KEY: Mixed formats/timezones or invalid strings.

08 row grain kya hai?

KEY: Har row exactly kis entity/event ko represent

09 Create DataFrame and inspect.

KEY: Coding answer: import pandas as pd

10 Filter score >= 85.

KEY: Coding answer: print(df.loc[df["score"] >= 85])

11 Group revenue by region.

KEY: Coding answer: df.groupby("region",

12 Fill numeric missing values with median.

KEY: Coding answer: df["age"] =

SCORE RULE

0-8: revise basics • 9-15: practice code • 16-20: explain aloud + build mini task.
Repeat until you can defend the reasoning.

SQL + DATABASES • Learn 1

Easy Hinglish • practical mental model • interview depth

Query order

Conceptual flow: FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → C LIMIT. Actual engine optimization may differ, but interview reasoning should clear.

Joins

INNER, LEFT, RIGHT/FULL concepts. Always ask what happens to unmatched r whether duplicate keys multiply records.

Aggregations

COUNT, SUM, AVG, MIN, MAX with GROUP BY. COUNT(*) and COUNT(column) NULLs exist.

WHERE vs HAVING

WHERE filters rows before aggregation; HAVING filters groups after aggregatio

Subqueries + CTE

CTE makes multi-step logic readable. Subquery can be scalar, correlated or table-like depending on placement.

SMART STUDY CHECK

Bina code dekhe: 1) concept 20 sec me explain karo 2) tiny example likho 3) one failure case bolo 4) one real AI/data use-case batao.

SQL + DATABASES • Learn 2

Easy Hinglish • practical mental model • interview depth

Window functions

ROW_NUMBER, RANK, DENSE_RANK, SUM OVER, LAG/LEAD. Great for top-N, r and change-over-time analysis.

Indexes

Indexes can speed reads but add storage and write/update cost. Explain base query patterns, selectivity and workload.

Transactions

ACID: atomicity, consistency, isolation, durability. AI apps often need transaction boundaries around chat history and business actions.

MySQL + SQLite

MySQL for server/client multi-user systems; SQLite is lightweight embedded relational database. Both support SQL but operational characteristics differ.

SQL interview mastery

Write query, explain rows at each step, state NULL behavior, join cardinality and expected output.

SMART STUDY CHECK

Bina code dekhe: 1) concept 20 sec me explain karo 2) tiny example likho 3) one failure case bolo 4) one real AI/data use-case batao.

INTERVIEW LAB • SQL + DATABASES

Smart answer = definition + example + practical use + edge case + follow-up

Q1 • WHERE vs HAVING?

BEST ANSWER
WHERE rows ko aggregate se pehle filter karta hai; HAVING groups ko aggregat
baad.
FOLLOW-UP / TIP
Revenue > 1L by region.
SPEAK: 30-45 sec, then give one project example.

Q2 • LEFT JOIN kab?

BEST ANSWER
Left table ke saare rows preserve karne hain, match ho ya na ho.
FOLLOW-UP / TIP
Customers with/without orders.
SPEAK: 30-45 sec, then give one project example.

Q3 • COUNT(*) vs COUNT(col)?

BEST ANSWER
COUNT(*) rows count karta hai; COUNT(col) NULL values ko exclude karta hai.
FOLLOW-UP / TIP
Null behavior.
SPEAK: 30-45 sec, then give one project example.

Q4 • window function?

BEST ANSWER
Related rows par calculation without collapsing rows.
FOLLOW-UP / TIP
Rank/running total.
SPEAK: 30-45 sec, then give one project example.

INTERVIEW UPGRADE

If interviewer asks “why?”, add a trade-off. If asks “how?”, give implementation steps. If asks “what if it fails?”, explain the failure path.

INTERVIEW LAB • SQL + DATABASES

Smart answer = definition + example + practical use + edge case + follow-up

Q5 • index trade-off?

BEST ANSWER
Reads fast ho sakte hain, but storage and writes/update cost increase hoti hai.
FOLLOW-UP / TIP
Workload based.
SPEAK: 30-45 sec, then give one project example.

Q6 • transaction?

BEST ANSWER
Logical unit of work jisme ACID guarantees apply hoti hain.
FOLLOW-UP / TIP
Payment or multi-step DB update.
SPEAK: 30-45 sec, then give one project example.

Q7 • SQL injection kaise avoid?

BEST ANSWER
Parameterized queries/ORM bindings; user input ko SQL string me concatenate karo.
FOLLOW-UP / TIP
Security.
SPEAK: 30-45 sec, then give one project example.

Q8 • join cardinality kyun?

BEST ANSWER
One-to-many join rows multiply kar sakta hai aur KPI double-count ho sakta hai.
FOLLOW-UP / TIP
Validate expected row counts.
SPEAK: 30-45 sec, then give one project example.

INTERVIEW UPGRADE

If interviewer asks “why?”, add a trade-off. If asks “how?”, give implementation steps. If asks “what if it fails?”, explain the failure path.

CODING LAB • SQL + DATABASES

Pehle solve karo → then answer compare karo → complexity + edge case explain karo.

EASY • PRACTICE

```
Find customers with orders.
ANSWER
SELECT DISTINCT c.customer_id FROM customers c JOIN orders o ON
c.customer_id=o.customer_id;
CHECK
Input edge cases + expected output + complexity.
```

EASY • PRACTICE

```
Revenue by region.
ANSWER
SELECT region, SUM(revenue) AS revenue FROM sales GROUP BY region;
CHECK
Input edge cases + expected output + complexity.
```

MEDIUM • PRACTICE

```
Regions with revenue > 100000.
ANSWER
SELECT region, SUM(revenue) AS revenue FROM sales GROUP BY region HAV
SUM(revenue)>100000;
CHECK
Input edge cases + expected output + complexity.
```

MEDIUM • PRACTICE

```
Top 3 products by revenue.
ANSWER
SELECT product_id, SUM(revenue) revenue FROM sales GROUP BY product_id
revenue DESC LIMIT 3;
CHECK
Input edge cases + expected output + complexity.
```

CODING INTERVIEW LINE

“Main pehle constraints aur edge cases clarify karunga, phir simplest correct solution, phir complexity aur test cases explain karunga.”

CODING LAB • SQL + DATABASES

Pehle solve karo → then answer compare karo → complexity + edge case explain karo.

HARD • PRACTICE

```
Rank sales within region.
ANSWER
SELECT region, employee, revenue, RANK() OVER(PARTITION BY region ORD
DESC) rnk FROM sales;
CHECK
Input edge cases + expected output + complexity.
```

HARD • PRACTICE

```
Running revenue total.
ANSWER
SELECT date, revenue, SUM(revenue) OVER(ORDER BY date) AS running_tota
sales;
CHECK
Input edge cases + expected output + complexity.
```

DEBUG • PRACTICE

```
Why is LEFT JOIN count larger?
ANSWER
Many-to-one assumption may be false; duplicate keys on right table ca
left rows.
CHECK
Input edge cases + expected output + complexity.
```

INTERVIEW CODE • PRACTICE

```
Second highest salary.
ANSWER
SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) F
employees);
CHECK
Input edge cases + expected output + complexity.
```

CODING INTERVIEW LINE

“Main pehle constraints aur edge cases clarify karunga, phir simplest correct solution, phir complexity aur test cases explain karunga.”

LOGIC + DEBUG LAB • SQL + DATABASES

Run karne se pehle predict → dry-run → reason → fix → retest.

PREDICT

What changes first: input, transformed data, or output?
CHECK / ANSWER
Trace data flow from boundary → validation → transformation → result.

TRACE

Pick one code example from today and write variable state after every step.
CHECK / ANSWER
Interviewers value reasoning, not only final output.

WHY

What is the most likely bug if the result is unexpectedly empty?
CHECK / ANSWER
Check input shape/type, filtering condition, join keys or model response.

FIX

What edge case breaks the simple solution?
CHECK / ANSWER
Empty input, NULL/None, duplicate IDs, invalid JSON, timeout or shape mismatc

DESIGN

Where should validation happen?
CHECK / ANSWER
At system boundaries before expensive or irreversible operations.

TRADE-OFF

What would you optimize first?
CHECK / ANSWER
Measure first; then target the actual bottleneck in latency, memory, cost or correctness.

MASTER CHECK

A topic is interview-ready when you can explain it simply, code it without copying, debug one failure, state a trade-off and connect it to your resume

SOLVED PRACTICE - SQL + Databases

Every coding-lab question now has its answer, expected result and a quick check - all in one place.

EASY

Find customers with orders.

```
SELECT DISTINCT c.customer_id FROM customers c JOIN orders o ON
c.customer_id=o.customer_id;
```

EXPECTED RESULT

Query returns the requested rows/aggregate.

EASY

Revenue by region.

```
SELECT region, SUM(revenue) AS revenue FROM sales GROUP BY region;
```

EXPECTED RESULT

[2, 4, 6] style filtered values.

MEDIUM

Regions with revenue > 100000.

```
SELECT region, SUM(revenue) AS revenue FROM sales GROUP BY region
HAVING
SUM(revenue)>100000;
```

EXPECTED RESULT

[2, 4, 6] style filtered values.

MEDIUM

Top 3 products by revenue.

```
SELECT product_id, SUM(revenue) revenue FROM sales GROUP BY
product_id ORDER BY
revenue DESC LIMIT 3;
```

EXPECTED RESULT

[2, 4, 6] style filtered values.

HARD

Rank sales within region.

```
SELECT region, employee, revenue, RANK() OVER(PARTITION BY region
ORDER BY revenue
DESC) rnk FROM sales;
```

EXPECTED RESULT

Query returns the requested rows/aggregate.

HARD

Running revenue total.

```
SELECT date, revenue, SUM(revenue) OVER(ORDER BY date) AS
running_total FROM
sales;
```

EXPECTED RESULT

[2, 4, 6] style filtered values.

DEBUG

Why is LEFT JOIN count larger?

```
Many-to-one assumption may be false; duplicate keys on right table
can multiply
left rows.
```

EXPECTED RESULT

Query returns the requested rows/aggregate.

INTERVIEW CODE

Second highest salary.

```
SELECT MAX(salary) FROM employees WHERE salary < (SELECT
MAX(salary) FROM
employees);
```

EXPECTED RESULT

Query returns the requested rows/aggregate.

PRACTICE HABIT

Practice rule: solve first, compare second, then change one input. Mark the question only after you can explain why the output changes.

MINI PROJECT - Customers With Orders

SQL + Databases ka concept ek runnable, interview-ready mini build me convert karo.

YE KYA KARTA HAI?

Customers aur orders join karke active customers find karta hai.

KIS KAAM AATA HAI?

CRM and sales analytics.

ARCHITECTURE

DB -> JOIN -> filter -> result

FILES / BUILD TARGET

query.sql Target: Preserve customers with no orders when asked

CORE IMPLEMENTATION - TYPE + RUN

```
SELECT c.id, c.name, o.order_id
FROM customers c
LEFT JOIN orders o
  ON o.customer_id = c.id;
```

SAMPLE OUTPUT `All customers remain visible; unmatched order_id is NULL.`

INTERVIEW EXPLANATION

Project answer: LEFT JOIN preserves all left-side customers; unmatched right side becomes NULL. Debug cue: Check whether INNER JOIN accidentally removes no-order customers. Follow-up: What would you improve first if this had 10x traffic/data?

BUILD STEPS

1. Create the file(s) shown above.
2. Run the exact sample once.
3. Change one input and observe the output.
4. Add one validation or failure test.

EDGE CASES / DEBUG

Test invalid input, boundary values and empty/missing data where relevant. Keep validation at the system boundary.

SQL + Databases

Use this pack after the section labs: explain -> build -> debug -> defend.

SECTION ANCHOR

Definition: Database se data correctly retrieve, join, aggregate and rank karna analytics/AI systems ki backbone hai.

Easy explanation: Query order and join cardinality matter more than syntax memorization.

Example: `SELECT region, SUM(revenue) FROM orders GROUP BY region;`

SIMPLE

Customers With Orders

Kya karta hai: Customers aur orders join karke active customers find karta hai.

Kis kaam aata hai: CRM and sales analytics.

Architecture: DB -> JOIN -> filter -> result

Files: `query.sql`

Build target: Preserve customers with no orders when asked

MEDIUM

Top Region Sales

Kya karta hai: Region-wise revenue calculate karke highest regions rank karta hai.

Kis kaam aata hai: Power BI/analytics data prep.

Architecture: Orders -> GROUP BY -> aggregate -> ORDER BY

Files: `sales.sql`

Build target: Define revenue and row grain

HARD

Within-Region Ranking

Kya karta hai: Each region me top customers rank karta hai with window functions.

Kis kaam aata hai: Advanced analytics and recommendation inputs.

Architecture: Aggregate -> window rank -> filter top-N

Files: `rank.sql`

Build target: Tie behavior explicit

Project rule: code khud type karo, output predict karo, edge case test karo, phir 60-second explanation do.

SQL + Databases - implementation lab

Three core implementations + project-tailored interview explanation + debugging cue.

SIMPLE | CUSTOMERS WITH ORDERS

```
SELECT c.id, c.name, o.order_id
FROM customers c
LEFT JOIN orders o
  ON o.customer_id = c.id;
```

INTERVIEW DEFENSE

Project answer: LEFT JOIN preserves all left-side customers; unmatched right side becomes NULL.

Debug cue: Check whether INNER JOIN accidentally removes no-order customers.

Follow-up: What would you improve first if this had 10x traffic/data?

MEDIUM | TOP REGION SALES

```
SELECT region, SUM(revenue) AS total_revenue
FROM orders
GROUP BY region
ORDER BY total_revenue DESC;
```

INTERVIEW DEFENSE

Project answer: Aggregation is meaningful only after the table grain is understood.

Debug cue: Duplicate line items can inflate totals.

Follow-up: What would you improve first if this had 10x traffic/data?

HARD | WITHIN-REGION RANKING

```
WITH s AS (
 SELECT region, customer_id, SUM(revenue) rev
 FROM orders GROUP BY region, customer_id)
SELECT *, DENSE_RANK() OVER(PARTITION BY region ORDER BY rev
DESC) rnk
FROM s;
```

INTERVIEW DEFENSE

Project answer: Window functions retain row detail while adding analytic context.

Debug cue: Choose RANK vs DENSE_RANK vs ROW_NUMBER based on tie semantics.

Follow-up: What would you improve first if this had 10x traffic/data?

SELF-CHECK

Can I explain the project without notes? Can I name one failure? Can I state time/space or cost trade-offs?

SQL + DATABASES • 20-Question Checkpoint

Answer first. Then use the compact answer key.

01 WHERE vs HAVING?

KEY: WHERE rows ko aggregate se pehle filter karta

02 LEFT JOIN kab?

KEY: Left table ke saare rows preserve karne hain,

03 COUNT(*) vs COUNT(col)?

KEY: COUNT(*) rows count karta hai; COUNT(col) NULL

04 window function?

KEY: Related rows par calculation without collapsing

05 index trade-off?

KEY: Reads fast ho sakte hain, but storage and

06 transaction?

KEY: Logical unit of work jisme ACID guarantees apply

07 SQL injection kaise avoid?

KEY: Parameterized queries/ORM bindings; user input

08 join cardinality kyun?

KEY: One-to-many join rows multiply kar sakta hai aur

09 Find customers with orders.

KEY: Coding answer: SELECT DISTINCT c.customer_id

10 Revenue by region.

KEY: Coding answer: SELECT region, SUM(revenue) AS

11 Regions with revenue > 100000.

KEY: Coding answer: SELECT region, SUM(revenue) AS

12 Top 3 products by revenue.

KEY: Coding answer: SELECT product_id, SUM(revenue)

SCORE RULE

0-8: revise basics • 9-15: practice code • 16-20: explain aloud + build mini task.
Repeat until you can defend the reasoning.

FLASK + REST APIs • Learn 1

Easy Hinglish • practical mental model • interview depth

HTTP basics

GET retrieve, POST create/process, PUT replace, PATCH partial update, DELE remove. Status codes should communicate outcome.

Flask mental model

Route receives request, validates input, runs service logic, returns response. Keep business logic outside route when app grows.

JSON + validation

Parse JSON, validate required fields/types, return structured errors. Never ass client input is correct.

REST design

Resources, nouns, consistent status codes, pagination/filtering, authentication and idempotency considerations.

AI API integration

Server receives prompt/input → validates → calls model provider → handles timeout/rate limit → normalizes response → logs metadata safely.

SMART STUDY CHECK

Bina code dekhe: 1) concept 20 sec me explain karo 2) tiny example likho 3) one failure case bolo 4) one real AI/data use-case batao.

FLASK + REST APIs • Learn 2

Easy Hinglish • practical mental model • interview depth

Error handling

Differentiate client errors, provider errors, transient failures and internal bugs. Retry only when operation is safe/idempotent.

Database integration

Use parameterized queries/ORM patterns; avoid SQL injection. Keep connection/session lifecycle clear.

Interview architecture

Frontend → Flask API → service layer → AI provider/database → response. Explain where validation, logging and retries live.

SMART STUDY CHECK

Bina code dekhe: 1) concept 20 sec me explain karo 2) tiny example likho 3) one failure case bolo 4) one real AI/data use-case batao.

INTERVIEW LAB • FLASK + REST APIs

Smart answer = definition + example + practical use + edge case + follow-up

Q1 • GET vs POST?

BEST ANSWER
GET data retrieve karne ke liye; POST new resource/action request ke liye commonly.
FOLLOW-UP / TIP
POST /chat.
SPEAK: 30-45 sec, then give one project example.

Q2 • route thin kyun?

BEST ANSWER
Business logic separate service layer me rakhne se testing and maintenance eas hoti hai.
FOLLOW-UP / TIP
Controller/service separation.
SPEAK: 30-45 sec, then give one project example.

Q3 • 401 vs 403?

BEST ANSWER
401 authentication missing/invalid; 403 authenticated client ko permission nah
FOLLOW-UP / TIP
API security.
SPEAK: 30-45 sec, then give one project example.

Q4 • AI API timeout?

BEST ANSWER
Timeout set, safe retry where appropriate, fallback/error response, logs and use friendly status.
FOLLOW-UP / TIP
Do not retry unsafe side effects blindly.
SPEAK: 30-45 sec, then give one project example.

INTERVIEW UPGRADE

If interviewer asks “why?”, add a trade-off. If asks “how?”, give implementation steps. If asks “what if it fails?”, explain the failure path.

INTERVIEW LAB • FLASK + REST APIs

Smart answer = definition + example + practical use + edge case + follow-up

Q5 • JSON validation?

BEST ANSWER
Required fields, types, limits and allowed values check karo before business logic.
FOLLOW-UP / TIP
Schema validation.
SPEAK: 30-45 sec, then give one project example.

Q6 • REST API scalable kaise?

BEST ANSWER
Stateless endpoints, pagination, caching where useful, DB optimization, async/background work and observability.
FOLLOW-UP / TIP
Context dependent.
SPEAK: 30-45 sec, then give one project example.

Q7 • Flask me DB safety?

BEST ANSWER
Parameterized queries/ORM and correct connection lifecycle.
FOLLOW-UP / TIP
Never concatenate raw SQL.
SPEAK: 30-45 sec, then give one project example.

Q8 • AI service abstraction?

BEST ANSWER
Provider SDK ko wrapper/service me isolate karo so provider change rest of app affect na kare.
FOLLOW-UP / TIP
OpenAI/Gemini swap.
SPEAK: 30-45 sec, then give one project example.

INTERVIEW UPGRADE

If interviewer asks “why?”, add a trade-off. If asks “how?”, give implementation steps. If asks “what if it fails?”, explain the failure path.

CODING LAB • FLASK + REST APIs

Pehle solve karo → then answer compare karo → complexity + edge case explain karo.

EASY • PRACTICE

```
Create /health endpoint.
ANSWER
@app.get("/health")
def health():
    return {"status":"ok"}
CHECK
Input edge cases + expected output + complexity.
```

EASY • PRACTICE

```
Read JSON input.
ANSWER
data = request.get_json(silent=True) or {}
name = data.get("name")
CHECK
Input edge cases + expected output + complexity.
```

MEDIUM • PRACTICE

```
Validate required field.
ANSWER
if not name:
    return {"error":"name is required"}, 400
CHECK
Input edge cases + expected output + complexity.
```

MEDIUM • PRACTICE

```
Return created status.
ANSWER
return {"id": new_id}, 201
CHECK
Input edge cases + expected output + complexity.
```

CODING INTERVIEW LINE

“Main pehle constraints aur edge cases clarify karunga, phir simplest correct solution, phir complexity aur test cases explain karunga.”

CODING LAB • FLASK + REST APIs

Pehle solve karo → then answer compare karo → complexity + edge case explain karo.

HARD • PRACTICE

```
AI provider wrapper pattern.
ANSWER
def generate_reply(prompt, client):
    # validate prompt before provider call
    return client.generate(prompt)
CHECK
Input edge cases + expected output + complexity.
```

HARD • PRACTICE

```
Handle provider timeout.
ANSWER
try:
    result = call_model(prompt, timeout=20)
except TimeoutError:
    return {"error":"provider timeout"}, 504
CHECK
Input edge cases + expected output + complexity.
```

DEBUG • PRACTICE

```
Route contains 150 lines — improve.
ANSWER
Move validation/service/database/provider logic into separate functio
and keep route orchestration thin.
CHECK
Input edge cases + expected output + complexity.
```

INTERVIEW CODE • PRACTICE

```
Parameterized SQL concept.
ANSWER
cursor.execute("SELECT * FROM users WHERE id = %s", (user_id,))
CHECK
Input edge cases + expected output + complexity.
```

CODING INTERVIEW LINE

“Main pehle constraints aur edge cases clarify karunga, phir simplest correct solution, phir complexity aur test cases explain karunga.”

LOGIC + DEBUG LAB • FLASK + REST APIs

Run karne se pehle predict → dry-run → reason → fix → retest.

PREDICT

What changes first: input, transformed data, or output?
CHECK / ANSWER
Trace data flow from boundary → validation → transformation → result.

TRACE

Pick one code example from today and write variable state after every step.
CHECK / ANSWER
Interviewers value reasoning, not only final output.

WHY

What is the most likely bug if the result is unexpectedly empty?
CHECK / ANSWER
Check input shape/type, filtering condition, join keys or model response.

FIX

What edge case breaks the simple solution?
CHECK / ANSWER
Empty input, NULL/None, duplicate IDs, invalid JSON, timeout or shape mismatc

DESIGN

Where should validation happen?
CHECK / ANSWER
At system boundaries before expensive or irreversible operations.

TRADE-OFF

What would you optimize first?
CHECK / ANSWER
Measure first; then target the actual bottleneck in latency, memory, cost or correctness.

MASTER CHECK

A topic is interview-ready when you can explain it simply, code it without copying, debug one failure, state a trade-off and connect it to your resume

FLASK + REST APIs • 20-Question Checkpoint

Answer first. Then use the compact answer key.

01 GET vs POST?

KEY: GET data retrieve karne ke liye; POST new

02 route thin kyun?

KEY: Business logic separate service layer me rakhne

03 401 vs 403?

KEY: 401 authentication missing/invalid; 403

04 AI API timeout?

KEY: Timeout set, safe retry where appropriate,

05 JSON validation?

KEY: Required fields, types, limits and allowed

06 REST API scalable kaise?

KEY: Stateless endpoints, pagination, caching where

07 Flask me DB safety?

KEY: Parameterized queries/ORM and correct connection

08 AI service abstraction?

KEY: Provider SDK ko wrapper/service me isolate karo

09 Create /health endpoint.

KEY: Coding answer: @app.get("/health")

10 Read JSON input.

KEY: Coding answer: data =

11 Validate required field.

KEY: Coding answer: if not name:

12 Return created status.

KEY: Coding answer: return {"id": new_id}, 201

SCORE RULE

0-8: revise basics • 9-15: practice code • 16-20: explain aloud + build mini task.
Repeat until you can defend the reasoning.

MACHINE LEARNING + SCIKIT-LEARN • Learn 1

Easy Hinglish • practical mental model • interview depth

ML workflow

Problem → target → split → baseline → preprocess → train → validate → evalua
error analysis → deploy/monitor.

Train/test leakage

Never fit preprocessing on test data. Use Pipeline/ColumnTransformer to keep
transformations inside training folds.

Classification metrics

Accuracy, precision, recall, F1, ROC-AUC/PR-AUC. Metric choice depends on c
balance and cost of errors.

Regression metrics

MAE, MSE, RMSE, R^2. Explain units and sensitivity to outliers.

Feature preprocessing

Numerical imputation/scaling; categorical encoding. Fit transforms only on
training data.

SMART STUDY CHECK

Bina code dekhe: 1) concept 20 sec me explain karo 2) tiny example likho 3) one
failure case bolo 4) one real AI/data use-case batao.

MACHINE LEARNING + SCIKIT-LEARN • Learn 2

Easy Hinglish • practical mental model • interview depth

Model selection

Baseline first. Compare models using consistent validation. Avoid choosing a complex model without evidence.

Cross-validation

Repeated train/validation splits to estimate generalization. Hyperparameter tu must respect validation design.

Class imbalance

Class weights, resampling, threshold tuning and suitable metrics. Do not blin oversample test data.

Interpretability

Feature importance is model-dependent; permutation importance and SHAP c different views. Explain limitations.

Deployment thinking

Serialize pipeline/model, validate input schema, version model and preprocessing, monitor latency and data drift.

SMART STUDY CHECK

Bina code dekhe: 1) concept 20 sec me explain karo 2) tiny example likho 3) one failure case bolo 4) one real AI/data use-case batao.

INTERVIEW LAB • MACHINE LEARNING + SCIKIT-LEARN

Smart answer = definition + example + practical use + edge case + follow-up

Q1 • data leakage?

BEST ANSWER
Validation/test information training process me accidentally use ho jana, resulting in unrealistically high metrics.
FOLLOW-UP / TIP
Fit scaler only on train.
SPEAK: 30-45 sec, then give one project example.

Q2 • accuracy kab misleading?

BEST ANSWER
Severe class imbalance me.
FOLLOW-UP / TIP
Use precision/recall/F1/PR-AUC based on cost.
SPEAK: 30-45 sec, then give one project example.

Q3 • Pipeline kyun?

BEST ANSWER
Preprocessing and model ko one reproducible object me combine karke leakag reduce karta hai.
FOLLOW-UP / TIP
CV-safe workflow.
SPEAK: 30-45 sec, then give one project example.

Q4 • baseline?

BEST ANSWER
Simple reference model/heuristic jisse improvement meaningful compare ho.
FOLLOW-UP / TIP
Majority class or mean predictor.
SPEAK: 30-45 sec, then give one project example.

INTERVIEW UPGRADE

If interviewer asks “why?”, add a trade-off. If asks “how?”, give implementation steps. If asks “what if it fails?”, explain the failure path.

INTERVIEW LAB • MACHINE LEARNING + SCIKIT-LEARN

Smart answer = definition + example + practical use + edge case + follow-up

Q5 • overfitting?

BEST ANSWER
Training performance strong but unseen data poor.
FOLLOW-UP / TIP
Regularization, simpler model, more data, CV.
SPEAK: 30-45 sec, then give one project example.

Q6 • precision?

BEST ANSWER
Predicted positives me kitne actually positive.
FOLLOW-UP / TIP
False positives costly.
SPEAK: 30-45 sec, then give one project example.

Q7 • recall?

BEST ANSWER
Actual positives me kitne detect hue.
FOLLOW-UP / TIP
False negatives costly.
SPEAK: 30-45 sec, then give one project example.

Q8 • model deployment me kya version?

BEST ANSWER
Code, model, preprocessing, config and ideally data/schema assumptions.
FOLLOW-UP / TIP
Reproducibility.
SPEAK: 30-45 sec, then give one project example.

INTERVIEW UPGRADE

If interviewer asks “why?”, add a trade-off. If asks “how?”, give implementation steps. If asks “what if it fails?”, explain the failure path.

CODING LAB • MACHINE LEARNING + SCIKIT-LEARN

Pehle solve karo → then answer compare karo → complexity + edge case explain karo.

EASY • PRACTICE

Train/test split.
ANSWER

```
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,rand
```

CHECK
Input edge cases + expected output + complexity.

EASY • PRACTICE

Classification report.
ANSWER

```
from sklearn.metrics import classification_report
print(classification_report(y_test, pred))
```

CHECK
Input edge cases + expected output + complexity.

MEDIUM • PRACTICE

Pipeline with scaler + logistic regression.
ANSWER

```
from sklearn.pipeline import Pipeline
pipe=Pipeline([("scale",StandardScaler()),("model",LogisticRegression
```

CHECK
Input edge cases + expected output + complexity.

MEDIUM • PRACTICE

Categorical preprocessing.
ANSWER

```
ColumnTransformer([("num",StandardScaler(),num_cols),("cat",OneHotEnco
```

CHECK
Input edge cases + expected output + complexity.

CODING INTERVIEW LINE

“Main pehle constraints aur edge cases clarify karunga, phir simplest correct solution, phir complexity aur test cases explain karunga.”

CODING LAB • MACHINE LEARNING + SCIKIT-LEARN

Pehle solve karo → then answer compare karo → complexity + edge case explain karo.

HARD • PRACTICE

```
Cross-validation score.
ANSWER
from sklearn.model_selection import cross_val_score
scores=cross_val_score(pipe,X,y,cv=5,scoring="f1")
CHECK
Input edge cases + expected output + complexity.
```

HARD • PRACTICE

```
Handle class imbalance.
ANSWER
LogisticRegression(class_weight="balanced")
CHECK
Input edge cases + expected output + complexity.
```

DEBUG • PRACTICE

```
Accuracy 99% but minority recall 10% — explain.
ANSWER
Likely class imbalance or threshold issue. Inspect confusion matrix a
recall/F1/PR-AUC aligned to business cost.
CHECK
Input edge cases + expected output + complexity.
```

INTERVIEW CODE • PRACTICE

```
Regression MAE.
ANSWER
from sklearn.metrics import mean_absolute_error
mae=mean_absolute_error(y_test,pred)
CHECK
Input edge cases + expected output + complexity.
```

CODING INTERVIEW LINE

“Main pehle constraints aur edge cases clarify karunga, phir simplest correct solution, phir complexity aur test cases explain karunga.”

LOGIC + DEBUG LAB • MACHINE LEARNING + SCIKIT-LEAR

Run karne se pehle predict → dry-run → reason → fix → retest.

PREDICT

What changes first: input, transformed data, or output?
CHECK / ANSWER
Trace data flow from boundary → validation → transformation → result.

TRACE

Pick one code example from today and write variable state after every step.
CHECK / ANSWER
Interviewers value reasoning, not only final output.

WHY

What is the most likely bug if the result is unexpectedly empty?
CHECK / ANSWER
Check input shape/type, filtering condition, join keys or model response.

FIX

What edge case breaks the simple solution?
CHECK / ANSWER
Empty input, NULL/None, duplicate IDs, invalid JSON, timeout or shape mismatc

DESIGN

Where should validation happen?
CHECK / ANSWER
At system boundaries before expensive or irreversible operations.

TRADE-OFF

What would you optimize first?
CHECK / ANSWER
Measure first; then target the actual bottleneck in latency, memory, cost or correctness.

MASTER CHECK

A topic is interview-ready when you can explain it simply, code it without copying, debug one failure, state a trade-off and connect it to your resume

SOLVED PRACTICE - Machine Learning + scikit-learn

Every coding-lab question now has its answer, expected result and a quick check - all in one place.

EASY

Train/test split.

```
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)
```

EXPECTED RESULT

Metric/validation result is printed.

EASY

Classification report.

```
from sklearn.metrics import classification_report
print(classification_report(y_test, pred))
```

EXPECTED RESULT

Metric/validation result is printed.

MEDIUM

Pipeline with scaler + logistic regression.

```
from sklearn.pipeline import Pipeline
pipe=Pipeline([("scale",StandardScaler()),("model",LogisticRegression())])
```

EXPECTED RESULT

Metric/validation result is printed.

MEDIUM

Categorical preprocessing.

```
ColumnTransformer([("num",StandardScaler(),num_cols),("cat",OneHotEncoder(handle_unknown="ignore"),cat_cols)])
```

EXPECTED RESULT

Metric/validation result is printed.

HARD

Cross-validation score.

```
from sklearn.model_selection import cross_val_score
scores=cross_val_score(pipe,X,y,cv=5,scoring="f1")
```

EXPECTED RESULT

Metric/validation result is printed.

HARD

Handle class imbalance.

```
LogisticRegression(class_weight="balanced")
```

EXPECTED RESULT

Metric/validation result is printed.

DEBUG

Accuracy 99% but minority recall 10% - explain.

```
Likely class imbalance or threshold issue. Inspect confusion matrix and use
recall/F1/PR-AUC aligned to business cost.
```

EXPECTED RESULT

Metric/validation result is printed.

INTERVIEW CODE

Regression MAE.

```
from sklearn.metrics import mean_absolute_error
mae=mean_absolute_error(y_test,pred)
```

EXPECTED RESULT

Metric/validation result is printed.

PRACTICE HABIT

Practice rule: solve first, compare second, then change one input. Mark the question only after you can explain why the output changes.

MINI PROJECT - Binary Classifier

Machine Learning + scikit-learn ka concept ek runnable, interview-ready mini build me convert karo.

YE KYA KARTA HAI?

Small tabular dataset par classification baseline train karta hai.

KIS KAAM AATA HAI?

ML interview practice and quick prototypes.

ARCHITECTURE

Data -> split -> model -> metric

FILES / BUILD TARGET

train.py Target: Report one classification metric

CORE IMPLEMENTATION - TYPE + RUN

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=.2,
random_state=42)
model = LogisticRegression(max_iter=1000).fit(Xtr, ytr)
print(model.score(Xte, yte))
```

SAMPLE OUTPUT `Test-set score printed after train/test split.`

INTERVIEW EXPLANATION

Project answer: Always establish a baseline and keep test data untouched until evaluation. Debug cue: Leakage often comes from preprocessing before the split. Follow-up: What would you improve first if this had 10x traffic/data?

BUILD STEPS

1. Create the file(s) shown above.
2. Run the exact sample once.
3. Change one input and observe the output.
4. Add one validation or failure test.

EDGE CASES / DEBUG

Test invalid input, boundary values and empty/missing data where relevant. Keep validation at the system boundary.

Machine Learning + scikit-learn

Use this pack after the section labs: explain -> build -> debug -> defend.

SECTION ANCHOR

Definition: ML workflow: target define -> split -> preprocess -> train -> validate -> metric -> error analysis.

Easy explanation: Baseline + leakage-safe evaluation + metric aligned to business cost.

Example: `model.fit(X_train,y_train); pred=model.predict(X_test)`

SIMPLE

Binary Classifier

Kya karta hai: Small tabular dataset par classification baseline train karta hai.

Kis kaam aata hai: ML interview practice and quick prototypes.

Architecture: Data -> split -> model -> metric

Files: `train.py`

Build target: Report one classification metric

MEDIUM

Leakage-Safe Pipeline

Kya karta hai: Scaler + model ko Pipeline me combine karta hai.

Kis kaam aata hai: Production-friendly tabular ML and cross-validation.

Architecture: Raw features -> pipeline preprocessing -> estimator -> CV

Files: `pipeline.py`

Build target: Fit preprocessing only on training folds

HARD

Cross-Validated Model Compare

Kya karta hai: Multiple estimators ko same CV splits and metric par compare karta hai.

Kis kaam aata hai: Model selection and interview system thinking.

Architecture: Dataset -> CV -> models -> metrics -> error analysis

Files: `compare.py`

Build target: Report mean/std, not one lucky split

Project rule: code khud type karo, output predict karo, edge case test karo, phir 60-second explanation do.

Machine Learning + scikit-learn - implementation la

Three core implementations + project-tailored interview explanation + debugging cue.

SIMPLE | BINARY CLASSIFIER

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
Xtr,Xte,ytr,yte=train_test_split(X,y,test_size=.2,random_stat
e=42)
m=LogisticRegression(max_iter=1000).fit(Xtr,ytr)
print(m.score(Xte,yte))
```

INTERVIEW DEFENSE

Project answer: Always establish a baseline and keep test data untouched until evaluation.

Debug cue: Leakage often comes from preprocessing before the split.

Follow-up: What would you improve first if this had 10x traffic/data?

MEDIUM | LEAKAGE-SAFE PIPELINE

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
pipe=Pipeline([("scale",StandardScaler()),("model",LogisticRe
gression())])
pipe.fit(X_train,y_train)
```

INTERVIEW DEFENSE

Project answer: Pipeline reduces leakage risk because transform steps are tied to model fitting.

Debug cue: Never fit the scaler on the full dataset before CV.

Follow-up: What would you improve first if this had 10x traffic/data?

HARD | CROSS-VALIDATED MODEL COMPARE

```
from sklearn.model_selection import cross_validate
for name,model in models.items():
    r=cross_validate(model,X,y,cv=5,scoring="f1")
    print(name,r["test_score"].mean(),r["test_score"].std())
```

INTERVIEW DEFENSE

Project answer: Mean/std reveals stability; choose metrics based on error costs.

Debug cue: Check class imbalance and stratification when appropriate.

Follow-up: What would you improve first if this had 10x traffic/data?

SELF-CHECK

Can I explain the project without notes? Can I name one failure? Can I state time/space or cost trade-offs?

MACHINE LEARNING + SCIKIT-LEARN • 20-Question Chec

Answer first. Then use the compact answer key.

01 data leakage?

KEY: Validation/test information training process me

02 accuracy kab misleading?

KEY: Severe class imbalance me.

03 Pipeline kyun?

KEY: Preprocessing and model ko one reproducible

04 baseline?

KEY: Simple reference model/heuristic jisse

05 overfitting?

KEY: Training performance strong but unseen data

06 precision?

KEY: Predicted positives me kitne actually positive.

07 recall?

KEY: Actual positives me kitne detect hue.

08 model deployment me kya version?

KEY: Code, model, preprocessing, config and ideally

09 Train/test split.

KEY: Coding answer: from sklearn.model_selection

10 Classification report.

KEY: Coding answer: from sklearn.metrics import

11 Pipeline with scaler + logistic regression.

KEY: Coding answer: from sklearn.pipeline import

12 Categorical preprocessing.

KEY: Coding answer:

SCORE RULE

0-8: revise basics • 9-15: practice code • 16-20: explain aloud + build mini task.
Repeat until you can defend the reasoning.

NLP + TRANSFORMERS • Learn 1

Easy Hinglish • practical mental model • interview depth

NLP pipeline

Normalize → tokenize → represent → model → decode/classify → evaluate. Act steps depend on task.

spaCy use cases

Tokenization, linguistic processing, named entities and rule/model pipelines. U where lightweight NLP pipelines are enough.

Transformers basics

Tokenizers convert text to model inputs; transformer attention lets tokens interact contextually. Different architectures target different tasks.

Embeddings

Dense vectors represent semantic information; similarity search can retrieve related text. Embedding quality depends on model and domain.

Fine-tuning vs prompting

Prompting changes instructions; fine-tuning changes model parameters for task/domain adaptation. Choose based on data, cost, latency and maintenance.

SMART STUDY CHECK

Bina code dekhe: 1) concept 20 sec me explain karo 2) tiny example likho 3) one failure case bolo 4) one real AI/data use-case batao.

NLP + TRANSFORMERS • Learn 2

Easy Hinglish • practical mental model • interview depth

NLP interview lab

Intent classification, text preprocessing, evaluation, confusion matrix and error analysis should be explained with your chatbot project.

SMART STUDY CHECK

Bina code dekhe: 1) concept 20 sec me explain karo 2) tiny example likho 3) one failure case bolo 4) one real AI/data use-case batao.

INTERVIEW LAB • NLP + TRANSFORMERS

Smart answer = definition + example + practical use + edge case + follow-up

Q1 • tokenizer?

BEST ANSWER
Text ko model-compatible token IDs/attention masks me convert karta hai.
FOLLOW-UP / TIP
Model input preparation.
SPEAK: 30-45 sec, then give one project example.

Q2 • embedding?

BEST ANSWER
Text/item ko dense vector me represent karta hai for semantic comparison.
FOLLOW-UP / TIP
RAG retrieval.
SPEAK: 30-45 sec, then give one project example.

Q3 • intent classification?

BEST ANSWER
User query ko predefined intent/category assign karna.
FOLLOW-UP / TIP
Chatbot routing.
SPEAK: 30-45 sec, then give one project example.

Q4 • spaCy vs Transformers?

BEST ANSWER
spaCy efficient NLP pipelines/rules/entities ke liye strong; transformers contextual deep models ke liye.
FOLLOW-UP / TIP
Task-dependent.
SPEAK: 30-45 sec, then give one project example.

INTERVIEW UPGRADE

If interviewer asks “why?”, add a trade-off. If asks “how?”, give implementation steps. If asks “what if it fails?”, explain the failure path.

INTERVIEW LAB • NLP + TRANSFORMERS

Smart answer = definition + example + practical use + edge case + follow-up

Q5 • fine-tune kyun?

BEST ANSWER
Task/domain behavior adapt karne ke liye model parameters update.
FOLLOW-UP / TIP
Requires data and training cost.
SPEAK: 30-45 sec, then give one project example.

Q6 • hallucination kya?

BEST ANSWER
Model unsupported or incorrect content confidently generate kar sakta hai.
FOLLOW-UP / TIP
Grounding/evaluation.
SPEAK: 30-45 sec, then give one project example.

Q7 • NLP evaluation?

BEST ANSWER
Classification: precision/recall/F1; generation: task-specific human/automated checks.
FOLLOW-UP / TIP
Always define target.
SPEAK: 30-45 sec, then give one project example.

Q8 • chatbot me NLP pipeline?

BEST ANSWER
Input → preprocessing/tokenization → intent/entity/context → response/tool/LLM output validation.
FOLLOW-UP / TIP
Explain your project.
SPEAK: 30-45 sec, then give one project example.

INTERVIEW UPGRADE

If interviewer asks “why?”, add a trade-off. If asks “how?”, give implementation steps. If asks “what if it fails?”, explain the failure path.

CODING LAB • NLP + TRANSFORMERS

Pehle solve karo → then answer compare karo → complexity + edge case explain karo.

EASY • PRACTICE

```
Simple tokenization concept.
ANSWER
tokens = text.lower().split()
CHECK
Input edge cases + expected output + complexity.
```

EASY • PRACTICE

```
Intent mapping.
ANSWER
intent = {"hello":"greeting","bye":"goodbye"}.get(text,"unknown")
CHECK
Input edge cases + expected output + complexity.
```

MEDIUM • PRACTICE

```
Count intents.
ANSWER
from collections import Counter
counts=Counter(labels)
CHECK
Input edge cases + expected output + complexity.
```

MEDIUM • PRACTICE

```
Confusion matrix.
ANSWER
from sklearn.metrics import confusion_matrix
cm=confusion_matrix(y_true,y_pred)
CHECK
Input edge cases + expected output + complexity.
```

CODING INTERVIEW LINE

“Main pehle constraints aur edge cases clarify karunga, phir simplest correct solution, phir complexity aur test cases explain karunga.”

CODING LAB • NLP + TRANSFORMERS

Pehle solve karo → then answer compare karo → complexity + edge case explain karo.

HARD • PRACTICE

```
Embedding similarity concept.
ANSWER
from sklearn.metrics.pairwise import cosine_similarity
sim=cosine_similarity(query_vec, doc_vectors)
CHECK
Input edge cases + expected output + complexity.
```

HARD • PRACTICE

```
Transformer pipeline concept.
ANSWER
from transformers import pipeline
clf=pipeline("sentiment-analysis")
print(clf("I like this"))
CHECK
Input edge cases + expected output + complexity.
```

DEBUG • PRACTICE

```
Model gets wrong intents despite high training score.
ANSWER
Check class imbalance, leakage, ambiguous labels, preprocessing misma
unseen language/phrasing.
CHECK
Input edge cases + expected output + complexity.
```

INTERVIEW CODE • PRACTICE

```
NLP preprocessing function.
ANSWER
def clean(text):
    return " ".join(text.lower().strip().split())
CHECK
Input edge cases + expected output + complexity.
```

CODING INTERVIEW LINE

“Main pehle constraints aur edge cases clarify karunga, phir simplest correct solution, phir complexity aur test cases explain karunga.”

LOGIC + DEBUG LAB • NLP + TRANSFORMERS

Run karne se pehle predict → dry-run → reason → fix → retest.

PREDICT

What changes first: input, transformed data, or output?
CHECK / ANSWER
Trace data flow from boundary → validation → transformation → result.

TRACE

Pick one code example from today and write variable state after every step.
CHECK / ANSWER
Interviewers value reasoning, not only final output.

WHY

What is the most likely bug if the result is unexpectedly empty?
CHECK / ANSWER
Check input shape/type, filtering condition, join keys or model response.

FIX

What edge case breaks the simple solution?
CHECK / ANSWER
Empty input, NULL/None, duplicate IDs, invalid JSON, timeout or shape mismatc

DESIGN

Where should validation happen?
CHECK / ANSWER
At system boundaries before expensive or irreversible operations.

TRADE-OFF

What would you optimize first?
CHECK / ANSWER
Measure first; then target the actual bottleneck in latency, memory, cost or correctness.

MASTER CHECK

A topic is interview-ready when you can explain it simply, code it without copying, debug one failure, state a trade-off and connect it to your resume

SOLVED PRACTICE - NLP + Transformers

Every coding-lab question now has its answer, expected result and a quick check - all in one place.

EASY

Simple tokenization concept.

```
tokens = text.lower().split()
```

EXPECTED RESULT

Structured text/model output is returned.

EASY

Intent mapping.

```
intent = {"hello":"greeting","bye":"goodbye"}.get(text,"unknown")
```

EXPECTED RESULT

Structured text/model output is returned.

MEDIUM

Count intents.

```
from collections import Counter
counts=Counter(labels)
```

EXPECTED RESULT

Structured text/model output is returned.

MEDIUM

Confusion matrix.

```
from sklearn.metrics import confusion_matrix
cm=confusion_matrix(y_true,y_pred)
```

EXPECTED RESULT

Structured text/model output is returned.

HARD

Embedding similarity concept.

```
from sklearn.metrics.pairwise import cosine_similarity
sim=cosine_similarity(query_vec, doc_vectors)
```

EXPECTED RESULT

Structured text/model output is returned.

HARD

Transformer pipeline concept.

```
from transformers import pipeline
clf=pipeline("sentiment-analysis")
print(clf("I like this"))
```

EXPECTED RESULT

Structured text/model output is returned.

DEBUG

Model gets wrong intents despite high training score.

```
Check class imbalance, leakage, ambiguous labels, preprocessing mismatch and
unseen language/phrasing.
```

EXPECTED RESULT

Structured text/model output is returned.

INTERVIEW CODE

NLP preprocessing function.

```
def clean(text):
    return " ".join(text.lower().strip().split())
```

EXPECTED RESULT

Structured text/model output is returned.

PRACTICE HABIT

Practice rule: solve first, compare second, then change one input. Mark the question only after you can explain why the output changes.

MINI PROJECT - Text Sentiment Demo

NLP + Transformers ka concept ek runnable, interview-ready mini build me convert karo.

YE KYA KARTA HAI?

Pretrained transformer pipeline se sentiment score nikalta hai.

KIS KAAM AATA HAI?

Proof-of-concept NLP.

ARCHITECTURE

Text -> tokenizer/model pipeline -> label

FILES / BUILD TARGET

sentiment.py Target: Handle empty text

CORE IMPLEMENTATION - TYPE + RUN

```
from transformers import pipeline
clf = pipeline("sentiment-analysis")
text = "I like this product"
print(clf(text))
```

SAMPLE OUTPUT `Example sentiment pipeline returns label + score.`

INTERVIEW EXPLANATION

Project answer: Pipeline abstracts tokenization and inference; production adds batching, monitoring and model selection. Debug cue: Model/device/dependency compatibility errors should be surfaced clearly.
Follow-up: What would you improve first if this had 10x traffic/data?

BUILD STEPS

1. Create the file(s) shown above.
2. Run the exact sample once.
3. Change one input and observe the output.
4. Add one validation or failure test.

EDGE CASES / DEBUG

Test invalid input, boundary values and empty/missing data where relevant. Keep validation at the system boundary.

NLP + Transformers

Use this pack after the section labs: explain -> build -> debug -> defend.

SECTION ANCHOR

Definition: Text ko tokenize, represent aur transform-based models se classify/summarize/extract kiya ja sakta hai.

Easy explanation: Start with task + baseline; then choose model, tokenization and evaluation.

Example: `from transformers import pipeline; clf = pipeline("sentiment-analysis")`

SIMPLE

Text Sentiment Demo

Kya karta hai: Pretrained transformer pipeline se sentiment score nikalta hai.

Kis kaam aata hai: Proof-of-concept NLP.

Architecture: Text -> tokenizer/model pipeline -> label

Files: `sentiment.py`

Build target: Handle empty text

MEDIUM

Entity Extraction Service

Kya karta hai: Input text se named entities extract karne ka service wrapper.

Kis kaam aata hai: Document processing and support analytics.

Architecture: Text -> NLP model -> entities -> normalized JSON

Files: `entities.py`

Build target: Stable JSON schema

HARD

Transformer Inference Batch

Kya karta hai: Many texts ko batches me infer karke normalized result store karta hai.

Kis kaam aata hai: Support intent classification or document tagging.

Architecture: Texts -> batches -> model -> normalized outputs -> DB

Files: `infer.py`

Build target: Batch size and max length configurable

Project rule: code khud type karo, output predict karo, edge case test karo, phir 60-second explanation do.

NLP + Transformers - implementation lab

Three core implementations + project-tailored interview explanation + debugging cue.

SIMPLE | TEXT SENTIMENT DEMO

```
from transformers import pipeline
clf=pipeline("sentiment-analysis")
text="I like this product"
print(clf(text))
```

INTERVIEW DEFENSE

Project answer: Pipeline abstracts tokenization and inference; production adds batching, monitoring and model selection.

Debug cue: Model/device/dependency compatibility errors should be surfaced clearly.

Follow-up: What would you improve first if this had 10x traffic/data?

MEDIUM | ENTITY EXTRACTION SERVICE

```
def to_entity(text, span, label):
    return {"text":text[span[0]:span[1]],"label":label}
print(to_entity("Ahetesham works in Pune", (0,8), "PERSON"))
```

INTERVIEW DEFENSE

Project answer: Normalized output contract downstream APIs and dashboards ke liye useful hai.

Debug cue: Token offsets must match the tokenizer/model convention.

Follow-up: What would you improve first if this had 10x traffic/data?

HARD | TRANSFORMER INFERENCE BATCH

```
def batches(items,bs):
    for i in range(0,len(items),bs): yield items[i:i+bs]
for batch in batches(texts,16):
    out = model(batch, truncation=True, max_length=256)
    save(normalize(out))
```

INTERVIEW DEFENSE

Project answer: Batching improves throughput, while max length controls memory/latency.

Debug cue: Watch for truncation, OOM and provider/model device mismatch.

Follow-up: What would you improve first if this had 10x traffic/data?

SELF-CHECK

Can I explain the project without notes? Can I name one failure? Can I state time/space or cost trade-offs?

NLP + TRANSFORMERS • 20-Question Checkpoint

Answer first. Then use the compact answer key.

01 tokenizer?

KEY: Text ko model-compatible token IDs/attention

02 embedding?

KEY: Text/item ko dense vector me represent karta hai

03 intent classification?

KEY: User query ko predefined intent/category assign

04 spaCy vs Transformers?

KEY: spaCy efficient NLP pipelines/rules/entities ke

05 fine-tune kyun?

KEY: Task/domain behavior adapt karne ke liye model

06 hallucination kya?

KEY: Model unsupported or incorrect content

07 NLP evaluation?

KEY: Classification: precision/recall/F1; generation:

08 chatbot me NLP pipeline?

KEY: Input → preprocessing/tokenization →

09 Simple tokenization concept.

KEY: Coding answer: tokens = text.lower().split()

10 Intent mapping.

KEY: Coding answer: intent =

11 Count intents.

KEY: Coding answer: from collections import Counter

12 Confusion matrix.

KEY: Coding answer: from sklearn.metrics import

SCORE RULE

0-8: revise basics • 9-15: practice code • 16-20: explain aloud + build mini task.
Repeat until you can defend the reasoning.

GENAI + LLM + RAG • Learn 1

Easy Hinglish • practical mental model • interview depth

LLM basics

Tokens in → contextual processing → tokens out. Model quality is not the sam factuality; external grounding may be required.

Prompt engineering

Clear role/task, context, constraints, output format and examples. Validate structured outputs rather than trusting free text.

Temperature + sampling

Sampling controls affect output variability. Lower randomness can help deterministic tasks; not a guarantee of correctness.

Tool/function calling

Model selects a declared tool and arguments; application executes tool and re result. Tool permissions must be validated server-side.

Embeddings + vector search

Chunk → embed → store → retrieve top-k → optionally rerank → generate with citations/context.

SMART STUDY CHECK

Bina code dekhe: 1) concept 20 sec me explain karo 2) tiny example likho 3) one failure case bolo 4) one real AI/data use-case batao.

GENAI + LLM + RAG • Learn 2

Easy Hinglish • practical mental model • interview depth

RAG failure modes

Bad chunking, poor retrieval, irrelevant context, stale documents, prompt injection and unsupported answers. Evaluate retrieval separately from gener

RAG vs fine-tuning

RAG is strong for changing/private knowledge; fine-tuning is for behavior/style/task adaptation. They can be combined.

Evaluation

Create representative questions, expected facts, retrieval metrics and answe quality checks. Track latency, cost and failure categories.

OpenAI/Gemini integration

Keep provider calls behind a service interface so the rest of the application do not depend on one SDK.

GenAI interview mastery

Always explain quality + cost + latency + security + evaluation, not only “API call se answer aata hai”.

SMART STUDY CHECK

Bina code dekhe: 1) concept 20 sec me explain karo 2) tiny example likho 3) one failure case bolo 4) one real AI/data use-case batao.

INTERVIEW LAB • GENAI + LLM + RAG

Smart answer = definition + example + practical use + edge case + follow-up

Q1 • RAG kya?

BEST ANSWER
Relevant external context retrieve karke model prompt me dena before genera
FOLLOW-UP / TIP
Private/changing knowledge.
SPEAK: 30-45 sec, then give one project example.

Q2 • chunking kyun?

BEST ANSWER
Documents ko retrievable units me todta hai. Too small loses context; too large hurts retrieval precision.
FOLLOW-UP / TIP
Tune with evaluation.
SPEAK: 30-45 sec, then give one project example.

Q3 • temperature?

BEST ANSWER
Sampling variability control karta hai; lower often more deterministic, but not a correctness guarantee.
FOLLOW-UP / TIP
Use case based.
SPEAK: 30-45 sec, then give one project example.

Q4 • tool calling?

BEST ANSWER
Model structured tool request deta hai; application validates and executes tool.
FOLLOW-UP / TIP
Never trust tool args blindly.
SPEAK: 30-45 sec, then give one project example.

INTERVIEW UPGRADE

If interviewer asks “why?”, add a trade-off. If asks “how?”, give implementation steps. If asks “what if it fails?”, explain the failure path.

INTERVIEW LAB • GENAI + LLM + RAG

Smart answer = definition + example + practical use + edge case + follow-up

Q5 • RAG failure?

BEST ANSWER
Retrieval wrong, chunking poor, stale docs, prompt injection, unsupported answ
FOLLOW-UP / TIP
Evaluate retrieval separately.
SPEAK: 30-45 sec, then give one project example.

Q6 • RAG vs fine-tune?

BEST ANSWER
RAG grounds changing knowledge; fine-tuning adapts behavior/task.
FOLLOW-UP / TIP
Different problems.
SPEAK: 30-45 sec, then give one project example.

Q7 • LLM production metrics?

BEST ANSWER
Quality, latency, cost, failure rate, safety/tool correctness and user satisfaction.
FOLLOW-UP / TIP
Not token count alone.
SPEAK: 30-45 sec, then give one project example.

Q8 • structured output?

BEST ANSWER
Constrain/validate output against schema before downstream automation.
FOLLOW-UP / TIP
Important for n8n/API workflows.
SPEAK: 30-45 sec, then give one project example.

INTERVIEW UPGRADE

If interviewer asks “why?”, add a trade-off. If asks “how?”, give implementation steps. If asks “what if it fails?”, explain the failure path.

CODING LAB • GENAI + LLM + RAG

Pehle solve karo → then answer compare karo → complexity + edge case explain karo.

EASY • PRACTICE

```
Prompt template function.
ANSWER
def prompt(question, context):
    return f"Answer only from context:
{context}
Q:{question}"
CHECK
Input edge cases + expected output + complexity.
```

EASY • PRACTICE

```
Structured result.
ANSWER
result={"answer":answer,"sources":sources,"confidence":score}
CHECK
Input edge cases + expected output + complexity.
```

MEDIUM • PRACTICE

```
Chunk text.
ANSWER
def chunks(text, size=500):
    return [text[i:i+size] for i in range(0,len(text),size)]
CHECK
Input edge cases + expected output + complexity.
```

MEDIUM • PRACTICE

```
Top-k retrieval concept.
ANSWER
scores = cosine_similarity(query_vec, doc_vecs)[0]
top_idx = scores.argsort()[-5:][::-1]
CHECK
Input edge cases + expected output + complexity.
```

CODING INTERVIEW LINE

“Main pehle constraints aur edge cases clarify karunga, phir simplest correct solution, phir complexity aur test cases explain karunga.”

CODING LAB • GENAI + LLM + RAG

Pehle solve karo → then answer compare karo → complexity + edge case explain karo.

HARD • PRACTICE

```
RAG pipeline.
ANSWER
docs=retrieve(query)
context="\n".join(docs)
answer=llm(prompt(query,context))
CHECK
Input edge cases + expected output + complexity.
```

HARD • PRACTICE

```
Validate JSON output.
ANSWER
import json
data=json.loads(model_text)
assert "answer" in data
CHECK
Input edge cases + expected output + complexity.
```

DEBUG • PRACTICE

```
RAG answers unrelated facts.
ANSWER
Inspect query embedding, chunk quality, top-k relevance, metadata fil
prompt grounding instructions.
CHECK
Input edge cases + expected output + complexity.
```

INTERVIEW CODE • PRACTICE

```
Token/cost logging wrapper concept.
ANSWER
start=time.perf_counter()
result=call_model(prompt)
latency=time.perf_counter()-start
log(latency=latency)
CHECK
Input edge cases + expected output + complexity.
```

CODING INTERVIEW LINE

“Main pehle constraints aur edge cases clarify karunga, phir simplest correct solution, phir complexity aur test cases explain karunga.”

LOGIC + DEBUG LAB • GENAI + LLM + RAG

Run karne se pehle predict → dry-run → reason → fix → retest.

PREDICT

What changes first: input, transformed data, or output?
CHECK / ANSWER
Trace data flow from boundary → validation → transformation → result.

TRACE

Pick one code example from today and write variable state after every step.
CHECK / ANSWER
Interviewers value reasoning, not only final output.

WHY

What is the most likely bug if the result is unexpectedly empty?
CHECK / ANSWER
Check input shape/type, filtering condition, join keys or model response.

FIX

What edge case breaks the simple solution?
CHECK / ANSWER
Empty input, NULL/None, duplicate IDs, invalid JSON, timeout or shape mismatc

DESIGN

Where should validation happen?
CHECK / ANSWER
At system boundaries before expensive or irreversible operations.

TRADE-OFF

What would you optimize first?
CHECK / ANSWER
Measure first; then target the actual bottleneck in latency, memory, cost or correctness.

MASTER CHECK

A topic is interview-ready when you can explain it simply, code it without copying, debug one failure, state a trade-off and connect it to your resume

SOLVED PRACTICE - Generative AI, LLMs & RAG

Every coding-lab question now has its answer, expected result and a quick check - all in one place.

EASY

Prompt template function.

```
def prompt(question, context):
    return f"Answer only from context:
{context}
Q:{question}"
```

EXPECTED RESULT
Grounded context is included before model generation.

EASY

Structured result.

```
result={"answer":answer,"sources":sources,"confidence":score}
```

EXPECTED RESULT
Grounded context is included before model generation.

MEDIUM

Chunk text.

```
def chunks(text, size=500):
    return [text[i:i+size] for i in range(0,len(text),size)]
```

EXPECTED RESULT
Grounded context is included before model generation.

MEDIUM

Top-k retrieval concept.

```
scores = cosine_similarity(query_vec, doc_vecs)[0]
top_idx = scores.argsort()[-5:][::-1]
```

EXPECTED RESULT
Grounded context is included before model generation.

HARD

RAG pipeline.

```
docs=retrieve(query)
context="\n".join(docs)
answer=llm(prompt(query,context))
```

EXPECTED RESULT
Grounded context is included before model generation.

HARD

Validate JSON output.

```
import json
data=json.loads(model_text)
assert "answer" in data
```

EXPECTED RESULT
Parsed dict or safe recovery path.

DEBUG

RAG answers unrelated facts.

```
Inspect query embedding, chunk quality, top-k relevance, metadata
filters and
prompt grounding instructions.
```

EXPECTED RESULT
Grounded context is included before model generation.

INTERVIEW CODE

Token/cost logging wrapper concept.

```
start=time.perf_counter()
result=call_model(prompt)
latency=time.perf_counter()-start
log(latency=latency)
```

EXPECTED RESULT
Grounded context is included before model generation.

PRACTICE HABIT

Practice rule: solve first, compare second, then change one input. Mark the question only after you can explain why the output changes.

MINI PROJECT - Prompt Helper

Generative AI, LLMs & RAG ka concept ek runnable, interview-ready mini build me convert karo.

YE KYA KARTA HAI?

Question aur context se reusable grounded prompt banata hai.

KIS KAAM AATA HAI?

Any LLM app prototype.

ARCHITECTURE

Question + context -> prompt template -> model

FILES / BUILD TARGET

prompting.py Target: No empty context

CORE IMPLEMENTATION - TYPE + RUN

```
def prompt(q, context):
    return f"Use only this context.\nContext: {context}\nQ: {q}"

print(prompt("What is X?", "X is Y"))
```

SAMPLE OUTPUT `Context + question rendered into a grounded prompt.`

INTERVIEW EXPLANATION

Project answer: Prompt is a contract; explicitly constrain what the model may use. Debug cue: Empty/irrelevant context should not be silently treated as truth. Follow-up: What would you improve first if this had 10x traffic/data?

BUILD STEPS

1. Create the file(s) shown above.
2. Run the exact sample once.
3. Change one input and observe the output.
4. Add one validation or failure test.

EDGE CASES / DEBUG

Test invalid input, boundary values and empty/missing data where relevant. Keep validation at the system boundary.

PROJECT LAB 22

Generative AI, LLMs & RAG

Use this pack after the section labs: explain -> build -> debug -> defend.

SECTION ANCHOR

Definition: LLM application ko input validation, prompting, retrieval, structured output and evaluation ke system ke roop me dekho.

Easy explanation: RAG = retrieve relevant external context, then generate grounded answer.

Example: `answer = llm(prompt(context, question))`

SIMPLE

Prompt Helper

Kya karta hai: Question aur context se reusable grounded prompt banata hai.

Kis kaam aata hai: Any LLM app prototype.

Architecture: Question + context -> prompt template -> model

Files: `prompting.py`

Build target: No empty context

MEDIUM

Structured Output Validator

Kya karta hai: Model text ko JSON parse karke required fields validate karta hai.

Kis kaam aata hai: Reliable automation and API integrations.

Architecture: LLM -> text -> JSON parse -> schema validate -> action

Files: `parse.py`

Build target: Malformed JSON safe failure

HARD

Mini RAG Pipeline

Kya karta hai: Documents chunk, retrieve top-k and pass context to a model.

Kis kaam aata hai: Company knowledge assistants and document QA.

Architecture: Docs -> chunk -> embed -> retrieve -> prompt -> model

Files: `rag.py`

Build target: Add source IDs and confidence metadata

Project rule: code khud type karo, output predict karo, edge case test karo, phir 60-second explanation do.

Generative AI, LLMs & RAG - implementation lab

Three core implementations + project-tailored interview explanation + debugging cue.

SIMPLE | PROMPT HELPER

```
def prompt(q, context):
    return f"Use only this context.\nContext:{context}\nQ:{q}
"
print(prompt("What is X?","X is Y"))
```

INTERVIEW DEFENSE

Project answer: Prompt is a contract; explicitly constrain what the model may use.

Debug cue: Empty/irrelevant context should not be silently treated as truth.

Follow-up: What would you improve first if this had 10x traffic/data?

MEDIUM | STRUCTURED OUTPUT VALIDATOR

```
import json
def parse(text):
    data=json.loads(text)
    if not isinstance(data.get("answer"),str): raise ValueErr
or("answer")
    return data
```

INTERVIEW DEFENSE

Project answer: Never trust raw model text at a side-effect boundary; parse and validate first.

Debug cue: Handle JSONDecodeError and missing/extra fields according to contract.

Follow-up: What would you improve first if this had 10x traffic/data?

HARD | MINI RAG PIPELINE

```
chunks = chunk(docs, 500)
doc_vecs = embed(chunks)
scores = [cosine(q_vec, v) for v in doc_vecs]
top = [chunks[i] for i in np.argsort(scores)[-5:][::-1]]
answer = llm(prompt(question, "\n".join(top)))
```

INTERVIEW DEFENSE

Project answer: RAG quality is a pipeline problem: retrieval, chunking, grounding and evaluation all matter.

Debug cue: When answers are wrong inspect retrieval before only changing the prompt.

Follow-up: What would you improve first if this had 10x traffic/data?

SELF-CHECK

Can I explain the project without notes? Can I name one failure? Can I state time/space or cost trade-offs?

GENAI + LLM + RAG • 20-Question Checkpoint

Answer first. Then use the compact answer key.

01 RAG kya?

KEY: Relevant external context retrieve karke model

02 chunking kyun?

KEY: Documents ko retrievable units me todta hai. Too

03 temperature?

KEY: Sampling variability control karta hai; lower

04 tool calling?

KEY: Model structured tool request deta hai;

05 RAG failure?

KEY: Retrieval wrong, chunking poor, stale docs,

06 RAG vs fine-tune?

KEY: RAG grounds changing knowledge; fine-tuning

07 LLM production metrics?

KEY: Quality, latency, cost, failure rate,

08 structured output?

KEY: Constrain/validate output against schema before

09 Prompt template function.

KEY: Coding answer: def prompt(question, context):

10 Structured result.

KEY: Coding answer:

11 Chunk text.

KEY: Coding answer: def chunks(text, size=500):

12 Top-k retrieval concept.

KEY: Coding answer: scores =

SCORE RULE

0-8: revise basics • 9-15: practice code • 16-20: explain aloud + build mini task.
Repeat until you can defend the reasoning.

AGENTIC AI + LANGCHAIN + N8N • Learn 1

Easy Hinglish • practical mental model • interview depth

Agent vs workflow

Workflow follows predefined steps; agent can choose among tools/steps base state and model decisions. Many reliable systems combine both.

Agent loop

Observe state → decide next action → call tool → receive result → update state stop/continue. Add explicit limits.

Tool design

Tools should have narrow purpose, typed inputs, clear outputs and permissio boundaries. Never let model directly execute unrestricted system actions.

Memory + state

Short-term conversation state differs from long-term user data. Store only wh needed and define retention/security.

LangChain concepts

Prompts, models, retrievers, tools, chains/runnables and memory/state patterns. Learn the abstraction, not only library syntax.

SMART STUDY CHECK

Bina code dekhe: 1) concept 20 sec me explain karo 2) tiny example likho 3) one failure case bolo 4) one real AI/data use-case batao.

AGENTIC AI + LANGCHAIN + N8N • Learn 2

Easy Hinglish • practical mental model • interview depth

n8n architecture

Trigger → validate → transform → AI node/agent → tool/API/database → condit output → error path.

Webhooks + REST

Validate webhook payload, authenticate source, handle retries/duplicate even return appropriate status quickly.

Agent failure handling

Timeouts, tool errors, malformed JSON, loops, hallucinated tool args, rate lim Add retry limits and fallbacks.

Human approval

For irreversible/high-impact actions, insert approval before final execution. Lo decision context and tool result.

Agent evaluation

Measure task success, tool correctness, latency, token cost, retries and unsafe/invalid actions.

SMART STUDY CHECK

Bina code dekhe: 1) concept 20 sec me explain karo 2) tiny example likho 3) one failure case bolo 4) one real AI/data use-case batao.

INTERVIEW LAB • AGENTIC AI + LANGCHAIN + N8N

Smart answer = definition + example + practical use + edge case + follow-up

Q1 • agent vs workflow?

BEST ANSWER
Workflow fixed orchestration; agent dynamically chooses actions based on state/model decisions.
FOLLOW-UP / TIP
Hybrid often practical.
SPEAK: 30-45 sec, then give one project example.

Q2 • agent loop?

BEST ANSWER
Observe → decide → tool → result → state update → stop/continue.
FOLLOW-UP / TIP
Bound steps.
SPEAK: 30-45 sec, then give one project example.

Q3 • tool safety?

BEST ANSWER
Narrow permissions, validated inputs, explicit outputs, authorization and audit logs.
FOLLOW-UP / TIP
No unrestricted execution.
SPEAK: 30-45 sec, then give one project example.

Q4 • n8n webhook risk?

BEST ANSWER
Unauthenticated/duplicate/malformed requests.
FOLLOW-UP / TIP
Authenticate, validate, idempotency.
SPEAK: 30-45 sec, then give one project example.

INTERVIEW UPGRADE

If interviewer asks “why?”, add a trade-off. If asks “how?”, give implementation steps. If asks “what if it fails?”, explain the failure path.

INTERVIEW LAB • AGENTIC AI + LANGCHAIN + N8N

Smart answer = definition + example + practical use + edge case + follow-up

Q5 • agent loop kaise prevent?

BEST ANSWER
Max iterations, state-based stop condition, timeouts and fallback.
FOLLOW-UP / TIP
Observability.
SPEAK: 30-45 sec, then give one project example.

Q6 • human approval kab?

BEST ANSWER
Irreversible, sensitive or high-impact actions.
FOLLOW-UP / TIP
Approval before execution.
SPEAK: 30-45 sec, then give one project example.

Q7 • agent evaluation?

BEST ANSWER
Task success, tool correctness, latency, cost, retries and unsafe actions.
FOLLOW-UP / TIP
Use test scenarios.
SPEAK: 30-45 sec, then give one project example.

Q8 • 60% reduction kaise explain?

BEST ANSWER
Baseline manual steps define karo, automated workflow measure karo, time/intervention comparison batao.
FOLLOW-UP / TIP
Do not invent measurement details.
SPEAK: 30-45 sec, then give one project example.

INTERVIEW UPGRADE

If interviewer asks “why?”, add a trade-off. If asks “how?”, give implementation steps. If asks “what if it fails?”, explain the failure path.

CODING LAB • AGENTIC AI + LANGCHAIN + N8N

Pehle solve karo → then answer compare karo → complexity + edge case explain karo.

EASY • PRACTICE

Bound agent loop.

ANSWER

```
for step in range(max_steps):
    action=decide(state)
    if action=="stop": break
```

CHECK

Input edge cases + expected output + complexity.

EASY • PRACTICE

Tool schema idea.

ANSWER

```
tool={"name":"search_customer","input":{"customer_id":"string"}}
```

CHECK

Input edge cases + expected output + complexity.

MEDIUM • PRACTICE

Validate tool argument.

ANSWER

```
if not isinstance(customer_id,str) or not customer_id:
    raise ValueError("invalid customer_id")
```

CHECK

Input edge cases + expected output + complexity.

MEDIUM • PRACTICE

Retry only transient errors.

ANSWER

```
for attempt in range(3):
    try: return call()
    except TimeoutError: continue
    except ValueError: raise
```

CHECK

Input edge cases + expected output + complexity.

CODING INTERVIEW LINE

“Main pehle constraints aur edge cases clarify karunga, phir simplest correct solution, phir complexity aur test cases explain karunga.”

CODING LAB • AGENTIC AI + LANGCHAIN + N8N

Pehle solve karo → then answer compare karo → complexity + edge case explain karo.

HARD • PRACTICE

```
Idempotent webhook concept.
ANSWER
event_id = payload["event_id"]
if already_processed(event_id): return {"status":"duplicate"}
CHECK
Input edge cases + expected output + complexity.
```

HARD • PRACTICE

```
Human approval gate.
ANSWER
if action.requires_approval:
    return {"status":"awaiting_approval","action":action}
CHECK
Input edge cases + expected output + complexity.
```

DEBUG • PRACTICE

```
Agent repeats same tool forever.
ANSWER
Track tool/action history, set max iterations, add state-based stop c
detect repeated identical actions.
CHECK
Input edge cases + expected output + complexity.
```

INTERVIEW CODE • PRACTICE

```
Audit log concept.
ANSWER
log_event(user_id, action, tool_name, success, latency)
CHECK
Input edge cases + expected output + complexity.
```

CODING INTERVIEW LINE

“Main pehle constraints aur edge cases clarify karunga, phir simplest correct solution, phir complexity aur test cases explain karunga.”

LOGIC + DEBUG LAB • AGENTIC AI + LANGCHAIN + N8N

Run karne se pehle predict → dry-run → reason → fix → retest.

PREDICT

What changes first: input, transformed data, or output?
CHECK / ANSWER
Trace data flow from boundary → validation → transformation → result.

TRACE

Pick one code example from today and write variable state after every step.
CHECK / ANSWER
Interviewers value reasoning, not only final output.

WHY

What is the most likely bug if the result is unexpectedly empty?
CHECK / ANSWER
Check input shape/type, filtering condition, join keys or model response.

FIX

What edge case breaks the simple solution?
CHECK / ANSWER
Empty input, NULL/None, duplicate IDs, invalid JSON, timeout or shape mismatc

DESIGN

Where should validation happen?
CHECK / ANSWER
At system boundaries before expensive or irreversible operations.

TRADE-OFF

What would you optimize first?
CHECK / ANSWER
Measure first; then target the actual bottleneck in latency, memory, cost or correctness.

MASTER CHECK

A topic is interview-ready when you can explain it simply, code it without copying, debug one failure, state a trade-off and connect it to your resume

SOLVED PRACTICE - Agentic AI + LangChain + n8n

Every coding-lab question now has its answer, expected result and a quick check - all in one place.

EASY

Bound agent loop.

```
for step in range(max_steps):
    action=decide(state)
    if action=="stop": break
```

EXPECTED RESULT

Tool call is bounded, validated and auditable.

EASY

Tool schema idea.

```
tool={"name":"search_customer","input":{"customer_id":"string"}}
```

EXPECTED RESULT

Tool call is bounded, validated and auditable.

MEDIUM

Validate tool argument.

```
if not isinstance(customer_id,str) or not customer_id:
    raise ValueError("invalid customer_id")
```

EXPECTED RESULT

Tool call is bounded, validated and auditable.

MEDIUM

Retry only transient errors.

```
for attempt in range(3):
    try: return call()
    except TimeoutError: continue
    except ValueError: raise
```

EXPECTED RESULT

Specific exception is handled without hiding unrelated failures.

HARD

Idempotent webhook concept.

```
event_id = payload["event_id"]
if already_processed(event_id): return {"status":"duplicate"}
```

EXPECTED RESULT

Validated request/response behavior with a bounded failure path.

HARD

Human approval gate.

```
if action.requires_approval:
    return {"status":"awaiting_approval","action":action}
```

EXPECTED RESULT

Tool call is bounded, validated and auditable.

DEBUG

Agent repeats same tool forever.

```
Track tool/action history, set max iterations, add state-based stop
conditions and
detect repeated identical actions.
```

EXPECTED RESULT

Tool call is bounded, validated and auditable.

INTERVIEW CODE

Audit log concept.

```
log_event(user_id, action, tool_name, success, latency)
```

EXPECTED RESULT

Tool call is bounded, validated and auditable.

PRACTICE HABIT

Practice rule: solve first, compare second, then change one input. Mark the question only after you can explain why the output changes.

MINI PROJECT - Tool Router

Agentic AI + LangChain + n8n ka concept ek runnable, interview-ready mini build me convert karo.

YE KYA KARTA HAI?

Intent ke basis par a narrow tool select karta hai.

KIS KAAM AATA HAI?

Agent-like routing without unrestricted autonomy.

ARCHITECTURE

Input -> router -> allowlisted tool -> result

FILES / BUILD TARGET

router.py Target: Allowlist tools explicitly

CORE IMPLEMENTATION - TYPE + RUN

```
TOOLS = {"weather": get_weather, "math": calc}

def route(name, arg):
    if name not in TOOLS:
        raise ValueError("tool not allowed")
    return TOOLS[name](arg)
```

SAMPLE OUTPUT `Allowed tool name executes; unknown tool is rejected.`

INTERVIEW EXPLANATION

Project answer: Tool contracts should be narrow, typed and permissioned. Debug cue: Never execute arbitrary model-generated function names. Follow-up: What would you improve first if this had 10x traffic/data?

BUILD STEPS

1. Create the file(s) shown above.
2. Run the exact sample once.
3. Change one input and observe the output.
4. Add one validation or failure test.

EDGE CASES / DEBUG

Test invalid input, boundary values and empty/missing data where relevant. Keep validation at the system boundary.

Agentic AI + LangChain + n8n

Use this pack after the section labs: explain -> build -> debug -> defend.

SECTION ANCHOR

Definition: Agentic systems observe state, choose tools/steps, execute, inspect results and stop within bounded limits.

Easy explanation: Agent != chatbot; reliability comes from bounded tools, validation, state and approvals.

Example: `state -> decide -> tool -> result -> state -> stop`

SIMPLE

Tool Router

Kya karta hai: Intent ke basis par a narrow tool select karta hai.

Kis kaam aata hai: Agent-like routing without unrestricted autonomy.

Architecture: Input -> router -> allowlisted tool -> result

Files: `router.py`

Build target: Allowlist tools explicitly

MEDIUM

Bounded Agent Loop

Kya karta hai: Limited steps ke andar observe-decide-tool cycle chalta hai.

Kis kaam aata hai: Research helpers and bounded automation.

Architecture: State -> model decision -> tool -> state -> stop after N

Files: `agent.py`

Build target: Max steps + timeout + tool validation

HARD

n8n Approval-Gated Automation

Kya karta hai: Webhook event ko validate karke AI route karta hai, then sensitive actions approval ke baad run hoti hain.

Kis kaam aata hai: Email/CRM/data-entry automation with AI decisions.

Architecture: Webhook -> validate -> AI/router -> approval -> API/DB -> audit

Files: `n8n workflow | validate.js`

Build target: Retry safe steps, approval for irreversible actions

Project rule: code khud type karo, output predict karo, edge case test karo, phir 60-second explanation do.

Agentic AI + LangChain + n8n - implementation lab

Three core implementations + project-tailored interview explanation + debugging cue.

SIMPLE | TOOL ROUTER

```
TOOLS={"weather":get_weather,"math":calc}
def route(name,arg):
    if name not in TOOLS: raise ValueError("tool")
    return TOOLS[name](arg)
```

INTERVIEW DEFENSE

Project answer: Tool contracts should be narrow, typed and permissioned.

Debug cue: Never execute arbitrary model-generated function names.

Follow-up: What would you improve first if this had 10x traffic/data?

MEDIUM | BOUNDED AGENT LOOP

```
for step in range(5):
    action = decide(state)
    if action["type"] == "final": break
    result = call_tool(action)
    state["last"] = result
else:
    raise TimeoutError("max steps")
```

INTERVIEW DEFENSE

Project answer: A reliable agent has explicit stop conditions, not infinite loops.

Debug cue: Log tool name, sanitized args, result status and step count.

Follow-up: What would you improve first if this had 10x traffic/data?

HARD | N8N APPROVAL-GATED AUTOMATION

```
const p = $json;
if (typeof p.event !== "string") throw new Error("event");
if (!p.data) throw new Error("data");
return [{ json: { ok: true, event:p.event, data:p.data } }];
```

INTERVIEW DEFENSE

Project answer: n8n handles orchestration; the model chooses within guardrails; approvals protect high-impact side effects.

Debug cue: Do not blindly retry non-idempotent actions.

Follow-up: What would you improve first if this had 10x traffic/data?

SELF-CHECK

Can I explain the project without notes? Can I name one failure? Can I state time/space or cost trade-offs?

AGENTIC AI + LANGCHAIN + N8N • 20-Question Checkpoi

Answer first. Then use the compact answer key.

01 agent vs workflow?

KEY: Workflow fixed orchestration; agent dynamically

02 agent loop?

KEY: Observe → decide → tool → result → state update

03 tool safety?

KEY: Narrow permissions, validated inputs, explicit

04 n8n webhook risk?

KEY: Unauthenticated/duplicate/malformed requests.

05 agent loop kaise prevent?

KEY: Max iterations, state-based stop condition,

06 human approval kab?

KEY: Irreversible, sensitive or high-impact actions.

07 agent evaluation?

KEY: Task success, tool correctness, latency, cost,

08 60% reduction kaise explain?

KEY: Baseline manual steps define karo, automated

09 Bound agent loop.

KEY: Coding answer: for step in range(max_steps):

10 Tool schema idea.

KEY: Coding answer:

11 Validate tool argument.

KEY: Coding answer: if not

12 Retry only transient errors.

KEY: Coding answer: for attempt in range(3):

SCORE RULE

0-8: revise basics • 9-15: practice code • 16-20: explain aloud + build mini task.
Repeat until you can defend the reasoning.

DATA ANALYTICS + POWER BI • Learn 1

Easy Hinglish • practical mental model • interview depth

EDA mindset

Start from business question, not chart. Define grain, dimensions, measures quality checks.

KPI design

Metric definition, numerator/denominator, time period, filters and business ow A KPI without definition can mislead.

Power BI model

Fact table + dimensions, relationships, star-schema thinking. Avoid ambiguo relationships.

DAX basics

Measures evaluate in filter context; CALCULATE changes filter context. Explain context vs filter context at interview level.

Excel analytics

Pivot tables, lookup concepts, cleaning and reconciliation. Always validate totals against source.

SMART STUDY CHECK

Bina code dekhe: 1) concept 20 sec me explain karo 2) tiny example likho 3) one failure case bolo 4) one real AI/data use-case batao.

DATA ANALYTICS + POWER BI • Learn 2

Easy Hinglish • practical mental model • interview depth

Dashboard storytelling

Headline insight → evidence → driver → action. Avoid dashboard overload.

30+ analytics challenges

Practice SQL + Pandas + Excel transformations and explain what business de
each output supports.

Interview lab

Given sales/operations data, define KPIs, clean data, build summary, explain one anomaly and propose next analysis.

SMART STUDY CHECK

Bina code dekhe: 1) concept 20 sec me explain karo 2) tiny example likho 3) one failure case bolo 4) one real AI/data use-case batao.

INTERVIEW LAB • DATA ANALYTICS + POWER B

Smart answer = definition + example + practical use + edge case + follow-up

Q1 • KPI kya?

BEST ANSWER
Business objective se linked measurable indicator with defined formula, scope a
period.
FOLLOW-UP / TIP
Revenue growth.
SPEAK: 30-45 sec, then give one project example.

Q2 • star schema?

BEST ANSWER
Fact table measures/events + dimension tables descriptive attributes.
FOLLOW-UP / TIP
Power BI modeling.
SPEAK: 30-45 sec, then give one project example.

Q3 • DAX CALCULATE?

BEST ANSWER
Filter context modify karke expression evaluate karta hai.
FOLLOW-UP / TIP
Core DAX concept.
SPEAK: 30-45 sec, then give one project example.

Q4 • EDA vs dashboard?

BEST ANSWER
EDA exploration/quality/pattern discovery; dashboard recurring decision
monitoring.
FOLLOW-UP / TIP
Different goals.
SPEAK: 30-45 sec, then give one project example.

INTERVIEW UPGRADE

If interviewer asks “why?”, add a trade-off. If asks “how?”, give implementation steps. If asks “what if it fails?”, explain the failure path.

INTERVIEW LAB • DATA ANALYTICS + POWER B

Smart answer = definition + example + practical use + edge case + follow-up

Q5 • data cleaning steps?

BEST ANSWER
Profile → standardize → missing/duplicates → types → outliers/anomalies → validate.
FOLLOW-UP / TIP
Document assumptions.
SPEAK: 30-45 sec, then give one project example.

Q6 • dashboard overload?

BEST ANSWER
Too many visuals reduce clarity.
FOLLOW-UP / TIP
Prioritize decision-critical KPIs.
SPEAK: 30-45 sec, then give one project example.

Q7 • pivot table use?

BEST ANSWER
Fast aggregation and slicing for business exploration.
FOLLOW-UP / TIP
Excel analytics.
SPEAK: 30-45 sec, then give one project example.

Q8 • insight ka format?

BEST ANSWER
Observation + evidence + likely driver + action/next analysis.
FOLLOW-UP / TIP
Avoid generic “sales increased”.
SPEAK: 30-45 sec, then give one project example.

INTERVIEW UPGRADE

If interviewer asks “why?”, add a trade-off. If asks “how?”, give implementation steps. If asks “what if it fails?”, explain the failure path.

CODING LAB • DATA ANALYTICS + POWER BI

Pehle solve karo → then answer compare karo → complexity + edge case explain karo.

EASY • PRACTICE

```
Revenue KPI.
ANSWER
revenue = df["revenue"].sum()
CHECK
Input edge cases + expected output + complexity.
```

EASY • PRACTICE

```
Missing-rate check.
ANSWER
missing_rate = df.isna().mean().sort_values(ascending=False)
CHECK
Input edge cases + expected output + complexity.
```

MEDIUM • PRACTICE

```
Top region.
ANSWER
df.groupby("region")["revenue"].sum().idxmax()
CHECK
Input edge cases + expected output + complexity.
```

MEDIUM • PRACTICE

```
Monthly trend.
ANSWER
df.assign(month=df["date"].dt.to_period("M")).groupby("month")["revenu
CHECK
Input edge cases + expected output + complexity.
```

CODING INTERVIEW LINE

“Main pehle constraints aur edge cases clarify karunga, phir simplest correct solution, phir complexity aur test cases explain karunga.”

CODING LAB • DATA ANALYTICS + POWER BI

Pehle solve karo → then answer compare karo → complexity + edge case explain karo.

HARD • PRACTICE

```
DAX measure concept.
ANSWER
Total Revenue := SUM(Sales[Revenue])
CHECK
Input edge cases + expected output + complexity.
```

HARD • PRACTICE

```
DAX filtered measure concept.
ANSWER
Online Revenue := CALCULATE([Total Revenue], Sales[Channel]="Online")
CHECK
Input edge cases + expected output + complexity.
```

DEBUG • PRACTICE

```
Dashboard total differs from source.
ANSWER
Check filters, relationships, duplicated fact rows, data refresh, dat
measure definition.
CHECK
Input edge cases + expected output + complexity.
```

INTERVIEW CODE • PRACTICE

```
Business insight template.
ANSWER
"Revenue grew 12%; growth was concentrated in Region A, while Region C
Next: inspect product mix and customer churn."
CHECK
Input edge cases + expected output + complexity.
```

CODING INTERVIEW LINE

“Main pehle constraints aur edge cases clarify karunga, phir simplest correct solution, phir complexity aur test cases explain karunga.”

LOGIC + DEBUG LAB • DATA ANALYTICS + POWER BI

Run karne se pehle predict → dry-run → reason → fix → retest.

PREDICT

What changes first: input, transformed data, or output?
CHECK / ANSWER
Trace data flow from boundary → validation → transformation → result.

TRACE

Pick one code example from today and write variable state after every step.
CHECK / ANSWER
Interviewers value reasoning, not only final output.

WHY

What is the most likely bug if the result is unexpectedly empty?
CHECK / ANSWER
Check input shape/type, filtering condition, join keys or model response.

FIX

What edge case breaks the simple solution?
CHECK / ANSWER
Empty input, NULL/None, duplicate IDs, invalid JSON, timeout or shape mismatc

DESIGN

Where should validation happen?
CHECK / ANSWER
At system boundaries before expensive or irreversible operations.

TRADE-OFF

What would you optimize first?
CHECK / ANSWER
Measure first; then target the actual bottleneck in latency, memory, cost or correctness.

MASTER CHECK

A topic is interview-ready when you can explain it simply, code it without copying, debug one failure, state a trade-off and connect it to your resume

SOLVED PRACTICE - Data Analytics + Power BI

Every coding-lab question now has its answer, expected result and a quick check - all in one place.

EASY

Revenue KPI.

```
revenue = df["revenue"].sum()
```

EXPECTED RESULT

[2, 4, 6] style filtered values.

EASY

Missing-rate check.

```
missing_rate = df.isna().mean().sort_values(ascending=False)
```

EXPECTED RESULT

KPI measure returns a business metric.

MEDIUM

Top region.

```
df.groupby("region")["revenue"].sum().idxmax()
```

EXPECTED RESULT

KPI measure returns a business metric.

MEDIUM

Monthly trend.

```
df.assign(month=df["date"].dt.to_period("M")).groupby("month")["reve
nue"].sum()
```

EXPECTED RESULT

KPI measure returns a business metric.

HARD

DAX measure concept.

```
Total Revenue := SUM(Sales[Revenue])
```

EXPECTED RESULT

KPI measure returns a business metric.

HARD

DAX filtered measure concept.

```
Online Revenue := CALCULATE([Total Revenue],
Sales[Channel]="Online")
```

EXPECTED RESULT

KPI measure returns a business metric.

DEBUG

Dashboard total differs from source.

```
Check filters, relationships, duplicated fact rows, data refresh,
date grain and
measure definition.
```

EXPECTED RESULT

KPI measure returns a business metric.

INTERVIEW CODE

Business insight template.

```
"Revenue grew 12%; growth was concentrated in Region A, while
Region C declined.
Next: inspect product mix and customer churn."
```

EXPECTED RESULT

KPI measure returns a business metric.

PRACTICE HABIT

Practice rule: solve first, compare second, then change one input. Mark the question only after you can explain why the output changes.

MINI PROJECT - Revenue KPI

Data Analytics + Power BI ka concept ek runnable, interview-ready mini build me convert karo.

YE KYA KARTA HAI?

Total revenue and order count calculate karta hai.

KIS KAAM AATA HAI?

Business dashboard foundation.

ARCHITECTURE

Data -> cleaned table -> measures -> card visuals

FILES / BUILD TARGET

model.pbix | sales.csv Target: 2 KPIs + one trend chart

CORE IMPLEMENTATION - TYPE + RUN

```
Total Revenue = SUM(Orders[Revenue])
Orders = COUNTROWS(Orders)
```

SAMPLE OUTPUT `Total Revenue and Order Count measures return KPI values.`

INTERVIEW EXPLANATION

Project answer: A KPI must answer a business question, not just look impressive. Debug cue: Check duplicates and filter context before trusting totals. Follow-up: What would you improve first if this had 10x traffic/data?

BUILD STEPS

1. Create the file(s) shown above.
2. Run the exact sample once.
3. Change one input and observe the output.
4. Add one validation or failure test.

EDGE CASES / DEBUG

Test invalid input, boundary values and empty/missing data where relevant. Keep validation at the system boundary.

PROJECT LAB 25

Data Analytics + Power BI

Use this pack after the section labs: explain -> build -> debug -> defend.

SECTION ANCHOR

Definition: Business question se start karke data ko KPI, EDA, model aur dashboard story me convert karo.

Easy explanation: Define grain, KPI numerator/denominator, filters and action.

Example: `Revenue = SUM(Orders[Revenue])`

SIMPLE

Revenue KPI

Kya karta hai: Total revenue and order count calculate karta hai.

Kis kaam aata hai: Business dashboard foundation.

Architecture: Data -> cleaned table -> measures -> card visuals

Files: `model.pbix | sales.csv`

Build target: 2 KPIs + one trend chart

MEDIUM

Sales EDA Dashboard

Kya karta hai: Region/product trends ko Pandas se prepare karke Power BI me visualize karta hai.

Kis kaam aata hai: Portfolio analytics and business reporting.

Architecture: CSV/SQL -> Pandas EDA -> clean output -> Power BI model

Files: `eda.ipynb | model.pbix`

Build target: Clean model + 4 visuals + insight headline

HARD

Executive KPI Storyboard

Kya karta hai: Star-style model, DAX measures and decision-focused dashboard combine karta hai.

Kis kaam aata hai: Professional BI interview and analytics portfolio.

Architecture: Sources -> model -> relationships -> DAX -> visuals -> actions

Files: `model.pbix | dax.txt`

Build target: KPI cards + drillthrough + variance story

Project rule: code khud type karo, output predict karo, edge case test karo, phir 60-second explanation do.

Data Analytics + Power BI - implementation lab

Three core implementations + project-tailored interview explanation + debugging cue.

SIMPLE | REVENUE KPI

```
Total Revenue = SUM(Orders[Revenue])
Orders = COUNTROWS(Orders)
```

INTERVIEW DEFENSE

Project answer: A KPI must answer a business question, not just look impressive.

Debug cue: Check duplicates and filter context before trusting totals.

Follow-up: What would you improve first if this had 10x traffic/data?

MEDIUM | SALES EDA DASHBOARD

```
df.groupby("region")["revenue"].sum().sort_values(ascending=F
alse)
# export cleaned table for BI
```

INTERVIEW DEFENSE

Project answer: The dashboard is the presentation layer; cleaning and metric definitions happen upstream.

Debug cue: Inspect row grain and date relationships before visual calculations.

Follow-up: What would you improve first if this had 10x traffic/data?

HARD | EXECUTIVE KPI STORYBOARD

```
Revenue LY = CALCULATE([Total Revenue], SAMEPERIODLASTYEAR(Da
te[Date]))
YoY % = DIVIDE([Total Revenue]-[Revenue LY],[Revenue LY])
```

INTERVIEW DEFENSE

Project answer: Strong dashboard story = headline insight -> evidence -> driver -> action.

Debug cue: Check filter context and date table completeness.

Follow-up: What would you improve first if this had 10x traffic/data?

SELF-CHECK

Can I explain the project without notes? Can I name one failure? Can I state time/space or cost trade-offs?

DATA ANALYTICS + POWER BI • 20-Question Checkpoint

Answer first. Then use the compact answer key.

01 KPI kya?

KEY: Business objective se linked measurable

02 star schema?

KEY: Fact table measures/events + dimension tables

03 DAX CALCULATE?

KEY: Filter context modify karke expression evaluate

04 EDA vs dashboard?

KEY: EDA exploration/quality/pattern discovery;

05 data cleaning steps?

KEY: Profile → standardize → missing/duplicates →

06 dashboard overload?

KEY: Too many visuals reduce clarity.

07 pivot table use?

KEY: Fast aggregation and slicing for business

08 insight ka format?

KEY: Observation + evidence + likely driver +

09 Revenue KPI.

KEY: Coding answer: revenue = df["revenue"].sum()

10 Missing-rate check.

KEY: Coding answer: missing_rate =

11 Top region.

KEY: Coding answer:

12 Monthly trend.

KEY: Coding answer:

SCORE RULE

0-8: revise basics • 9-15: practice code • 16-20: explain aloud + build mini task.
Repeat until you can defend the reasoning.

MLOPS + DOCKER + GIT + AWS • Learn 1

Easy Hinglish • practical mental model • interview depth

Git fundamentals

Working tree → staging → commit → branch → pull/merge. Commit messages describe intent.

Docker basics

Image packages application/runtime; container is running instance. Dockerfile defines build instructions.

Environment variables

Secrets/config should not be hard-coded. .env for local development; deployr secret stores for production.

MLOps mindset

Version code/data/model, automate tests/builds, monitor model behavior and rollback.

AWS fundamentals

Know compute/storage/networking at conceptual level; explain why a service is selected rather than memorizing names.

SMART STUDY CHECK

Bina code dekhe: 1) concept 20 sec me explain karo 2) tiny example likho 3) one failure case bolo 4) one real AI/data use-case batao.

MLOPS + DOCKER + GIT + AWS • Learn 2

Easy Hinglish • practical mental model • interview depth

Observability

Logs, metrics and traces. For AI apps track latency, provider errors, token usa and user-visible failures.

Security basics

Input validation, authentication, least privilege, secret management, depende updates and prompt/tool security.

Deployment interview

Explain from local code → Git → build → container → cloud runtime → logs/monitoring → rollback.

SMART STUDY CHECK

Bina code dekhe: 1) concept 20 sec me explain karo 2) tiny example likho 3) one failure case bolo 4) one real AI/data use-case batao.

INTERVIEW LAB • MLOPS + DOCKER + GIT + AV

Smart answer = definition + example + practical use + edge case + follow-up

Q1 • image vs container?

BEST ANSWER
Image packaged artifact; container running instance.
FOLLOW-UP / TIP
Docker.
SPEAK: 30-45 sec, then give one project example.

Q2 • why Git branches?

BEST ANSWER
Isolate work and enable review/integration.
FOLLOW-UP / TIP
Team workflow.
SPEAK: 30-45 sec, then give one project example.

Q3 • secret management?

BEST ANSWER
Environment/secret store; never commit keys.
FOLLOW-UP / TIP
Rotate if exposed.
SPEAK: 30-45 sec, then give one project example.

Q4 • MLOps?

BEST ANSWER
Practices to reproducibly build, deploy, monitor and maintain ML systems.
FOLLOW-UP / TIP
Beyond training.
SPEAK: 30-45 sec, then give one project example.

INTERVIEW UPGRADE

If interviewer asks “why?”, add a trade-off. If asks “how?”, give implementation steps. If asks “what if it fails?”, explain the failure path.

INTERVIEW LAB • MLOPS + DOCKER + GIT + AW

Smart answer = definition + example + practical use + edge case + follow-up

Q5 • logs vs metrics?

BEST ANSWER
Logs are event/context records; metrics numeric time-series indicators.
FOLLOW-UP / TIP
Both needed.
SPEAK: 30-45 sec, then give one project example.

Q6 • rollback?

BEST ANSWER
Return service/model to known-good version after failure.
FOLLOW-UP / TIP
Version artifacts.
SPEAK: 30-45 sec, then give one project example.

Q7 • Docker benefit?

BEST ANSWER
Consistent runtime packaging across environments.
FOLLOW-UP / TIP
Not a magic scaling solution.
SPEAK: 30-45 sec, then give one project example.

Q8 • AWS interview approach?

BEST ANSWER
Explain compute/storage/network/service choice by workload rather than memo
product names.
FOLLOW-UP / TIP
State assumptions.
SPEAK: 30-45 sec, then give one project example.

INTERVIEW UPGRADE

If interviewer asks “why?”, add a trade-off. If asks “how?”, give implementation steps. If asks “what if it fails?”, explain the failure path.

CODING LAB • MLOPS + DOCKER + GIT + AWS

Pehle solve karo → then answer compare karo → complexity + edge case explain karo.

EASY • PRACTICE

Dockerfile idea.
ANSWER

```
FROM python:3.12-slim
COPY . /app
WORKDIR /app
RUN pip install -r requirements.txt
CMD ["python","app.py"]
```

CHECK
Input edge cases + expected output + complexity.

EASY • PRACTICE

Git sequence.
ANSWER

```
git status
git add .
git commit -m "add validation"
git push
```

CHECK
Input edge cases + expected output + complexity.

MEDIUM • PRACTICE

Health check endpoint.
ANSWER
Return dependency-light status first; optionally expose deeper readin
separately.
CHECK
Input edge cases + expected output + complexity.

MEDIUM • PRACTICE

Config separation.
ANSWER
Read environment variables at startup; keep defaults non-secret and va
required production config.
CHECK
Input edge cases + expected output + complexity.

CODING INTERVIEW LINE

“Main pehle constraints aur edge cases clarify karunga, phir simplest correct solution, phir complexity aur test cases explain karunga.”

CODING LAB • MLOPS + DOCKER + GIT + AWS

Pehle solve karo → then answer compare karo → complexity + edge case explain karo.

HARD • PRACTICE

```
Model artifact versioning.
ANSWER
Store model + preprocessing + metadata/version together so inference
transformation assumptions.
CHECK
Input edge cases + expected output + complexity.
```

HARD • PRACTICE

```
Monitoring event.
ANSWER
log(model_version, latency_ms, status, input_schema_version, provider_
CHECK
Input edge cases + expected output + complexity.
```

DEBUG • PRACTICE

```
Works locally, fails in container.
ANSWER
Check Python version, dependencies, paths, env vars, ports, file case
and startup command.
CHECK
Input edge cases + expected output + complexity.
```

INTERVIEW CODE • PRACTICE

```
Safe secret rule.
ANSWER
API_KEY = os.getenv("API_KEY")
if not API_KEY: raise RuntimeError("Missing API_KEY")
CHECK
Input edge cases + expected output + complexity.
```

CODING INTERVIEW LINE

“Main pehle constraints aur edge cases clarify karunga, phir simplest correct solution, phir complexity aur test cases explain karunga.”

LOGIC + DEBUG LAB • MLOPS + DOCKER + GIT + AWS

Run karne se pehle predict → dry-run → reason → fix → retest.

PREDICT

What changes first: input, transformed data, or output?
CHECK / ANSWER
Trace data flow from boundary → validation → transformation → result.

TRACE

Pick one code example from today and write variable state after every step.
CHECK / ANSWER
Interviewers value reasoning, not only final output.

WHY

What is the most likely bug if the result is unexpectedly empty?
CHECK / ANSWER
Check input shape/type, filtering condition, join keys or model response.

FIX

What edge case breaks the simple solution?
CHECK / ANSWER
Empty input, NULL/None, duplicate IDs, invalid JSON, timeout or shape mismatc

DESIGN

Where should validation happen?
CHECK / ANSWER
At system boundaries before expensive or irreversible operations.

TRADE-OFF

What would you optimize first?
CHECK / ANSWER
Measure first; then target the actual bottleneck in latency, memory, cost or correctness.

MASTER CHECK

A topic is interview-ready when you can explain it simply, code it without copying, debug one failure, state a trade-off and connect it to your resume

SOLVED PRACTICE - MLOps + Docker + Git + AWS

Every coding-lab question now has its answer, expected result and a quick check - all in one place.

EASY

Dockerfile idea.

```
FROM python:3.12-slim
COPY . /app
WORKDIR /app
RUN pip install -r requirements.txt
CMD ["python","app.py"]
```

EXPECTED RESULT
Text is loaded with explicit UTF-8 encoding.

EASY

Git sequence.

```
git status
git add .
git commit -m "add validation"
git push
```

EXPECTED RESULT
Repeatable runtime/config/observability behavior.

MEDIUM

Health check endpoint.

```
Return dependency-light status first; optionally expose deeper
readiness
separately.
```

EXPECTED RESULT
Repeatable runtime/config/observability behavior.

MEDIUM

Config separation.

```
Read environment variables at startup; keep defaults non-secret and
validate
required production config.
```

EXPECTED RESULT
Repeatable runtime/config/observability behavior.

HARD

Model artifact versioning.

```
Store model + preprocessing + metadata/version together so
inference uses the same
transformation assumptions.
```

EXPECTED RESULT
Repeatable runtime/config/observability behavior.

HARD

Monitoring event.

```
log(model_version, latency_ms, status, input_schema_version,
provider_error)
```

EXPECTED RESULT
[2, 4, 6] style filtered values.

DEBUG

Works locally, fails in container.

```
Check Python version, dependencies, paths, env vars, ports, file
case-sensitivity
and startup command.
```

EXPECTED RESULT
Repeatable runtime/config/observability behavior.

INTERVIEW CODE

Safe secret rule.

```
API_KEY = os.getenv("API_KEY")
if not API_KEY: raise RuntimeError("Missing API_KEY")
```

EXPECTED RESULT
Repeatable runtime/config/observability behavior.

PRACTICE HABIT

Practice rule: solve first, compare second, then change one input. Mark the question only after you can explain why the output changes.

MINI PROJECT - Containerized Python App

MLOps + Docker + Git + AWS ka concept ek runnable, interview-ready mini build me convert karo.

YE KYA KARTA HAI?

Python service ko Docker image me package karta hai.

KIS KAAM AATA HAI?

Consistent local/prod runtime.

ARCHITECTURE

Source -> Dockerfile -> image -> container

FILES / BUILD TARGET

Dockerfile | requirements.txt | app.py Target: Non-root, minimal runtime basics

CORE IMPLEMENTATION - TYPE + RUN

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

SAMPLE OUTPUT `Docker image builds a repeatable Python runtime.`

INTERVIEW EXPLANATION

Project answer: Image packages app plus runtime dependencies; container is the running instance. Debug cue: Check working directory, dependencies and exposed port. Follow-up: What would you improve first if this had 10x traffic/data?

BUILD STEPS

1. Create the file(s) shown above.
2. Run the exact sample once.
3. Change one input and observe the output.
4. Add one validation or failure test.

EDGE CASES / DEBUG

Test invalid input, boundary values and empty/missing data where relevant. Keep validation at the system boundary.

PROJECT LAB 26

MLOps + Docker + Git + AWS

Use this pack after the section labs: explain -> build -> debug -> defend.

SECTION ANCHOR

Definition: AI app ko repeatably build, version, test, package and observe karna deployment skill ka core hai.

Easy explanation: Artifact + environment + observability + rollback.

Example: `git -> test -> build -> image -> deploy -> monitor`

SIMPLE

Containerized Python App

Kya karta hai: Python service ko Docker image me package karta hai.

Kis kaam aata hai: Consistent local/prod runtime.

Architecture: Source -> Dockerfile -> image -> container

Files: `Dockerfile | requirements.txt | app.py`

Build target: Non-root, minimal runtime basics

MEDIUM

Git Feature Workflow

Kya karta hai: Feature branch se change develop, review and merge karta hai.

Kis kaam aata hai: Team software engineering and portfolio hygiene.

Architecture: main -> branch -> commit -> PR -> merge -> release

Files: `.gitignore | README.md`

Build target: Small commits + clean README

HARD

AI Service Observability Pack

Kya karta hai: Latency, provider errors and request counts ko structured events me capture karta hai.

Kis kaam aata hai: Production AI monitoring and incident debugging.

Architecture: Service -> structured logs -> metrics -> alerts -> dashboard

Files: `logging.py | metrics.py`

Build target: Track latency/error/token-cost fields

Project rule: code khud type karo, output predict karo, edge case test karo, phir 60-second explanation do.

MLOps + Docker + Git + AWS - implementation lab

Three core implementations + project-tailored interview explanation + debugging cue.

SIMPLE | CONTAINERIZED PYTHON APP

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python","app.py"]
```

INTERVIEW DEFENSE

Project answer: Image packages app plus runtime dependencies; container is the running instance.

Debug cue: Check working directory, dependencies and exposed port.

Follow-up: What would you improve first if this had 10x traffic/data?

MEDIUM | GIT FEATURE WORKFLOW

```
git switch -c feature/chat
git add .
git commit -m "Add chat endpoint"
git push -u origin feature/chat
```

INTERVIEW DEFENSE

Project answer: Branches isolate work and make review/rollback easier.

Debug cue: Check git status and diff before commit.

Follow-up: What would you improve first if this had 10x traffic/data?

HARD | AI SERVICE OBSERVABILITY PACK

```
event={"route":"/chat","latency_ms":312,"provider":"openai","
ok":True}
log(event)
if event["latency_ms"] > 3000: alert("slow")
```

INTERVIEW DEFENSE

Project answer: Monitoring should answer what failed, where, how often and with what user impact.

Debug cue: Avoid logging secrets, prompts with sensitive data or raw tokens by default.

Follow-up: What would you improve first if this had 10x traffic/data?

SELF-CHECK

Can I explain the project without notes? Can I name one failure? Can I state time/space or cost trade-offs?

MLOPS + DOCKER + GIT + AWS • 20-Question Checkpoint

Answer first. Then use the compact answer key.

01 image vs container?

KEY: Image packaged artifact; container running

02 why Git branches?

KEY: Isolate work and enable review/integration.

03 secret management?

KEY: Environment/secret store; never commit keys.

04 MLOps?

KEY: Practices to reproducibly build, deploy, monitor

05 logs vs metrics?

KEY: Logs are event/context records; metrics numeric

06 rollback?

KEY: Return service/model to known-good version after

07 Docker benefit?

KEY: Consistent runtime packaging across

08 AWS interview approach?

KEY: Explain compute/storage/network/service choice

09 Dockerfile idea.

KEY: Coding answer: FROM python:3.12-slim

10 Git sequence.

KEY: Coding answer: git status

11 Health check endpoint.

KEY: Coding answer: Return dependency-light status

12 Config separation.

KEY: Coding answer: Read environment variables at

SCORE RULE

0-8: revise basics • 9-15: practice code • 16-20: explain aloud + build mini task.
Repeat until you can defend the reasoning.

RESUME PROJECT INTERVIEW LABS • Learn 1

Easy Hinglish • practical mental model • interview depth

AI chatbot 60-sec pitch

Problem → Python/Flask/NLP → REST → MySQL/SQLite → SpaCy/Transformers session/chat history → future OpenAI/Gemini integration.

Chatbot deep questions

Why Flask? Why REST? Why DB? How intent classification works? How session continuity works? How do you handle invalid input/timeouts?

Agentic automation pitch

Trigger → n8n workflow → LLM agent → conditional routing → REST/API/databa → monitoring/error handling → human boundary.

Agentic deep questions

Why agent vs workflow? How avoid loops? What if tool fails? How validate mod output? How measure 60% reduction?

Power BI project pitch

30-day workshop/internship → KPI-driven reports → real-world datasets → visualization → recurring report views.

SMART STUDY CHECK

Bina code dekhe: 1) concept 20 sec me explain karo 2) tiny example likho 3) one failure case bolo 4) one real AI/data use-case batao.

RESUME PROJECT INTERVIEW LABS • Learn 2

Easy Hinglish • practical mental model • interview depth

Analytics project pitch

30+ challenges → SQL + Pandas + NumPy + Excel pivots → EDA/data cleanir BI KPIs → business insights.

Project honesty rule

Interview answers must match what you can explain and demonstrate. If a to integrated conceptually rather than deeply, say so and explain your learning p

Project follow-up drill

For every bullet in resume, prepare: what, why, how, bug, trade-off, result, limitation, next improvement.

SMART STUDY CHECK

Bina code dekhe: 1) concept 20 sec me explain karo 2) tiny example likho 3) one failure case bolo 4) one real AI/data use-case batao.

INTERVIEW LAB • RESUME PROJECT INTERVIEW LABS

Smart answer = definition + example + practical use + edge case + follow-up

Q1 • Apne AI chatbot ko 60 sec me explain karo.

BEST ANSWER
Problem → stack → architecture → key implementation → result → limitation → n
step.
FOLLOW-UP / TIP
Use only claims you can defend.
SPEAK: 30-45 sec, then give one project example.

Q2 • Flask kyun?

BEST ANSWER
Lightweight Python web framework, easy REST endpoint creation and integratio
Python AI/data ecosystem.
FOLLOW-UP / TIP
Mention your architecture.
SPEAK: 30-45 sec, then give one project example.

Q3 • chat history DB me kyun?

BEST ANSWER
Persistence, session continuity, analytics and retrieval of previous interactions.
FOLLOW-UP / TIP
Explain schema at high level.
SPEAK: 30-45 sec, then give one project example.

Q4 • agentic automation me agent kyun?

BEST ANSWER
When workflow needs context-dependent tool/route selection; fixed determinisi
steps can remain ordinary workflow nodes.
FOLLOW-UP / TIP
Hybrid design.
SPEAK: 30-45 sec, then give one project example.

INTERVIEW UPGRADE

If interviewer asks “why?”, add a trade-off. If asks “how?”, give implementation steps. If asks “what if it fails?”, explain the failure path.

INTERVIEW LAB • RESUME PROJECT INTERVIEW LABS

Smart answer = definition + example + practical use + edge case + follow-up

Q5 • 60% reduction kaise prove?

BEST ANSWER
Before/after manual intervention/time measurement and scope define karo.
FOLLOW-UP / TIP
Be precise about methodology.
SPEAK: 30-45 sec, then give one project example.

Q6 • Power BI dashboard ka business value?

BEST ANSWER
Turns cleaned structured data into KPI/trend views supporting recurring decision
FOLLOW-UP / TIP
Give one concrete insight example.
SPEAK: 30-45 sec, then give one project example.

Q7 • project me biggest bug?

BEST ANSWER
Describe a real bug you actually encountered; if not asked, do not invent. Expla
reproduction, root cause and fix.
FOLLOW-UP / TIP
Honesty matters.
SPEAK: 30-45 sec, then give one project example.

Q8 • next improvement?

BEST ANSWER
Testing, monitoring, schema validation, caching, security, evaluation or scalability — choose based on actual project gaps.
FOLLOW-UP / TIP
Explain trade-off.
SPEAK: 30-45 sec, then give one project example.

INTERVIEW UPGRADE

If interviewer asks “why?”, add a trade-off. If asks “how?”, give implementation steps. If asks “what if it fails?”, explain the failure path.

CODING LAB • RESUME PROJECT INTERVIEW LA

Pehle solve karo → then answer compare karo → complexity + edge case explain karo.

EASY • PRACTICE

Project pitch template.
ANSWER
problem → stack → architecture → contribution → result → limitation →
CHECK
Input edge cases + expected output + complexity.

EASY • PRACTICE

Chatbot architecture.
ANSWER
UI → Flask → validation → NLP/LLM service → DB → response
CHECK
Input edge cases + expected output + complexity.

MEDIUM • PRACTICE

Agentic architecture.
ANSWER
Trigger → n8n → state/context → agent → tools → validation → action →
CHECK
Input edge cases + expected output + complexity.

MEDIUM • PRACTICE

Analytics architecture.
ANSWER
Source → cleaning → EDA → SQL/Pandas → model/dashboard → KPI → insight
CHECK
Input edge cases + expected output + complexity.

CODING INTERVIEW LINE

“Main pehle constraints aur edge cases clarify karunga, phir simplest correct solution, phir complexity aur test cases explain karunga.”

CODING LAB • RESUME PROJECT INTERVIEW LA

Pehle solve karo → then answer compare karo → complexity + edge case explain karo.

HARD • PRACTICE

Failure-path checklist.
ANSWER
invalid input, timeout, provider error, DB error, duplicate webhook, model output, stale data
CHECK
Input edge cases + expected output + complexity.

HARD • PRACTICE

Interview trade-off.
ANSWER
Simple workflow vs agent: choose deterministic nodes for fixed steps a decisions only where variability adds value.
CHECK
Input edge cases + expected output + complexity.

DEBUG • PRACTICE

Resume claim challenge.
ANSWER
For each bullet write evidence: dataset, code, metric, test or demo. a detail under pressure.
CHECK
Input edge cases + expected output + complexity.

INTERVIEW CODE • PRACTICE

60-sec answer skeleton.
ANSWER
"I built X to solve Y. I used A/B/C. My main contribution was Z. The h was Q, which I solved by R. The measurable result was S. Next I would
CHECK
Input edge cases + expected output + complexity.

CODING INTERVIEW LINE

“Main pehle constraints aur edge cases clarify karunga, phir simplest correct solution, phir complexity aur test cases explain karunga.”

LOGIC + DEBUG LAB • RESUME PROJECT INTERVIEW LABS

Run karne se pehle predict → dry-run → reason → fix → retest.

PREDICT

What changes first: input, transformed data, or output?
CHECK / ANSWER
Trace data flow from boundary → validation → transformation → result.

TRACE

Pick one code example from today and write variable state after every step.
CHECK / ANSWER
Interviewers value reasoning, not only final output.

WHY

What is the most likely bug if the result is unexpectedly empty?
CHECK / ANSWER
Check input shape/type, filtering condition, join keys or model response.

FIX

What edge case breaks the simple solution?
CHECK / ANSWER
Empty input, NULL/None, duplicate IDs, invalid JSON, timeout or shape mismatc

DESIGN

Where should validation happen?
CHECK / ANSWER
At system boundaries before expensive or irreversible operations.

TRADE-OFF

What would you optimize first?
CHECK / ANSWER
Measure first; then target the actual bottleneck in latency, memory, cost or correctness.

MASTER CHECK

A topic is interview-ready when you can explain it simply, code it without copying, debug one failure, state a trade-off and connect it to your resume

SOLVED PRACTICE - Resume Project Interview Labs

Every coding-lab question now has its answer, expected result and a quick check - all in one place.

EASY

Project pitch template.

```
problem → stack → architecture → contribution → result → limitation
→ next step
```

EXPECTED RESULT

Clear project story: problem -> stack -> contribution -> result.

EASY

Chatbot architecture.

```
UI → Flask → validation → NLP/LLM service → DB → response
```

EXPECTED RESULT

Clear project story: problem -> stack -> contribution -> result.

MEDIUM

Agentic architecture.

```
Trigger → n8n → state/context → agent → tools → validation → action
→ logs
```

EXPECTED RESULT

Clear project story: problem -> stack -> contribution -> result.

MEDIUM

Analytics architecture.

```
Source → cleaning → EDA → SQL/Pandas → model/dashboard → KPI →
insight
```

EXPECTED RESULT

Clear project story: problem -> stack -> contribution -> result.

HARD

Failure-path checklist.

```
invalid input, timeout, provider error, DB error, duplicate
webhook, malformed
model output, stale data
```

EXPECTED RESULT

Clear project story: problem -> stack -> contribution -> result.

HARD

Interview trade-off.

```
Simple workflow vs agent: choose deterministic nodes for fixed
steps and agent
decisions only where variability adds value.
```

EXPECTED RESULT

Clear project story: problem -> stack -> contribution -> result.

DEBUG

Resume claim challenge.

```
For each bullet write evidence: dataset, code, metric, test or
demo. Never invent
a detail under pressure.
```

EXPECTED RESULT

Clear project story: problem -> stack -> contribution -> result.

INTERVIEW CODE

60-sec answer skeleton.

```
"I built X to solve Y. I used A/B/C. My main contribution was Z.
The hardest issue
was Q, which I solved by R. The measurable result was S. Next I
would improve T."
```

EXPECTED RESULT

Clear project story: problem -> stack -> contribution -> result.

PRACTICE HABIT

Practice rule: solve first, compare second, then change one input. Mark the question only after you can explain why the output changes.

MINI PROJECT - Power BI Dashboard Development

Resume Project Interview Labs ka concept ek runnable, interview-ready mini build me convert karo.

YE KYA KARTA HAI?

KPI-driven Power BI reporting project based on real-world datasets.

KIS KAAM AATA HAI?

Business analytics, KPI reporting and data storytelling.

ARCHITECTURE

Dataset -> cleaning/model -> DAX -> visuals -> business insight

FILES / BUILD TARGET

model.pbix | data.xlsx/csv Target: Explain one KPI, one data-cleaning choice and one insight

CORE IMPLEMENTATION - TYPE + RUN

```
Total Revenue = SUM(Orders[Revenue])
Order Count = COUNTROWS(Orders)
AOV = DIVIDE([Total Revenue], [Order Count])
```

SAMPLE OUTPUT `Revenue, Order Count and AOV measures shown.`

INTERVIEW EXPLANATION

Project answer: Resume alignment: describe the business question, data prep, KPI definitions and dashboard decisions. Debug cue: Verify grain, relationships and filter context before claiming numbers. Follow-up: What would you improve first if this had 10x traffic/data?

BUILD STEPS

1. Create the file(s) shown above.
2. Run the exact sample once.
3. Change one input and observe the output.
4. Add one validation or failure test.

EDGE CASES / DEBUG

Test invalid input, boundary values and empty/missing data where relevant. Keep validation at the system boundary.

Resume Project Interview Labs

Use this pack after the section labs: explain -> build -> debug -> defend.

SECTION ANCHOR

Definition: Resume ke actual projects ko stack se nahi, problem, architecture, contribution, failure cases aur measurable result se explain karo.

Easy explanation: Every bullet should survive a why/how/bug/trade-off follow-up.

Example: `Project story = Problem -> Stack -> Architecture -> Contribution -> Result -> Limitation`

SIMPLE

Power BI Dashboard Development

Kya karta hai: KPI-driven Power BI reporting project based on real-world datasets.

Kis kaam aata hai: Business analytics, KPI reporting and data storytelling.

Architecture: Dataset -> cleaning/model -> DAX -> visuals -> business insight

Files: `model.pbix | data.xlsx/csv`

Build target: Explain one KPI, one data-cleaning choice and one insight

MEDIUM

AI-Powered Professional Chatbot

Kya karta hai: Python + Flask + NLP + REST chatbot with persistent history and modular routing.

Kis kaam aata hai: Real-time user support, intent classification and contextual responses.

Architecture: HTML/CSS -> Flask REST -> NLP/intent -> service/LLM -> MySQL/SQLite history

Files: `app.py | routes.py | nlp.py | models.py | templates/`

Build target: Explain one request end-to-end and one failure path

HARD

Agentic AI Automation Workflow with n8n

Kya karta hai: n8n workflow that combines LLM reasoning, webhooks, REST APIs and database connectors for multi-step automation.

Kis kaam aata hai: Email/CRM/data-entry automation and intelligent routing.

Architecture: Webhook -> validate -> AI decision -> tools/APIs/DB -> condition -> output/approval -> audit

Files: `workflow.json | validate.js | README.md`

Build target: Defend retries, idempotency, approvals, timeouts and the 60% claim only if measured

Project rule: code khud type karo, output predict karo, edge case test karo, phir 60-second explanation do.

Resume Project Interview Labs - implementation lab

Three core implementations + project-tailored interview explanation + debugging cue.

SIMPLE | POWER BI DASHBOARD DEVELOPMENT

```
Total Revenue = SUM(Orders[Revenue])
Order Count = COUNTROWS(Orders)
AOV = DIVIDE([Total Revenue],[Order Count])
```

INTERVIEW DEFENSE

Project answer: Resume alignment: describe the business question, data prep, KPI definitions and dashboard decisions.

Debug cue: Verify grain, relationships and filter context before claiming numbers.

Follow-up: What would you improve first if this had 10x traffic/data?

MEDIUM | AI-POWERED PROFESSIONAL CHATBOT

```
@app.post("/chat")
def chat():
    msg=request.json["message"]
    intent=classify_intent(msg)
    reply=generate_response(intent,msg)
    save_history(msg,reply)
    return {"reply":reply,"intent":intent}
```

INTERVIEW DEFENSE

Project answer: Say what you personally implemented: request handling, intent routing, persistence, response generation and modular design.

Debug cue: Trace request -> validation -> intent -> model/service -> DB -> response.

Follow-up: What would you improve first if this had 10x traffic/data?

HARD | AGENTIC AI AUTOMATION WORKFLOW WITH N8N

```
const p=$json;
if(!p.event || !p.data) throw new Error("invalid payload");
// AI decision result must match an allowlisted action
const allowed=["email","crm_update","ignore"];
if(!allowed.includes(p.action)) throw new Error("blocked acti
on");
return [{json:p}];
```

INTERVIEW DEFENSE

Project answer: Resume-aligned story: multi-step automation, webhook/API/database integration, conditional routing and error handling. Use the 60% manual-intervention reduction only when you can explain how it was measured.

Debug cue: For side effects, do not blindly retry; use idempotency keys and human approval when impact is high.

Follow-up: What would you improve first if this had 10x traffic/data?

SELF-CHECK

Can I explain the project without notes? Can I name one failure? Can I state time/space or cost trade-offs?

RESUME PROJECT INTERVIEW LABS • 20-Question Checkp

Answer first. Then use the compact answer key.

01 Apne AI chatbot ko 60 sec me explain karo.

KEY: Problem → stack → architecture → key

02 Flask kyun?

KEY: Lightweight Python web framework, easy REST

03 chat history DB me kyun?

KEY: Persistence, session continuity, analytics and

04 agentic automation me agent kyun?

KEY: When workflow needs context-dependent tool/route

05 60% reduction kaise prove?

KEY: Before/after manual intervention/time

06 Power BI dashboard ka business value?

KEY: Turns cleaned structured data into KPI/trend

07 project me biggest bug?

KEY: Describe a real bug you actually encountered; if

08 next improvement?

KEY: Testing, monitoring, schema validation, caching,

09 Project pitch template.

KEY: Coding answer: problem → stack → architecture →

10 Chatbot architecture.

KEY: Coding answer: UI → Flask → validation → NLP/LLM

11 Agentic architecture.

KEY: Coding answer: Trigger → n8n → state/context →

12 Analytics architecture.

KEY: Coding answer: Source → cleaning → EDA →

SCORE RULE

0-8: revise basics • 9-15: practice code • 16-20: explain aloud + build mini task.
Repeat until you can defend the reasoning.

INTERVIEW MASTER LAB 01

Cross-skill questions combining Python + data + AI + APIs + agents + analytics.

Python → Data

Pandas merge ke baad rows double ho gaye. Sabse pehle kya check karoge?
SMART ANSWER
Join key uniqueness, expected cardinality, duplicate IDs, pre/post row counts and grain.

AI → API

LLM call timeout ho raha hai. Production fix kya hoga?
SMART ANSWER
Timeouts, bounded retries for safe operations, fallback, structured errors, logging and provider health.

Agentic AI

Agent same tool repeat kar raha hai.
SMART ANSWER
Track action history, max steps, stop conditions, duplicate-action detection and fallback.

ML

Validation score unrealistic high hai.
SMART ANSWER
Leakage, duplicated samples, target leakage, preprocessing fit and split strategy inspect karo.

SQL

Revenue suddenly 2x ho gaya after JOIN.
SMART ANSWER
Join cardinality and duplicate keys; compare row counts before/after and aggrega at correct grain.

RAG

Retrieved context irrelevant hai.
SMART ANSWER
Embedding model, chunking, metadata filters, query formulation, top-k and reranking evaluate karo.

Flask

Client malformed JSON bhejta hai.
SMART ANSWER
Validate boundary, return 4xx structured error, do not call downstream AI/DB with invalid payload.

Power BI

Dashboard source total se match nahi karta.
SMART ANSWER
Filters, relationships, duplicate facts, date context, refresh status and DAX measure definition.

Resume

60% manual reduction ko defend kaise karoge?
SMART ANSWER
Define baseline, automated scope, measurement method and period. Only state you actually measured.

Behavior

“Mujhe answer nahi pata” ko professionally kaise handle?
SMART ANSWER
Clarify assumptions, reason from known principles, state uncertainty and explain how you would verify.

FOLLOW-UP RULE

Har answer ke baad interviewer ke 3 likely follow-ups socho: WHY? HOW? WHAT IF IT FAILS? Phir apne project se example connect karo.

INTERVIEW MASTER LAB 02

Cross-skill questions combining Python + data + AI + APIs + agents + analytics.

Python → Data

Pandas merge ke baad rows double ho gaye. Sabse pehle kya check karoge?
SMART ANSWER
Join key uniqueness, expected cardinality, duplicate IDs, pre/post row counts and grain.

AI → API

LLM call timeout ho raha hai. Production fix kya hoga?
SMART ANSWER
Timeouts, bounded retries for safe operations, fallback, structured errors, logging and provider health.

Agentic AI

Agent same tool repeat kar raha hai.
SMART ANSWER
Track action history, max steps, stop conditions, duplicate-action detection and fallback.

ML

Validation score unrealistic high hai.
SMART ANSWER
Leakage, duplicated samples, target leakage, preprocessing fit and split strategy inspect karo.

SQL

Revenue suddenly 2x ho gaya after JOIN.
SMART ANSWER
Join cardinality and duplicate keys; compare row counts before/after and aggrega at correct grain.

RAG

Retrieved context irrelevant hai.
SMART ANSWER
Embedding model, chunking, metadata filters, query formulation, top-k and reranking evaluate karo.

Flask

Client malformed JSON bhejta hai.
SMART ANSWER
Validate boundary, return 4xx structured error, do not call downstream AI/DB with invalid payload.

Power BI

Dashboard source total se match nahi karta.
SMART ANSWER
Filters, relationships, duplicate facts, date context, refresh status and DAX measure definition.

Resume

60% manual reduction ko defend kaise karoge?
SMART ANSWER
Define baseline, automated scope, measurement method and period. Only state you actually measured.

Behavior

“Mujhe answer nahi pata” ko professionally kaise handle?
SMART ANSWER
Clarify assumptions, reason from known principles, state uncertainty and explain how you would verify.

FOLLOW-UP RULE

Har answer ke baad interviewer ke 3 likely follow-ups socho: WHY? HOW? WHAT IF IT FAILS? Phir apne project se example connect karo.

INTERVIEW MASTER LAB 03

Cross-skill questions combining Python + data + AI + APIs + agents + analytics.

Python → Data

Pandas merge ke baad rows double ho gaye. Sabse pehle kya check karoge?
SMART ANSWER
Join key uniqueness, expected cardinality, duplicate IDs, pre/post row counts and grain.

AI → API

LLM call timeout ho raha hai. Production fix kya hoga?
SMART ANSWER
Timeouts, bounded retries for safe operations, fallback, structured errors, logging and provider health.

Agentic AI

Agent same tool repeat kar raha hai.
SMART ANSWER
Track action history, max steps, stop conditions, duplicate-action detection and fallback.

ML

Validation score unrealistic high hai.
SMART ANSWER
Leakage, duplicated samples, target leakage, preprocessing fit and split strategy inspect karo.

SQL

Revenue suddenly 2x ho gaya after JOIN.
SMART ANSWER
Join cardinality and duplicate keys; compare row counts before/after and aggrega at correct grain.

RAG

Retrieved context irrelevant hai.
SMART ANSWER
Embedding model, chunking, metadata filters, query formulation, top-k and reranking evaluate karo.

Flask

Client malformed JSON bhejta hai.
SMART ANSWER
Validate boundary, return 4xx structured error, do not call downstream AI/DB with invalid payload.

Power BI

Dashboard source total se match nahi karta.
SMART ANSWER
Filters, relationships, duplicate facts, date context, refresh status and DAX measure definition.

Resume

60% manual reduction ko defend kaise karoge?
SMART ANSWER
Define baseline, automated scope, measurement method and period. Only state you actually measured.

Behavior

“Mujhe answer nahi pata” ko professionally kaise handle?
SMART ANSWER
Clarify assumptions, reason from known principles, state uncertainty and explain how you would verify.

FOLLOW-UP RULE

Har answer ke baad interviewer ke 3 likely follow-ups socho: WHY? HOW? WHAT IF IT FAILS? Phir apne project se example connect karo.

INTERVIEW MASTER LAB 04

Cross-skill questions combining Python + data + AI + APIs + agents + analytics.

Python → Data

Pandas merge ke baad rows double ho gaye. Sabse pehle kya check karoge?
SMART ANSWER
Join key uniqueness, expected cardinality, duplicate IDs, pre/post row counts and grain.

AI → API

LLM call timeout ho raha hai. Production fix kya hoga?
SMART ANSWER
Timeouts, bounded retries for safe operations, fallback, structured errors, logging and provider health.

Agentic AI

Agent same tool repeat kar raha hai.
SMART ANSWER
Track action history, max steps, stop conditions, duplicate-action detection and fallback.

ML

Validation score unrealistic high hai.
SMART ANSWER
Leakage, duplicated samples, target leakage, preprocessing fit and split strategy inspect karo.

SQL

Revenue suddenly 2x ho gaya after JOIN.
SMART ANSWER
Join cardinality and duplicate keys; compare row counts before/after and aggrega at correct grain.

RAG

Retrieved context irrelevant hai.
SMART ANSWER
Embedding model, chunking, metadata filters, query formulation, top-k and reranking evaluate karo.

Flask

Client malformed JSON bhejta hai.
SMART ANSWER
Validate boundary, return 4xx structured error, do not call downstream AI/DB with invalid payload.

Power BI

Dashboard source total se match nahi karta.
SMART ANSWER
Filters, relationships, duplicate facts, date context, refresh status and DAX measure definition.

Resume

60% manual reduction ko defend kaise karoge?
SMART ANSWER
Define baseline, automated scope, measurement method and period. Only state you actually measured.

Behavior

“Mujhe answer nahi pata” ko professionally kaise handle?
SMART ANSWER
Clarify assumptions, reason from known principles, state uncertainty and explain how you would verify.

FOLLOW-UP RULE

Har answer ke baad interviewer ke 3 likely follow-ups socho: WHY? HOW? WHAT IF IT FAILS? Phir apne project se example connect karo.

INTERVIEW MASTER LAB 05

Cross-skill questions combining Python + data + AI + APIs + agents + analytics.

Python → Data

Pandas merge ke baad rows double ho gaye. Sabse pehle kya check karoge?
SMART ANSWER
Join key uniqueness, expected cardinality, duplicate IDs, pre/post row counts and grain.

AI → API

LLM call timeout ho raha hai. Production fix kya hoga?
SMART ANSWER
Timeouts, bounded retries for safe operations, fallback, structured errors, logging and provider health.

Agentic AI

Agent same tool repeat kar raha hai.
SMART ANSWER
Track action history, max steps, stop conditions, duplicate-action detection and fallback.

ML

Validation score unrealistic high hai.
SMART ANSWER
Leakage, duplicated samples, target leakage, preprocessing fit and split strategy inspect karo.

SQL

Revenue suddenly 2x ho gaya after JOIN.
SMART ANSWER
Join cardinality and duplicate keys; compare row counts before/after and aggrega at correct grain.

RAG

Retrieved context irrelevant hai.
SMART ANSWER
Embedding model, chunking, metadata filters, query formulation, top-k and reranking evaluate karo.

Flask

Client malformed JSON bhejta hai.
SMART ANSWER
Validate boundary, return 4xx structured error, do not call downstream AI/DB wit invalid payload.

Power BI

Dashboard source total se match nahi karta.
SMART ANSWER
Filters, relationships, duplicate facts, date context, refresh status and DAX measure definition.

Resume

60% manual reduction ko defend kaise karoge?
SMART ANSWER
Define baseline, automated scope, measurement method and period. Only state you actually measured.

Behavior

“Mujhe answer nahi pata” ko professionally kaise handle?
SMART ANSWER
Clarify assumptions, reason from known principles, state uncertainty and explain how you would verify.

FOLLOW-UP RULE

Har answer ke baad interviewer ke 3 likely follow-ups socho: WHY? HOW? WHAT IF IT FAILS? Phir apne project se example connect karo.

INTERVIEW MASTER LAB 06

Cross-skill questions combining Python + data + AI + APIs + agents + analytics.

Python → Data

Pandas merge ke baad rows double ho gaye. Sabse pehle kya check karoge?
SMART ANSWER
Join key uniqueness, expected cardinality, duplicate IDs, pre/post row counts and grain.

AI → API

LLM call timeout ho raha hai. Production fix kya hoga?
SMART ANSWER
Timeouts, bounded retries for safe operations, fallback, structured errors, logging and provider health.

Agentic AI

Agent same tool repeat kar raha hai.
SMART ANSWER
Track action history, max steps, stop conditions, duplicate-action detection and fallback.

ML

Validation score unrealistic high hai.
SMART ANSWER
Leakage, duplicated samples, target leakage, preprocessing fit and split strategy inspect karo.

SQL

Revenue suddenly 2x ho gaya after JOIN.
SMART ANSWER
Join cardinality and duplicate keys; compare row counts before/after and aggrega at correct grain.

RAG

Retrieved context irrelevant hai.
SMART ANSWER
Embedding model, chunking, metadata filters, query formulation, top-k and reranking evaluate karo.

Flask

Client malformed JSON bhejta hai.
SMART ANSWER
Validate boundary, return 4xx structured error, do not call downstream AI/DB with invalid payload.

Power BI

Dashboard source total se match nahi karta.
SMART ANSWER
Filters, relationships, duplicate facts, date context, refresh status and DAX measure definition.

Resume

60% manual reduction ko defend kaise karoge?
SMART ANSWER
Define baseline, automated scope, measurement method and period. Only state you actually measured.

Behavior

“Mujhe answer nahi pata” ko professionally kaise handle?
SMART ANSWER
Clarify assumptions, reason from known principles, state uncertainty and explain how you would verify.

FOLLOW-UP RULE

Har answer ke baad interviewer ke 3 likely follow-ups socho: WHY? HOW? WHAT IF IT FAILS? Phir apne project se example connect karo.

INTERVIEW MASTER LAB 07

Cross-skill questions combining Python + data + AI + APIs + agents + analytics.

Python → Data

Pandas merge ke baad rows double ho gaye. Sabse pehle kya check karoge?
SMART ANSWER
Join key uniqueness, expected cardinality, duplicate IDs, pre/post row counts and grain.

AI → API

LLM call timeout ho raha hai. Production fix kya hoga?
SMART ANSWER
Timeouts, bounded retries for safe operations, fallback, structured errors, logging and provider health.

Agentic AI

Agent same tool repeat kar raha hai.
SMART ANSWER
Track action history, max steps, stop conditions, duplicate-action detection and fallback.

ML

Validation score unrealistic high hai.
SMART ANSWER
Leakage, duplicated samples, target leakage, preprocessing fit and split strategy inspect karo.

SQL

Revenue suddenly 2x ho gaya after JOIN.
SMART ANSWER
Join cardinality and duplicate keys; compare row counts before/after and aggrega at correct grain.

RAG

Retrieved context irrelevant hai.
SMART ANSWER
Embedding model, chunking, metadata filters, query formulation, top-k and reranking evaluate karo.

Flask

Client malformed JSON bhejta hai.
SMART ANSWER
Validate boundary, return 4xx structured error, do not call downstream AI/DB with invalid payload.

Power BI

Dashboard source total se match nahi karta.
SMART ANSWER
Filters, relationships, duplicate facts, date context, refresh status and DAX measure definition.

Resume

60% manual reduction ko defend kaise karoge?
SMART ANSWER
Define baseline, automated scope, measurement method and period. Only state you actually measured.

Behavior

“Mujhe answer nahi pata” ko professionally kaise handle?
SMART ANSWER
Clarify assumptions, reason from known principles, state uncertainty and explain how you would verify.

FOLLOW-UP RULE

Har answer ke baad interviewer ke 3 likely follow-ups socho: WHY? HOW? WHAT IF IT FAILS? Phir apne project se example connect karo.

INTERVIEW MASTER LAB 08

Cross-skill questions combining Python + data + AI + APIs + agents + analytics.

Python → Data

Pandas merge ke baad rows double ho gaye. Sabse pehle kya check karoge?
SMART ANSWER
Join key uniqueness, expected cardinality, duplicate IDs, pre/post row counts and grain.

AI → API

LLM call timeout ho raha hai. Production fix kya hoga?
SMART ANSWER
Timeouts, bounded retries for safe operations, fallback, structured errors, logging and provider health.

Agentic AI

Agent same tool repeat kar raha hai.
SMART ANSWER
Track action history, max steps, stop conditions, duplicate-action detection and fallback.

ML

Validation score unrealistic high hai.
SMART ANSWER
Leakage, duplicated samples, target leakage, preprocessing fit and split strategy inspect karo.

SQL

Revenue suddenly 2x ho gaya after JOIN.
SMART ANSWER
Join cardinality and duplicate keys; compare row counts before/after and aggrega at correct grain.

RAG

Retrieved context irrelevant hai.
SMART ANSWER
Embedding model, chunking, metadata filters, query formulation, top-k and reranking evaluate karo.

Flask

Client malformed JSON bhejta hai.
SMART ANSWER
Validate boundary, return 4xx structured error, do not call downstream AI/DB with invalid payload.

Power BI

Dashboard source total se match nahi karta.
SMART ANSWER
Filters, relationships, duplicate facts, date context, refresh status and DAX measure definition.

Resume

60% manual reduction ko defend kaise karoge?
SMART ANSWER
Define baseline, automated scope, measurement method and period. Only state you actually measured.

Behavior

“Mujhe answer nahi pata” ko professionally kaise handle?
SMART ANSWER
Clarify assumptions, reason from known principles, state uncertainty and explain how you would verify.

FOLLOW-UP RULE

Har answer ke baad interviewer ke 3 likely follow-ups socho: WHY? HOW? WHAT IF IT FAILS? Phir apne project se example connect karo.

INTERVIEW MASTER LAB 09

Cross-skill questions combining Python + data + AI + APIs + agents + analytics.

Python → Data

Pandas merge ke baad rows double ho gaye. Sabse pehle kya check karoge?
SMART ANSWER
Join key uniqueness, expected cardinality, duplicate IDs, pre/post row counts and grain.

AI → API

LLM call timeout ho raha hai. Production fix kya hoga?
SMART ANSWER
Timeouts, bounded retries for safe operations, fallback, structured errors, logging and provider health.

Agentic AI

Agent same tool repeat kar raha hai.
SMART ANSWER
Track action history, max steps, stop conditions, duplicate-action detection and fallback.

ML

Validation score unrealistic high hai.
SMART ANSWER
Leakage, duplicated samples, target leakage, preprocessing fit and split strategy inspect karo.

SQL

Revenue suddenly 2x ho gaya after JOIN.
SMART ANSWER
Join cardinality and duplicate keys; compare row counts before/after and aggrega at correct grain.

RAG

Retrieved context irrelevant hai.
SMART ANSWER
Embedding model, chunking, metadata filters, query formulation, top-k and reranking evaluate karo.

Flask

Client malformed JSON bhejta hai.
SMART ANSWER
Validate boundary, return 4xx structured error, do not call downstream AI/DB with invalid payload.

Power BI

Dashboard source total se match nahi karta.
SMART ANSWER
Filters, relationships, duplicate facts, date context, refresh status and DAX measure definition.

Resume

60% manual reduction ko defend kaise karoge?
SMART ANSWER
Define baseline, automated scope, measurement method and period. Only state you actually measured.

Behavior

“Mujhe answer nahi pata” ko professionally kaise handle?
SMART ANSWER
Clarify assumptions, reason from known principles, state uncertainty and explain how you would verify.

FOLLOW-UP RULE

Har answer ke baad interviewer ke 3 likely follow-ups socho: WHY? HOW? WHAT IF IT FAILS? Phir apne project se example connect karo.

INTERVIEW MASTER LAB 10

Cross-skill questions combining Python + data + AI + APIs + agents + analytics.

Python → Data

Pandas merge ke baad rows double ho gaye. Sabse pehle kya check karoge?
SMART ANSWER
Join key uniqueness, expected cardinality, duplicate IDs, pre/post row counts and grain.

AI → API

LLM call timeout ho raha hai. Production fix kya hoga?
SMART ANSWER
Timeouts, bounded retries for safe operations, fallback, structured errors, logging and provider health.

Agentic AI

Agent same tool repeat kar raha hai.
SMART ANSWER
Track action history, max steps, stop conditions, duplicate-action detection and fallback.

ML

Validation score unrealistic high hai.
SMART ANSWER
Leakage, duplicated samples, target leakage, preprocessing fit and split strategy inspect karo.

SQL

Revenue suddenly 2x ho gaya after JOIN.
SMART ANSWER
Join cardinality and duplicate keys; compare row counts before/after and aggrega at correct grain.

RAG

Retrieved context irrelevant hai.
SMART ANSWER
Embedding model, chunking, metadata filters, query formulation, top-k and reranking evaluate karo.

Flask

Client malformed JSON bhejta hai.
SMART ANSWER
Validate boundary, return 4xx structured error, do not call downstream AI/DB with invalid payload.

Power BI

Dashboard source total se match nahi karta.
SMART ANSWER
Filters, relationships, duplicate facts, date context, refresh status and DAX measure definition.

Resume

60% manual reduction ko defend kaise karoge?
SMART ANSWER
Define baseline, automated scope, measurement method and period. Only state you actually measured.

Behavior

“Mujhe answer nahi pata” ko professionally kaise handle?
SMART ANSWER
Clarify assumptions, reason from known principles, state uncertainty and explain how you would verify.

FOLLOW-UP RULE

Har answer ke baad interviewer ke 3 likely follow-ups socho: WHY? HOW? WHAT IF IT FAILS? Phir apne project se example connect karo.

FINAL REVISION

One-Page Skill Matrix

Resume-aligned revision order.

Python

Functions • OOP • exceptions • files • modules • generators • decorators • • testing • DSA

Data

NumPy • Pandas • SQL • EDA • statistics • Excel • Power BI/DAX

AI/ML

Scikit-learn • preprocessing • metrics • NLP • spaCy • Transformers • TensorFlow/PyTorch concepts

GenAI

LLM • prompting • embeddings • RAG • tool calling • evaluation • cost/laten

Agents

LangChain • agent loop • state • tools • n8n • webhooks • REST • automati guardrails

Engineering

Flask • APIs • MySQL/SQLite • Git/GitHub • Docker • AWS concepts • MLOps

DAILY RULE

1 concept • 3 coding problems • 5 interview answers aloud • 2 output predictions •
1 debugging task • 1 resume-project explanation.

FINAL REVISION

Python Function Cheat Sheet

Interview-focused syntax and decisions.

SIGNATURES

```
def f(x, y=0):
    return x+y
def f(*args, **kwargs):
    ...
def f(x, *, mode="safe"):
    ...
Rule: explicit interface > clever interface.
```

FUNCTION DESIGN

Input validation → core logic → return → caller handles presentation.
Avoid hidden global state.
Avoid mutable defaults.
Prefer small single-responsibility functions.
Add type hints when useful.
Test boundaries.

ADVANCED

Decorator = wrap behavior
Generator = lazy values
Closure = function remembers enclosing scope
Higher-order function = accepts/returns function
Recursion = function calls itself
Pure function = same input → same output, no observable side effects

INTERVIEW ANSWER

“I would first define the function contract: expected input, output and invalid cases. Then I would implement the simplest readable logic, test edge cases, a state time/space complexity. If the function is used in production, I would also consider logging, validation and observability.”

FINAL REVISION

NumPy + Pandas Interview Cheat Sheet

Shape-first thinking + grain-first thinking.

NUMPY

shape → ndim → dtype → axis
Vectorize when possible.
Broadcast only compatible shapes.
Understand view/copy.
Use @ / matmul for matrix multiplication.
Check NaN before statistics.
Benchmark claims of speed.

PANDAS

inspect → clean → transform → aggregate → validate
Always define row grain.
Check missingness.
Check join cardinality.
Prefer vectorized ops over row-wise apply.
Validate totals before dashboarding.

5 RAPID QUESTIONS

1 axis=0 means? 2 loc vs iloc? 3 Why merge duplicates? 4 Why fit scaler only on train? 5 Why is row grain important?

ANSWER KEY

1 columns-wise aggregation in 2D • 2 labels vs positions • 3 many-to-many/duplicate keys • 4 prevent leakage • 5 prevents incorrect joins/KPIs.

FINAL REVISION

GenAI + Agentic AI Cheat Sheet

Reliable AI application > impressive demo.

RAG

ingest → clean → chunk → embed → index → retrieve → rerank (optional) → pr
generate → validate → cite/evaluate
Failures: bad chunks, poor retrieval, stale data, prompt injection, unsupporte
answers.

AGENT

state → decide → tool → observe → update → stop
Guardrails: typed tools, permission checks, max steps, timeout, retries, audit
logs, human approval for high-impact actions.

LLM APP

Provider wrapper → prompt/schema → model → output validation → business
Track latency, cost, errors and quality.

N8N

Trigger → validate → transform → AI/agent → API/DB → condition → action → er
branch → monitoring.

INTERVIEW LINE

“I don’t treat the LLM as a trusted source. I validate inputs, constrain tool permissions, validate outputs, handle provider failures, and evaluate quality on representative tasks.”

FINAL CHECKPOINT

100-Question Final Interview Challenge

Use this as the final gate before applying/interviewing.

01	Explain LEGB with a practical bug.	02	Why mutable defaults are dangerous?
03	return vs print?	04	*args vs **kwargs?
05	Decorator use case?	06	Generator vs list?
07	Shallow vs deep copy?	08	Exception hierarchy idea?
09	Context manager?	10	OOP composition vs inheritance?
11	NumPy shape?	12	axis=0 vs axis=1?
13	Broadcasting?	14	Vectorization?
15	view vs copy?	16	Pandas loc vs iloc?
17	groupby?	18	merge cardinality?
19	missing-value strategy?	20	row grain?
21	WHERE vs HAVING?	22	INNER vs LEFT JOIN?
23	COUNT(*) vs COUNT(col)?	24	Window function?
25	Index trade-off?	26	Transaction/ACID?
27	SQL injection prevention?	28	MySQL vs SQLite?
29	Flask route design?	30	HTTP status codes?
31	JSON validation?	32	API timeout?
33	Provider retry?	34	Service layer?

PASS CONDITION

Can explain 80% without notes + solve 70% coding tasks + defend all resume bullets
+ explain one failure path for each major system.

FINAL CHECKPOINT

100-QUESTION CHALLENGE • 2/3

Answer aloud first. Then verify with your own notes/projects.

35	Authentication vs authorization?	36	ML leakage?
37	Baseline?	38	Precision vs recall?
39	F1?	40	Cross-validation?
41	Pipeline?	42	Class imbalance?
43	Overfitting?	44	Regression metrics?
45	NLP intent classification?	46	Tokenizer?
47	Embedding?	48	Transformer?
49	Fine-tuning vs prompting?	50	Hallucination?
51	RAG?	52	Chunking trade-off?
53	Top-k?	54	Reranking?
55	Structured output?	56	Tool calling?
57	Agent vs workflow?	58	Agent loop?
59	Agent memory?	60	Agent safety?
61	n8n webhook validation?	62	Idempotency?
63	Human approval?	64	Power BI star schema?
65	DAX CALCULATE?	66	KPI definition?
67	EDA vs dashboard?	68	Docker image vs container?

PASS CONDITION

Can explain 80% without notes + solve 70% coding tasks + defend all resume bullets
+ explain one failure path for each major system.

FINAL CHECKPOINT

100-QUESTION CHALLENGE • 3/3

Answer aloud first. Then verify with your own notes/projects.

69	Git branch?	70	Secret management?
71	MLOps?	72	Monitoring?
73	Rollback?	74	AWS service selection approach?
75	Explain chatbot architecture.	76	Explain agentic automation architecture.
77	Explain 30+ analytics work.	78	Defend 60% reduction.
79	Biggest project challenge?	80	Next project improvement?
81	Why Python for AI?	82	Why Flask?
83	Why Pandas?	84	Why NumPy?
85	Why SQL?	86	Why RAG?
87	When not to use an agent?	88	How to reduce LLM cost?
89	How to reduce latency?	90	How to improve reliability?
91	How to debug wrong model output?	92	How to debug wrong dashboard totals?
93	How to debug API 500?	94	How to debug merge row explosion?
95	How to test an AI workflow?	96	How to explain an unknown answer?
97	How to handle conflicting requirements?	98	How to communicate trade-offs?
99	How to answer “tell me about yourself”?	100	Give your 60-second AI Engineer pitch.

PASS CONDITION

Can explain 80% without notes + solve 70% coding tasks + defend all resume bullets
+ explain one failure path for each major system.

OFFICIAL RESOURCE HUB

Python + Data Foundations

Keep this page open while you study. The links are intended as source-of-truth references.

Python

OPEN DOCS

https://docs.python.org/3/

What: Tutorial, language reference and standard library.

Use it for: Use when a Python behavior needs authoritative confirmation.

NumPy

OPEN DOCS

https://numpy.org/doc/stable/

What: Arrays, broadcasting, indexing, data types, linear algebra.

Use it for: Use for shape-first numeric debugging and reference.

pandas

OPEN DOCS

https://pandas.pydata.org/docs/

What: DataFrame, missing data, merge, groupby, time series, I/O.

Use it for: Use for EDA and transformation details.

scikit-learn

OPEN DOCS

https://scikit-learn.org/stable/

What: ML estimators, preprocessing, model selection and common pitfalls.

Use it for: Use for leakage-safe workflows and API details.

SOURCE PRACTICE RULE

Search within the official docs first; then update your code against the version you are actually using.

ML + NLP

Keep this page open while you study. The links are intended as source-of-truth references.

Transformers

OPEN DOCS

https://huggingface.co/docs/transformers/

What: Pretrained models, pipelines, tokenization and fine-tuning concepts.

Use it for: Use for NLP/Transformer implementation reference.

OpenAI API

OPEN DOCS

https://platform.openai.com/docs/quickstart

What: Official SDK/API quickstart and API patterns.

Use it for: Use current official guidance; keep keys out of source control.

LangChain

OPEN DOCS

https://docs.langchain.com/

What: Agents, tools, prompts, integrations and orchestration concepts.

Use it for: Use to understand abstractions; avoid learning syntax without architecture.

n8n

OPEN DOCS

https://docs.n8n.io/

What: Workflow automation, webhooks, integrations and AI features.

Use it for: Use for node configuration and workflow design.

SOURCE PRACTICE RULE

Search within the official docs first; then update your code against the version you are actually using.

OFFICIAL RESOURCE HUB

GenAI + Agentic + APIs

Keep this page open while you study. The links are intended as source-of-truth references.

Flask

OPEN DOCS

https://flask.palletsprojects.com/en/stable/

What: Routing, request handling and application patterns.

Use it for: Use for lightweight Python REST services.

FastAPI

OPEN DOCS

https://fastapi.tiangolo.com/

What: Typed API development, validation and interactive docs.

Use it for: Use when building typed production APIs.

SQLAlchemy

OPEN DOCS

https://docs.sqlalchemy.org/en/20/

What: ORM/Core, connections and database integration.

Use it for: Use for Python database access details.

Docker

OPEN DOCS

https://docs.docker.com/get-started/

What: Images, containers, Dockerfiles and container workflows.

Use it for: Use for reproducible AI service environments.

SOURCE PRACTICE RULE

Search within the official docs first; then update your code against the version you are actually using.

Engineering + BI

Keep this page open while you study. The links are intended as source-of-truth references.

Git

OPEN DOCS

https://git-scm.com/docs/git

What: Commands, branches, history, collaboration.

Use it for: Use for exact command behavior and workflows.

Power BI

OPEN DOCS

https://learn.microsoft.com/en-us/power-bi/fundamentals/

What: Power BI reports, dashboards and learning paths.

Use it for: Use for dashboard and model concepts.

DAX

OPEN DOCS

https://learn.microsoft.com/en-us/dax/

What: DAX formulas, functions, context and troubleshooting.

Use it for: Use when KPI measures behave unexpectedly.

PyTorch

OPEN DOCS

https://pytorch.org/docs/stable/

What: Tensors, modules, training and deployment concepts.

Use it for: Use for deep learning reference when needed.

SOURCE PRACTICE RULE

Search within the official docs first; then update your code against the version you are actually using.

Round 1 - Questions 1-10

Answer aloud first. Then reveal/check the model answer below each prompt.

PYTHON

01. What is the difference between == and is?

MODEL ANSWER: == checks value equality; is checks object identity. Prefer is None for None checks.

PYTHON

02. Explain LEGB with one practical bug.

MODEL ANSWER: Local -> Enclosing -> Global -> Built-in. A nested function may read an outer name; accidental globals can make tests/order unpredictable.

PYTHON

03. Why are mutable default arguments risky?

MODEL ANSWER: The same default object is reused across calls. Use None, then create the list/dict inside the function.

PYTHON

04. Generator vs list?

MODEL ANSWER: A generator yields values lazily and uses less memory for large streams; a list stores all values eagerly.

PYTHON

05. What is shallow-copy aliasing?

MODEL ANSWER: The outer container is copied but nested mutable objects can still be shared.

PYTHON

06. How do you debug an unexpected TypeError?

MODEL ANSWER: Reproduce, inspect values/types, isolate the smallest failing expression, change one thing, rerun and add a regression test.

DATA

07. loc vs iloc in Pandas?

MODEL ANSWER: loc uses labels; iloc uses integer positions.

DATA

08. Why can merge multiply rows?

MODEL ANSWER: A one-to-many or many-to-many join increases row count. Validate key uniqueness/cardinality before aggregating.

DATA

09. axis=0 vs axis=1 in NumPy?

MODEL ANSWER: axis=0 reduces across rows and typically keeps column-wise results; axis=1 reduces across columns and keeps row-wise results.

DATA

10. Why is shape important in NumPy?

MODEL ANSWER: Shape determines broadcasting compatibility and matrix multiplication dimensions.

Round 2 - Questions 11-20

Answer aloud first. Then reveal/check the model answer below each prompt.

SQL

11. WHERE vs HAVING?

MODEL ANSWER: WHERE filters rows before aggregation; HAVING filters groups after GROUP BY.

SQL

12. INNER vs LEFT JOIN?

MODEL ANSWER: INNER keeps matched rows; LEFT keeps all left rows and fills unmatched right columns with NULL.

SQL

13. What is a window function?

MODEL ANSWER: It computes analytics across related rows without collapsing them, e.g. rank, running sum or lag.

ML

14. What is data leakage?

MODEL ANSWER: Information from outside the training fold leaks into model fitting, making validation unrealistically strong.

ML

15. Why keep a baseline?

MODEL ANSWER: A simple baseline gives a reference point and prevents complicated models from appearing useful without evidence.

ML

16. Precision vs recall?

MODEL ANSWER: Precision asks how many predicted positives were correct; recall asks how many actual positives were found.

NLP

17. What does tokenization do?

MODEL ANSWER: It converts text into model-readable token IDs/subwords.

NLP

18. Why can max length matter?

MODEL ANSWER: Longer sequences increase memory/latency; truncation can also remove useful information.

GENAI

19. What is RAG?

MODEL ANSWER: Retrieve relevant external context and pass it to the generator so answers can be grounded in that retrieved information.

GENAI

20. Fine-tune vs RAG?

MODEL ANSWER: RAG adds external context at inference and is useful for changing knowledge; fine-tuning changes model behavior/weights for a task or style.

Round 3 - Questions 21-30

Answer aloud first. Then reveal/check the model answer below each prompt.

AGENTS

21. Agent vs workflow?

MODEL ANSWER: A workflow follows predefined steps; an agent can choose among actions based on state/model decisions, but reliable systems still bound tools and steps.

AGENTS

22. How do you stop an agent loop?

MODEL ANSWER: Max steps, timeouts, explicit stop conditions, tool errors and fallback states.

APIS

23. What belongs in a production API answer?

MODEL ANSWER: Contract, validation, status codes, timeouts, auth, secrets, logging, observability and failure handling.

APIS

24. Why use request timeouts?

MODEL ANSWER: Provider/network failures otherwise keep resources occupied and increase tail latency.

RESUME

25. Explain your AI chatbot in 60 seconds.

MODEL ANSWER: Problem -> Flask/REST entry -> NLP/intent routing -> response generation -> MySQL/SQLite history -> frontend -> one failure case -> next improvement.

RESUME

26. Why Flask for the chatbot?

MODEL ANSWER: It provided a lightweight Python web layer with clear routing and easy REST integration; the service layer remained modular.

RESUME

27. How did you use SpaCy/Transformers?

MODEL ANSWER: For NLP-based query processing/intent recognition and routing. Explain the exact components you actually implemented.

RESUME

28. Explain the n8n automation architecture.

MODEL ANSWER: Webhook -> validation -> AI decision -> allowed tool/API/DB action -> condition/error path -> audit/approval where needed.

RESUME

29. How would you defend the 60% reduction claim?

MODEL ANSWER: State the baseline, measurement period, activities counted and calculation. Use the number only if you can reproduce the measurement.

BEHAVIOR

30. What if you do not know an answer?

MODEL ANSWER: Say what you know, state the assumption, reason from principles and avoid inventing details.

Round 4 - Questions 31-40

Answer aloud first. Then reveal/check the model answer below each prompt.

CODING

31. What is your coding interview process?

MODEL ANSWER: Clarify inputs/outputs/constraints -> examples -> simplest correct approach -> code -> complexity -> tests -> edge cases.

CODING

32. Two-sum pattern?

MODEL ANSWER: Hash map for complement lookup; average O(n) time and O(n) space.

CODING

33. When use sliding window?

MODEL ANSWER: For contiguous subarray/substring problems where window state can be updated incrementally.

CODING

34. Binary search requirement?

MODEL ANSWER: Sorted or monotonic search space and a condition that lets you discard half the search space.

SYSTEM

35. How would you design a resilient LLM endpoint?

MODEL ANSWER: Validate input -> bounded provider call -> timeout/retry policy -> structured output validation -> logging/metrics -> safe error response.

SYSTEM

36. What should agent state contain?

MODEL ANSWER: Only needed context: current task, validated inputs, tool results, step count and decision metadata; avoid unnecessary long-term personal data.

SYSTEM

37. Where should human approval be added?

MODEL ANSWER: Before irreversible/high-impact side effects when automated mistakes would be costly.

SYSTEM

38. What do you monitor in an AI app?

MODEL ANSWER: Quality/task success, latency, provider errors, cost/token usage, retries, tool correctness and safety/invalid actions.

CAREER

39. Which project should you explain first?

MODEL ANSWER: Pick the project most relevant to the interviewer question, then explain the architecture and your specific contribution.

Tasks 1-5

Time target: 35 minutes. Explain constraints before coding.

EASY 1

Write a function that returns the first non-repeating character.

Expected approach: Answer: use a count map, then scan again. O(n) time.

REFERENCE

```
def first_unique(s):
    c={}
    for ch in s: c[ch]=c.get(ch,0)+1
    for ch in s:
        if c[ch]==1: return ch
    return None
```

EASY 2

Given two string numbers, return their numeric sum.

Expected approach: Cast with int() after validating the input.

REFERENCE

```
a,b="12","8"
print(int(a)+int(b))
```

MEDIUM 3

Normalize a Pandas email column and count missing values.

Expected approach: Use .str.strip().str.lower() and isna().sum().

REFERENCE

```
df["email"]=df["email"].str.strip().str.lower()
missing=df["email"].isna().sum()
```

MEDIUM 4

Find top 3 regions by total revenue.

Expected approach: Group, aggregate, sort descending and head(3).

REFERENCE

```
out=(df.groupby("region")["revenue"].sum()
      .sort_values(ascending=False).head(3))
```

MEDIUM 5

Write safe JSON parsing for model output.

Expected approach: Catch JSONDecodeError and validate required fields.

REFERENCE

```
import json
def parse(s):
    d=json.loads(s)
    if "answer" not in d: raise ValueError("answer")
    return d
```

Tasks 6-10

Harder tasks focus on AI/data/API reasoning, not syntax alone.

HARD 6

Implement cosine similarity safely for NumPy vectors.

Expected approach: Guard zero norms and check matching dimensions.

REFERENCE

```
den=np.linalg.norm(a)*np.linalg.norm(b)
score=0.0 if den==0 else float(a@b/den)
print(score)
```

HARD 7

Write a bounded tool-call loop.

Expected approach: Validate allowlisted tools and enforce max steps.

REFERENCE

```
for step in range(5):
    action=decide(state)
    if action["type"]=="final": break
    assert action["tool"] in TOOLS
    state["result"]=TOOLS[action["tool"]](action["arg
s"])
```

HARD 8

Design a Flask /chat error path for provider timeout.

Expected approach: Catch TimeoutError and return a stable 504 response while logging latency.

REFERENCE

```
try:
    out=service.answer(payload)
    return {"answer":out},200
except TimeoutError:
    log("provider_timeout")
    return {"error":"provider_timeout"},504
```

DEBUG 9

Pandas merge doubles row count. What do you inspect first?

Expected approach: Join key duplicates and expected cardinality. Use validate="many_to_one" or "one_to_one" when valid.

REFERENCE

```
merged=left.merge(right,on="id",how="left",validate="
many_to_one")
```

DEBUG 10

LLM output is valid-looking but unrelated to the question. What do you inspect?

Expected approach: Retrieval relevance, chunking, metadata filters, prompt grounding and model output validation.

SCORE

1 point each. Add +1 bonus per task where you clearly state complexity + edge case. Target: 8/10 or higher before the final mock.

Interview answers 01-20

Use after speaking. Compare the structure, not word-for-word phrasing.

01. What is the difference between == and is?
== checks value equality; is checks object identity. Prefer is None for None checks.

02. Explain LEGB with one practical bug.
Local -> Enclosing -> Global -> Built-in. A nested function may read an outer name; accidental globals can make tests/order unpredictable.

03. Why are mutable default arguments risky?
The same default object is reused across calls. Use None, then create the list/dict inside the function.

04. Generator vs list?
A generator yields values lazily and uses less memory for large streams; a list stores all values eagerly.

05. What is shallow-copy aliasing?
The outer container is copied but nested mutable objects can still be shared.

06. How do you debug an unexpected TypeError?
Reproduce, inspect values/types, isolate the smallest failing expression, change one thing, rerun and add a regression test.

07. loc vs iloc in Pandas?
loc uses labels; iloc uses integer positions.

08. Why can merge multiply rows?
A one-to-many or many-to-many join increases row count. Validate key uniqueness/cardinality before aggregating.

09. axis=0 vs axis=1 in NumPy?
axis=0 reduces across rows and typically keeps column-wise results; axis=1 reduces across columns and keeps row-wise results.

10. Why is shape important in NumPy?
Shape determines broadcasting compatibility and matrix multiplication dimensions.

11. WHERE vs HAVING?
WHERE filters rows before aggregation; HAVING filters groups after GROUP BY.

12. INNER vs LEFT JOIN?
INNER keeps matched rows; LEFT keeps all left rows and fills unmatched right columns with NULL.

13. What is a window function?
It computes analytics across related rows without collapsing them, e.g. rank, running sum or lag.

14. What is data leakage?
Information from outside the training fold leaks into model fitting, making validation unrealistically strong.

15. Why keep a baseline?
A simple baseline gives a reference point and prevents complicated models from appearing useful without evidence.

16. Precision vs recall?
Precision asks how many predicted positives were correct; recall asks how many actual positives were found.

17. What does tokenization do?
It converts text into model-readable token IDs/subwords.

18. Why can max length matter?
Longer sequences increase memory/latency; truncation can also remove useful information.

19. What is RAG?
Retrieve relevant external context and pass it to the generator so answers can be grounded in that retrieved information.

20. Fine-tune vs RAG?
RAG adds external context at inference and is useful for changing knowledge; fine-tuning changes model behavior/weights for a task or style.

Interview answers 21-40 + score rubric

Finish by explaining two resume projects and one failure path without notes.

21. Agent vs workflow?
A workflow follows predefined steps; an agent can choose among actions based on state/model decisions, but reliable systems still bound tools and steps.

22. How do you stop an agent loop?
Max steps, timeouts, explicit stop conditions, tool errors and fallback states.

23. What belongs in a production API answer?
Contract, validation, status codes, timeouts, auth, secrets, logging, observability and failure handling.

24. Why use request timeouts?
Provider/network failures otherwise keep resources occupied and increase tail latency.

25. Explain your AI chatbot in 60 seconds.
Problem -> Flask/REST entry -> NLP/intent routing -> response generation -> MySQL/SQLite history -> frontend -> one failure case -> next improvement.

26. Why Flask for the chatbot?
It provided a lightweight Python web layer with clear routing and easy REST integration; the service layer remained modular.

27. How did you use SpaCy/Transformers?
For NLP-based query processing/intent recognition and routing. Explain the exact components you actually implemented.

28. Explain the n8n automation architecture.
Webhook -> validation -> AI decision -> allowed tool/API/DB action -> condition/error path -> audit/approval where needed.

29. How would you defend the 60% reduction claim?
State the baseline, measurement period, activities counted and calculation. Use the number only if you can reproduce the measurement.

30. What if you do not know an answer?
Say what you know, state the assumption, reason from principles and avoid inventing details.

31. What is your coding interview process?
Clarify inputs/outputs/constraints -> examples -> simplest correct approach -> code -> complexity -> tests -> edge cases.

32. Two-sum pattern?
Hash map for complement lookup; average O(n) time and O(n) space.

33. When use sliding window?
For contiguous subarray/substring problems where window state can be updated incrementally.

34. Binary search requirement?
Sorted or monotonic search space and a condition that lets you discard half the search space.

35. How would you design a resilient LLM endpoint?
Validate input -> bounded provider call -> timeout/retry policy -> structured output validation -> logging/metrics -> safe error response.

36. What should agent state contain?
Only needed context: current task, validated inputs, tool results, step count and decision metadata; avoid unnecessary long-term personal data.

37. Where should human approval be added?
Before irreversible/high-impact side effects when automated mistakes would be costly.

38. What do you monitor in an AI app?
Quality/task success, latency, provider errors, cost/token usage, retries, tool correctness and safety/invalid actions.

39. Which project should you explain first?
Pick the project most relevant to the interviewer question, then explain the architecture and your specific contribution.

READY CHECK

Interview: 32+/40 clear answers | Coding: 8+/10 | Project defense: 3 projects + 3 failure cases | Then repeat weak modules.